

Study Guide: Introduction to Algorithms (CLRS, 4th Edition)

Generated 2026-06-01. Subject: Computer Science / Algorithms. Exam format: Mixed (MCQ, Short Answer, Problem-Solving, Essay). Coverage: comprehensive. Target length: ~10000 lines.

Chapter-by-Chapter Breakdown

Ch. 1 – The Role of Algorithms in Computing

Named Entities (Terms & Definitions)

- **Algorithm:** Any well-defined computational procedure that takes some value(s) as input and produces some value(s) as output in a finite amount of time; a sequence of computational steps that transform the input into the output.
- **Computational problem:** A problem whose statement specifies the desired input/output relationship for problem instances, typically of arbitrarily large size.
- **Problem instance:** The input (satisfying whatever constraints are imposed in the problem statement) needed to compute a solution to the problem.
- **Instance:** The input to a problem, e.g., the input sequence $\langle 31, 41, 59, 26, 41, 58 \rangle$ is an instance of the sorting problem.
- **Correct algorithm:** An algorithm that, for every problem instance provided as input, halts (finishes its computing in finite time) and outputs the correct solution to the problem instance.
- **Incorrect algorithm:** An algorithm that might not halt at all on some input instances, or it might halt with an incorrect answer.
- **Sorting problem (formally):** Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$. Output: A permutation (reordering) of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.
- **Monotonically increasing order:** The order of a sorted sequence where elements are non-decreasing.
- **Data structure:** A way to store and organize data in order to facilitate access and modifications.
- **Satellite data:** Data associated with a key (e.g., all fields of a record besides the sort key).
- **Record:** A key plus its associated satellite data.
- **NP-complete:** A class of problems for which no efficient algorithm is known; if an efficient algorithm exists for any one NP-complete problem, then efficient algorithms exist for all of them.

- **Approximation algorithm:** An algorithm that gives a good, but not necessarily the best possible, solution (used when exact solution is NP-complete).
- **Traveling-salesperson problem:** Given a central depot and delivery addresses, select an order of delivery stops that yields the lowest overall distance traveled by each truck; known to be NP-complete.
- **Online algorithm:** Algorithms that receive their input over time, rather than having all the input present at the start, and must decide how to proceed without knowing future data.
- **Multicore computers:** Chips designed to contain not just one but several processing “cores” (a type of parallel computer).
- **Task-parallel algorithms:** Algorithms designed for multicore computers that take advantage of multiple processing cores.
- **Efficiency:** Usually measured by speed (time), but other measures include memory usage, communication bandwidth, and energy consumption.
- **Discrete Fourier transform:** Converts the time domain to the frequency domain; approximates a signal as a weighted sum of sinusoids; applications in signal processing, data compression, multiplying large polynomials and integers.
- **Fast Fourier Transform (FFT):** An efficient algorithm for the discrete Fourier transform problem.
- **Linear programming:** A method for allocating scarce resources in the most beneficial way (Ch. 29).
- **Dynamic programming:** Important technique for solving problems involving determining similarity between DNA sequences (Ch. 14).
- **Topological sorting:** Given a mechanical design with parts that may include instances of other parts, list the parts in order so that each part appears before any part that uses it (Ch. 20).
- **Shortest path:** Finding the shortest route from one intersection to another on a road map modeled as a graph (Ch. 22).
- **Clustering algorithm:** Used to determine whether an image represents a cancerous tumor or benign one based on similarity (Ch. 33).
- **Huffman coding:** Compresses files by encoding characters by bit sequences of various lengths, with frequently occurring characters encoded by shorter bit sequences (Ch. 15).
- **Human Genome Project:** Goals: identify all roughly 30,000 genes in human DNA, determine sequences of roughly 3 billion chemical base pairs, store this information, develop tools for data analysis. Uses sophisticated algorithms including dynamic programming.
- **Public-key cryptography and digital signatures:** Core technologies for electronic commerce, based on numerical algorithms and number theory (Ch. 31).

Processes / Algorithms / Pathways

Insertion Sort (introduced conceptually)

- **Goal:** Sort a sequence of numbers into monotonically increasing order.
- **Complexity:** Time roughly $c_1 \cdot n^2$ for n items (proportional to n^2).
- **Key insight:** Insertion sort typically has a smaller constant factor than merge sort.

Merge Sort (introduced conceptually)

- **Goal:** Sort a sequence of numbers into monotonically increasing order.
- **Complexity:** Time roughly $c_2 \cdot n \cdot \lg n$ (proportional to $n \cdot \lg n$, where $\lg n = \log_2 n$).
- **Key insight:** Merge sort's advantage of $\lg n$ versus n more than compensates for difference in constant factors once n becomes large enough.
- **Crossover point:** No matter how much smaller c_1 is than c_2 , there is always a crossover point beyond which merge sort is faster.

Classifications & Hierarchies

- **Problem types:** (1) Problems with many candidate solutions, most of which do not solve the problem at hand; (2) Problems with practical applications (shortest path, topological sorting, etc.).
- **Algorithm types:** Correct vs. incorrect algorithms (controllable error rate).
- **Computational models:** Sequential (RAM model) vs. parallel (multicore, task-parallel) vs. online.
- **Computing technology components:** Hardware design, GUIs, object-oriented systems, web technologies, networking, machine learning – all rely on algorithms at their core.

Comparisons & Trade-offs

Dimension	Insertion Sort	Merge Sort
Running time	$\sim c_1 \cdot n^2$	$\sim c_2 \cdot n \cdot \lg n$
Constant factor	Typically smaller ($c_1 < c_2$)	Typically larger ($c_2 > c_1$)
Advantage	Small input sizes	Large input sizes
Growth rate	n (linear factor)	$\lg n$ (logarithmic factor)

Dimension	Insertion Sort	Merge Sort
Crossover	Always exists a point where merge sort wins	—

- Computer A (10 billion instr/sec, insertion sort, $2n^2$ instructions) vs Computer B (10 million instr/sec, merge sort, $50n \cdot \lg n$ instructions): For 10M numbers, Computer B runs $>17\times$ faster than Computer A despite being $1000\times$ slower in raw power. For 100M numbers, insertion sort takes >23 days, merge sort <4 hours.

Formulas & Equations

Efficiency comparison formula

(Computer A time for $n=10M$) = $(2 \cdot (10^7)^2 \text{ instructions}) / (10^{10} \text{ instr/sec}) = 20,000 \text{ sec} \approx 5.56 \text{ hours}$
 (Computer B time for $n=10M$) = $(50 \cdot 10^7 \cdot \lg(10^7) \text{ instructions}) / (10^6 \text{ instr/sec}) \approx 1,163 \text{ sec} \approx 19.4 \text{ min}$

Rules, Laws & Theorems

Property of NP-complete problems

- **Statement:** If an efficient algorithm exists for any one NP-complete problem, then efficient algorithms exist for all of them.
- **Significance:** A small change to the problem statement can cause a big change to the efficiency of the best known algorithm.

Reason to study algorithms even with infinite/free resources

- **Statement:** You would still like to be certain that your solution method terminates and does so with the correct answer.

Algorithm as a technology

- **Statement:** Total system performance depends on choosing efficient algorithms as much as on choosing fast hardware.

Edge Cases & Pitfalls

- “Incorrect algorithms can sometimes be useful, if you can control their error rate” – e.g., algorithms for finding large prime numbers with controllable error rate (Ch. 31).
- No efficient algorithm known for NP-complete problems, but nobody has ever proven one cannot exist.

- Physical limitations prevent ever-increasing clock speeds because power density increases superlinearly with clock speed (chips risk melting).
- For some problems (e.g., linear programming) the input arrives over time, requiring online algorithms.

Case Studies & Examples

Insertion sort vs. merge sort comparison (concrete)

- **What:** Computer A (10 billion instructions/sec) runs insertion sort with $2n^2$ instructions; Computer B (10 million instructions/sec) runs merge sort with $50n \cdot \lg n$ instructions. Input: 10 million numbers. Result: Computer A $\approx$ 5.56 hours, Computer B $\approx$ 19.4 minutes. Computer B $> 17\times$ faster than Computer A.
- **Significance:** Algorithmic efficiency can overcome massive hardware disadvantages.

Traveling-salesperson problem

- **What:** Delivery company with central depot needs order of stops minimizing distance. NP-complete. Solution uses approximation algorithms (Ch. 35).

Diagrams & Visuals

- [none in this chapter]

End-of-Chapter Material

Exercises 1.1 1.1-1: Describe your own real-world example that requires sorting. Describe one that requires finding the shortest distance between two points. 1.1-2: Other than speed, what other measures of efficiency might you need to consider in a real-world setting? 1.1-3: Select a data structure that you have seen, and discuss its strengths and limitations. 1.1-4: How are the shortest-path and traveling-salesperson problems similar? How are they different? 1.1-5: Suggest a real-world problem in which only the best solution will do. Then come up with one in which “approximately” the best solution is good enough. 1.1-6: Describe a real-world problem in which sometimes the entire input is available before you need to solve the problem, but other times the input is not entirely available in advance and arrives over time.

Exercises 1.2 1.2-1: Give an example of an application that requires algorithmic content at the application level, and discuss the function of the algorithms involved. 1.2-2: Suppose that for inputs of size n on a particular computer, insertion sort runs in $8n^2$ steps and merge sort runs in $64n \lg n$ steps. For which values of n does insertion sort beat merge sort? 1.2-3:

What is the smallest value of n such that an algorithm whose running time is $100n^2$ runs faster than an algorithm whose running time is 2^n on the same machine?

Problem 1-1: Comparison of running times For each function $f(n)$ and time t in the following table, determine the largest size n of a problem that can be solved in time t , assuming that the algorithm to solve the problem takes $f(n)$ microseconds. [Table includes times: 1 second, 1 minute, 1 hour, 1 day, 1 month, 1 year, 1 century; functions: $\lg n$, $\sqrt{n}$, n , $n \lg n$, n^2 , n^3 , 2^n , $n!$]

Ch. 2 – Getting Started

Named Entities (Terms & Definitions)

- **Keys:** The numbers to be sorted (often associated with satellite data).
- **Satellite data:** Additional data associated with a key.
- **Record:** A key plus its satellite data.
- **Pseudocode:** Description of algorithms similar to C, C++, Java, Python, or JavaScript; employs expressive methods most clear and concise; ignores aspects of software engineering (data abstraction, modularity, error handling) to convey essence concisely.
- **Loop invariant:** A property that holds before each iteration of a loop; used to prove algorithm correctness.
- **Subarray:** A contiguous portion of an array, denoted $A[i:j]$ (includes $A[i]$ through $A[j]$).
- **Insertion sort:** An efficient algorithm for sorting a small number of elements; works like sorting a hand of playing cards.
- **RAM (random-access machine) model:** Generic one-processor model of computation where instructions execute sequentially with no concurrent operations; each instruction/data access takes constant time.
- **Input size:** The number of items in the input (for sorting) or total number of bits (for integer multiplication); for graphs, characterized by both vertices and edges.
- **Running time:** The number of instructions and data accesses executed on a particular input.
- **Worst-case running time:** The longest running time for any input of size n .
- **Best-case running time:** The shortest running time for any input of size n .
- **Average-case running time:** The expected running time over a distribution of inputs.
- **Order of growth:** The rate at which running time increases with input size; only the leading term matters for large n .
- **Θ -notation (informal):** "Roughly proportional when n is large"; $\Theta(n^2)$ means "roughly proportional to n^2 when n is large."

- **Merge sort:** A sorting algorithm based on the divide-and-conquer method.
- **Incremental method:** For each element $A[i]$, insert it into its proper place in the sorted subarray $A[1:i]$ (used by insertion sort).
- **Divide-and-conquer method:** Break problem into subproblems similar to original but smaller, solve subproblems recursively, combine solutions to create solution to original problem.
- **Divide:** The problem into one or more subproblems that are smaller instances of the same problem.
- **Conquer:** The subproblems by solving them recursively.
- **Combine:** The subproblem solutions to form a solution to the original problem.
- **Base case:** When the problem is small enough to solve directly without recursing.
- **Recursive case:** When the problem requires recursion.
- **Recurrence (recurrence equation):** An equation that describes the overall running time on a problem of size n in terms of the running time on smaller inputs.
- **Recursion tree:** A tree whose nodes represent the costs incurred at various levels of the recursion.

Processes / Algorithms / Pathways

INSERTION-SORT (A, n)

- **Goal:** Sort array $A[1:n]$ into monotonically increasing order.
- **Input:** Array A of n numbers.
- **Output:** Array A sorted in place.
- **Steps:**
 1. for $i = 2$ to n
 2. $key = A[i]$
 3. // Insert $A[i]$ into the sorted subarray $A[1:i-1]$
 4. $j = i - 1$
 5. while $j > 0$ and $A[j] > key$
 6. $A[j+1] = A[j]$
 7. $j = j - 1$
 8. $A[j+1] = key$
- **Complexity:** Best-case: $\Theta(n)$ (when already sorted); Worst-case: $\Theta(n^2)$ (when reverse sorted); Average-case: $\Theta(n^2)$.
- **Example:** Input $\langle 5, 2, 4, 6, 1, 3 \rangle$. $i=2$: $key=2$, compare with $A[1]=5$, shift 5 right, insert 2 $\rightarrow \langle 2, 5, 4, 6, 1, 3 \rangle$. $i=3$: $key=4$, compare with 5 shift, insert $\rightarrow \langle 2, 4, 5, 6, 1, 3 \rangle$. $i=4$: $key=6$, no shift $\rightarrow \langle 2, 4, 5, 6, 1, 3 \rangle$. $i=5$: $key=1$, shift all 4, insert 1 $\rightarrow \langle 1, 2, 4, 5, 6, 3 \rangle$. $i=6$: $key=3$, shift 4, 5, 6, insert 3 $\rightarrow \langle 1, 2, 3, 4, 5, 6 \rangle$.

Loop Invariant for Insertion Sort

- **Statement:** At the start of each iteration of the for loop (lines 1-8), the subarray $A[1:i-1]$ consists of the elements originally in $A[1:i-1]$, but in sorted order.
- **Initialization:** Prior to first iteration ($i=2$), subarray $A[1:1]$ is the single original element, which is trivially sorted.
- **Maintenance:** The for loop body moves elements right until proper position for $A[i]$ is found, then inserts key. The subarray $A[1:i]$ then contains original elements in sorted order.
- **Termination:** i exceeds n ($i = n+1$). Substituting, $A[1:n]$ consists of original elements in sorted order – algorithm is correct.

MERGE (A, p, q, r)

- **Goal:** Merge two adjacent sorted subarrays $A[p:q]$ and $A[q+1:r]$ into a single sorted subarray $A[p:r]$.
- **Input:** Array A , indices p, q, r with $p \leq q < r$; assumes $A[p:q]$ and $A[q+1:r]$ are sorted.
- **Output:** $A[p:r]$ contains merged sorted elements.
- **Steps:**
 1. $nL = q - p + 1$ // length of $A[p:q]$
 2. $nR = r - q$ // length of $A[q+1:r]$
 3. let $L[0:nL-1]$ and $R[0:nR-1]$ be new arrays
 4. for $i = 0$ to $nL-1$
 5. $L[i] = A[p + i]$
 6. for $j = 0$ to $nR-1$
 7. $R[j] = A[q + j + 1]$
 8. $i = 0, j = 0, k = p$
 9. while $i < nL$ and $j < nR$
 10. if $L[i] \leq R[j]$
 11. $A[k] = L[i]; i = i + 1$
 12. else $A[k] = R[j]; j = j + 1$
 13. $k = k + 1$
 14. // Copy remainder of L
 15. while $i < nL$: $A[k] = L[i]; i++; k++$
 16. // Copy remainder of R
 17. while $j < nR$: $A[k] = R[j]; j++; k++$
- **Complexity:** $\Theta(n)$ time where $n = r - p + 1$.
- **Example:** Merge $A[9:16] = [2, 4, 6, 7, 1, 2, 3, 5]$. $L = [2, 4, 6, 7]$, $R = [1, 2, 3, 5]$. Compare 2 vs 1 $\rightarrow A[9]=1$; 2 vs 2 $\rightarrow A[10]=2$; 4 vs 2 $\rightarrow A[11]=2$; 4 vs 3 $\rightarrow A[12]=3$; 4 vs 5 $\rightarrow A[13]=4$; 6 vs 5 $\rightarrow A[14]=5$; then copy remaining L : $A[15]=6, A[16]=7$.

MERGE-SORT (A, p, r)

- **Goal:** Sort subarray $A[p:r]$ recursively.
- **Input:** Array A , indices p, r (inclusive).

- **Output:** $A[p:r]$ sorted.
- **Steps:**
 1. if $p \geq r$ // zero or one element? return
 2. $q = \lfloor (p + r)/2 \rfloor$ // midpoint
 3. MERGE-SORT(A, p, q) // recursively sort $A[p:q]$
 4. MERGE-SORT($A, q+1, r$) // recursively sort $A[q+1:r]$
 5. MERGE(A, p, q, r) // merge sorted halves
- **Initial call:** MERGE-SORT($A, 1, n$)
- **Complexity:** $T(n) = 2T(n/2) + \Theta(n) \rightarrow T(n) = \Theta(n \lg n)$.
- **Example:** $A = \langle 12, 3, 7, 9, 14, 6, 11, 2 \rangle$. Divide: $\langle 12, 3, 7, 9 \rangle$ and $\langle 14, 6, 11, 2 \rangle$. Recursively divide to single elements, then merge: $\langle 3, 12 \rangle, \langle 7, 9 \rangle, \langle 6, 14 \rangle, \langle 2, 11 \rangle \rightarrow \langle 3, 7, 9, 12 \rangle, \langle 2, 6, 11, 14 \rangle \rightarrow \langle 2, 3, 6, 7, 9, 11, 12, 14 \rangle$.

Pseudocode Conventions

- Indentation indicates block structure.
- for, while, repeat-until, if-else constructs similar to C/C++/Java/Python/JavaScript.
- Loop counter retains its value after loop exits (value that first exceeded bound).
- "to" for incrementing loop; "downto" for decrementing; "by" for other increments.
- "///" for comments.
- Variables are local to the procedure.
- Array access: $A[i]$. 1-origin indexing most common (specify bounds explicitly).
- ":" denotes subarray: $A[i:j]$ includes $A[i]$ through $A[j]$.
- Objects composed of attributes accessed via dot notation ($x.f$).
- Arrays/objects treated as pointers (references).
- NIL for null pointer.
- Parameters passed by value (but arrays by pointer).
- return can return multiple values.
- Boolean operators "and" and "or" are short-circuiting.
- "error" keyword indicates erroneous conditions.

Comparisons & Trade-offs

Dimension	Insertion Sort	Merge Sort
Design method	Incremental	Divide-and-conquer
Worst-case time	$\Theta(n^2)$	$\Theta(n \lg n)$
Best-case time	$\Theta(n)$	$\Theta(n \lg n)$
In-place?	Yes	No (needs temporary arrays)
Small inputs	Faster	Slower
Large inputs	Slower	Faster

Formulas & Equations

Insertion sort worst-case running time

$$T(n) = c \cdot n + c \cdot (n-1) + c \cdot (n-1) + c \cdot (\sum_{i=1}^{n-1} i) + c \cdot (\sum_{i=1}^{n-1} (i-1)) + c \cdot (\sum_{i=1}^{n-1} (i-1)) + c \cdot (n-1) = an^2 + bn + c \rightarrow \theta(n^2)$$

Insertion sort best-case running time

$$T(n) = c \cdot n + c \cdot (n-1) + c \cdot (n-1) + c \cdot (n-1) + c \cdot (n-1) = an + b \rightarrow \theta(n)$$

Merge sort recurrence

$$T(n) = 2T(n/2) + \theta(n) \rightarrow T(n) = \theta(n \lg n)$$

General divide-and-conquer recurrence

$T(n) = aT(n/b) + D(n) + C(n)$ - a = number of subproblems - n/b = size of each subproblem - $D(n)$ = time to divide - $C(n)$ = time to combine

Summation formulas

$$\sum_{i=1}^n i = n(n+1)/2 - 1 = n^2/2 + n/2 - 1 \quad \sum_{i=1}^n (i-1) = n(n-1)/2 = n^2/2 - n/2$$

Rules, Laws & Theorems

Loop Invariant (for proving correctness)

- **Statement:** Three properties must hold: Initialization (true prior to first iteration), Maintenance (if true before an iteration, remains true before the next iteration), Termination (loop terminates, and the invariant gives a useful property).
- **Relation to induction:** Initialization = base case; Maintenance = inductive step. Induction stops when loop terminates.

RAM Model Assumptions

- Each instruction/data access takes constant time.
- Instructions: arithmetic (add, subtract, multiply, divide, remainder, floor, ceiling), data movement (load, store, copy), control (conditional/unconditional branch, subroutine call/return).
- Data types: integer, floating point, character.
- Each word of data has limited bits ($c \cdot \log n$ bits for some constant $c \geq 1$).

- Does not account for memory hierarchy (caches, virtual memory).

Edge Cases & Pitfalls

- When n is not an exact power of 2, MERGE-SORT creates subarrays whose lengths differ by 1 (e.g., dividing length 7 gives length 4 and 3). Merging still takes $\Theta(n)$ time.
- For loops: the loop counter retains its value after loop exits (unlike some C++/Java contexts).
- Subarray $A[i:j]$ includes $A[j]$ (unlike Python's exclusive upper bound).
- Boolean short-circuiting: in " x and y ", y is evaluated only if x is TRUE (for "and") or FALSE (for "or").
- When computing 2^n , treat as constant time only when n is no more than the number of bits in a computer word.

Case Studies & Examples

Insertion sort on $\langle 5, 2, 4, 6, 1, 3 \rangle$

- See steps in algorithm walkthrough above. Figure 2.2 shows the complete operation.

Merge sort on $A = \langle 12, 3, 7, 9, 14, 6, 11, 2 \rangle$

- Initial call: MERGE-SORT($A, 1, 8$). Recursively divides into halves until single elements. Merge steps combine sorted subarrays bottom-up. Final result: sorted array.

Diagrams & Visuals

- **Figure 2.1:** Sorting a hand of cards using insertion sort (analogy).
- **Figure 2.2:** Operation of INSERTION-SORT on $\langle 5, 2, 4, 6, 1, 3 \rangle$. Blue rectangle = key, tan rectangles = compared values, orange arrows = shifts, blue arrows = key insertion. Shows 6 array positions over (a)-(e) iterations, (f) final sorted array.
- **Figure 2.3:** MERGE($A, 9, 12, 16$) merging $\langle 2, 4, 6, 7 \rangle$ (L) and $\langle 1, 2, 3, 5 \rangle$ (R). Shows (a)-(g) iterations of while loop, (h) termination state.
- **Figure 2.4:** Merge sort on array $\langle 12, 3, 7, 9, 14, 6, 11, 2 \rangle$. Shows indices p, q, r for each subarray. Italic numbers indicate order of procedure calls.
- **Figure 2.5:** Recursion tree for recurrence $T(n) = 2T(n/2) + c \cdot n$. Height = $\lg n + 1$. Each level above leaves costs $c \cdot n$; leaf level costs $c \cdot n$. Total: $c \cdot n \lg n + c \cdot n = \Theta(n \lg n)$.

End-of-Chapter Material

Exercises 2.1 2.1-1: Using Figure 2.2 as a model, illustrate the operation of INSERTION-SORT on an array initially containing the sequence $\langle 31, 41, 59, 26, 41, 58 \rangle$.

2.1-2: Consider the procedure SUM-ARRAY. It computes the sum of the n numbers in array $A[1:n]$. State a loop invariant for this procedure, and use its initialization, maintenance, and termination properties to show that SUM-ARRAY returns the sum.

```
SUM-ARRAY(A, n)
1 sum = 0
2 for i = 1 to n
3   sum = sum + A[i]
4 return sum
```

2.1-3: Rewrite INSERTION-SORT to sort into monotonically decreasing instead of monotonically increasing order.

2.1-4: Consider the searching problem: Input: A sequence of n numbers $\langle a_1, \dots, a_n \rangle$ stored in array $A[1:n]$ and a value x . Output: An index i such that $x = A[i]$ or NIL if x not in A . Write pseudocode for linear search. Using a loop invariant, prove your algorithm is correct.

2.1-5: Consider the problem of adding two n -bit binary integers a and b , stored in two n -element arrays $A[0:n-1]$ and $B[0:n-1]$. Write a procedure ADD-BINARY-INTEGERS that takes input arrays A and B , along with length n , and returns array C holding the sum.

Exercises 2.2 2.2-1: Express the function $n^3/1000 + 100n^2 - 100n + 3$ in terms of Θ -notation.

2.2-2: Consider selection sort: find smallest element of $A[1:n]$ and exchange with $A[1]$; find smallest of $A[2:n]$ and exchange with $A[2]$; continue for first $n-1$ elements. Write pseudocode. What loop invariant does it maintain? Why only $n-1$ elements? Give worst-case running time in Θ -notation. Is best-case any better?

2.2-3: Consider linear search again (Ex. 2.1-4). How many elements need to be checked on average (element equally likely to be any element)? How about worst case? Using Θ -notation, give average-case and worst-case running times.

2.2-4: How can you modify any sorting algorithm to have a good best-case running time?

Exercises 2.3 2.3-1: Using Figure 2.4 as a model, illustrate the operation of merge sort on the array $\langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$.

2.3-2: The test in line 1 of MERGE-SORT reads "if $p \geq r$ " rather than "if $p \neq r$." Argue that as long as initial call has $n \geq 1$, "if $p \neq r$ " suffices.

2.3-3: State a loop invariant for the while loop of lines 12-18 of MERGE. Show how to use it, along with the while loops of lines 20-23 and 24-27, to prove MERGE is correct.

2.3-4: Use mathematical induction to show that when $n \geq 2$ is an exact power of 2, the solution to $T(n) = 2T(n/2) + n$ is $T(n) = n \lg n$.

2.3-5: Write pseudocode for a recursive version of insertion sort (sort $A[1:n-1]$, then insert $A[n]$). Give a recurrence for its worst-case running time.

2.3-6: Write pseudocode (iterative or recursive) for binary search. Argue that worst-case running time is $\Theta(\lg n)$.

2.3-7: Would using binary search instead of linear search in INSERTION-SORT improve overall worst-case running time to $\Theta(n \lg n)$? Why or why not?

2.3-8: Describe an algorithm that, given a set S of n integers and another integer x , determines whether S contains two elements that sum to exactly x . Should take $\Theta(n \lg n)$ time in the worst case.

Problems

2-1: **Insertion sort on small arrays in merge sort** - a. Show insertion sort can sort n/k sublists, each of length k , in $\Theta(nk)$ worst-case time. - b. Show how to merge the sublists in $\Theta(n \lg(n/k))$ worst-case time. - c. Given modified algorithm runs in $\Theta(nk + n \lg(n/k))$ time, what is the largest value of k as a function of n for which it has same running time as standard merge sort? - d. How should you choose k in practice?

2-2: **Correctness of bubblesort** - BUBBLESORT(A, n):

```
1 for i = 1 to n-1
2   for j = n downto i+1
3     if  $A[j] < A[j-1]$ 
4       exchange  $A[j]$  with  $A[j-1]$ 
```

- 1. Prove that after BUBBLESORT, $A'[1] \leq A'[2] \leq \dots \leq A'[n]$. What else to prove?
- 1. State precisely a loop invariant for the for loop in lines 2-4, and prove it holds.
- 1. Using termination condition from (b), state a loop invariant for lines 1-4 to prove inequality.
- 1. What is worst-case running time? How does it compare with insertion sort?

2-3: **Correctness of Horner's rule** - Given coefficients $a_0, a_1, \dots, a_n$ for polynomial $P(x) = \sum_{i=0}^n a_i x^i$. - Horner's rule: $P(x) = a_0 + x(a_1 + x(a_2 + \dots + x(a_{n-1} + x a_n) \dots))$ - HORNER(A,n,x) computes P(x) in n steps.

```
HORNER(A,n,x)
1 p = 0
2 for i = n downto 0
3   p = A[i] + x.p
4 return p
```

- 1. Running time in Θ -notation?
- 1. Write naive polynomial-evaluation algorithm. Running time? Compare.
- 1. Use loop invariant to show termination yields P(x).

2-4: **Inversions** - If $i < j$ and $A[i] > A[j]$, then (i,j) is an inversion of A. - a. List the five inversions of $\langle 2, 3, 8, 6, 1 \rangle$. - b. What array with elements from $\{1, 2, \dots, n\}$ has the most inversions? How many? - c. What is the relationship between running time of insertion sort and number of inversions? Justify. - d. Give algorithm that determines number of inversions in $\Theta(n \lg n)$ worst-case time. (Hint: modify merge sort.)

Ch. 3 – Characterizing Running Times

Named Entities (Terms & Definitions)

- **Asymptotic efficiency:** How running time of an algorithm increases with input size in the limit, as input size increases without bound.
- **O -notation (big-oh):** Characterizes an upper bound on asymptotic behavior; says a function grows no faster than a certain rate.
- **Ω -notation (big-omega):** Characterizes a lower bound on asymptotic behavior; says a function grows at least as fast as a certain rate.
- **Θ -notation (theta):** Characterizes a tight bound on asymptotic behavior; says a function grows precisely at a certain rate (to within constant factors from above and below).
- **o -notation (little-oh):** Denotes an upper bound that is not asymptotically tight.
- **ω -notation (little-omega):** Denotes a lower bound that is not asymptotically tight.
- **Asymptotically nonnegative:** $f(n)$ must be nonnegative whenever n is sufficiently large.
- **Asymptotically positive:** A function that is positive for all sufficiently large n .
- **Anonymous function:** When asymptotic notation appears in a formula, it stands for some unnamed function.
- **Watershed function** (introduced contextually): $n^{\lg_b a}$ in master recurrences.

- **Polynomially bounded:** $f(n) = O(n^k)$ for some constant k .
- **Polylogarithmically bounded:** $f(n) = O(\lg^k n)$ for some constant k .
- **Monotonically increasing:** $m \leq n$ implies $f(m) \leq f(n)$.
- **Monotonically decreasing:** $m \leq n$ implies $f(m) \geq f(n)$.
- **Strictly increasing:** $m < n$ implies $f(m) < f(n)$.
- **Strictly decreasing:** $m < n$ implies $f(m) > f(n)$.
- **Floor:** $\lfloor x \rfloor$ = greatest integer less than or equal to x .
- **Ceiling:** $\lceil x \rceil$ = least integer greater than or equal to x .
- **Modular arithmetic:** $a \bmod n$ = remainder of a/n ; $a \equiv b \pmod{n}$ if $(a \bmod n) = (b \bmod n)$.
- **Polynomial of degree d :** $p(n) = \sum_{i=0}^d a_i n^i$ where $a_d \neq 0$.
- **Exponential:** a^n for $a > 0$.
- **Logarithms:** $\lg n = \log_2 n$, $\ln n = \log_e n$, $\lg^k n = (\lg n)^k$, $\lg \lg n = \lg(\lg n)$.
- **Factorial:** $n! = 1 \cdot 2 \cdot 3 \cdots n$ for $n \geq 0$; $0! = 1$.
- **Functional iteration:** $f^{(i)}(n) = f(f(\dots f(n)\dots))$ with $f^{(0)}(n) = n$.
- **Iterated logarithm:** $\lg^* n = \min\{i \geq 0: \lg^{(i)} n \leq 1\}$.
- **Fibonacci numbers:** $F_0 = 0$, $F_1 = 1$, $F_i = F_{i-1} + F_{i-2}$ for $i \geq 2$.
- **Golden ratio:** $\phi = (1 + \sqrt{5})/2 \approx 1.61803$.
- **Golden ratio conjugate:** $\bar{\phi} = (1 - \sqrt{5})/2 \approx -0.61803$.
- **Stirling's approximation:** $n! = \sqrt{2\pi n} (n/e)^n (1 + O(1/n))$.
- **Natural logarithm base:** $e \approx 2.71828$.

Processes / Algorithms / Pathways

Formal Definitions of Asymptotic Notation

O -notation: $O(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0\}$. - Function grows no faster than $g(n)$. - **Example:** $4n^2 + 100n + 500 = O(n^2)$. Choose $c = 5.05$, $n_0 = 100$, or $c = 19$, $n_0 = 10$, or $c = 604$, $n_0 = 1$. - **Non-member example:** $n^3 - 100n^2 \neq O(n^2)$ because no c , n_0 satisfy $n - 100 \leq c$ for all $n \geq n_0$.

Ω -notation: $\Omega(g(n)) = \{f(n): \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0\}$. - Function grows at least as fast as $g(n)$. - **Example:** $4n^2 + 100n + 500 = \Omega(n^2)$ with $c = 4$, any n_0 . $n^2/100 - 100n - 500 = \Omega(n^2)$ with $c = 2.49 \times 10^{-5}$, $n_0 = 10,005$ or $c = 0.0089$, $n_0 = 100,000$.

Θ -notation: $\Theta(g(n)) = \{f(n): \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0\}$. - Function grows precisely at rate $g(n)$ to within constant factors.

o -notation: $o(g(n)) = \{f(n): \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq f(n) < cg(n) \text{ for all } n \geq n_0\}$. - **Example:** $2n = o(n^2)$, but $2n^2 \neq o(n^2)$. - Limit characterization: $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$.

ω -notation: $\omega(g(n)) = \{f(n) : \text{for any positive constant } c > 0, \text{ there exists a constant } n_0 > 0 \text{ such that } 0 \leq cg(n) < f(n) \text{ for all } n \geq n_0\}$. - **Example:** $n^2/2 = \omega(n)$, but $n^2/2 \neq \omega(n^2)$. - Limit characterization: $\lim_{n \rightarrow \infty} f(n)/g(n) = \infty$.

Lower-bound argument for INSERTION-SORT worst case

- Assume n is multiple of 3. Divide array into groups of $n/3$ positions. If the $n/3$ largest values occupy the first $n/3$ positions, they must each move through the middle $n/3$ positions (each requiring at least $n/3$ executions of line 6). Time $\geq (n/3)(n/3) = n^2/9 = \Omega(n^2)$.
- Generalization: If αn largest values start in first αn positions, each must pass through middle $(1-2\alpha)n$ positions, requiring time $\geq \alpha(1-2\alpha)n^2$. For $0 < \alpha < 1/2$. Max at $\alpha = 1/4$ (value = $n^2/8$).

Comparisons & Trade-offs

Notation	Bound type	Example	Analogy with reals
$O(g(n))$	Upper bound (asymptotic $\leq$)	$f(n) \leq c \cdot g(n)$	$a \leq b$
$\Omega(g(n))$	Lower bound (asymptotic $\geq$)	$f(n) \geq c \cdot g(n)$	$a \geq b$
$\Theta(g(n))$	Tight bound (asymptotic $=$)	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$	$a = b$
$o(g(n))$	Non-tight upper bound (asymptotic $<$)	$f(n) < c \cdot g(n)$ for all $c > 0$	$a < b$
$\omega(g(n))$	Non-tight lower bound (asymptotic $>$)	$f(n) > c \cdot g(n)$ for all $c > 0$	$a > b$

Formulas & Equations

Theorem 3.1 (Asymptotic tight bound)

- **Statement:** For any two functions $f(n)$ and $g(n)$, $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.
- **Proof:** Exercise 3.2-4.

Transitivity

- $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n)) \implies f(n) = \Theta(h(n))$
- $f(n) = O(g(n))$ and $g(n) = O(h(n)) \implies f(n) = O(h(n))$
- $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n)) \implies f(n) = \Omega(h(n))$
- $f(n) = o(g(n))$ and $g(n) = o(h(n)) \implies f(n) = o(h(n))$
- $f(n) = \omega(g(n))$ and $g(n) = \omega(h(n)) \implies f(n) = \omega(h(n))$

Reflexivity

- $f(n) = \Theta(f(n))$, $f(n) = O(f(n))$, $f(n) = \Omega(f(n))$

Symmetry

- $f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$

Transpose symmetry

- $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$
- $f(n) = o(g(n))$ if and only if $g(n) = \omega(f(n))$

No trichotomy

- For any two real numbers a, b , exactly one holds: $a < b$, $a = b$, $a > b$.
- For functions, not always comparable (e.g., n and $n^{1+\sin n}$ oscillates between 0 and 2).

Floor and ceiling properties

- $\lfloor x \rfloor$ = greatest integer $\leq x$; $\lceil x \rceil$ = least integer $\geq x$.
- For any integer n : $\lfloor n \rfloor = \lceil n \rceil = n$.
- For all real x : $x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$.
- For any real $x \geq 0$ and integers $a, b > 0$: $\lfloor \lfloor x/a \rfloor / b \rfloor = \lfloor x / (ab) \rfloor$ and $\lceil \lceil x/a \rceil / b \rceil = \lceil x / (ab) \rceil$.
- For any integer n and real x : $\lfloor n/x \rfloor = n$ if and only if $x \leq n$; $\lceil n/x \rceil = n$ if and only if $n < x$.

Modular arithmetic

- $a \bmod n = a - n \lfloor a/n \rfloor$ for any integer a and positive integer n .
- $a \equiv b \pmod{n}$ if $(a \bmod n) = (b \bmod n)$, i.e., n divides $(b - a)$.

Polynomial

- $p(n) = \sum_{d=0}^k a_d n^d$, $a_d > 0 \rightarrow p(n) = \Theta(n^d)$.
- Asymptotically positive if $a_d > 0$.
- Monotonically increasing for $a \geq 0$ (n^a), decreasing for $a \leq 0$.

Exponential identities

- $a^0 = 1$, $a^1 = a$, $a^{-1} = 1/a$, $(a^m)^n = a^{mn}$, $a^{m \cdot a} n = a^{m+n}$.
- For $a > 1$ and any real b : $\lim_{n \rightarrow \infty} n^b / a^n = 0$ (exponentials dominate polynomials).

Exponential series

- $e^x = \sum_{k=0}^{\infty} x^k/k! = 1 + x + x^2/2! + x^3/3! + \dots$
- $e^x \geq 1 + x$ for all real x (equality only at $x = 0$).
- For $|x| \leq 1$: $1 + x \leq e^x \leq 1 + x + x^2$.
- $e^x = 1 + x + \theta(x^2)$ as $x \rightarrow 0$.

Logarithmic identities

- $\lg n = \log_2 n$, $\ln n = \log_e n$, $\lg^k n = (\lg n)^k$, $\lg \lg n = \lg(\lg n)$.
- $\log_b a = \log_c a / \log_c b$.
- $\log_b(1/a) = -\log_b a$.
- $\log_b a = 1 / \log_a b$.
- $a^{\{\log_b c\}} = c^{\{\log_b a\}}$.
- $\ln(1+x) = x - x^2/2 + x^3/3 - x^4/4 + x^5/5 - \dots$ for $|x| < 1$.
- For $x > -1$: $x/(1+x) \leq \ln(1+x) \leq x$ (equality only at $x = 0$).
- For $a > 0$, $b > 0$: any positive polynomial grows faster than any polylogarithmic function: $\lim_{n \rightarrow \infty} \lg^b n / n^a = 0$.

Stirling's approximation

$n! = \sqrt{2\pi n}(n/e)^n(1 + \theta(1/n))$ - Leads to: $n! = o(n^n)$, $n! = \omega(2^n)$, $\lg(n!) = \theta(n \lg n)$. - Also: $n! = \sqrt{2\pi n}(n/e)^n \cdot e^{\{\alpha_n\}}$ where $1/(12n+1) < \alpha_n < 1/(12n)$ for $n \geq 1$.

Fibonacci numbers and golden ratio

- $\phi = (1 + \sqrt{5})/2$ (≈ 1.61803), $\bar{\phi} = (1 - \sqrt{5})/2$ (≈ -0.61803).
- $x^2 = x + 1$ roots: $\phi^2 = \phi + 1$, $\bar{\phi}^2 = \bar{\phi} + 1$.
- $F_i = (\phi^i - \bar{\phi}^i)/\sqrt{5}$.
- Since $|\bar{\phi}| < 1$, $|\bar{\phi}^i|/\sqrt{5} < 1/\sqrt{5} < 1/2$, so $F_i = \phi^i/\sqrt{5} + 1/2$.
- Fibonacci numbers grow exponentially.

Rules, Laws & Theorems

- **Theorem 3.1:** $f(n) = \theta(g(n))$ iff $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.
- **Polynomial vs exponential:** Any exponential function with base > 1 grows faster than any polynomial function: $n^a = o(a^n)$ for $a > 1$.
- **Polynomial vs polylogarithmic:** Any positive polynomial grows faster than any polylogarithmic function: $\lg^b n = o(n^a)$ for $a > 0$.
- **Trichotomy fails:** Two functions may not be asymptotically comparable (e.g., n and $n^{1+\sin n}$).

Edge Cases & Pitfalls

- **Theta overstatement:** "Insertion sort's running time is $\theta(n^2)$ " is incorrect (it's $\theta(n)$ in best case). Correct: "worst-case running time is $\theta(n^2)$ ".

- **0-not-necessarily-tight:** $O(n^2)$ -time algorithm might actually run in $\Theta(n)$ time. O -notation is only an upper bound.
- **Conflating O and Θ :** "An $O(n \lg n)$ -time algorithm runs faster than an $O(n^2)$ -time algorithm" – maybe not, since the $O(n^2)$ algorithm might actually run in $\Theta(n)$ time.
- **Using asymptotic notation in inductive hypothesis:** Error prone because constants can change. Always use explicit constants in substitution proofs.
- **Asymptotic notation in equations:** When used in formulas, represents anonymous functions. Number of anonymous functions = number of times notation appears.
- **Floating point precision:** Many numbers cannot be represented exactly in floating point.
- **$O(1)$ for base cases:** When stating recurrences as $T(n) = O(1)$ for $n < 3$, this is an abuse since formal definition only applies for $n \geq n_0$.
- **Functions defined only on subset:** Asymptotic notation still applies on the domain where function is defined.

Case Studies & Examples

Insertion sort $\Omega(n^2)$ worst-case lower bound

- **What:** Assume $n/3$ largest values occupy first $n/3$ array positions. After sorting, these values must end up in last $n/3$ positions. Each must pass through middle $n/3$ positions one at a time via line 6. At least $(n/3) \times (n/3) = n^2/9$ total operations = $\Omega(n^2)$.

$4n^2 + 100n + 500 = O(n^2)$ proof

- **What:** Need c, n_0 such that $4n^2 + 100n + 500 \leq cn^2$ for $n \geq n_0$. Dividing by n^2 : $4 + 100/n + 500/n^2 \leq c$. With $n_0 = 100$, $c = 5.05$ works.

$n^3 - 100n^2 \neq O(n^2)$

- **What:** Would require $n - 100 \leq c$ for all $n \geq n_0$. Impossible because n grows without bound.

$n^2/100 - 100n - 500 = \Omega(n^2)$

- **What:** Need $1/100 - 100/n - 500/n^2 \geq c$. With $n_0 = 100,000$, $c = 0.0089$ works.

Insertion sort $O(n^2)$ upper bound

- **What:** Outer for loop runs $n-1$ times; inner while loop iterates at most $i-1 \leq n-1$ times per outer iteration. Total inner iterations $\leq (n-1)(n-1) < n^2$. Each iteration constant time, so $O(n^2)$.

MERGE-SORT recurrence $O(n \lg n)$

- $T(n) = 2T(n/2) + \Theta(n)$ – each level costs $c \cdot n$, there are $\lg n + 1$ levels, so total = $c \cdot n \lg n + c \cdot n = \Theta(n \lg n)$.

Diagrams & Visuals

- **Figure 3.1:** $\Omega(n^2)$ lower bound for insertion sort. Shows $n/3$ largest values in first $n/3$ positions must each move through middle $n/3$ positions. Arrow diagram: first $n/3 \rightarrow$ middle $n/3 \rightarrow$ last $n/3$.
- **Figure 3.2(a):** O -notation – $f(n)$ on or below $c g(n)$ for $n \geq n_0$.
- **Figure 3.2(b):** Ω -notation – $f(n)$ on or above $c g(n)$ for $n \geq n_0$.
- **Figure 3.2(c):** Θ -notation – $f(n)$ between $c_1 g(n)$ and $c_2 g(n)$ for $n \geq n_0$.

End-of-Chapter Material

Exercises 3.1 3.1-1: Modify lower-bound argument for insertion sort to handle input sizes that are not necessarily a multiple of 3. 3.1-2: Using reasoning similar to insertion sort, analyze running time of selection sort (Ex. 2.2-2). 3.1-3: Suppose α is a fraction in range $0 < \alpha < 1$. Generalize lower-bound argument: αn largest values start in first αn positions. What restriction on α ? What value of α maximizes number of times through middle $(1-2\alpha)n$ positions?

Exercises 3.2 3.2-1: Prove $\max\{f(n), g(n)\} = \Theta(f(n) + g(n))$ for asymptotically nonnegative f, g . 3.2-2: Explain why “The running time of algorithm A is at least $O(n^2)$ ” is meaningless. 3.2-3: Is $2^{n+1} = O(2^n)$? Is $2^{2n} = O(2^n)$? 3.2-4: Prove Theorem 3.1. 3.2-5: Prove running time of algorithm is $\Theta(g(n))$ iff worst-case running time is $O(g(n))$ and best-case running time is $\Omega(g(n))$. 3.2-6: Prove $o(g(n)) \cap \omega(g(n))$ is empty set. 3.2-7: Give definitions for $\Omega(g(n, m))$ and $\Theta(g(n, m))$.

Exercises 3.3 3.3-1: Show that if $f(n)$ and $g(n)$ are monotonically increasing, then so are $f(n)+g(n)$ and $f(g(n))$. If also nonnegative, then $f(n) \cdot g(n)$ is monotonically increasing. 3.3-2: Prove $\lceil \alpha n \rceil + \lceil (1-\alpha)n \rceil = n$ for any integer n and real $\alpha \in [0, 1]$. 3.3-3: Use equation (3.14) to show $(n + o(n))^k = \Theta(n^k)$ for any real k . Conclude $\lceil n \rceil^k = \Theta(n^k)$ and $\lfloor n \rfloor^k = \Theta(n^k)$. 3.3-4: Prove (a) Equation (3.21); (b) Equations (3.26)-(3.28); (c) $\lg(\Theta(n)) = \Theta(\lg n)$. 3.3-5: Is $\lceil \lg n \rceil!$ polynomially bounded? Is $\lceil \lg \lg n \rceil!$ polynomially bounded? 3.3-6: Which is asymptotically larger: $\lg(\lg^* n)$ or $\lg^*(\lg n)$? 3.3-7: Show that ϕ and $\bar{\phi}$ both satisfy $x^2 = x + 1$. 3.3-8: Prove by induction $F_i = (\phi^i - \bar{\phi}^i)/\sqrt{5}$. 3.3-9: Show that $k \lg k = \Theta(n)$ implies $k = \Theta(n/\lg n)$.

Problems

3-1: Asymptotic behavior of polynomials (see end-of-chapter for full statement) - Let $p(n)$ be a degree- d polynomial ($a_d > 0$). For constant k : - a. If $k \geq d \rightarrow p(n) = O(n^k)$ - b. If $k \leq d \rightarrow p(n) = \Omega(n^k)$ - c. If $k = d \rightarrow p(n) = \Theta(n^k)$ - d. If $k > d \rightarrow p(n) = o(n^k)$ - e. If $k < d \rightarrow p(n) = \omega(n^k)$

3-2: Relative asymptotic growths - Determine for each pair (A, B) whether A is $O/o/\Omega/\omega/\Theta$ of B . Assume $k \geq 1$, $\varepsilon > 0$, $c > 1$ constants.

3-3: Ordering by asymptotic growth rates - a. Rank 30 functions by order of growth ($g_i = \Omega(g_j)$, etc.). Partition into equivalence classes ($f = \Theta(g)$). - Functions: $\lg(\lg^* n)$, $2^{\lg^* n}$, $(\sqrt{2})^{\lg n}$, n^2 , $n!$, $(\lg n)!$, $(3/2)^n$, n^3 , $\lg^2 n$, $\lg(n!)$, $n^{\{1/\lg n\}}$, $\ln \ln n$, $\lg^* n$, $n \cdot 2^n$, $n^{\{\lg \lg n\}}$, $\ln n$, 1 , $2^{\{\lg n\}}$, $(\lg n)^{\{\lg n\}}$, e^n , $4^{\{\lg n\}}$, $(n+1)!$, $\lg^*(\lg n)$, $2^{\{\sqrt{2 \lg n}\}}$, n , 2^n , $n \lg n$, $2^{\{2^{n-1}\}}$ - b. Give an example of a nonnegative function $f(n)$ that is neither $O(g_i(n))$ nor $\Omega(g_i(n))$ for any g_i in part (a).

3-4: Asymptotic notation properties - Prove or disprove: - a. $f(n) = O(g(n))$ implies $g(n) = O(f(n))$ - b. $f(n) + g(n) = \Theta(\min\{f(n), g(n)\})$ - c. $f(n) = O(g(n))$ implies $\lg f(n) = O(\lg g(n))$ ($\lg g(n) \geq 1$, $f(n) \geq 1$) - d. $f(n) = O(g(n))$ implies $2^{\{f(n)\}} = O(2^{\{g(n)\}})$ - e. $f(n) = O((f(n))^2)$ - f. $f(n) = O(g(n))$ implies $g(n) = \Omega(f(n))$ - g. $f(n) = \Theta(f(n/2))$ - h. $f(n) + o(f(n)) = \Theta(f(n))$

3-5: Manipulating asymptotic notation - Prove: - a. $\Theta(\Theta(f(n))) = \Theta(f(n))$ - b. $\Theta(f(n)) + O(f(n)) = \Theta(f(n))$ - c. $\Theta(f(n)) + \Theta(g(n)) = \Theta(f(n) + g(n))$ - d. $\Theta(f(n)) \cdot \Theta(g(n)) = \Theta(f(n) \cdot g(n))$ - e. For constants $a_i, b_i > 0$ and k_i, k_i : $(a_i n + b_i)^{\{k_i\}} (a_i n + b_i)^{\{k_i\}} \dots = \Theta(n^{\{k_i + k_i + \dots\}})$ - f. For $S \subseteq \mathbb{Z}$: $\sum_{s \in S} \Theta(f(s)) = \Theta(\sum_{s \in S} f(s))$ - g. Show product version does not necessarily hold (give counterexample)

3-6: Variations on O and Ω - Ω_∞ : $f(n) \geq cg(n) \geq 0$ for infinitely many integers n . - a. Show for asymptotically nonnegative f, g : $f(n) = O(g(n))$ or $f(n) = \Omega_\infty(g(n))$ (or both). - b. Show two asymptotically nonnegative functions for which neither $f = O(g)$ nor $f = \Omega(g)$. - c. Advantages/disadvantages of Ω_∞ vs Ω . - O' : $f(n) = O'(g(n))$ iff $|f(n)| = O(g(n))$. - d. What happens to each direction of Theorem 3.1 if we substitute O' for O but still use Ω ? - $\tilde{O}$ (soft-oh): $f(n) = \tilde{O}(g(n))$ if $\exists c, k, n_0$: $0 \leq f(n) \leq c \cdot g(n) \cdot \lg^k(n)$ for $n \geq n_0$. - e. Define $\tilde{\Omega}$ and $\tilde{\Theta}$ similarly. Prove analog to Theorem 3.1.

3-7: Iterated functions - For each $f(n)$ and constant c , give tight bound on $f^*(n) = \min\{i: f^{(i)}(n) \leq c\}$: - a. $f(n) = n-1$, $c=0$ - b. $f(n) = \lg n$, $c=1$ - c. $f(n) = n/2$, $c=1$ - d. $f(n) = n/2$, $c=2$ - e. $f(n) = \sqrt{n}$, $c=2$ - f. $f(n) = n^{\{1/3\}}$, $c=2$ - g. $f(n) = n/\lg n$, $c=2$

Ch. 4 – Divide-and-Conquer

Named Entities (Terms & Definitions)

- **Divide-and-conquer method:** Solve a problem recursively: Divide (break into smaller subproblems), Conquer (solve subproblems recursively), Combine (subproblem solutions to form original solution).
- **Base case:** Problem small enough to solve directly without recursing.
- **Recursive case:** Problem requires recursion (three steps: divide, conquer, combine).
- **Recurrence:** An equation that describes a function in terms of its value on other (typically smaller) arguments.
- **Recursive case (recurrence):** Involves recursive invocation of the function on different (usually smaller) inputs.
- **Base case (recurrence):** Does not involve recursive invocation.
- **Well defined recurrence:** At least one function satisfies it.
- **Ill defined recurrence:** No function satisfies it.
- **Algorithmic recurrence:** A recurrence $T(n)$ such that for every sufficiently large threshold constant $n_0 > 0$: (1) For all $n < n_0$, $T(n) = \Theta(1)$; (2) For all $n \geq n_0$, every path of recursion terminates in a defined base case within a finite number of invocations.
- **Driving function $f(n)$:** The non-recursive cost in a master recurrence $T(n) = aT(n/b) + f(n)$; encompasses cost of dividing and combining.
- **Master recurrence:** $T(n) = aT(n/b) + f(n)$, where $a > 0$ and $b > 1$ are constants.
- **Watershed function:** $n^{\log_b a}$ – the function against which $f(n)$ is compared in the master theorem.
- **Polynomial-growth condition:** A function $f(n)$ satisfies this if there exists a constant n_0 such that for every constant $\phi \geq 1$, there exists $d > 1$ (depending on ϕ) such that $f(n)/d \leq f(\psi n) \leq d \cdot f(n)$ for all $1 \leq \psi \leq \phi$ and $n \geq n_0$.
- **Substitution method:** Guess form of solution, use mathematical induction to prove correct, solve for constants.
- **Recursion-tree method:** Models recurrence as a tree whose nodes represent costs at various levels; sum costs to solve.
- **Master method:** “Cookbook” method for solving recurrences of the form $T(n) = aT(n/b) + f(n)$. Memorize 3 cases.
- **Akra-Bazzi method:** General method for solving divide-and-conquer recurrences; involves calculus; handles subproblems of different sizes.
- **Continuous master theorem:** Variant of master theorem defined over sufficiently large positive real numbers (no floors/ceilings).
- **Dense matrix:** A matrix where most of the n^2 entries are not 0.
- **Sparse matrix:** A matrix where most entries are 0 and nonzero entries stored compactly.

- **Monge array:** An $m \times n$ array A of reals such that for all $i < k$ and $j < l$: $A[i,j] + A[k,l] \leq A[i,l] + A[k,j]$ (sum of upper-left + lower-right $\leq$ sum of lower-left + upper-right).
- **Generating function (formal power series):** $F(z) = \sum_{i=0}^{\infty} F_i z^i$ where F_i is the i -th Fibonacci number.
- **Regularity condition:** $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n (required for master theorem case 3).

Processes / Algorithms / Pathways

MATRIX-MULTIPLY (A, B, C, n)

- **Goal:** Compute $C = C + A \cdot B$ for $n \times n$ matrices.
- **Input:** Three $n \times n$ matrices A, B, C .
- **Output:** C updated to $C + A \cdot B$.
- **Steps:**
 1. for $i = 1$ to n
 2. for $j = 1$ to n
 3. $c_{ij} = c_{ij} + \sum_{k=1}^n a_{ik} \cdot b_{kj}$
- **Complexity:** $\Theta(n^3)$ (triply nested loops, each exactly n iterations).
- **Note:** To compute $C = A \cdot B$ only, initialize C to 0 first ($\Theta(n^2)$ time).

MATRIX-MULTIPLY-RECURSIVE (A, B, C, n)

- **Goal:** Compute $C = C + A \cdot B$ for $n \times n$ matrices using divide-and-conquer.
- **Input:** Three $n \times n$ matrices A, B, C (n assumed exact power of 2).
- **Output:** C updated to $C + A \cdot B$.
- **Steps:**
 1. if $n == 1$: $c_{ij} = c_{ij} + a_{ij} \cdot b_{ij}$; return
 2. Partition A, B, C into $n/2 \times n/2$ submatrices $A_{11}, A_{12}, A_{21}, A_{22}; B_{11}, B_{12}, B_{21}, B_{22}; C_{11}, C_{12}, C_{21}, C_{22}$
 3. // Compute $C_{11} = C_{11} + A_{11} \cdot B_{11} + A_{12} \cdot B_{21}$
 4. MATRIX-MULTIPLY-RECURSIVE($A_{11}, B_{11}, C_{11}, n/2$)
 5. MATRIX-MULTIPLY-RECURSIVE($A_{11}, B_{12}, C_{12}, n/2$)
 6. MATRIX-MULTIPLY-RECURSIVE($A_{12}, B_{11}, C_{21}, n/2$)
 7. MATRIX-MULTIPLY-RECURSIVE($A_{12}, B_{12}, C_{22}, n/2$)
 8. // Compute $C_{21} = C_{21} + A_{21} \cdot B_{11} + A_{22} \cdot B_{21}$
 9. MATRIX-MULTIPLY-RECURSIVE($A_{21}, B_{11}, C_{21}, n/2$)
 10. MATRIX-MULTIPLY-RECURSIVE($A_{21}, B_{12}, C_{22}, n/2$)
 11. MATRIX-MULTIPLY-RECURSIVE($A_{22}, B_{11}, C_{21}, n/2$)
 12. MATRIX-MULTIPLY-RECURSIVE($A_{22}, B_{12}, C_{22}, n/2$)
- **Complexity:** $T(n) = 8T(n/2) + \Theta(1) \rightarrow T(n) = \Theta(n^3)$.
- **Key insight:** 8 recursive multiplications of $n/2 \times n/2$ matrices. Submatrix partitioning by index calculation ($\Theta(1)$ time).

STRASSEN'S ALGORITHM for Matrix Multiplication

- **Goal:** Compute $C = C + A \cdot B$ for $n \times n$ matrices (n exact power of 2) asymptotically faster than $\Theta(n^3)$.
- **Input:** Three $n \times n$ matrices A, B, C .
- **Output:** C updated to $C + A \cdot B$.
- **Steps:**
 1. If $n=1$: single scalar multiplication and addition.
Return.
 2. Partition A, B, C into $n/2 \times n/2$ submatrices ($\Theta(1)$ by index calculation).
 3. Create 10 $n/2 \times n/2$ matrices $S_1, \dots, S_{10}$ (sums/differences of submatrices) ($\Theta(n^2)$):
 - $S_1 = B_{11} - B_{21}$
 - $S_2 = A_{11} + A_{21}$
 - $S_3 = A_{12} + A_{22}$
 - $S_4 = B_{12} - B_{22}$
 - $S_5 = A_{11} + A_{22}$
 - $S_6 = B_{11} + B_{22}$
 - $S_7 = A_{12} - A_{22}$
 - $S_8 = B_{12} + B_{22}$
 - $S_9 = A_{11} - A_{21}$
 - $S_{10} = B_{11} + B_{21}$
 4. Recursively compute 7 matrix products $P_1, \dots, P_7$ ($n/2 \times n/2$):
[$7T(n/2)$ time]
 - $P_1 = A_{11} \cdot S_1 (= A_{11} \cdot B_{11} - A_{11} \cdot B_{21})$
 - $P_2 = S_2 \cdot B_{12} (= A_{11} \cdot B_{12} + A_{21} \cdot B_{12})$
 - $P_3 = S_3 \cdot B_{12} (= A_{12} \cdot B_{12} + A_{22} \cdot B_{12})$
 - $P_4 = A_{11} \cdot S_4 (= A_{11} \cdot B_{12} - A_{11} \cdot B_{22})$
 - $P_5 = S_5 \cdot S_6 (= A_{11} \cdot B_{11} + A_{12} \cdot B_{11} + A_{21} \cdot B_{11} + A_{22} \cdot B_{11})$
 - $P_6 = S_7 \cdot S_8 (= A_{12} \cdot B_{12} + A_{12} \cdot B_{22} - A_{22} \cdot B_{12} - A_{22} \cdot B_{22})$
 - $P_7 = S_9 \cdot S_{10} (= A_{11} \cdot B_{11} + A_{11} \cdot B_{21} - A_{21} \cdot B_{11} - A_{21} \cdot B_{21})$
 5. Combine ($\Theta(n^2)$ time):
 - $C_{11} = C_{11} + P_1 + P_2 - P_4 + P_5$
 - $C_{12} = C_{12} + P_1 + P_3$
 - $C_{21} = C_{21} + P_2 + P_6$
 - $C_{22} = C_{22} + P_3 + P_7 - P_5 - P_6$
- **Complexity:** $T(n) = 7T(n/2) + \Theta(n^2) \rightarrow T(n) = \Theta(n^{\lg 7}) = \Theta(n^{2.81})$.
- **Key insight:** Reduces recursive multiplications from 8 to 7 at cost of 10 matrix additions/subtractions (18 total submatrix additions). Bushier recursion tree avoided.
- **Analogy:** $x^2 - y^2 = (x+y)(x-y)$: 2 multiplications $\rightarrow$ 1 multiplication + 2 additions.

Master Method (Three Cases)

- **Case 1:** $f(n) = O(n^{\log_b a - \epsilon})$ for $\epsilon > 0 \rightarrow T(n) = O(n^{\log_b a})$.
 - Watershed grows polynomially faster. Costs increase geometrically root→leaves. Leaf cost dominates.
- **Case 2:** $f(n) = \Theta(n^{\log_b a} \lg^k n)$ for $k \geq 0 \rightarrow T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$.
 - Watershed and driving function grow at nearly same rate. Most common: $k=0 \rightarrow T(n) = \Theta(n^{\log_b a} \lg n)$. Each level costs same; $\Theta(\lg n)$ levels.
- **Case 3:** $f(n) = \Omega(n^{\log_b a + \epsilon})$ for $\epsilon > 0$, and $af(n/b) \leq cf(n)$ for some $c < 1$ (regularity condition) $\rightarrow T(n) = O(f(n))$.
 - Driving function grows polynomially faster. Costs decrease geometrically root→leaves. Root dominates.

Substitution Method

- **Steps:** (1) Guess form of solution using symbolic constants. (2) Use mathematical induction to show solution works, find constants.
- **Key technique:** Prove O -bound and Ω -bound separately, then combine for Θ -bound.
- **Subtracting a lower-order term:** When inductive proof fails, strengthen hypothesis by subtracting a lower-order term (e.g., guess $cn-d$ instead of cn). This works because the lower-order term gets subtracted per recursive call.

Recursion-Tree Method

- **Steps:** Draw tree where each node represents cost of a subproblem. Sum costs per level. Sum all per-level costs. Use to generate guess, then verify with substitution method.

Akra-Bazzi Method

- **For recurrences:** $T(n) = \sum_{i=1}^k a_i T(n/b_i) + f(n)$, $a_i > 0$, $b_i > 1$.
- **Steps:** (1) Find unique real p such that $\sum a_i b_i^{-p} = 1$. (2) Solution: $T(n) = \Theta(n^p + n^p \int_1^n f(x)/x^{p+1} dx)$.
- **Requirement:** $f(n)$ must satisfy polynomial-growth condition to ignore floors/ceilings.

Comparisons & Trade-offs

Algorithm	Recursive multiplications	Additions	Running time
Simple recursive (M-M-R)	8 $(n/2 \times n/2)$	4	$\Theta(n^3)$
Strassen	7 $(n/2 \times n/2)$	18	$\Theta(n^{\lg 7}) \approx O(n^{2.81})$

Recurrence solving method	Applicability	Difficulty	Key feature
Substitution method	Most general	Hard (need to guess)	Robust, use explicit constants
Recursion-tree method	Many recurrences	Medium	Good for generating guesses
Master method	$T(n)=aT(n/b)+f(n)$	Easy (3 cases)	Limited to equal-sized subproblems
Akra-Bazzi method	General (unequal subproblems)	Hard (calculus)	Most general, handles unequal splits

Formulas & Equations

Matrix multiplication (standard)

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

Matrix partition (4 quadrants)

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} \Rightarrow$$

$$\begin{aligned} C_{11} &= A_{11} \cdot B_{11} + A_{12} \cdot B_{21} & C_{12} &= A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\ C_{21} &= A_{21} \cdot B_{11} + A_{22} \cdot B_{21} & C_{22} &= A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{aligned}$$

Simple recursive MM recurrence

$$T(n) = 8T(n/2) + \Theta(1) \rightarrow T(n) = \Theta(n^3)$$

Strassen's recurrence

$$T(n) = 7T(n/2) + \Theta(n^2) \rightarrow T(n) = \Theta(n^{\lg 7}) = \Theta(n^{2.81})$$

Master recurrence (general)

$$T(n) = aT(n/b) + f(n), \quad a > 0, \quad b > 1$$

Master theorem (three cases)

- Case 1: $f(n) = O(n^{\log_b a - \epsilon}) \rightarrow T(n) = \Theta(n^{\log_b a})$
- Case 2: $f(n) = \Theta(n^{\log_b a} \lg^k n) \rightarrow T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$
- Case 3: $f(n) = \Omega(n^{\log_b a + \epsilon})$ and $af(n/b) \leq cf(n)$ for $c < 1 \rightarrow T(n) = \Theta(f(n))$

Recursion-tree summation (Lemma 4.2)

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\lceil \log_b n \rceil} a^j f(n/b^j)$$

Akra-Bazzi solution

$T(n) = \Theta(n^p + n^p \int_1^n f(x)/x^{p+1} dx)$ where p satisfies $\sum a_i b_i^{-p} = 1$.

Monge array property

$A[i, j] + A[k, l] \leq A[i, l] + A[k, j]$ for all $i < k, j < l$.

Generating function for Fibonacci

$F(z) = \sum_{i=0}^{\infty} F_i z^i = z + zF(z) + z^2F(z) = z/(1 - z - z^2)$

Rules, Laws & Theorems

Master Theorem (Theorem 4.1)

- Let $a > 0$ and $b > 1$ be constants, $f(n)$ defined and nonnegative on sufficiently large reals. Define $T(n) = aT(n/b) + f(n)$ where $aT(n/b)$ means $a'T(\lfloor n/b \rfloor) + a''T(\lceil n/b \rceil)$ with $a = a' + a''$.
 - Case 1: $\exists \epsilon > 0: f(n) = O(n^{\log_b a - \epsilon}) \rightarrow T(n) = \Theta(n^{\log_b a})$
 - Case 2: $\exists k \geq 0: f(n) = \Theta(n^{\log_b a} \lg^k n) \rightarrow T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$
 - Case 3: $\exists \epsilon > 0: f(n) = \Omega(n^{\log_b a + \epsilon})$ and $af(n/b) \leq cf(n)$ for some $c < 1 \rightarrow T(n) = \Theta(f(n))$

Theorem 4.5 (Floors and ceilings)

- If $f(n)$ in Akra-Bazzi recurrence satisfies polynomial-growth condition, then replacing $T(n/b_i)$ with $T(\lfloor n/b_i \rfloor)$ or $T(\lceil n/b_i \rceil)$ does not change asymptotic solution.

Polynomial-growth condition (formal)

- A function $f(n)$ satisfies PGC if $\exists n_0$ such that $\exists \phi \geq 1, \exists d > 1: f(n)/d \leq f(\psi n) \leq d \cdot f(n)$ for all $1 \leq \psi \leq \phi$ and $n \geq n_0$.

Edge Cases & Pitfalls

- Master method doesn't apply:** Gap between cases 1 and 2 when $f(n) = n^{\log_b a} / \lg n$ (grows logarithmically slower, not polynomially slower). Gap between cases 2 and 3 when f grows polylogarithmically faster but not polynomially faster.
- Regularity condition may fail:** E.g., $f(n) = 2^{\lfloor \lg n \rfloor}$ satisfies case 3 conditions except regularity.
- Recurrence on integers vs reals:** Floors/ceilings don't affect asymptotic solution for most algorithm recurrences (Theorem 4.5).

- **Matrix partitioning methods:** Copying ($\Theta(n^2)$ time) vs index calculation ($\Theta(1)$ time) – makes no difference asymptotically for matrix multiplication, but can for matrix addition.
- **Asymptotic notation in inductive hypothesis:** Dangerous – constants can change. Always name constants explicitly.
- **Master method for n not exact power of 2:** OK – implicit floors/ceilings don't change asymptotic bounds.
- **When subproblems have different sizes:** Master method does not apply; use Akra-Bazzi instead.

Case Studies & Examples

Master method applications

- $T(n) = 9T(n/3) + n$: $a=9$, $b=3$, $n^{\log_3 9} = n^2$. $f(n) = n = O(n^{2-\epsilon})$ with $\epsilon \leq 1 \rightarrow$ Case 1 $\rightarrow T(n) = \Theta(n^2)$.
- $T(n) = T(2n/3) + 1$: $a=1$, $b=3/2$, $n^{\log_{3/2} 1} = n^0 = 1$. $f(n) = 1 = \Theta(1) \rightarrow$ Case 2 with $k=0 \rightarrow T(n) = \Theta(\lg n)$.
- $T(n) = 3T(n/4) + n \lg n$: $a=3$, $b=4$, $n^{\log_4 3} \approx n^{0.793}$. $f(n) = n \lg n = \Omega(n^{0.793+\epsilon})$ with $\epsilon \approx 0.2$. Regularity: $3(n/4) \lg(n/4) \leq (3/4)n \lg n$, $c=3/4$. $\rightarrow$ Case 3 $\rightarrow T(n) = \Theta(n \lg n)$.
- $T(n) = 2T(n/2) + n \lg n$: $a=2$, $b=2$, $n^{\log_2 2} = n$. $f(n) = n \lg n = \Theta(n \lg n)$. $k=1 \rightarrow$ Case 2 $\rightarrow T(n) = \Theta(n \lg^2 n)$.
- $T(n) = 2T(n/2) + n$: $a=2$, $b=2$, $n^{\log_2 2} = n$. $f(n) = n = \Theta(n)$ $\rightarrow$ Case 2, $k=0 \rightarrow T(n) = \Theta(n \lg n)$.
- $T(n) = 8T(n/2) + \Theta(1)$: $a=8$, $b=2$, $n^{\log_2 8} = n^3$. $f(n) = \Theta(1) = O(n^{3-\epsilon}) \rightarrow$ Case 1 $\rightarrow T(n) = \Theta(n^3)$.
- $T(n) = 7T(n/2) + \Theta(n^2)$: $a=7$, $b=2$, $n^{\lg 7} \approx n^{2.81}$. $f(n) = \Theta(n^2) = O(n^{\lg 7 - \epsilon})$ with $\epsilon \approx 0.8 \rightarrow$ Case 1 $\rightarrow T(n) = \Theta(n^{\lg 7})$.

$T(n) = 2T(n/2) + n$ – merge sort recurrence

- Recursion tree: height = $\lg n$, each of $\lg n$ levels costs n , leaves cost $\Theta(n)$. Total = $\Theta(n \lg n)$.

$T(n) = 3T(n/4) + cn^2$ recursion tree

- Root cost: cn^2 . Depth i : 3^i nodes, each cost $c(n/4^i)^2 \rightarrow$ per-level cost = $(3/16)^i \cdot cn^2$. Geometric series: $\text{sum} \leq (16/13)cn^2$. Leaves: $3^{\log_4 n} = n^{\log_4 3}$. Total: $O(n^2) \rightarrow$ verified by substitution with $d \geq (16/13)c$.

$T(n) = T(n/3) + T(2n/3) + cn$ (irregular)

- Height: $\log_{3/2} n = \Theta(\lg n)$. Each level $\leq cn$. Internal nodes total: $O(n \lg n)$. Leaves: $L(n) = L(n/3) + L(2n/3)$, $L(n) = \Theta(n)$. Therefore total: $O(n \lg n) + \Theta(n) = O(n \lg n)$. Verify $\Theta(n \lg n)$ by substitution.

Strassen's example: compute $\begin{bmatrix} 1 & 3 \\ 7 & 5 \end{bmatrix} \times \begin{bmatrix} 6 & 8 \\ 4 & 2 \end{bmatrix}$

- Show your work (Exercise 4.2-1).

Substitution method: $T(n) = 2T(n/2) + n$

- Guess $T(n) = O(n \lg n)$. Inductive hypothesis: $T(n) \leq cn \lg n$.
 $T(n) \leq 2c(n/2) \lg(n/2) + n \leq cn \lg(n/2) + n = cn \lg n - cn + n \leq cn \lg n$ for $c \geq 1$.

$x^2 - y^2$ trick (motivation for Strassen)

- $x^2 - y^2 = (x+y)(x-y)$: requires 1 multiplication + 2 additions instead of 2 multiplications + 1 subtraction. For large matrices, multiplication cost dominates.

Chip testing (Problem 4-6)

- More than $n/2$ chips good. Use $n/2$ pairwise tests to reduce to $\leq n/2$ chips while preserving majority-good property. Recurse to find one good chip. Then test remaining.

Monge arrays (Problem 4-7)

- Leftmost minimum per row is nondecreasing: $f(1) \leq f(2) \leq \dots \leq f(m)$. Divide-and-conquer algorithm on even rows, then compute odd rows.

Diagrams & Visuals

- **Figure 4.1:** Recursion tree for $T(n) = 3T(n/4) + cn^2$. Height = $\log_4 n$. Per-level costs $(3/16)^i \cdot cn^2$. Leaves: $n^{\log_4 3}$.
- **Figure 4.2:** Recursion tree for $T(n) = T(n/3) + T(2n/3) + cn$. Height = $\Theta(\lg n)$ (longest path down right edge). Per-level costs $\leq cn$.
- **Figure 4.3:** Recursion tree for $T(n) = aT(n/b) + f(n)$. Complete a -ary tree with $n^{\log_b a}$ leaves. Height = $\lceil \log_b n \rceil + 1$.

End-of-Chapter Material

Exercises 4.1 4.1-1: Generalize MATRIX-MULTIPLY-RECURSIVE for n not exact power of 2. Give recurrence. Argue $\Theta(n^3)$. 4.1-2: How fast to multiply $k \times n$ matrix by $n \times k$ matrix? $n \times k$ by $k \times n$? Which is faster? 4.1-3: If using copying instead of index calculation, how does recurrence change? Solution? 4.1-4: Write pseudocode for MATRIX-ADD-RECURSIVE (divide-and-conquer matrix addition). Give recurrence and solve. What if $\Theta(n^2)$ -time copying instead of $\Theta(1)$?

Exercises 4.2 4.2-1: Use Strassen's algorithm to compute $\begin{bmatrix} 1 & 3 \\ 5 \end{bmatrix} \times \begin{bmatrix} 6 & 8 \\ 4 & 2 \end{bmatrix}$. Show work. 4.2-2: Write pseudocode for Strassen's algorithm. 4.2-3: Largest k such that multiplying 3×3 matrices using k multiplications yields $O(n^{\lg 7})$ time? What is running time? 4.2-4: Pan's methods: 68×68 in 132,464; 70×70 in 143,640; 72×72 in 155,424 multiplications. Which is best asymptotically? Compare to Strassen. 4.2-5: Multiply complex numbers $a+bi$ and $c+di$ using only 3 real multiplications. 4.2-6: Given $\Theta(n^\alpha)$ -time squaring algorithm ($\alpha \geq 2$), show how to multiply different $n \times n$ matrices in $\Theta(n^\alpha)$ time.

Exercises 4.3 4.3-1: Show by substitution: a. $T(n) = T(n-1) + n \rightarrow O(n^2)$ b. $T(n) = T(n/2) + \Theta(1) \rightarrow O(\lg n)$ c. $T(n) = 2T(n/2) + n \rightarrow \Theta(n \lg n)$ d. $T(n) = 2T(n/2+17) + n \rightarrow O(n \lg n)$ e. $T(n) = 2T(n/3) + \Theta(n) \rightarrow \Theta(n)$ f. $T(n) = 4T(n/2) + \Theta(n) \rightarrow \Theta(n^2)$ 4.3-2: $T(n) = 4T(n/2) + n = \Theta(n^2)$. Show substitution fails with $T(n) \leq cn^2$. Show subtracting lower-order term works. 4.3-3: $T(n) = 2T(n-1) + 1 = O(2^n)$. Show substitution fails with $T(n) \leq c \cdot 2^n$. Show subtracting lower-order term works.

Exercises 4.4 4.4-1: For each recurrence, sketch recursion tree, guess asymptotic upper bound, verify by substitution: a. $T(n) = T(n/2) + n^3$ b. $T(n) = 4T(n/3) + n$ c. $T(n) = 4T(n/2) + n$ d. $T(n) = 3T(n-1) + 1$ 4.4-2: Use substitution to prove $L(n) = \Omega(n)$ for recurrence (4.15). Conclude $L(n) = \Theta(n)$. 4.4-3: Use substitution to prove $T(n) = \Omega(n \lg n)$ for recurrence (4.14). Conclude $T(n) = \Theta(n \lg n)$. 4.4-4: Use recursion tree to solve $T(n) = T(\alpha n) + T((1-\alpha)n) + \Theta(n)$ for $0 < \alpha < 1$.

Exercises 4.5 4.5-1: Use master method for: a. $T(n) = 2T(n/4) + 1$ b. $T(n) = 2T(n/4) + \sqrt{n}$ c. $T(n) = 2T(n/4) + n$ d. $T(n) = 2T(n/4) + n^2$ 4.5-2: Professor Caesar's algorithm: divide into $n/4 \times n/4$ submatrices, $\Theta(n^2)$ for divide/combine. What is largest a (subproblems) for which it could be asymptotically faster than Strassen? 4.5-3: Use master method to show binary search $T(n) = T(n/2) + \Theta(1) = \Theta(\lg n)$. 4.5-4: Show $f(n) = \lg n$ fails regularity condition for $a=1, b=2$. 4.5-5: Show $f(n) = 2^{\lfloor \lg n \rfloor}$ satisfies case 3 conditions except regularity.

Exercises 4.6 4.6-1: Show lower bound for case 2 summation. 4.6-2: Show regularity condition implies existence of $\epsilon > 0$ for case 3. 4.6-3: Show for $f(n) = n/\lg n$, the summation in (4.19) has solution $\Theta(n \lg \lg n)$.

Exercises 4.7 4.7-1: Show scaling driving function by constant doesn't affect asymptotic solution. 4.7-2: Show $f(n)=n^2$ satisfies PGC; $f(n)=2^n$ does not. 4.7-3: Show PGC implies asymptotic positivity. 4.7-4: Example of function not satisfying PGC but $f(\Theta(n)) = \Theta(f(n))$. 4.7-5: Use Akra-Bazzi for: a. $T(n) = T(n/2) + T(n/3) + T(n/6) + n \lg n$ b. $T(n) = 3T(n/3) + 8T(n/4) + n^2/\lg n$ c. $T(n) = (2/3)T(n/3) + (1/3)T(2n/3) + \lg n$ d. $T(n) = (1/3)T(n/3) + 1/n$ e. $T(n) = 3T(n/3) + 3T(2n/3) + n^2$ 4.7-6: Use Akra-Bazzi to prove the continuous master theorem.

Problems

4-1: **Recurrence examples** – Tight bounds for: a. $T(n) = 2T(n/2) + n^3$ b. $T(n) = T(8n/11) + n$ c. $T(n) = 16T(n/4) + n^2$ d. $T(n) = 4T(n/2) + n^2 \lg n$ e. $T(n) = 8T(n/3) + n^2$ f. $T(n) = 7T(n/2) + n^2 \lg n$ g. $T(n) = T(n-1) + 1/n$ h. $T(n) = T(n-2) + n^2$

4-2: **Parameter-passing costs** – Three strategies (by pointer $\Theta(1)$, by copy $\Theta(N)$, by copy subrange $\Theta(n)$). Give recurrences for binary search, MERGE-SORT, MATRIX-MULTIPLY-RECURSIVE. Solve with tight bounds.

4-3: **Solving recurrences with change of variables** a-d: $T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n$. Define $m = \lg n$, $S(m) = T(2^m)$. Solve: $T(n) = \Theta(\lg n \lg \lg n)$. e. $T(n) = T(\sqrt{n}) + \Theta(\lg n)$ f. $T(n) = \sqrt{n} \cdot T(\sqrt{n}) + n$

4-4: **More recurrence examples** – Tight bounds: a. $T(n) = 5T(n/3) + n \lg n$ b. $T(n) = 3T(n/3) + n/\lg n$ c. $T(n) = \sqrt{n} \cdot T(\sqrt{n}) + n$ d. $T(n) = 2T(n/2 - 2) + n/2$ e. $T(n) = 2T(n/2) + n/\lg n$ f. $T(n) = T(n/2) + T(n/4) + T(n/8) + n$ g. $T(n) = T(n-1) + 1/n$ h. $T(n) = T(n-1) + \lg n$ i. $T(n) = T(n-2) + 1/\lg n$ j. $T(n) = T(n/2) + n \cdot 2^n$

4-5: **Fibonacci numbers** – Generating function $F(z) = \sum F_i z^i$. a. Show $F(z) = z + zF(z) + z^2F(z)$ b. Show $F(z) = z/(1-z-z^2) = (1/\sqrt{5})(1/(1-\phi z) - 1/(1-\phi^2 z))$ c. Show $F(z) = \sum (1/\sqrt{5})(\phi^i - \phi^{i-2})z^i$ d. Prove $F_i = \phi^i / \sqrt{5}$ rounded to nearest integer e. Prove $F_{i+2} \geq \phi^i$ for $i \geq 0$

4-6: **Chip testing** – Chips test each other; good chips tell truth. a. If $\geq n/2$ chips bad, cannot necessarily determine good chips. b. $\lfloor n/2 \rfloor$ pairwise tests sufficient to reduce to $\leq \lfloor n/2 \rfloor$ chips with majority good. c. Recursively identify one good chip. Recurrence: tests needed. d. Identify all good chips with additional $\Theta(n)$ tests.

4-7: **Monge arrays** a. Array is Monge iff condition holds for all adjacent rows/columns. b. Fix a non-Monge array to be Monge. c. Prove $f(1) \leq f(2) \leq \dots \leq f(m)$ for leftmost minima. d. Compute leftmost minima in odd rows given even rows ($O(m+n)$ time). e. Recurrence $\rightarrow O(m + n \log m)$.

Ch. 5 – Probabilistic Analysis and Randomized Algorithms

Named Entities (Terms & Definitions)

- **Probabilistic analysis:** The use of probability in the analysis of problems; computing average-case running time over a distribution of inputs.
- **Average-case running time:** The expected value of running time over the distribution of possible inputs.

- **Randomized algorithm:** An algorithm whose behavior is determined not only by its input but also by values produced by a random-number generator.
- **Expected running time:** The expectation of running time over the distribution of values returned by the random-number generator (for randomized algorithms).
- **Random-number generator:** $\text{RANDOM}(a,b)$ returns an integer between a and b inclusive, each equally likely, independent of previous calls.
- **Uniform random permutation:** A permutation where each of the $n!$ possible permutations appears with equal probability ($1/n!$).
- **Indicator random variable:** $I\{A\} = 1$ if event A occurs, 0 otherwise. $E[I\{A\}] = \Pr\{A\}$ (Lemma 5.1).
- **Linearity of expectation:** $E[\sum X_i] = \sum E[X_i]$ even when random variables are dependent.
- **Birthday paradox:** Only 23 people needed for $>50\%$ chance two share a birthday; 28 people for expected number of pairs ≥ 1 .
- **Balls and bins model:** Randomly toss balls into b bins; each toss independent, equally likely any bin. Used for hashing analysis.
- **Coupon collector's problem:** Expected number of tosses to get one ball in each bin $= b \cdot H_b \approx b \ln b$, where $H_b = 1 + 1/2 + \dots + 1/b$.
- **Geometric distribution:** Number of trials until first success; probability of p ; expected value $= 1/p$.
- **Binomial distribution:** $b(k; n, p) = C(n,k) p^k (1-p)^{n-k}$.
- **Secretary problem:** Variant of hiring problem where you hire exactly once; use strategy of interviewing k candidates then hiring first better than all seen. Optimal $k = n/e$, success probability $\geq 1/e$.
- **k-permutation:** A sequence containing k of n elements, no repetitions. There are $n!/(n-k)!$ such permutations.
- **Hat-check problem:** n customers give hats to hat-check person; hats returned randomly. Expected number receiving own hat $= 1$.
- **Inversion:** If $i < j$ and $A[i] > A[j]$, then (i,j) is an inversion.
- **Bernoulli trial:** A trial with two outcomes (success/failure), probability p of success.

Processes / Algorithms / Pathways

HIRE-ASSISTANT (n)

- **Goal:** Hire the best-qualified office assistant from n candidates.
- **Input:** n candidates interviewed sequentially.
- **Output:** A hiring process where current best is always replaced when better candidate appears.
- **Steps:**
 1. $\text{best} = 0$ // dummy least-qualified candidate
 2. for $i = 1$ to n

3. interview candidate i
 4. if candidate i is better than candidate best
 5. best = i
 6. hire candidate i
- **Cost:** Interview cost c_i (per candidate), hiring cost c_h (per hire). Total = $O(c_i \cdot n + c_h \cdot m)$ where m = number hired.
 - **Worst case:** Candidates in strictly increasing order $\rightarrow$ hire n times $\rightarrow O(c_h \cdot n)$.
 - **Average case:** Random order $\rightarrow$ expected hires $\approx \ln n \rightarrow O(c_h \ln n)$.
 - **Proof using indicator variables:** Let $X_i = I\{\text{candidate } i \text{ hired}\}$. $\Pr\{\text{candidate } i \text{ hired}\} = 1/i$ (equally likely best of first i). $E[X] = \sum_{i=1}^n 1/i = H_n = \ln n + O(1)$.

RANDOMIZED-HIRE-ASSISTANT (n)

- **Goal:** Hire best-qualified assistant with expected $O(c_h \ln n)$ cost regardless of input.
- **Steps:**
 1. Randomly permute the list of candidates
 2. HIRE-ASSISTANT(n)
- **Key insight:** Random permutation enforces uniform random order. No input produces worst-case behavior; only “unlucky” random permutation.
- **Expected hiring cost:** $O(c_h \ln n)$ (Lemma 5.3).

RANDOMLY-PERMUTE (A, n)

- **Goal:** Produce a uniform random permutation of $A[1:n]$ in place.
- **Steps:**
 1. for $i = 1$ to n
 2. swap $A[i]$ with $A[\text{RANDOM}(i, n)]$
- **Complexity:** $\Theta(n)$ time.
- **Correctness proof (loop invariant):** Prior to i -th iteration, for each possible $(i-1)$ -permutation, subarray $A[1:i-1]$ contains it with probability $(n-i+1)!/n!$. At termination ($i=n+1$), $A[1:n]$ is any n -permutation with probability $1/n!$.
- **Key:** Each element $A[i]$ chosen uniformly from $A[i..n]$; after i -th iteration, $A[i]$ never altered.

ONLINE-MAXIMUM (k, n)

- **Goal:** Hire the best-qualified candidate by interviewing exactly once (online hiring problem).
- **Input:** k (number to reject initially), n (total candidates).
- **Output:** Index of candidate hired.
- **Steps:**
 1. best-score = $-\infty$
 2. for $i = 1$ to k

3. if $\text{score}(i) > \text{best-score}$: $\text{best-score} = \text{score}(i)$
 4. for $i = k+1$ to n
 5. if $\text{score}(i) > \text{best-score}$: return i
 6. return n // hire last if none better
- **Success probability:** Max when $k = n/e$. $\Pr\{\text{success}\} \geq 1/e \approx 0.368$.
 - **Derivation:** $\Pr\{\text{success}\} = (k/n)(H_{n-1} - H_{k-1}) \approx (k/n)(\ln n - \ln k)$. Set derivative = 0 $\rightarrow \ln k = \ln n - 1 \rightarrow k = n/e$.

Formulas & Equations

Indicator random variable

$I\{A\} = \{ 1 \text{ if } A \text{ occurs, } 0 \text{ otherwise} \}$ $E[I\{A\}] = \Pr\{A\}$ (Lemma 5.1)

Linearity of expectation

$E[\sum X_i] = \sum E[X_i]$ (holds even if X_i are dependent)

Expected number of hires

$E[X] = \sum_{i=1}^n 1/i = H_n = \ln n + \gamma + O(1/n)$ where $\gamma \approx 0.57721$ (Euler's constant)

Birthday paradox – exact probability

$\Pr\{B_k\} = \Pr\{\text{all } k \text{ birthdays distinct}\} = \prod_{i=1}^{k-1} (1 - i/n)$
 $\Pr\{B_k\} \leq e^{-k(k-1)/(2n)}$ Threshold: $k(k-1) \geq 2n \ln 2 \rightarrow k \geq 23$
 for $n=365$ (≥ 31 for Mars $n=669$)

Birthday paradox – indicator variable approximation

$E[X] = C(k, 2)/n = k(k-1)/(2n)$ Expected pairs with same birthday ≥ 1 when $k \geq \sqrt{2n}$. For $n=365$: $k \geq 28$.

Balls and bins

- Expected balls in given bin: n/b
- Expected tosses until given bin gets a ball: b (geometric distribution)
- Expected tosses until every bin has ≥ 1 ball: $b \cdot H_b \approx b \ln b$ (coupon collector)

Streaks – upper bound

- $\Pr\{\text{streak of length } \geq 2 \lceil \lg n \rceil \text{ begins at position } i\} = 1/2^{2 \lceil \lg n \rceil} \leq 1/n^2$
- $\Pr\{\text{streak of length } \geq 2 \lceil \lg n \rceil \text{ occurs anywhere}\} \leq n \cdot 1/n^2 = 1/n$
- $E[\text{length of longest streak}] = O(\lg n)$

Streaks – lower bound

- Partition flips into groups of $\lceil (\lg n)/2 \rceil$. $\Pr\{\text{group all heads}\} = 1/2^{\lceil (\lg n)/2 \rceil} \geq 1/\sqrt{n}$
- $\Pr\{\text{no group all heads}\} \leq (1 - 1/\sqrt{n})^{\lceil n/(\lg n) \rceil} \leq e^{-\sqrt{n}/\lg n}$
→ small
- $\Pr\{\text{streak} \geq \lceil (\lg n)/2 \rceil\} \geq 1 - O(1/n)$
- $E[\text{length of longest streak}] = \Omega(\lg n)$

Online hiring – probability of success

$\Pr\{S\} = (k/n)(H_{n-1} - H_{k-1}) \approx (k/n)(\ln n - \ln k)$ - Maximized at $k = n/e$, giving $\Pr\{S\} \geq 1/e$.

Coupon collector expectation

$$E[n] = b \cdot (1 + 1/2 + 1/3 + \dots + 1/b) = b \cdot H_b \approx b \ln b$$

Rules, Laws & Theorems

Lemma 5.1 (Indicator random variable expectation)

- **Statement:** $E[I\{A\}] = \Pr\{A\}$.
- **Proof:** By definition of expectation: $E[I\{A\}] = 1 \cdot \Pr\{A\} + 0 \cdot \Pr\{\bar{A}\} = \Pr\{A\}$.

Lemma 5.2 (HIRE-ASSISTANT average-case cost)

- **Statement:** Assuming random order of candidates, HIRE-ASSISTANT has average-case total hiring cost $O(c_h \ln n)$.
- **Proof:** Expected hires $\approx \ln n$; follows from definition of hiring cost.

Lemma 5.3 (RANDOMIZED-HIRE-ASSISTANT expected cost)

- **Statement:** Expected hiring cost of RANDOMIZED-HIRE-ASSISTANT is $O(c_h \ln n)$.
- **Proof:** Random permutation achieves identical situation as probabilistic analysis.

Lemma 5.4 (RANDOMLY-PERMUTE correctness)

- **Statement:** RANDOMLY-PERMUTE computes a uniform random permutation.
- **Proof:** Loop invariant: prior to i -th iteration, each $(i-1)$ -permutation in $A[1:i-1]$ with probability $(n-i+1)!/n!$. At termination: each n -permutation with probability $1/n!$.

Edge Cases & Pitfalls

- **Average-case vs expected running time:** Average-case = probability distribution over inputs. Expected = algorithm itself makes random choices.
- **Uniform random permutation guarantee:** Showing each element has $1/n$ probability of ending up in position j is INSUFFICIENT to prove uniform random permutation (Ex. 5.3-4).
- **Weak vs strong randomness:** Pairwise independence of birthdays is NOT sufficient for the birthday paradox analysis (Ex. 5.4-4).
- **Biased random:** BIASED-RANDOM outputs 1 with unknown probability p . Can convert to unbiased by calling twice: if outputs (0,1) return 0; if (1,0) return 1; otherwise repeat.
- **Random sampling:** RANDOM-SAMPLE (m, n) returns random m -subset with only m calls to RANDOM, rather than n calls followed by taking first m .
- **PERMUTE-WITH-ALL does NOT produce uniform random permutation** – because it can swap with already-placed elements, creating bias.
- **Inductive hypothesis with asymptotic notation:** Dangerous because constants change. Always name constants explicitly.

Case Studies & Examples

Birthday paradox

- **What:** 23 people needed for $\geq 50\%$ chance of shared birthday; 28 for expected pairs ≥ 1 .
- **Mars:** 669 days $\rightarrow$ 31 Martians for 50% chance; 38 for expected pairs ≥ 1 .
- **Significance:** Surprisingly few people relative to 365 days.

Balls and bins / Coupon collector

- **What:** Expected $b \ln b$ tosses to get one ball in every bin. Used in hashing analysis.

Streaks of consecutive heads

- **What:** In n fair coin flips, longest streak expected $\Theta(\lg n)$. Upper: $2 \lg n$. Lower: $\lg n / 2$.
- **Example:** $n=1000$ flips, ≥ 20 heads streak probability $\leq 1/1000$; ≥ 30 heads $\leq 1/1,000,000$.

Online hiring (secretary problem)

- **What:** Interview and reject first k candidates; hire first thereafter better than all before. Optimal $k = n/e$, success probability $\geq 1/e \approx 36.8\%$.

Hiring problem deterministic vs randomized

- $A[] = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]$: 10 hires (worst case).
- $A[] = [10, 9, 8, 7, 6, 5, 4, 3, 2, 1]$: 1 hire (best case).
- $A[] = [5, 2, 1, 8, 4, 7, 10, 9, 3, 6]$: 3 hires.
- With randomization, even $A[]$ can produce any permutation depending on random choices.

Hat-check problem

- **What:** Expected number of customers receiving own hat = 1 (sum of indicator variables, each with probability $1/n$).

Expected number of inversions

- **What:** For uniform random permutation of $1..n$, expected inversions = $n(n-1)/4$.

End-of-Chapter Material

Exercises 5.1 5.1-1: Show that the assumption in HIRE-ASSISTANT line 4 (always determine best) implies a total order on candidate ranks. □ 5.1-2: Implement RANDOM(a,b) using only RANDOM(0,1). Expected time? □ 5.1-3: Given BIASED-RANDOM (outputs 1 with unknown p), devise algorithm returning 0 or 1 each with probability $1/2$. Expected time as function of p ?

Exercises 5.2 5.2-1: Probability of hiring exactly once? Exactly n times? (Random order) 5.2-2: Probability of hiring exactly twice? 5.2-3: Use indicator random variables to compute expected sum of n dice. 5.2-4: Verify linearity of expectation holds even without independence: (a) two independent dice; (b) second die = first; (c) second die = $7 - \text{first}$. Expected sum each case? 5.2-5: Hat-check problem: expected number getting own hat back? 5.2-6: Expected number of inversions in uniform random permutation?

Exercises 5.3 5.3-1: Modify RANDOMLY-PERMUTE so loop invariant applies to nonempty subarray from start; modify proof. 5.3-2: Does PERMUTE-WITHOUT-IDENTITY (swap $A[i]$ with $\text{RANDOM}(i+1,n)$) produce all permutations except identity? (Answer: No – it also cannot produce permutations where any element is fixed.) 5.3-3: Does PERMUTE-WITH-ALL (swap $A[i]$ with $\text{RANDOM}(1,n)$) produce uniform random permutation? (Answer: No – there are n^n equally likely outcomes, which cannot be distributed evenly among $n!$ permutations.) 5.3-4: Show PERMUTE-BY-CYCLE: each element has $1/n$ probability at any position, but permutation is NOT uniformly random. (It generates only n cyclic shifts.) 5.3-5: Show RANDOM-SAMPLE(m,n) returns random m -subset with only m calls to RANDOM (instead of n calls + take first m).

□ **Exercises 5.4** 5.4-1: How many people needed for probability $\geq 1/2$ that someone shares YOUR birthday? That at least two have birthday July 4? 5.4-2: How many people for probability ≥ 0.99 of

shared birthday? Expected pairs for that many? 5.4-3: Expected number of ball tosses until some bin has 2 balls? □ 5.4-4: Is mutual independence of birthdays required for birthday paradox, or is pairwise independence sufficient? Justify. □ 5.4-5: How many people for likely three with same birthday? □ 5.4-6: Probability that k-string over a set of size n forms a k-permutation? Relation to birthday paradox? □ 5.4-7: Toss n balls into n bins. Expected empty bins? Expected bins with exactly one ball? □ 5.4-8: Sharpen lower bound: probability $\geq 1 - 1/n$ that streak of length $\lg n - 2 \lg \lg n$ occurs.

Problems

5-1: **Probabilistic counting** (Morris's algorithm) - b-bit counter, count n_i for value i . INCREMENT: with prob $1/(n_{i+1} - n_i)$ increase counter; else unchanged. - a. Show expected value after n INCREMENTS = n . - b. For $n_i = 100i$, estimate variance after n operations.

5-2: **Searching an unsorted array** - RANDOM-SEARCH: pick random index repeatedly until find x or all checked. - a. Write pseudocode (ensure termination). - b. Exactly one x : expected picks until found = n . - c. k occurrences: expected picks = n/k . - d. No x : expected picks until all checked = $n \cdot H_n \approx n \ln n$ (coupon collector). - DETERMINISTIC-SEARCH (linear scan): - e. Exactly one x : average-case = $n/2$, worst-case = n . - f. k occurrences: average-case = $n/(k+1)$, worst-case = $n - k + 1$. - g. No x : average-case = worst-case = n . - SCRAMBLE-SEARCH (randomly permute then linear search): - h. For $k=0$: worst=expected= n ; $k=1$: worst= n , expected= $(n+1)/2$; $k \geq 1$: worst= $n - k + 1$, expected= $(n+1)/(k+1)$. - i. Which algorithm would you use? Explain.

Ch. 6 – Heapsort

Named Entities (Terms & Definitions)

- **Heap (binary heap)**: An array object that can be viewed as a nearly complete binary tree. Each node corresponds to an array element. The tree is completely filled on all levels except possibly the lowest, which is filled from the left up to a point.
- **A.heap-size**: Attribute representing how many elements in the heap are stored within array A . Only elements in $A[1 : A.\text{heap-size}]$, where $0 \leq A.\text{heap-size} \leq n$, are valid elements. If $A.\text{heap-size} = 0$, the heap is empty.
- **Max-heap**: A binary heap where for every node i other than the root, $\text{APARENT}(i) \geq A[i]$. The largest element is stored at the root.
- **Min-heap**: A binary heap where for every node i other than the root, $\text{APARENT}(i) \leq A[i]$. The smallest element is at the root.
- **Max-heap property**: For every node i other than the root, $\text{APARENT}(i) \geq A[i]$. The value of a node is at most the value of its parent.

- **Min-heap property:** For every node i other than the root, $A[PARENT(i)] \leq A[i]$.
- **Height of a node in a heap:** Number of edges on the longest simple downward path from the node to a leaf.
- **Height of the heap:** Height of its root. A heap of n elements has height $\Theta(\lg n)$ (Exercise 6.1-2).
- **Priority queue:** A data structure for maintaining a set S of elements, each with an associated value called a key.
- **Max-priority queue:** Supports INSERT, MAXIMUM, EXTRACT-MAX, and INCREASE-KEY.
- **Min-priority queue:** Supports INSERT, MINIMUM, EXTRACT-MIN, and DECREASE-KEY.
- **Handle:** Additional information stored in objects and heap elements that maps between application objects and array indices. Overhead is $O(1)$ per access.
- **Key:** The associated value of an element in a priority queue that determines its priority.
- **d-ary heap:** Like a binary heap but nonleaf nodes have d children instead of two.
- **Young tableau:** An $m \times n$ matrix with entries sorted left-to-right in each row and top-to-bottom in each column. Some entries may be ∞ (nonexistent).

Processes / Algorithms / Pathways

PARENT(i)

- **Goal:** Compute the index of the parent of a node
- **Input/Output:** Input index i , output $\lfloor i/2 \rfloor$
- **Steps:** (1) return $\lfloor i/2 \rfloor$
- **Complexity:** $O(1)$

LEFT(i)

- **Goal:** Compute the index of the left child of a node
- **Input/Output:** Input index i , output $2i$
- **Complexity:** $O(1)$ (can be done in one instruction by shifting binary representation left one bit)

RIGHT(i)

- **Goal:** Compute the index of the right child of a node
- **Input/Output:** Input index i , output $2i+1$
- **Complexity:** $O(1)$

MAX-HEAPIFY(A, i)

- **Goal:** Maintain the max-heap property; lets the value at $A[i]$ "float down"
- **Input/Output:** Input array A with heap-size attribute and index i . Assumes LEFT(i) and RIGHT(i) are roots of max-heaps

but $A[i]$ might be smaller than its children. Restores max-heap property at subtree rooted at i .

- **Steps:** (1) $l = \text{LEFT}(i)$; $r = \text{RIGHT}(i)$ (2) if $l \leq A.\text{heap-size}$ and $A[l] > A[i]$, $\text{largest} = l$; else $\text{largest} = i$ (3) if $r \leq A.\text{heap-size}$ and $A[r] > A[\text{largest}]$, $\text{largest} = r$ (4) if $\text{largest} \neq i$, exchange $A[i]$ with $A[\text{largest}]$ and call $\text{MAX-HEAPIFY}(A, \text{largest})$ recursively
- **Complexity:** Time $O(\lg n) = O(h)$ where h is height of node. Worst-case recurrence: $T(n) \leq T(2n/3) + \Theta(1)$, solved by case 2 of master theorem $\rightarrow T(n) = O(\lg n)$. Worst-case $\Omega(\lg n)$ (Exercise 6.2-7).
- **Example:** $\text{MAX-HEAPIFY}(A, 2)$ on heap of size 10 where $A[2]$ violates max-heap property. Compare $A[2]$ with children $A[4]$ and $A[5]$, swap with largest child, recurse on that child's position until property holds.

BUILD-MAX-HEAP(A, n)

- **Goal:** Convert an unordered array into a max-heap
- **Input/Output:** Input array $A[1:n]$, output max-heap in A
- **Steps:** (1) $A.\text{heap-size} = n$ (2) for $i = \lfloor n/2 \rfloor$ downto 1 (3) $\text{MAX-HEAPIFY}(A, i)$
- **Complexity:** Time $O(n)$ (tight analysis using the fact that MAX-HEAPIFY time varies with height; $O(n)$ not $O(n \lg n)$). Linear time.
- **Loop invariant:** At start of each iteration of for loop, each node $i+1, i+2, \dots, n$ is the root of a max-heap.
- **Tighter analysis:** Time for MAX-HEAPIFY on node of height h is $O(h)$. At most $\lfloor n/2^{h+1} \rfloor$ nodes of height h . Summation: $\sum_{h=0}^{\lfloor \lg n \rfloor} \lfloor n/2^{h+1} \rfloor O(h) = O(n \sum_{h=0}^{\lfloor \lg n \rfloor} \lfloor \lg n \rfloor h / 2^h) = O(n)$.
- **Example:** BUILD-MAX-HEAP on $A = \langle 5, 3, 17, 10, 84, 19, 6, 22, 9 \rangle$. Start at $i = \lfloor 9/2 \rfloor = 4$, call MAX-HEAPIFY on nodes 4, 3, 2, 1 in order.

HEAPSORT(A, n)

- **Goal:** Sort array A in place
- **Input/Output:** Input array $A[1:n]$, output sorted A
- **Steps:** (1) $\text{BUILD-MAX-HEAP}(A, n)$ (2) for $i = n$ downto 2 (3) exchange $A[1]$ with $A[i]$ (4) $A.\text{heap-size} = A.\text{heap-size} - 1$ (5) $\text{MAX-HEAPIFY}(A, 1)$
- **Complexity:** Time $O(n \lg n)$ [$\text{BUILD-MAX-HEAP } O(n) + n-1$ calls to MAX-HEAPIFY each $O(\lg n)$]. Worst-case $\Omega(n \lg n)$ (Exercise 6.4-5). Best-case $\Omega(n \lg n)$ when all elements distinct.
- **Example:** HEAPSORT on $A = \langle 5, 13, 2, 25, 7, 17, 20, 8, 4 \rangle$. Build max-heap, repeatedly swap root (max) with last element, reduce heap size, restore heap property.
- **Loop invariant:** At start of each iteration of for loop (lines 2-5), $A[1:i]$ is a max-heap containing the i smallest elements of $A[1:n]$, and $A[i+1:n]$ contains the $n-i$ largest elements of $A[1:n]$, sorted.

MAX-HEAP-MAXIMUM(A)

- **Goal:** Return element with largest key
- **Input/Output:** Input array A, output A[1]
- **Steps:** (1) if A.heap-size < 1 error "heap underflow" (2) return A[1]
- **Complexity:** $\Theta(1)$

MAX-HEAP-EXTRACT-MAX(A)

- **Goal:** Remove and return element with largest key
- **Steps:** (1) max = MAX-HEAP-MAXIMUM(A) (2) A[1] = A[A.heap-size] (3) A.heap-size = A.heap-size - 1 (4) MAX-HEAPIFY(A, 1) (5) return max
- **Complexity:** $O(\lg n)$
- **Example:** EXTRACT-MAX on heap A = [15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1]. Returns 15, replaces root with last element (1), heapify restores property.

MAX-HEAP-INCREASE-KEY(A, x, k)

- **Goal:** Increase key of element x to new value k (assumed $\geq$ current key)
- **Steps:** (1) if $k < x.key$ error "new key is smaller than current key" (2) $x.key = k$ (3) find index i of x (4) while $i > 1$ and $PARENT(i).key < A[i].key$ (5) exchange A[i] with $PARENT(i)$, updating mapping (6) $i = PARENT(i)$
- **Complexity:** $O(\lg n)$
- **Loop invariant:** At start of each iteration of while loop: (a) if PARENT(i) and LEFT(i) exist, $PARENT(i).key \geq LEFT(i).key$; (b) if PARENT(i) and RIGHT(i) exist, $PARENT(i).key \geq RIGHT(i).key$; (c) subarray A[1:A.heap-size] satisfies max-heap property except one possible violation: A[i].key may be $> PARENT(i).key$.

MAX-HEAP-INSERT(A, x, n)

- **Goal:** Insert new element x into max-heap
- **Steps:** (1) if A.heap-size == n error "heap overflow" (2) A.heap-size = A.heap-size + 1 (3) $k = x.key$ (4) $x.key = -\infty$ (5) A[A.heap-size] = x (6) map x to index heap-size (7) MAX-HEAP-INCREASE-KEY(A, x, k)
- **Complexity:** $O(\lg n)$ plus overhead for mapping

BUILD-MAX-HEAP'(A, n) (alternative)

- **Steps:** (1) A.heap-size = 1 (2) for i = 2 to n (3) MAX-HEAP-INSERT(A, A[i], n)
- **Note:** Does NOT always create the same heap as BUILD-MAX-HEAP on the same input (counterexample exists). Worst-case time: $\Theta(n \lg n)$.

Classifications & Hierarchies

- **Binary Heaps:** Max-heap (used by heapsort) vs. Min-heap (used by priority queues)
- **Priority Queues:** Max-priority queue vs. Min-priority queue
- **Heap types:** Binary heap, d-ary heap, Fibonacci heap (INSERT and DECREASE-KEY in $O(1)$ amortized time), strict Fibonacci heap (actual $O(1)$ times), van Emde Boas trees ($O(\lg \lg n)$ for INSERT/DELETE/SEARCH/MINIMUM/MAXIMUM/PREDECESSOR/SUCCESSOR for unique keys in $\{0, \dots, n-1\}$), radix heap (monotone EXTRACT-MIN in $O(\lg C)$ amortized, DECREASE-KEY $O(1)$)

Comparisons & Trade-offs

Dimension	Heapsort	Merge Sort	Insertion Sort
Worst-case time	$O(n \lg n)$	$O(n \lg n)$	$O(n^2)$
In-place	Yes	No	Yes
Stable	No	Yes	Yes
Practical speed	Usually beaten by quicksort	—	Better on nearly sorted data
Dimension	Max-heap	Min-heap	
Property	$\text{PARENT}(i) \geq A[i]$	$\text{PARENT}(i) \leq A[i]$	
Root	Largest element	Smallest element	
Use	Heapsort, max-priority queues Min-priority queues		
Priority Queue Implementation	Time		
Binary heap	$O(\lg n)$ per operation		
Fibonacci heap (INSERT/DECREASE-KEY)	$O(1)$ amortized		
van Emde Boas tree	$O(\lg \lg n)$		

Formulas & Equations

Height of heap

$\text{height} = \lceil \lg n \rceil$ - n = number of elements

Children's subtree size bound

$\text{size} \leq 2n/3$ - Used in MAX-HEAPIFY recurrence

Nodes of height h

$\leq \lceil n/2^{(h+1)} \rceil$ - n = heap size, h = height

BUILD-MAX-HEAP total cost

$\sum_{h=0}^{\lceil \lg n \rceil} \lceil n/2^{(h+1)} \rceil O(h) = O(n)$

Rules, Laws & Theorems

Heap property (max-heap)

- **Statement:** For every node i other than the root, $A[\text{PARENT}(i)] \geq A[i]$
- **Condition:** Valid max-heap

Heap property (min-heap)

- **Statement:** For every node i other than the root, $A[\text{PARENT}(i)] \leq A[i]$
- **Condition:** Valid min-heap

Leaf indices

- **Statement:** Leaves are nodes indexed by $\lceil n/2 \rceil + 1, \lceil n/2 \rceil + 2, \dots, n$

Edge Cases & Pitfalls

- The term “heap” in this context refers to the data structure, not garbage-collected storage
- MAX-HEAPIFY has a worst-case lower bound of $\Omega(\lg n)$ – can recurse along entire root-to-leaf path
- Calling MAX-HEAPIFY(A, i) when $A[i]$ is larger than its children does nothing
- Calling MAX-HEAPIFY(A, i) for $i > A.\text{heap-size}/2$ does nothing (node is a leaf)
- The while loop in MAX-HEAP-INCREASE-KEY cannot be replaced by MAX-HEAPIFY because MAX-HEAPIFY moves elements downward while INCREASE-KEY needs to move upward
- Heap overflow/underflow conditions must be checked in priority queue operations
- BUILD-MAX-HEAP' (using insertion) is $\Theta(n \lg n)$ worst-case, not $O(n)$
- Alternate mapping strategies (handles vs. hash tables) have different trade-offs: handles are $O(1)$ but require per-access overhead; hash tables add expected $O(1)$ but worst-case $\Theta(n)$

Case Studies & Examples

MAX-HEAPIFY Example

- **What:** Figure 6.2 shows MAX-HEAPIFY($A, 2$) on heap of size 10. Initial state has $A[2]$ violating property. After swapping $A[2] \leftrightarrow A[4]$, property is restored at node 2 but node 4 now violates it. Recursive call MAX-HEAPIFY($A, 4$) swaps $A[4] \leftrightarrow A[9]$. No further changes needed.

HEAPSORT Example

- **What:** Figure 6.4 shows HEAPSORT operation. After BUILD-MAX-HEAP, root (largest) is swapped to $A[n]$, heap-size decremented, MAX-HEAPIFY restores property, repeat. Blue nodes are heap; tan nodes (right side) are sorted final elements.

MAX-HEAP-INCREASE-KEY Example

- **What:** Figure 6.5 shows increasing a node's key to 15. The node is compared to parent and swapped upward iteratively until property holds.

Diagrams & Visuals

[Figure 6.1: Max-heap as binary tree and array. Tree with values: 16 at root (index 1), left child 14 (index 2), right child 10 (index 3), and so on. Array representation shows parent-child relationships with lines above/below. Tree height = 3.]

[Figure 6.2: MAX-HEAPIFY(A,2) action. Step (a): $A[2]$ =violating node in blue. (b): Exchange $A[2] \leftrightarrow A[4]$. (c): Recursive call on $A[4]$ fixes remaining problem.]

[Figure 6.3: BUILD-MAX-HEAP on 10-element array. Shows each iteration $i = 5, 4, 3, 2, 1$ with the node indexed by i in blue, progressively building the max-heap.]

[Figure 6.4: HEAPSORT after BUILD-MAX-HEAP. Shows 10 steps (b)-(j) as the heap shrinks (blue nodes) and sorted portion grows (tan nodes). Final (k) shows sorted array.]

[Figure 6.5: MAX-HEAP-INCREASE-KEY. (a) Original heap with node to increase. (b) Key increased to 15. (c) After one iteration of while loop, node and parent swapped. (d) Final state with max-heap property restored.]

End-of-Chapter Material

Key Terms: heap, max-heap, min-heap, heap property, height, heapify, priority queue, key, handle, d-ary heap, Young tableau

Exercises: - 6.1-1: Min and max number of elements in heap of height h ? Answer: minimum = 2^h , maximum = $2^{(h+1)} - 1$ - 6.1-2: Show an n -element heap has height $\lceil \lg n \rceil$ - 6.1-3: Show in any subtree of a max-heap, the root contains the largest value - 6.1-4: Where in a max-heap might the smallest element reside? Answer: at a leaf - 6.1-5: At which levels might the k th largest element reside for $2 \leq k \leq \lceil n/2 \rceil$? Answer: within the first $\lceil \lg k \rceil$ levels - 6.1-6: Is a sorted array a min-heap? Answer: yes, if sorted in increasing order - 6.1-7: Is $\langle 33, 19, 20, 15, 13, 10, 2, 13, 16, 12 \rangle$ a max-heap? Answer: no ($16 > 15$ at

positions 9,4) - 6.1-8: Show leaves are $\lceil n/2 \rceil + 1, \lceil n/2 \rceil + 2, \dots, n$ - 6.2-1: Illustrate MAX-HEAPIFY(A,3) on A = [27,17,3,16,13,10,1,5,7,12,4,8,9,0] - 6.2-2: Each child of root of n-node heap roots subtree of at most $2n/3$ nodes; smallest $\alpha = 2/3$ - 6.2-3: Write MIN-HEAPIFY pseudocode; same running time as MAX-HEAPIFY - 6.2-4: Effect of MAX-HEAPIFY(A,i) when $A[i] >$ children: nothing happens - 6.2-5: Effect of MAX-HEAPIFY(A,i) for $i > A.\text{heap-size}/2$: nothing (leaf) - 6.2-6: Write iterative MAX-HEAPIFY using loop instead of recursion - 6.2-7: Show worst-case $\Omega(\lg n)$ for MAX-HEAPIFY - 6.3-1: Illustrate BUILD-MAX-HEAP on A = [5,3,17,10,84,19,6,22,9] - 6.3-2: Show $\lceil n/2^{(h+1)} \rceil \geq 1/2$ for $0 \leq h \leq \lceil \lg n \rceil$ - 6.3-3: Why does loop index decrease in BUILD-MAX-HEAP? Answer: children have higher indices; to maintain invariant that children are roots of max-heaps - 6.3-4: Show at most $\lceil n/2^{(h+1)} \rceil$ nodes of height h - 6.4-1: Illustrate HEAPSORT on A = [5,13,2,25,7,17,20,8,4] - 6.4-2: Argue correctness using given loop invariant - 6.4-3: Running time on already sorted increasing: $O(n \lg n)$; decreasing: $O(n \lg n)$ - 6.4-4: Show worst-case $\Omega(n \lg n)$ - 6.4-5: Show best-case $\Omega(n \lg n)$ with distinct elements - 6.5-1: MAX-HEAP-EXTRACT-MAX on A = [15,13,9,5,12,8,7,4,0,6,2,1] - 6.5-2: MAX-HEAP-INSERT(A,10) on same heap - 6.5-3: Write min-priority queue procedures - 6.5-4: Write MAX-HEAP-DECREASE-KEY – runs in $O(\lg n)$ time - 6.5-5: Why set key to $-\infty$ in line 5 of MAX-HEAP-INSERT? Answer: to satisfy precondition of MAX-HEAP-INCREASE-KEY - 6.5-6: Explain flaw in replacing while loop with MAX-HEAPIFY - 6.5-7: Argue correctness of MAX-HEAP-INCREASE-KEY using loop invariant - 6.5-8: Reduce three assignments to one using insertion sort technique - 6.5-9: Implement FIFO queue and stack with priority queue - 6.5-10: Implement MAX-HEAP-DELETE in $O(\lg n)$ - 6.5-11: Merge k sorted lists in $O(n \lg k)$ using min-heap for k-way merging

Problems: - 6-1: Building a heap using insertion (BUILD-MAX-HEAP') – counterexample for same heap; $\Theta(n \lg n)$ worst-case - 6-2: Analysis of d-ary heaps – height = $\lceil \log_d (n(d-1)+1) \rceil - 1$; EXTRACT-MAX $O(d \log_d n)$; INCREASE-KEY $O(\log_d n)$; INSERT $O(\log_d n)$ - 6-3: Young tableaux – $m \times n$ matrix with sorted rows/columns; EXTRACT-MIN $O(m+n)$; INSERT $O(m+n)$; sort n^2 numbers $O(n^3)$; find element $O(m+n)$

Ch. 7 – Quicksort

Named Entities (Terms & Definitions)

- **Quicksort:** A divide-and-conquer sorting algorithm that sorts in place. Worst-case $\Theta(n^2)$, expected $\Theta(n \lg n)$ with small constant factors.
- **Pivot:** The element (typically $A[r]$ in the basic version) around which the array is partitioned.
- **Partitioning:** The divide step of quicksort that rearranges the array into two subarrays (low side and high side) around the pivot.
- **Low side:** Subarray $A[p:q-1]$ containing elements $\leq$ pivot.

- **High side:** Subarray $A[q+1:r]$ containing elements $\geq$ pivot.
- **Hoare partition:** The original partitioning algorithm by C. A. R. Hoare. Uses $A[p]$ as pivot, two indices i and j moving toward each other.
- **Tail-recursion elimination:** An optimization that replaces the second recursive call with a loop to reduce stack depth.
- **Median-of-3 method:** Choosing the pivot as the median of three randomly selected elements from the subarray to improve balance.
- **Stooge sort:** A deceptively simple sorting algorithm that recursively sorts the first two-thirds, last two-thirds, and first two-thirds again.
- **Fuzzy sorting:** Sorting intervals $[a_i, b_i]$ to produce a permutation where there exist c_i in each interval such that $c_1 \leq c_2 \leq \dots \leq c_n$.

Processes / Algorithms / Pathways

PARTITION(A, p, r)

- **Goal:** Partition subarray in place, returning index of pivot
- **Input/Output:** Input subarray $A[p:r]$, output rearranged array with pivot in final position, returns index q
- **Steps:** (1) $x = A[r]$ (pivot) (2) $i = p-1$ (3) for $j = p$ to $r-1$ (4) if $A[j] \leq x$ (5) $i = i+1$ (6) exchange $A[i]$ with $A[j]$ (7) exchange $A[i+1]$ with $A[r]$ (8) return $i+1$
- **Complexity:** $\Theta(n)$ where $n = r-p+1$
- **Loop invariant:** At start of each iteration: (1) if $p \leq k \leq i$, $A[k] \leq x$; (2) if $i+1 \leq k \leq j-1$, $A[k] > x$; (3) if $k = r$, $A[k] = x$
- **Example:** PARTITION on $A = [13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11]$. Pivot = 11. Process elements from $p=1$ to $r-1=11$, swapping elements ≤ 11 into low side.

QUICKSORT(A, p, r)

- **Goal:** Sort subarray $A[p:r]$
- **Input/Output:** Input array A , indices p, r ; output sorted subarray
- **Steps:** (1) if $p < r$ (2) $q = \text{PARTITION}(A, p, r)$ (3) $\text{QUICKSORT}(A, p, q-1)$ (4) $\text{QUICKSORT}(A, q+1, r)$
- **Complexity:** Worst-case $\Theta(n^2)$ (when partition produces subproblems of sizes $n-1$ and 0 at every level). Best-case $\Theta(n \lg n)$ (when partition splits evenly). Average-case $\Theta(n \lg n)$ (assuming distinct elements).

RANDOMIZED-PARTITION(A, p, r)

- **Goal:** Choose pivot randomly
- **Steps:** (1) $i = \text{RANDOM}(p, r)$ (2) exchange $A[r]$ with $A[i]$ (3) return $\text{PARTITION}(A, p, r)$
- **Effect:** No particular input elicits worst-case behavior

RANDOMIZED-QUICKSORT(A, p, r)

- **Goal:** Randomized version of quicksort
- **Steps:** (1) if $p < r$ (2) $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ (3) $\text{RANDOMIZED-QUICKSORT}(A, p, q-1)$ (4) $\text{RANDOMIZED-QUICKSORT}(A, q+1, r)$

HOARE-PARTITION(A, p, r)

- **Goal:** Original Hoare partitioning algorithm
- **Steps:** (1) $x = A[p]$ (2) $i = p-1$ (3) $j = r+1$ (4) while TRUE (5) repeat $j = j-1$ until $A[j] \leq x$ (8) repeat $i = i+1$ until $A[i] \geq x$ (11) if $i < j$ exchange $A[i]$ with $A[j]$ (13) else return j
- **Properties:** Returns j where $p \leq j < r$. Every element of $A[p:j] \leq$ every element of $A[j+1:r]$. Pivot value goes into one of the two partitions (neither partition is empty).
- **Advantage over PARTITION:** Better performance when all elements are equal (no unnecessary swaps).

TRE-QUICKSORT(A, p, r)

- **Goal:** Quicksort with tail-recursion elimination
- **Steps:** (1) while $p < r$ (2) $q = \text{PARTITION}(A, p, r)$ (3) $\text{TRE-QUICKSORT}(A, p, q-1)$ (4) $p = q+1$
- **Stack depth:** Can be $\Theta(n)$ worst-case; can be modified to $O(\lg n)$ by always recursing on the smaller subarray first (or using an explicit stack).

STOOGE-SORT(A, p, r)

- **Goal:** Another sorting algorithm
- **Steps:** (1) if $A[p] > A[r]$ exchange $A[p]$ with $A[r]$ (2) if $p+1 < r$ (3) $k = \lceil (r-p+1)/3 \rceil$ (4) $\text{STOOGE-SORT}(A, p, r-k)$ (5) $\text{STOOGE-SORT}(A, p+k, r)$ (6) $\text{STOOGE-SORT}(A, p, r-k)$
- **Complexity:** Recurrence $T(n) = 3T(2n/3) + \Theta(1) \rightarrow T(n) = \Theta(n^{\log_{3/2} 3}) \approx \Theta(n^{2.71})$. Much worse than merge sort, heapsort, quicksort.

Classifications & Hierarchies

- **Sorting algorithms by approach:** Insertion sort (incremental) vs. Merge sort (divide-and-conquer) vs. Quicksort (divide-and-conquer in-place)
- **Partitioning strategies:** Lomuto partition (this chapter) vs. Hoare partition (Problem 7-1)
- **Pivot selection:** Fixed ($A[r]$) vs. Randomized vs. Median-of-3

Comparisons & Trade-offs

Dimension	Quicksort	Merge Sort	Insertion Sort
Worst-case	$\Theta(n^2)$	$\Theta(n \lg n)$	$\Theta(n^2)$
Best-case	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$O(n)$
Expected time	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n^2)$
In-place	Yes	No (needs $\Theta(n)$ extra)	Yes
Stable	No	Yes	Yes
Constant factors	Small	Larger	Very small for small n
Virtual memory	Works well	—	—
Partition	Pivot choice	Properties	
PARTITION (Lomuto)	$A[r]$	Returns single index q ; $A[p:q-1] \leq A[q] < A[q+1:r]$	
HOARE-PARTITION	$A[p]$	Returns index j ; $A[p:j] \leq A[j+1:r]$; pivot not at boundary	

Formulas & Equations

Best-case recurrence

$$T(n) = 2T(n/2) + \Theta(n) \rightarrow T(n) = \Theta(n \lg n)$$

Worst-case recurrence

$$T(n) = T(n-1) + \Theta(n) \rightarrow T(n) = \Theta(n^2)$$

9-to-1 proportional split recurrence

$$T(n) = T(9n/10) + T(n/10) + \Theta(n) \rightarrow T(n) = O(n \lg n)$$

General recurrence (worst-case analysis)

$$T(n) = \max\{T(q) + T(n-1-q) : 0 \leq q \leq n-1\} + \Theta(n)$$

Probability of comparison in RANDOMIZED-QUICKSORT

$$\Pr\{z_i \text{ compared with } z_j\} = 2/(j - i + 1) \text{ - For distinct elements } z_1 < z_2 < \dots < z_n$$

Expected comparisons

$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n 2/(j-i+1) = O(n \lg n)$ -
Derivation: $E[X] = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} 2/(k+1) < \sum_{i=1}^{n-1} \sum_{k=1}^n 2/k = \sum_{i=1}^{n-1} O(\lg n) = O(n \lg n)$

Rules, Laws & Theorems

Lemma 7.1 (Running time and comparisons)

- **Statement:** Running time of QUICKSORT on n-element array is $O(n + X)$, where X is number of element comparisons performed.
- **Proof:** At most n calls to PARTITION. Each call takes $O(1) +$ time proportional to number of for-loop iterations. Each iteration performs one comparison. Total time $O(n + X)$.

Lemma 7.2 (When comparisons occur)

- **Statement:** During execution of RANDOMIZED-QUICKSORT on array of n distinct elements $z_1 < \dots < z_n$, z_i is compared with z_j ($i < j$) iff one of them is chosen as pivot before any other element in $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$. No two elements are compared twice.

Lemma 7.3 (Probability of comparison)

- **Statement:** For distinct elements $z_1 < \dots < z_n$, $\Pr\{z_i \text{ compared with } z_j\} = 2/(j - i + 1)$.

Theorem 7.4 (Expected running time)

- **Statement:** Expected running time of RANDOMIZED-QUICKSORT on n distinct elements is $O(n \lg n)$.

Edge Cases & Pitfalls

- Worst-case occurs when input is already sorted or reverse sorted (always picks max/min pivot)
- If all elements have the same value, PARTITION returns r, leading to $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$
- The stack depth can be $\Theta(n)$ worst-case; tail-recursion elimination can reduce to $\Theta(\lg n)$ by always recursing on smaller subarray first
- Hoare partition is generally more efficient in practice (about 3× fewer swaps than Lomuto)
- For nearly sorted input, insertion sort beats quicksort
- The analysis assumes distinct elements; equal elements can be handled by converting to ordered pairs $(A[i], i)$
- A “killer adversary” can produce $\Theta(n^2)$ behavior on virtually any implementation (McIlroy 1999)

Case Studies & Examples

PARTITION Example

- **What:** Figure 7.1 shows PARTITION on 8-element array. Initial: [2,8,7,1,3,5,6,4] with pivot=4. After partitioning, array becomes [2,1,3,4,7,5,6,8] with pivot at index 4. Low side [2,1,3], high side [7,5,6,8].

Example of Lemma 7.2

- **What:** Input 1..10 in arbitrary order. First pivot = 7. Partition into {1,2,3,4,5,6} and {8,9,10}. 7 is compared with all others. 2 and 9 are never compared because first pivot from $Z_{\{2,9\}}$ is 7.

Bad split followed by good split (Figure 7.5)

- **What:** Root partition produces sizes 0 and $n-1$ (bad). Next level does best-case split on $n-1$. Combined cost $\Theta(n) + \Theta(n-1) = \Theta(n)$, same as balanced split. Bad split's cost is absorbed into good split's cost.

Diagrams & Visuals

[Figure 7.1: PARTITION on sample 8-element array. Shows tan (low side), blue (high side), white (unexamined), yellow (pivot). Steps (a)-(i) show progressive partitioning.]

[Figure 7.2: The four regions maintained by PARTITION. Tan $A[p:i] \leq x$, Blue $A[i+1:j-1] > x$, White $A[j:r-1]$ unknown, Yellow $A[r] = x$.]

[Figure 7.3: Two cases per iteration. (a) $A[j] > x$: increment j only. (b) $A[j] \leq x$: increment i , swap $A[i] \leftrightarrow A[j]$, increment j .]

[Figure 7.4: Recursion tree for 9-to-1 split. Each level cost = n . Depth = $\log_{10/9} n$. Total $O(n \lg n)$.]

[Figure 7.5: (a) Bad split (0, $n-1$) followed by good split on $n-1$. (b) Single well-balanced level. Total cost same asymptotically.]

End-of-Chapter Material

Exercises: - 7.1-1: Illustrate PARTITION on $A = [13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11]$ - 7.1-2: PARTITION returns r when all values equal; modify to return midpoint - 7.1-3: Running time of PARTITION on size n is $\Theta(n)$ - 7.1-4: Modify QUICKSORT for decreasing order (reverse comparison) - 7.2-1: Prove $T(n) = T(n-1) + \Theta(n) \rightarrow \Theta(n^2)$ by substitution - 7.2-2: Running time when all values equal: $\Theta(n^2)$ - 7.2-3: Running time on decreasing order

with distinct elements: $\Theta(n^2)$ - 7.2-4: Why insertion sort beats quicksort on almost-sorted input - 7.2-5: Constant proportion $\alpha:\beta$ split $\rightarrow$ min depth $\approx \log_{\{1/\alpha\}} n$, max depth $\approx \log_{\{1/\beta\}} n$ - 7.2-6: For any constant $0 < \alpha \leq 1/2$, probability of split at least as balanced as $1-\alpha:\alpha \approx 1-2\alpha$ - 7.3-1: Why analyze expected time rather than worst-case? Answer: worst-case is same for all inputs; randomization eliminates bad inputs but not bad random choices - 7.3-2: Number of RANDOM calls: worst-case $\Theta(n)$ (every element a pivot), best-case $\Theta(\lg n)$ - 7.4-1: Show recurrence has lower bound $\Omega(n^2)$ - 7.4-2: Show best-case running time $\Omega(n \lg n)$ - 7.4-3: Show $q^2 + (n-q-1)^2$ maximized at $q=0$ or $q=n-1$ - 7.4-4: Show expected running time $\Omega(n \lg n)$ - 7.4-5: Coarsening with insertion sort for subarrays $< k \rightarrow O(nk + n \lg(n/k))$ expected; choose $k = \Theta(\lg n)$ in practice - 7.4-6: Probability of worse than α -to- $(1-\alpha)$ split with median-of-3

Problems: - 7-1: Hoare partition correctness – proves procedure never accesses outside subarray; returns j with $p \leq j < r$; $A[p:j] \leq A[j+1:r]$; rewrite QUICKSORT to use HOARE-PARTITION - 7-2: Quicksort with equal element values – (a) all equal: $\Theta(n^2)$; (b) PARTITION' returns two indices q, t for equal range; (c) RANDOMIZED-PARTITION' + QUICKSORT'; (d) adjust analysis - 7-3: Alternative quicksort analysis – expected running time using indicator variables for pivot selection; $E[X_i] = 1/n$; derive $E[T(n)] = O(n \lg n)$ - 7-4: Stooge sort – recurrence $T(n) = 3T(2n/3) + \Theta(1) \rightarrow T(n) = \Theta(n^{\log_{\{3/2\}} 3}) \approx \Theta(n^{2.71})$ - 7-5: Stack depth – TRE-QUICKSORT can have $\Theta(n)$ stack depth; modify to always recurse on smaller subarray first for $\Theta(\lg n)$ worst-case depth - 7-6: Median-of-3 partition – exact formula $p_i = (i-1)(n-i)/C(n, 3)$; increases likelihood of picking median by factor $\sim 3/2$ - 7-7: Fuzzy sorting of intervals – general case $\Theta(n \lg n)$ expected; all intervals overlap $\rightarrow \Theta(n)$ expected

Ch. 8 – Sorting in Linear Time

Named Entities (Terms & Definitions)

- **Comparison sort:** A sorting algorithm that determines sorted order based only on comparisons between input elements (e.g., insertion sort, merge sort, heapsort, quicksort).
- **Decision tree:** A full binary tree representing comparisons between elements performed by a particular comparison sort on input of a given size. Each internal node annotated $i:j$ (comparing $a_i \leq a_j$), each leaf by a permutation $\pi(1), \pi(2), \dots, \pi(n)$.
- **Counting sort:** A sorting algorithm that assumes each input element is an integer in range 0 to k . Runs in $\Theta(n+k)$ time. Stable.
- **Radix sort:** Sorts by sorting on the least significant digit first, using a stable sort per digit. Runs in $\Theta(d(n+k))$ for d -digit numbers with k possible values per digit.
- **Bucket sort:** Assumes input drawn from uniform distribution over $[0,1)$. Divides $[0,1)$ into n equal-sized buckets,

distributes elements, sorts each bucket with insertion sort. Average-case $O(n)$.

- **Stable sort:** A sorting algorithm where elements with the same value appear in the output in the same order as in the input.
- **Stability:** Property that equal elements retain their relative input order.
- **0-1 sorting lemma:** If an oblivious compare-exchange algorithm correctly sorts all input sequences consisting of only 0s and 1s, then it correctly sorts all inputs containing arbitrary values.
- **Oblivious compare-exchange algorithm:** Operates solely by a prespecified sequence of compare-exchange operations; indices are determined in advance, independent of values.
- **Compare-exchange operation:** On $A[i]$ and $A[j]$ ($i < j$): if $A[i] > A[j]$, exchange them. After operation, $A[i] \leq A[j]$.
- **Columsort:** An oblivious compare-exchange algorithm that sorts an $r \times s$ array ($n = rs$) in eight steps. Restrictions: r even, s divides r , $r \geq 2s^2$.
- **Clean area:** In 0-1 sorting lemma context, an area known to contain all 0s or all 1s, or empty.
- **Dirty area:** An area that might contain mixed 0s and 1s.
- **k-sorted array:** An array where for all $i = 1, \dots, n-k$, $(\sum_{j=i}^{i+k-1} A[j])/k \leq (\sum_{j=i+1}^{i+k} A[j])/k$. Equivalent to $A[i] \leq A[i+k]$ for all i .

Processes / Algorithms / Pathways

COUNTING-SORT(A, n, k)

- **Goal:** Sort array of integers in range $0..k$
- **Input/Output:** Input array $A[1:n]$, size n , upper bound k ; returns sorted array $B[1:n]$
- **Steps:** (1) let $B[1:n]$ and $C[0:k]$ be new arrays (2) for $i=0$ to k : $C[i] = 0$ (3) for $j=1$ to n : $C[A[j]] = C[A[j]] + 1$ (4) for $i=1$ to k : $C[i] = C[i] + C[i-1]$ (5) for $j=n$ downto 1: $B[C[A[j]]] = A[j]$; $C[A[j]] = C[A[j]] - 1$ (6) return B
- **Complexity:** $\Theta(n+k)$. When $k = O(n)$, $\Theta(n)$.
- **Properties:** Stable (if iterating from end of A). Not in-place.
- **Example:** COUNTING-SORT on $A = [6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2]$ with $k=6$. Compute frequencies, cumulative counts, place elements in output from right to left.

RADIX-SORT(A, n, d)

- **Goal:** Sort d -digit numbers
- **Steps:** (1) for $i = 1$ to d (2) use a stable sort to sort array A on digit i
- **Complexity:** $\Theta(d(n+k))$ when using counting sort with k possible values per digit. With b -bit numbers and r bits per digit: $\Theta((b/r)(n + 2^r))$. Optimal when $r = \lceil \lg n \rceil$ for $b \geq \lceil \lg n \rceil$, yielding $\Theta(bn/\lg n)$. For $b = O(\lg n)$ and $r \approx \lg n$: $\Theta(n)$.

- **Example:** Radix sort on seven 3-digit numbers. First sort on least significant digit, then tens digit, then hundreds digit. After 3 passes, numbers are fully sorted.
- **Lemma 8.3:** Given n d -digit numbers with each digit taking up to k values, RADIX-SORT correctly sorts in $\Theta(d(n+k))$ time.
- **Lemma 8.4:** Given n b -bit numbers and any positive integer $r \leq b$, RADIX-SORT correctly sorts in $\Theta((b/r)(n + 2^r))$ time.

BUCKET-SORT(A, n)

- **Goal:** Sort numbers uniformly distributed in $[0,1)$
- **Steps:** (1) let $B[0:n-1]$ be new array (2) for $i=0$ to $n-1$: make $B[i]$ empty list (3) for $i=1$ to n : insert $A[i]$ into list $B[\lfloor n \cdot A[i] \rfloor]$ (4) for $i=0$ to $n-1$: sort list $B[i]$ with insertion sort (5) concatenate lists $B[0], B[1], \dots, B[n-1]$ together in order (6) return concatenated lists
- **Complexity:** Average-case $\Theta(n)$ (when input is uniform). Worst-case $\Theta(n^2)$ (all elements in one bucket). Can be changed to $O(n \lg n)$ worst-case by using $O(n \lg n)$ sort per bucket instead of insertion sort.
- **Analysis:** Let n_i be number of elements in bucket i . $E[n_i] = 1$, $E[n_i^2] = 2 - 1/n$. Total expected time: $\Theta(n) + \sum O(E[n_i^2]) = \Theta(n) + n \cdot O(2 - 1/n) = \Theta(n)$.
- **Example:** BUCKET-SORT on $A = [79, .13, .16, .64, .39, .20, .89, .53, .71, .42]$. Distribute into 10 buckets, sort each with insertion sort, concatenate.

COMPARE-EXCHANGE(A, i, j)

- **Goal:** Compare and exchange if out of order
- **Steps:** (1) if $A[i] > A[j]$ (2) exchange $A[i]$ with $A[j]$
- **Postcondition:** $A[i] \leq A[j]$

COMPARE-EXCHANGE-INSERTION-SORT(A, n)

- **Steps:** (1) for $i=2$ to n (2) for $j=i-1$ downto 1 (3) COMPARE-EXCHANGE($A, j, j+1$)
- **Complexity:** $\Theta(n^2)$ in all cases (unlike standard insertion sort)

Columnsort (8 steps)

- **Goal:** Sort $n = rs$ elements in column-major order
- **Restrictions:** r even, s divides r , $r \geq 2s^2$
- **Steps:** (1) Sort each column (2) Transpose array, reshape to $r \times s$ (3) Sort each column (4) Inverse of step 2 permutation (5) Sort each column (6) Shift: top half of each column $\rightarrow$ bottom half of same column; bottom half $\rightarrow$ top half of next column; leave top of leftmost empty; bottom of last column $\rightarrow$ top of new rightmost column (7) Sort each column (8) Inverse of step 6 permutation

- **Properties:** After steps 1-3: clean rows of 0s top, clean rows of 1s bottom, at most s dirty rows between. After step 4: clean 0s area, clean 1s area, at most s^2 dirty elements in middle. Steps 5-8 clean up.

Classifications & Hierarchies

- **Comparison sorts:** Insertion sort, merge sort, heapsort, quicksort $\rightarrow$ all $\Omega(n \lg n)$ worst-case
- **Linear-time non-comparison sorts:** Counting sort, radix sort, bucket sort $\rightarrow$ beat $\Omega(n \lg n)$ lower bound by using additional information about input (integers in small range, uniform distribution, etc.)
- **Stable sorts:** Counting sort (yes), merge sort (yes), insertion sort (yes), heapsort (no), quicksort (no)

Comparisons & Trade-offs

Dimension	Counting Sort	Radix Sort	Bucket Sort	Comparison Sorts
Assumption	Integers $0..k$	d -digit numbers	Uniform $[0,1)$	None
Time	$\Theta(n+k)$	$\Theta(d(n+k))$	$\Theta(n)$ avg, $\Theta(n^2)$ worst	$\Omega(n \lg n)$
In-place	No	No	No	Yes (quicksort, heapsort)
Stable	Yes	Yes (if stable sub-sort)	Depends	Varies
Cache efficiency	Lower	Lower	Lower	Higher (quicksort)

Radix sort parameters

Time
 r small (b/r) large $\rightarrow$ time increases
 $r = \lceil \lg n \rceil$ $\Theta(bn/\lg n)$ optimal for $b \geq \lg n$
 $r = b$, $b < \lceil \lg n \rceil$ $\Theta(n)$ asymptotically optimal

Formulas & Equations

Sorting lower bound

$h \geq \lg(n!) = \Omega(n \lg n)$ - Any comparison sort requires $\Omega(n \lg n)$ comparisons in worst case - Proof: $n! \leq 1 \leq 2^h \rightarrow h \geq \lg(n!)$

Counting sort time

$\Theta(n + k)$ - n = number of elements, k = maximum value

Radix sort time (general)

$\Theta(d(n + k))$ - d = number of digits, k = values per digit

Radix sort time (b-bit numbers, r-bit digits)

$$\Theta((b/r)(n + 2^r))$$

Bucket sort expected time

$$\Theta(n) + \sum_{i=0}^{n-1} \Theta(E[n_i^2]) = \Theta(n) + n \cdot O(2 - 1/n) = \Theta(n)$$

Where $E[n_i] = 1$, $\text{Var}[n_i] = 1 - 1/n$, $E[n_i^2] = 2 - 1/n$

Probability of success in bucket i (Bernoulli)

$p = 1/n$ - Each element equally likely to fall in any bucket

Rules, Laws & Theorems

Theorem 8.1 (Lower bound for comparison sorts)

- **Statement:** Any comparison sort algorithm requires $\Omega(n \lg n)$ comparisons in the worst case.
- **Proof:** Decision tree of height h has at most 2^h leaves. Correct algorithm has at least $n!$ leaves. Thus $n! \leq 2^h$, so $h \geq \lg(n!) = \Omega(n \lg n)$.

Corollary 8.2 (Asymptotically optimal comparison sorts)

- **Statement:** Heapsort and merge sort are asymptotically optimal comparison sorts.
- **Proof:** Their $O(n \lg n)$ upper bounds match the $\Omega(n \lg n)$ lower bound.

Lemma 8.3 (Radix sort correctness/time)

- **Statement:** Given n d -digit numbers with each digit taking up to k possible values, RADIX-SORT correctly sorts in $\Theta(d(n+k))$ time.

Lemma 8.4 (Radix sort with b-bit numbers)

- **Statement:** Given n b -bit numbers and any positive integer $r \leq b$, RADIX-SORT correctly sorts in $\Theta((b/r)(n + 2^r))$ time.

0-1 Sorting Lemma

- **Statement:** If an oblivious compare-exchange algorithm correctly sorts all input sequences consisting of only 0s and 1s, then it correctly sorts all inputs containing arbitrary values.
- **Proof:** By contrapositive: if algorithm fails on arbitrary input, it fails on some 0-1 input (constructed by setting $B[p]=0$ for smallest misplaced element, 1 otherwise).

Edge Cases & Pitfalls

- Counting sort is stable only if the loop in line 11 iterates from n down to 1 (reverse). If from 1 to n , algorithm still works but is not stable.
- Radix sort's digit sorts must be stable for correctness.
- Bucket sort worst-case is $\Theta(n^2)$ when all elements fall into same bucket. Can be mitigated by using an $O(n \lg n)$ sort within each bucket.
- Radix sort is not in-place; uses extra space for counting sort's output array.
- Quicksort often uses hardware caches more effectively than radix sort.
- The lower bound for comparison sorts (Theorem 8.1) assumes all input elements are distinct. For nondistinct elements, lower bound still holds because distinct case is a subset.
- When $b < \lceil \lg n \rceil$, radix sort can be $\Theta(n)$, asymptotically optimal; when $b \geq \lceil \lg n \rceil$, optimal running time is $\Theta(bn/\lg n)$.
- Columnsort requires $r \geq 2s^2$; if s does not divide r , need $r \geq 4s^2$ or modify step 1.

Case Studies & Examples

Decision tree for insertion sort on 3 elements (Figure 8.1)

- **What:** Shows all comparison paths sorting a_1, a_2, a_3 . 6 leaves ($3! = 6$). Highlighted path for input $\langle 6, 8, 5 \rangle$: $1:2 \rightarrow \text{left}$ ($a_1 \leq a_2$), $2:3 \rightarrow \text{right}$ ($a_2 > a_3$), $1:3 \rightarrow \text{right}$ ($a_1 > a_3$) $\rightarrow$ ordering $a_1 \leq a_3 \leq a_2 \rightarrow$ permutation $\langle 3, 1, 2 \rangle$.

Counting sort on 8 elements (Figure 8.2)

- **What:** Input array $A[1:8]$, $k=5$. After frequency count: $C=[2,0,2,3,0,1]$. After cumulative: $C=[2,2,4,7,7,8]$. Process right-to-left: place each $A[j]$ into $B[C[A[j]]]$, decrement count.

Radix sort on 7 three-digit numbers (Figure 8.3)

- **What:** Input: 329, 457, 657, 839, 436, 720, 355. Sort on LSD (units): 720, 355, 436, 457, 657, 329, 839. Sort on tens: 720, 329, 436, 839, 355, 457, 657. Sort on hundreds: 329, 355, 436, 457, 657, 720, 839. Fully sorted.

Bucket sort on 10 numbers (Figure 8.4)

- **What:** Input $A = [.78, .17, .39, .26, .72, .94, .21, .12, .23, .68]$. Distributed into 10 buckets (index = $\lfloor 10 \cdot A[i] \rfloor$). Bucket 1: .17, .12; Bucket 2: .26, .21, .23; Bucket 3: .39; Bucket 6: .68; Bucket 7: .78, .72; Bucket 9: .94. Each bucket sorted by insertion sort, concatenated.

Diagrams & Visuals

[Figure 8.1: Decision tree for insertion sort on 3 elements. Internal nodes $i:j$, leaves with permutations. Height = worst-case comparisons.]

[Figure 8.2: Counting sort on 8-element array with $k=5$. Shows A, C after each phase. Final B sorted.]

[Figure 8.3: Radix sort on seven 3-digit numbers. Shows input and after each of 3 digit-sort passes.]

[Figure 8.4: Bucket sort with $n=10$. Input array $A[1:10]$, array $B[0:9]$ of sorted lists (buckets).]

[Figure 8.5: Columnsort with 6 rows, 3 columns. Shows all 8 steps: (a) input, (b) after sorting columns, (c) after transpose/reshape, (d) after sorting columns, (e) after inverse perm, (f) after sorting columns, (g) after shifting, (h) after sorting columns, (i) after inverse perm – fully sorted in column-major order.]

End-of-Chapter Material

Exercises: - 8.1-1: Smallest possible depth of leaf in decision tree? Answer: $n-1$ (after $n-1$ comparisons, input is already sorted) - 8.1-2: Tight bounds on $\lg(n!)$ without Stirling: $\sum_{k=1}^n \lg k$ - 8.1-3: No comparison sort runs in linear time for at least half, $1/n$, or $1/2^n$ fraction of inputs - 8.1-4: $\Omega(n \lg n)$ lower bound still holds with partial knowledge ($i \bmod 4 = 0$ elements known) - 8.2-1: Illustrate COUNTING-SORT on $A = [6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2]$ - 8.2-2: Prove counting sort is stable - 8.2-3: Forward loop still works but is not stable - 8.2-4: Prove loop invariant for counting sort - 8.2-5: Modify counting sort to sort in-place (just A and C, no B) - 8.2-6: Preprocess to answer range queries $[a:b]$ in $O(1)$ with $O(n+k)$ preprocessing - 8.2-7: Counting sort with fractional parts (at most d decimal digits) $\rightarrow O(n + 10^d k)$ - 8.3-1: Illustrate radix sort on list of English words - 8.3-2: Stable sorts: insertion sort, merge sort; heapsort and quicksort are not. Make any comparison sort stable by storing original index with each element and using it as tiebreaker. Adds $O(n)$ space and time. - 8.3-3: Induction proof of radix sort correctness – needs stability assumption at the induction step - 8.3-4: Reduce $2d$ passes to $d+1$ by combining preprocessing and digit extraction - 8.3-5: Sort n integers in range $0..n^3-1$ in $O(n)$ time using radix sort with $r = \lceil \lg n \rceil$ (2 passes with base n) - 8.3-6: Most-significant-digit-first card sorting: worst-case sorting passes = d ; piles to track = up to 10^d - 8.4-1: Illustrate bucket sort on $A = [79, .13, .16, .64, .39, .20, .89, .53, .71, .42]$ - 8.4-2: Worst-case $O(n^2)$; change to $O(n \lg n)$ worst-case by using $O(n \lg n)$ sort per bucket - 8.4-3: $E[X^2]$ for two fair coin flips: $E[X^2] = 1.5$; $E^2[X] = 1$ - 8.4-4: Modify bucket sort for $A[i] = x_i^2 + y_i^2$ where

$x_i, y_i \in [0,1]$: use buckets based on distance² from origin (sizes $\propto$ area) - 8.4-5: Sort points in unit disk by distance from origin in $O(n)$ average time using bucket sizes reflecting area proportion - 8.4-6: Sort from continuous probability distribution P in linear average time: transform via $P(x)$ to uniform $[0,1]$ then bucket sort

Problems: - 8-1: Probabilistic lower bounds – proves $\Omega(n \lg n)$ average-case for deterministic comparison sorts; randomized sorts have deterministic counterpart with no more expected comparisons - 8-2: Sorting in place in linear time – keys 0/1: (a) stable counting sort; (b) in-place partition; (c) stable and in-place (harder); (d-e) radix sort with these subroutines - 8-3: Sorting variable-length items – (a) total digits n , sort in $O(n)$; (b) total characters n , sort strings in $O(n)$ - 8-4: Water jugs – (a) $O(n^2)$ deterministic; (b) $\Omega(n \lg n)$ lower bound; (c) randomized $O(n \lg n)$ expected - 8-5: Average sorting (k -sorted arrays) – (d) $O(n \lg(n/k))$ to k -sort; (e) sort k -sorted array in $O(n \lg k)$; (f) $\Omega(n \lg n)$ lower bound for constant k - 8-6: Lower bound on merging sorted lists – $2n-1$ comparisons worst-case; $2n-o(n)$ via decision tree; $2n-1$ via consecutive elements argument - 8-7: 0-1 sorting lemma and columnsort – proves columnsort correctly sorts under restrictions; handles s not dividing r (requires $r \geq 4s^2$)

Ch. 9 – Medians and Order Statistics

Named Entities (Terms & Definitions)

- **Order statistic:** The i -th order statistic of a set of n elements is the i -th smallest element.
- **Minimum:** First order statistic ($i = 1$).
- **Maximum:** n -th order statistic ($i = n$).
- **Median:** The “halfway point” of the set. When n is odd, unique at $i = (n+1)/2$. When n even, lower median at $i = n/2$ and upper median at $i = n/2+1$. Text uses “the median” to refer to lower median.
- **Selection problem:** Given a set A of n distinct numbers and an integer i ($1 \leq i \leq n$), find the element $x \in A$ that is larger than exactly $i-1$ other elements.
- **Helpful partitioning:** A partitioning where $|A(j)| \leq (3/4)|A(j-1)|$, i.e., at least $1/4$ of remaining elements are removed from consideration.
- **Middle half:** All but the smallest $\lceil n/4 \rceil - 1$ and greatest $\lceil n/4 \rceil - 1$ elements of the subarray (if it were sorted).
- **Generation (in SELECT analysis):** Sequence of consecutively partitioned sets starting after a helpful partitioning and ending before the next helpful partitioning.
- **Weighted (lower) median:** An element x_k with positive weights summing to 1 such that sum of weights of elements $< x_k < 1/2$ and sum of weights of elements $\leq x_k \geq 1/2$.
- **Post-office location problem:** Given points p_i with weights w_i , find point p minimizing $\sum w_i \cdot d(p, p_i)$.

- **k-th quantiles:** The k-1 order statistics that divide the sorted set into k equal-sized sets (to within 1).

Processes / Algorithms / Pathways

MINIMUM(A, n)

- **Goal:** Find minimum of array A
- **Steps:** (1) $\text{min} = A[1]$ (2) for $i = 2$ to n (3) if $\text{min} > A[i]$ (4) $\text{min} = A[i]$ (5) return min
- **Complexity:** $n-1$ comparisons – optimal (lower bound: $n-1$, since every non-winner must lose at least one comparison)

Simultaneous Minimum and Maximum

- **Goal:** Find both min and max of n elements
- **Complexity:** At most $3\lceil n/2 \rceil$ comparisons
- **Method:** Process elements in pairs. Compare two elements with each other, then compare smaller with current min, larger with current max. 3 comparisons per 2 elements.
- **Setup:** n odd: set $\text{min}=\text{max}=\text{first element}$, process remaining in pairs. n even: compare first 2 elements to initialize min/max, process rest in pairs.
- **Total:** n odd: $3\lceil n/2 \rceil$ comparisons. n even: $1 + 3(n-2)/2 = 3n/2 - 2$ comparisons.
- **Lower bound:** $\lceil 3n/2 \rceil - 2$ comparisons (Exercise 9.1-4)

RANDOMIZED-SELECT(A, p, r, i)

- **Goal:** Find i -th smallest element of $A[p:r]$
- **Input/Output:** Returns i -th smallest element; $1 \leq i \leq r-p+1$
- **Steps:** (1) if $p == r$ return $A[p]$ (2) $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ (3) $k = q-p+1$ (4) if $i == k$ return $A[q]$ (5) elseif $i < k$ return $\text{RANDOMIZED-SELECT}(A, p, q-1, i)$ (6) else return $\text{RANDOMIZED-SELECT}(A, q+1, r, i-k)$
- **Complexity:** Expected $\Theta(n)$ (worst-case $\Theta(n^2)$)
- **Example:** Figure 9.1 shows successive partitionings narrowing the subarray. The answer is the tan element where $p = r = 5$ and $i = 1$.
- **Key difference from quicksort:** Works on only one side of partition (not both).

SELECT(A, p, r, i) – Worst-case linear time

- **Goal:** Find i -th smallest element with worst-case $\Theta(n)$ time
- **Steps:**
 1. while $(r-p+1) \bmod 5 \neq 0$:
 - find minimum of $A[p:r]$, put at $A[p]$
 - if $i == 1$ return $A[p]$
 - else $p = p+1, i = i-1$
 1. $g = (r-p+1)/5$ (number of 5-element groups)

2. for $j = p$ to $p+g-1$: sort 5-element group $[A[j], A[j+g], A[j+2g], A[j+3g], A[j+4g]]$ in place
 3. $x = \text{SELECT}(A, p+2g, p+3g-1, \lceil g/2 \rceil)$ (median of medians)
 4. $q = \text{PARTITION-AROUND}(A, p, r, x)$
 5. $k = q-p+1$
 6. if $i == k$ return $A[q]$
 7. elseif $i < k$ return $\text{SELECT}(A, p, q-1, i)$
 8. else return $\text{SELECT}(A, q+1, r, i-k)$
- **Complexity:** $\Theta(n)$ worst-case. Recurrence: $T(n) \leq T(n/5) + T(7n/10) + \Theta(n)$. Solved by substitution: $T(n) \leq 9cn/10 + \Theta(n) \leq cn$ for sufficiently large c .
 - **Key idea:** Groups of 5, median of medians guarantees each side of partition has at least $3g/2 \geq 3n/10$ elements, so max subproblem size is $7n/10$.

SELECT3(A, p, r, i)

- **Goal:** Alternative worst-case linear-time selection using groups of 3 with a second level of grouping
- **Steps:** Similar to SELECT but divides into groups of 3, then groups of 3 subgroups, recursively finds median of subgroup medians.
- **Complexity:** $\Theta(n)$ worst-case.

Simpler-Randomized-Select(A, p, r, i)

- **Goal:** Simplified version without checking $i == k$ (Professor Mendel's variant)
- **Steps:** (1) if $p == r$ return $A[p]$ (2) $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ (3) $k = q-p+1$ (4) if $i \leq k$ return **Simpler-Randomized-Select**(A, p, q, i) (5) else return **Simpler-Randomized-Select**(A, q+1, r, i-k)
- **Issue:** In worst case, never terminates (can recurse on subarray that still includes the pivot, leading to infinite recursion). Expected time: $\Theta(n)$.

Classifications & Hierarchies

- **Selection algorithms:**
 - Naïve: Sort then index $\rightarrow \Theta(n \lg n)$
 - Min/Max only: $n-1$ comparisons (optimal)
 - Simultaneous min/max: $\leq 3 \lceil n/2 \rceil$ comparisons
 - Expected linear: **RANDOMIZED-SELECT** $\rightarrow \Theta(n)$ expected, $\Theta(n^2)$ worst-case
 - Worst-case linear: **SELECT** (groups of 5) $\rightarrow \Theta(n)$ worst-case
- **Medians:** Lower median ($\lceil (n+1)/2 \rceil$) vs. Upper median ($\lfloor (n+1)/2 \rfloor$)

Comparisons & Trade-offs

Algorithm	Time Complexity	Notes
Sort + index	$O(n \lg n)$	Simplest
MINIMUM	$\theta(n)$	$n-1$ comparisons, optimal
Simultaneous min/max	$\theta(n)$	$\leq 3 \lceil n/2 \rceil$ comparisons
RANDOMIZED-SELECT	$\theta(n)$ expected, $\theta(n^2)$ worst	Practical, simple
SELECT (groups of 5)	$\theta(n)$ worst	Theoretical, large constants

Approach Comparisons to find median

Upper bound (Schönhage et al.)	$< 3n$
Upper bound (Dor & Zwick)	$< 2.95n$
Lower bound (Bent & John)	$2n$
Lower bound (Dor & Zwick)	$(2+\epsilon)n$

Formulas & Equations

Selection problem bound

$$T(n) = T(n/5) + T(7n/10) + \theta(n) \rightarrow T(n) = \theta(n)$$

RANDOMIZED-SELECT expected time intuition

$$T(n) = T(3n/4) + \theta(n) \rightarrow T(n) = \theta(n) \text{ - When pivot always falls in middle half (probability } \geq 1/2)$$

Helpful partitioning

$$|A(j)| \leq (3/4)|A(j-1)| \text{ - Probability of helpful partitioning } \geq 1/2 \text{ (Lemma 9.1)}$$

Number of helpful partitionings needed

$$m = \lceil \log_{4/3} n \rceil \text{ - After at most this many helpful partitionings, only one element remains}$$

Expected number of sets per generation

$$E[X_k] \leq 2 \text{ - } X_k = \text{number of sets in } k\text{-th generation (geometric distribution with } p \geq 1/2)$$

Expected total comparisons

$$E[\text{total comparisons}] < n \cdot \sum_{k=0}^{\infty} (3/4)^k \cdot E[X_k] \leq n \cdot 2 \cdot 1/(1-3/4) = 8n \text{ - Thus expected comparisons} = O(n)$$

Weighted median conditions

$$\sum_{\{x_i < x_k\}} w_i < 1/2 \text{ and } \sum_{\{x_i \leq x_k\}} w_i \geq 1/2$$

Small order statistics recurrence

$U_i(n) = \lceil n/2 \rceil + U_i(\lceil n/2 \rceil) + S(2i)$ - Where $S(2i)$ = cost of finding median of $2i$ elements; yields $U_i(n) = n + O(S(2i) \lg(n/i))$

Rules, Laws & Theorems

Lower bound for finding minimum

- **Statement:** $n-1$ comparisons are necessary to find the minimum of n elements.
- **Proof:** By tournament argument. Every element except the winner must lose at least one comparison.

Lemma 9.1 (Helpful partitioning probability)

- **Statement:** A partitioning is helpful with probability at least $1/2$.
- **Proof:** Pivot in middle half (probability $\geq 1/2$) guarantees at least $\lceil n/4 \rceil$ elements removed $\rightarrow$ helpful.

Theorem 9.2 (RANDOMIZED-SELECT expected time)

- **Statement:** RANDOMIZED-SELECT on n distinct elements has expected running time $\Theta(n)$.

Theorem 9.3 (SELECT worst-case time)

- **Statement:** Running time of SELECT on n elements is $\Theta(n)$.
- **Proof:** By substitution: $T(n) \leq T(n/5) + T(7n/10) + \Theta(n)$. Choose c large enough that $T(n) \leq cn$.

Edge Cases & Pitfalls

- In RANDOMIZED-SELECT, the recursive call with $i-k$ on the high side: the pivot is excluded from both recursive calls (correct).
- SIMPLER-RANDOMIZED-SELECT can infinitely recurse because if pivot is smaller than i -th element, the recursive call includes the pivot again in the subarray $A[p:q]$.
- SELECT's while loop (lines 1-10) ensures n is divisible by 5 before the core algorithm runs; this loop runs at most 4 times.
- If using groups of 3 instead of 5, SELECT does NOT run in linear time – it runs in $O(n \lg n)$ because recursion gives $T(n) \leq T(n/3) + T(2n/3) + \Theta(n) = O(n \lg n)$.

- Groups must be of odd size ≥ 5 for the median-of-medians approach to yield linear time; groups of 3 fail because the guaranteed reduction isn't sufficient.
- SELECT3 (with nested groups of 3 then 3) does achieve $O(n)$ time.
- The linear-time selection algorithms are not subject to the $\Omega(n \lg n)$ lower bound because they solve selection without sorting all elements.
- To enforce distinct elements: convert each value $A[i]$ to ordered pair $(A[i], i)$.

Case Studies & Examples

Weighted Median Example

- **What:** Elements $x_i = [3, 8, 2, 5, 4, 1, 6]$ with weights $w_i = [0.12, 0.35, 0.025, 0.08, 0.15, 0.075, 0.2]$. Median ($i=4$) = 4, but weighted median = 6. Sum of weights less than 6: $0.12 + 0.025 + 0.08 + 0.15 + 0.075 = 0.45 < 0.5$. Only element greater than 6 is 8 with weight $0.35 \leq 0.5$.

RANDOMIZED-SELECT Example (Figure 9.1)

- **What:** Successive partitionings narrow the subarray. Shows $A(0)$ through $A(5)$ with p, r, i values. Helpful partitionings marked. Answer is the element where $p = r = 5$ and $i = 1$.

Oil Pipeline Problem (Professor Olay, Figure 9.4)

- **What:** Find optimal east-west pipeline location minimizing total north-south spur length. Optimal is median of y -coordinates. Can be found in linear time using SELECT.

Diagrams & Visuals

[Figure 9.1: RANDOMIZED-SELECT action. Shows successive partitionings narrowing subarray $A[p:r]$. Tan = current subarray, dark tan = pivot, blue = outside subarray. Answer is tan element where $p=r=5$.]

[Figure 9.2: Generations in SELECT analysis. Vertical lines represent sets $A(j)$. Black sets = results of helpful partitionings ($\leq 3/4$ previous size). Orange sets = within a generation. Generation k starts with $A(h_k)$, ends before next helpful partitioning.]

[Figure 9.3: SELECT algorithm's pivot selection with groups of 5. g groups of 5 elements each shown as columns sorted bottom-to-top. Medians in red. Pivot x (median of medians) labeled. Blue background = elements $\leq x$. Yellow background = elements $\geq x$. Green = pivot. White = unknown side.]

End-of-Chapter Material

Exercises: - 9.1-1: Find second smallest with $n + \lceil \lg n \rceil - 2$ comparisons (find min via tournament, second smallest is among those compared with min) - 9.1-2: Find neither min nor max: 1 comparison (compare any 2 elements, pick the smaller if not min or larger if not max; but to guarantee: compare 3 elements, result is element between them) - 9.1-3: Fastest 3 horses out of 25: 7 races (5 heats + final + one more) - 9.1-4: Lower bound for simultaneous min/max: $\lceil 3n/2 \rceil - 2$ comparisons - 9.2-1: Show RANDOMIZED-SELECT never makes recursive call to 0-length array - 9.2-2: Write iterative version of RANDOMIZED-SELECT - 9.2-3: Describe worst-case partition sequence on $A = \langle 2, 3, 0, 5, 7, 9, 1, 8, 6, 4 \rangle$ - 9.2-4: Expected running time does not depend on input order (induction on n) - 9.3-1: SELECT works in linear time with groups of 7 $\rightarrow$ recurrence $T(n) \leq T(n/7) + T(5n/7) + \theta(n) = \theta(n)$ - 9.3-2: Base case for $n \geq n_0$ instead of while loop; recurrence still $\theta(n)$ - 9.3-3: Use SELECT as subroutine to choose pivot for quicksort $\rightarrow O(n \lg n)$ worst-case - 9.3-4: Finding i -th smallest gives $i-1$ smaller and $n-i$ larger elements free (no extra comparisons) - 9.3-5: Median of 5 elements in 6 comparisons - 9.3-6: Use black-box median to find arbitrary order statistic in linear time - 9.3-7: Oil pipeline: optimal location = median of y -coordinates; find in $O(n)$ with SELECT - 9.3-8: Find k -th quantiles in $O(n \lg k)$ time (recursively partition) - 9.3-9: Find k numbers closest to median in $O(n)$ time - 9.3-10: Median of two sorted arrays in $O(\lg n)$ time

Problems: - 9-1: Largest i numbers – (a) sort $O(n \lg n + i)$; (b) max-priority queue $O(n + i \lg n)$; (c) order-statistic + partition $O(n + i \lg i)$ - 9-2: SIMPLER-RANDOMIZED-SELECT – worst-case never terminates; expected time $O(n)$ - 9-3: Weighted median – (a) median = weighted median with $w_i = 1/n$; (b) $O(n \lg n)$ via sorting; (c) $O(n)$ via SELECT; (d) weighted median solves 1D post-office; (e) 2D Manhattan: independent medians of x and y coordinates - 9-4: Small order statistics – $U_i(n) = n + O(S(2i) \lg(n/i))$; for constant i : $n + O(\lg n)$; for $i = n/k$: $n + O(S(2n/k) \lg k)$ - 9-5: Alternative analysis of randomized selection – using indicator variables X_{ijk} : $E[X_{ijk}]$ depends on i, j, k ; $E[X_i] \leq 4n \rightarrow O(n)$ expected - 9-6: Select with groups of 3 – (a) any odd constant > 3 gives linear; (b) groups of 3 gives $O(n \lg n)$; SELECT3 (nested 3 then 3) gives $O(n)$

Comprehensive Extraction: Group 3

Ch. 10 – Elementary Data Structures

Named Entities (Terms & Definitions)

- **Elementary data structures:** Simple data structures using pointers – arrays, matrices, stacks, queues, linked lists, and rooted trees.
- **Array:** A contiguous sequence of bytes in memory. If first element has index s , array starts at memory address a , each element occupies b bytes, then the i -th element occupies bytes $a + b(i - s)$ through $a + b(i - s + 1) - 1$.
- **Row-major order:** Matrix stored row by row in a single array.
- **Column-major order:** Matrix stored column by column in a single array.
- **Block representation:** Matrix divided into blocks, each block stored contiguously.
- **Stack:** Dynamic set implementing LIFO (last-in, first-out) policy. INSERT = PUSH, DELETE = POP.
- **Queue:** Dynamic set implementing FIFO (first-in, first-out) policy. INSERT = ENQUEUE, DELETE = DEQUEUE.
- **Deque:** Double-ended queue allowing insertion and deletion at both ends.
- **Linked list:** Data structure in which objects are arranged in a linear order determined by pointers in each object, not array indices.
- **Doubly linked list:** Each element has attributes key, next, and prev. $x.next$ points to successor, $x.prev$ points to predecessor.
- **Singly linked list:** Each element has a next pointer but not a prev pointer.
- **Sorted list:** Linear order of list corresponds to linear order of keys.
- **Unsorted list:** Elements can appear in any order.
- **Circular list:** prev pointer of head points to tail, next pointer of tail points to head.
- **Sentinel:** A dummy object that allows simplification of boundary conditions. In a linked list L , the sentinel is an object $L.nil$ that represents NIL but has all attributes of other objects.
- **Compact list:** An n -element singly linked list represented with two arrays key and next, stored only in positions 1 through n .
- **Rooted tree:** Represented by linked data structures; each node is an object with key attribute and pointers.
- **Binary tree:** Uses attributes p (parent), left, and right.
- **Left-child, right-sibling representation:** Each node has pointers left-child (leftmost child) and right-sibling (sibling immediately to the right). Uses $O(n)$ space for any n -node rooted tree.

Processes / Algorithms / Pathways

STACK-EMPTY(S)

- **Goal:** Test whether stack is empty
- **Input/Output:** Stack S → returns TRUE if S.top == 0, else FALSE
- **Steps:** (1) if S.top == 0 return TRUE else return FALSE
- **Complexity:** O(1)

PUSH(S, x)

- **Goal:** Insert element x onto stack
- **Input/Output:** Stack S, element x → modifies S
- **Steps:** (1) if S.top == S.size error "overflow" (2) else S.top = S.top + 1 (3) S[S.top] = x
- **Complexity:** O(1)
- **Example:** PUSH(S, 17) on stack with 4 elements: S.top becomes 5, S[5] = 17 (Figure 10.2(b))

POP(S)

- **Goal:** Delete and return top element from stack
- **Input/Output:** Stack S → returns element
- **Steps:** (1) if STACK-EMPTY(S) error "underflow" (2) else S.top = S.top - 1 (3) return S[S.top + 1]
- **Complexity:** O(1)
- **Example:** POP(S) from stack with elements [17,3,9,?] returns 3 (most recently pushed), S.top becomes 4 (Figure 10.2(c))

ENQUEUE(Q, x)

- **Goal:** Insert element x at tail of queue
- **Input/Output:** Queue Q, element x → modifies Q
- **Steps:** (1) Q[Q.tail] = x (2) if Q.tail == Q.size then Q.tail = 1 else Q.tail = Q.tail + 1
- **Complexity:** O(1)
- **Example:** ENQUEUE(Q, 17), ENQUEUE(Q, 3), ENQUEUE(Q, 5) on queue with 5 elements – Figure 10.3(b)

DEQUEUE(Q)

- **Goal:** Delete and return head element from queue
- **Input/Output:** Queue Q → returns element
- **Steps:** (1) x = Q[Q.head] (2) if Q.head == Q.size then Q.head = 1 else Q.head = Q.head + 1 (3) return x
- **Complexity:** O(1)
- **Example:** DEQUEUE(Q) returns 15 (formerly at head), new head has key 6 – Figure 10.3(c)

LIST-SEARCH(L, k)

- **Goal:** Find first element with key k in list L by simple linear search
- **Input/Output:** List L, key k → pointer to element or NIL
- **Steps:** (1) $x = L.head$ (2) while $x \neq NIL$ and $x.key \neq k$: $x = x.next$ (3) return x
- **Complexity:** $O(n)$ worst-case
- **Example:** LIST-SEARCH(L, 4) in {1,4,9,16} returns pointer to third element; LIST-SEARCH(L, 7) returns NIL

LIST-PREPEND(L, x)

- **Goal:** Insert element x at front of linked list
- **Input/Output:** List L, element x (key set) → modifies L
- **Steps:** (1) $x.next = L.head$ (2) $x.prev = NIL$ (3) if $L.head \neq NIL$: $L.head.prev = x$ (4) $L.head = x$
- **Complexity:** $O(1)$
- **Example:** After LIST-PREPEND(L, x) with $x.key = 25$, new head has key 25, points to old head with key 9 – Figure 10.4(b)

LIST-INSERT(x, y)

- **Goal:** Splice element x into list immediately following element y
- **Input/Output:** Element x (to insert), element y (to follow) → modifies list
- **Steps:** (1) $x.next = y.next$ (2) $x.prev = y$ (3) if $y.next \neq NIL$: $y.next.prev = x$ (4) $y.next = x$
- **Complexity:** $O(1)$ – never references list object L
- **Example:** LIST-INSERT(x, y) where $x.key = 36$ and y points to node with key 9 – Figure 10.4(c)

LIST-DELETE(L, x)

- **Goal:** Remove element x from linked list L
- **Input/Output:** List L, pointer to element x → modifies L
- **Steps:** (1) if $x.prev \neq NIL$: $x.prev.next = x.next$ else: $L.head = x.next$ (2) if $x.next \neq NIL$: $x.next.prev = x.prev$
- **Complexity:** $O(1)$ (but call to LIST-SEARCH first makes worst-case $O(n)$ when deleting by key)
- **Example:** LIST-DELETE(L, x) where x points to object with key 4 – Figure 10.4(d)

LIST-DELETE'(x)

- **Goal:** Delete element from circular doubly linked list with sentinel
- **Input/Output:** Element x → modifies list (no list parameter needed)
- **Steps:** (1) $x.prev.next = x.next$ (2) $x.next.prev = x.prev$
- **Complexity:** $O(1)$

LIST-INSERT'(x, y)

- **Goal:** Insert element x into circular doubly linked list with sentinel, following object y
- **Steps:** (1) x.next = y.next (2) x.prev = y (3) y.next.prev = x (4) y.next = x

LIST-SEARCH'(L, k)

- **Goal:** Search circular doubly linked list with sentinel for key k
- **Input/Output:** List L (with sentinel), key k → pointer to element or NIL
- **Steps:** (1) L.nil.key = k (store key in sentinel) (2) x = L.nil.next (3) while x.key != k: x = x.next (4) if x == L.nil: return NIL else return x
- **Complexity:** Same asymptotic as LIST-SEARCH, but eliminates one comparison per iteration
- **Key insight:** Guarantees key is found somewhere (in sentinel if not really in list), saving boundary check in loop

Tree walks

- **Inorder tree walk** (INORDER-TREE-WALK): Prints root between left and right subtrees – produces sorted order
- **Preorder tree walk:** Prints root before left and right subtrees
- **Postorder tree walk:** Prints root after left and right subtrees

INORDER-TREE-WALK(x)

- **Goal:** Print all keys in binary search tree in sorted order
- **Input/Output:** Node x → prints keys
- **Steps:** (1) if x != NIL: (2) INORDER-TREE-WALK(x.left) (3) print x.key (4) INORDER-TREE-WALK(x.right)
- **Complexity:** $\Theta(n)$ on n-node tree (Theorem 12.1)

COMPACT-LIST-SEARCH(key, next, head, n, k)

- **Goal:** Search compact sorted linked list using random skips for expected $O(\sqrt{n})$ time
- **Input/Output:** Arrays key[1:n], next[1:n], head index, size n, key k → index or NIL
- **Steps:** (1) i = head (2) while i != NIL and key[i] < k: (3) j = RANDOM(1,n) (4) if key[i] < key[j] and key[j] ≤ k: i = j (5) if key[i] == k: return i (6) i = next[i] (7) if i == NIL or key[i] > k: return NIL else return i
- **Complexity:** Expected $O(\sqrt{n})$ time
- **Assumptions:** All keys distinct, list is sorted, compact

Data Structures

Array

- **Properties:** Contiguous memory, $O(1)$ element access, must be same element size (or store pointers)
- **Formulas:** With 1-origin: element i at bytes $a + b(i-1)$ through $a + bi - 1$. With 0-origin: element i at bytes $a + bi$ through $a + b(i+1) - 1$
- **Operations:** Access $O(1)$, Insert/delete at front: $O(n)$ worst-case

Matrix (2D array)

- **Properties:** $m \times n$ matrix
- **Row-major order:** Single array index $s + n(i - s) + (j - s)$; when $s=1$: $n(i-1)+j$; when $s=0$: $ni+j$
- **Column-major order:** Single array index $s + m(j - s) + (i - s)$; when $s=1$: $i + m(j-1)$; when $s=0$: $i + mj$
- **Multiple-array representation:** Each row (or column) in its own array, with an array of pointers
- **Block representation:** Matrix divided into blocks stored contiguously
- **Single-array typically more efficient; multiple-array allows ragged arrays**

Stack (array-based)

- **Properties:** LIFO policy, array $S[1:n]$, attributes $S.top$ and $S.size$
- **Operations:** STACK-EMPTY $O(1)$, PUSH $O(1)$, POP $O(1)$
- **Edge cases:** $S.top = 0 \rightarrow$ empty stack; PUSH on full stack $\rightarrow$ overflow; POP on empty $\rightarrow$ underflow

Queue (array-based)

- **Properties:** FIFO policy, array $Q[1:n]$, attributes $Q.head$, $Q.tail$, $Q.size$. Circular wrap-around.
- **Operations:** ENQUEUE $O(1)$, DEQUEUE $O(1)$
- **Empty:** $Q.head = Q.tail$; Full: $Q.head = Q.tail + 1$ or $(Q.head = 1 \text{ and } Q.tail = Q.size)$
- **Edge cases:** Queue underflow (dequeue from empty), queue overflow (enqueue to full)

Doubly Linked List

- **Properties:** Linear order by pointers, each node has key, next, prev; L.head points to first element
- **Operations:** SEARCH $O(n)$, PREPEND $O(1)$, INSERT $O(1)$, DELETE $O(1)$ (given pointer)
- **When to use:** Need fast insert/delete anywhere; don't need indexed access

Circular Doubly Linked List with Sentinel

- **Properties:** L.nil replaces NIL; L.head eliminated (use L.nil.next); empty list: L.nil.next = L.nil.prev = L.nil
- **Operations:** INSERT' $O(1)$, DELETE' $O(1)$, SEARCH' $O(n)$ but with lower constant factor
- **Memory trade-off:** Extra storage for sentinel; not ideal for many small lists

Binary Tree

- **Properties:** Each node has p, left, right; T.root points to root
- **Operations:** Tree walks (inorder/preorder/postorder) $O(n)$

Rooted Tree with Unbounded Branching (Left-Child, Right-Sibling)

- **Properties:** Each node x has x.left-child (leftmost child) and x.right-sibling (sibling to right); $O(n)$ space
- **Operations:** Access children in linear time, access parent in $O(1)$ via parent pointer
- **When to use:** Arbitrary branching factor where allocating k pointers per node is wasteful

Classifications & Hierarchies

List types: - Singly vs. Doubly linked - Sorted vs. Unsorted - Circular vs. Non-circular - With sentinel vs. Without sentinel

Matrix storage schemes: - Single-array: row-major, column-major - Multiple-array: row-major (one array per row), column-major (one array per column) - Block representation

Tree representations: - Binary tree (left, right, p) - Left-child, right-sibling (for arbitrary branching) - Array-based (heap, complete binary tree) - Parent-only pointers

Comparisons & Trade-offs

Dimension	Array Doubly Linked List	
Access k-th element	$O(1)$	$O(k)$
Insert at front	$O(n)$	$O(1)$
Delete first element	$O(n)$	$O(1)$
Memory overhead	Low	One pointer per element (singly) or two (doubly)
Dimension	Single-array matrix	Multiple-array matrix
	More efficient	Less efficient

Dimension	Single-array matrix	Multiple-array matrix
Efficiency on modern machines		
Flexibility	Less flexible	More flexible (ragged arrays)
Dimension	With sentinel	Without sentinel
Code simplicity	Simpler	Boundary conditions needed
Memory	Extra sentinel object	No extra storage
Speed	Better constant factor	Same asymptotic

Formulas & Equations

Array element address (1-origin, s=1)

i-th element occupies bytes $a + b(i - 1)$ through $a + bi - 1$

Array element address (0-origin, s=0)

i-th element occupies bytes $a + bi$ through $a + b(i + 1) - 1$

Row-major index (1-origin)

$\text{index} = n(i - 1) + j$

Column-major index (1-origin)

$\text{index} = i + m(j - 1)$

Row-major index (0-origin)

$\text{index} = ni + j$

Column-major index (0-origin)

$\text{index} = i + mj$

Row-major index (general, starting at s)

$\text{index} = s + n(i - s) + (j - s)$

Column-major index (general, starting at s)

$\text{index} = s + m(j - s) + (i - s)$

Rules, Laws & Theorems

Theorem 12.1 (Inorder Tree Walk Time)

- **Statement:** If x is root of an n -node subtree, INORDER-TREE-WALK(x) takes $\Theta(n)$ time.
- **Proof:** $T(0) = c$; $T(n) \leq T(k) + T(n-k-1) + d$; by substitution $T(n) \leq (c+d)n + c$.

Edge Cases & Pitfalls

- **Stack overflow:** Attempting PUSH when $S.top == S.size$
- **Stack underflow:** Attempting POP when stack is empty
- **Queue overflow:** Enqueue when queue is full ($Q.head = Q.tail + 1$)
- **Queue underflow:** Dequeue when queue is empty ($Q.head = Q.tail$)
- **Sentinel:** Never delete the sentinel unless deleting entire list
- **Array with variable-size elements:** Must store pointers, not elements directly
- **Linked list SEARCH:** $\Theta(n)$ worst-case, no binary search possible
- **Singly linked list DELETE:** $\Theta(n)$ worst-case (need predecessor)
- **Compact list:** All keys must be distinct for random skips to help

Case Studies & Examples

Stack operation sequence (Exercise 10.1-2)

Sequence: PUSH($S, 4$), PUSH($S, 1$), PUSH($S, 3$), POP(S), PUSH($S, 8$), POP(S) on initially empty stack $S[1:6]$. After PUSH(4): $S.top=1$, $S[1]=4$ After PUSH(1): $S.top=2$, $S[2]=1$ After PUSH(3): $S.top=3$, $S[3]=3$ After POP: returns 3, $S.top=2$ After PUSH(8): $S.top=3$, $S[3]=8$ After POP: returns 8, $S.top=2$

Queue operation sequence (Exercise 10.1-4)

Sequence: ENQUEUE($Q, 4$), ENQUEUE($Q, 1$), ENQUEUE($Q, 3$), DEQUEUE(Q), ENQUEUE($Q, 8$), DEQUEUE(Q) on initially empty $Q[1:6]$. Initially: $Q.head = Q.tail = 1$ Enqueue 4: $Q[1]=4$, $Q.tail=2$ Enqueue 1: $Q[2]=1$, $Q.tail=3$ Enqueue 3: $Q[3]=3$, $Q.tail=4$ Dequeue: returns 4, $Q.head=2$ Enqueue 8: $Q[4]=8$, $Q.tail=5$ Dequeue: returns 1, $Q.head=3$

Linked list insertion/deletion example (Figure 10.4)

List $L = \{1, 4, 9, 16\}$ (doubly linked). After LIST-PREPEND(L, x) with $x.key=25$: head becomes $25 \rightarrow 9 \rightarrow 4 \rightarrow 16 \rightarrow 1$. After LIST-INSERT(x, y) with $x.key=36$ following y (key 9): $25 \rightarrow 9 \rightarrow 36 \rightarrow 4 \rightarrow 16 \rightarrow 1$. After LIST-DELETE(L, x) where x points to key 4: $25 \rightarrow 9 \rightarrow 36 \rightarrow 16 \rightarrow 1$.

Numerical example: Row-major vs column-major ($M[2,1]$ in 2×3 matrix with 1-origin)

Row-major: $3(2-1)+1 = 4$ Column-major: $2+2(1-1) = 2$

Diagrams & Visuals

Figure 10.1: Four ways to store a 2×3 matrix. (a) Row-major single array. (b) Column-major single array. (c) Row-major with one array per row + pointer array. (d) Column-major with one array per column + pointer array.

Figure 10.2: Array implementation of stack S. (a) Stack with 4 elements, top = 9. (b) After PUSH(17) and PUSH(3). (c) After POP returns 3 (most recently pushed).

Figure 10.3: Queue implemented using array Q[1:12]. (a) 5 elements at Q[7:11]. (b) After ENQUEUE(17,3,5). (c) After DEQUEUE returns 15.

Figure 10.4: (a) Doubly linked list {1,4,9,16}. (b) After LIST-PREPEND with key 25. (c) After LIST-INSERT with key 36 following key 9. (d) After LIST-DELETE of key 4.

Figure 10.5: Circular doubly linked list with sentinel L.nil. (a) Empty list: L.nil.next = L.nil.prev = L.nil. (b) List {1,4,9,16} with head 9, tail 1. (c-e) After various INSERT'/DELETE' operations.

Figure 10.6: Binary tree T representation with p (top), left (lower left), right (lower right) attributes.

Figure 10.7: Left-child, right-sibling representation with p (top), left-child (lower left), right-sibling (lower right).

End-of-Chapter Material

Exercises 10.1: - **10.1-1:** For $m \times n$ matrix with m, n powers of 2, rows/cols indexed from 0, stored as 2×2 block matrix in single array (0-origin). Show how to construct binary representation of index from binary of i and j . - **10.1-2:** Illustrate PUSH(S,4), PUSH(S,1), PUSH(S,3), POP(S), PUSH(S,8), POP(S) on initially empty stack S[1:6]. - **10.1-3:** Implement two stacks in one array A[1:n] so neither overflows unless total elements = n. PUSH/POP $O(1)$. - **10.1-4:** Illustrate ENQUEUE(Q,4), ENQUEUE(Q,1), ENQUEUE(Q,3), DEQUEUE(Q), ENQUEUE(Q,8), DEQUEUE(Q) on empty Q[1:6]. - **10.1-5:** Rewrite ENQUEUE and DEQUEUE to detect underflow and overflow. - **10.1-6:** Write four $O(1)$ -time procedures for deque (double-ended queue) insertion/deletion at both ends. - **10.1-7:** Implement a queue using two stacks. Analyze running time. - **10.1-8:** Implement a stack using two queues. Analyze running time.

Exercises 10.2: - **10.2-1:** INSERT on singly linked list is $O(1)$ but DELETE is $O(n)$ worst-case. - **10.2-2:** Implement stack with singly linked list; PUSH/POP $O(1)$. Need any extra attributes? - **10.2-3:** Implement queue with singly linked list; ENQUEUE/DEQUEUE $O(1)$. Need extra attributes? - **10.2-4:** Support UNION of two disjoint sets in $O(1)$ using suitable list structure. - **10.2-5:** $O(n)$ -time nonrecursive procedure to reverse singly linked list using constant extra storage. - **10.2-6** □: Implement doubly linked list using one pointer $x.\text{np} = x.\text{next} \text{ XOR } x.\text{prev}$. Describe SEARCH, INSERT, DELETE. Reverse in $O(1)$.

Exercises 10.3: - **10.3-1:** Draw binary tree rooted at index 6 from given attributes table. - **10.3-2:** $O(n)$ -time recursive procedure to print keys of n -node binary tree. - **10.3-3:** $O(n)$ -time nonrecursive procedure using stack to print keys. - **10.3-4:** $O(n)$ -time procedure to print all keys of arbitrary rooted tree (left-child, right-sibling). - **10.3-5** □: $O(n)$ -time nonrecursive procedure, constant extra space, no modification, no stack. - **10.3-6** □: Use only two pointers + one boolean per node to access parent/children in linear time.

Problems: - **10-1 Comparison among lists:** Fill table of asymptotic worst-case running times for SEARCH, INSERT, DELETE, SUCCESSOR, PREDECESSOR, MINIMUM, MAXIMUM across unsorted/singly, sorted/singly, unsorted/doubly, sorted/doubly linked lists. - **10-2 Mergeable heaps using linked lists:** Implement mergeable heap (MAKE-HEAP, INSERT, MINIMUM, EXTRACT-MIN, UNION) using linked lists. (a) Sorted lists. (b) Unsorted lists. (c) Unsorted, disjoint sets. - **10-3 Searching a sorted compact list:** Analyze COMPACT-LIST-SEARCH. Prove expected $O(\sqrt{n})$ time. Parts (a)-(h): compare with COMPACT-LIST-SEARCH', derive bounds, show $E[X_t] \leq n/(t+1)$, conclude $O(\sqrt{n})$ expected time.

Ch. 11 – Hash Tables

Named Entities (Terms & Definitions)

- **Hash table:** Effective data structure for implementing dictionaries; generalized ordinary array. Average $O(1)$ for dictionary operations.
- **Direct-address table:** Array $T[0:m-1]$ where slot k points to element with key k ; works when universe U is small.
- **Slot:** Each position in a hash table.
- **Hash function h :** Maps universe U of keys into slots of hash table $T[0:m-1]$; $h: U \rightarrow \{0, 1, \dots, m-1\}$.
- **Hash value:** $h(k)$ is the hash value of key k .
- **Collision:** Two keys may hash to the same slot.
- **Independent uniform hash function:** For each k in U , $h(k)$ is an element randomly and independently chosen uniformly from $\{0, \dots, m-1\}$. Also called a random oracle.
- **Independent uniform hashing:** Using an independent uniform hash function.

- **Chaining:** Collision resolution where each nonempty slot points to a linked list of all keys with that hash value.
- **Load factor α :** $\alpha = n/m$, the average number of elements stored in a chain.
- **Open addressing:** Collision resolution where all elements occupy the hash table itself; no storage outside table.
- **Probe:** Successive examination of hash table slots during insertion/search.
- **Probe sequence:** Sequence $\langle h(k,0), h(k,1), \dots, h(k,m-1) \rangle$, a permutation of $\langle 0,1,\dots,m-1 \rangle$.
- **Double hashing:** $h(k,i) = (h_0(k) + ih_1(k)) \bmod m$; uses two auxiliary hash functions.
- **Linear probing:** $h(k,i) = (h_0(k) + i) \bmod m$; simplest open addressing; $h_0(k) = 1$ for all k .
- **Independent uniform permutation hashing:** Each key's probe sequence is equally likely to be any of the $m!$ permutations.
- **Static hashing:** Single fixed hash function; works well for some input sets but vulnerable to adversarial keys.
- **Random hashing:** Choose hash function randomly from a family at runtime, independent of keys.
- **Universal hashing:** Family H of hash functions is universal if for any distinct keys k_1, k_2 , the number of functions $h \in H$ with $h(k_1) = h(k_2)$ is at most $|H|/m$.
- **ϵ -universal:** For any distinct keys k_1, k_2 , $\Pr[h(k_1) = h(k_2)] \leq \epsilon$.
- **d -independent:** For any distinct keys $k_1, \dots, k_d$ and any slots $q_1, \dots, q_d$, $\Pr[h(k_i) = q_i \text{ for all } i] = 1/m^d$.
- **Uniform family H :** For any key k and slot q , $\Pr[h(k) = q] = 1/m$.
- **Division method:** $h(k) = k \bmod m$.
- **Multiplication method:** $h(k) = \lfloor m(ka \bmod 1) \rfloor$ where $0 < A < 1$.
- **Multiply-shift method:** $h_\ell(k) = ((ka) \bmod 2^w) \gg (w-\ell)$ where $m = 2^\ell$.
- **Cryptographic hash functions:** Complex pseudorandom functions like SHA-256; useful for implementing approximate random oracle.
- **Wee hash function:** Simple hash function based on addition, multiplication, and swapping halves of a word; related to RC6.
- **Primary clustering:** Long runs of occupied slots build up in linear probing, increasing average search time.
- **Free list:** A linked list of all unused slots within the hash table itself.
- **Bit vector:** An array of bits (0 or 1); more space-efficient than array of pointers.

Processes / Algorithms / Pathways

DIRECT-ADDRESS-SEARCH(T, k)

- **Steps:** (1) return $T[k]$
- **Complexity:** $O(1)$

DIRECT-ADDRESS-INSERT(T, x)

- **Steps:** (1) $T[x.key] = x$
- **Complexity:** $O(1)$

DIRECT-ADDRESS-DELETE(T, x)

- **Steps:** (1) $T[x.key] = \text{NIL}$
- **Complexity:** $O(1)$

CHAINED-HASH-INSERT(T, x)

- **Steps:** (1) $\text{LIST-PREPEND}(T[h(x.key)], x)$
- **Complexity:** $O(1)$ worst-case (assuming element not already present)

CHAINED-HASH-SEARCH(T, k)

- **Steps:** (1) return $\text{LIST-SEARCH}(T[h(k)], k)$
- **Complexity:** $\Theta(1+\alpha)$ average under independent uniform hashing; $\Theta(n)$ worst-case

CHAINED-HASH-DELETE(T, x)

- **Steps:** (1) $\text{LIST-DELETE}(T[h(x.key)], x)$
- **Complexity:** $O(1)$ if lists are doubly linked

HASH-INSERT(T, k) (open addressing)

- **Steps:** (1) $i = 0$ (2) repeat: $q = h(k, i)$; if $T[q] == \text{NIL}$: $T[q] = k$; return q ; else $i = i+1$ (3) until $i == m$ (4) error "hash table overflow"
- **Complexity:** Expected $O(1/(1-\alpha))$ probes

HASH-SEARCH(T, k) (open addressing)

- **Steps:** (1) $i = 0$ (2) repeat: $q = h(k, i)$; if $T[q] == k$: return q ; $i = i+1$ (3) until $T[q] == \text{NIL}$ or $i == m$ (4) return NIL
- **Termination condition:** Finds empty slot $\rightarrow$ key not present

LINEAR-PROBING-HASH-DELETE(T, q)

- **Goal:** Delete key stored at position q in linear-probing hash table without using DELETED marker
- **Steps:** (1) while TRUE: (2) $T[q] = \text{NIL}$ (3) $q' = q$ (4) repeat: $q' = (q'+1) \bmod m$; $k' = T[q']$; if $k' == \text{NIL}$: return; until $g(k', q) < g(k', q')$ (5) $T[q] = k'$; $q = q'$
- **Key concept:** Inverse function $g(k, q) = (q - h(k)) \bmod m$ maps key k and slot q to probe number. If $g(k', q) < g(k', q')$, then slot q was probed before q' during k' insertion, so k' must move to q .

HASH-DELETE (with DELETED marker)

- **Steps:** Fill deleted key's slot with special value DELETED; HASH-INSERT treats DELETED as empty; HASH-SEARCH passes over DELETED values
- **Pitfall:** Search times no longer depend on load factor α

Division method hash

$h(k) = k \bmod m$ - Fast (single division operation) - Works well when m is prime not too close to an exact power of 2 - May complicate applications (constrains table size to be prime)

Multiplication method hash

$h(k) = \lfloor m(kA \bmod 1) \rfloor = \lfloor m(kA - \lfloor kA \rfloor) \rfloor$ - Where $0 < A < 1$ (constant) - Value of m is not critical - Can choose m independently of A

Multiply-shift hash

- For $m = 2^\ell$ ($\ell \leq w$, w = bits in machine word)
- Choose odd w -bit integer a
- $h_\ell(k) = ((ka) \bmod 2^w) \gg (w - \ell)$
- Implemented with 3 machine instructions: multiply, subtract, logical right shift
- Example: $k=123456$, $\ell=14$, $m=16384$, $w=32$, $a=2654435769 \rightarrow ka = 327706022297664$, $r_\ell=76300$, $r_\ell=17612864$, $h_\ell(k)=67$
- $2/m$ -universal (Theorem 11.5)

Number-theoretic universal hash family H_{pm}

$h_b(k) = ((ak + b) \bmod p) \bmod m$ - p is prime, $p > m$, $p >$ all keys
- $a \in \{1, 2, \dots, p-1\}$, $b \in \{0, 1, \dots, p-1\}$ - Family size: $p(p-1)$ hash functions - Theorem 11.4: H_{pm} is universal - Proof: distinct $k_1, k_2 \rightarrow r_1 \neq r_2 \bmod p$; $(a, b) \leftrightarrow (r_1, r_2)$ 1-to-1; collision prob $\leq 1/m$

Multiply-shift $2/m$ -universal family

$H = \{h_\ell : h_\ell(k) = ((ka) \bmod 2^w) \gg (w - \ell), a \text{ odd}\}$ - Theorem 11.5:
This family is $2/m$ -universal - Recommended for practice: fast and provably good

Cryptographic hash function usage

$h(k) = \text{SHA-256}(a \parallel k) \bmod m$ - a is a "salt" string prepended to input - SHA-256 produces 256-bit output for any input - AES-NI instructions provide fast hardware implementation

Wee hash function (short inputs, $t \leq w$ bits)

$h_{\{a,b,t,r\}}(k) = f_a^{+r}(k + b + 2^t) \bmod m - f_a(k) = \text{swap}((2k^2 + ak) \bmod 2^w) - \text{swap}(x) = (x \ll (w/2)) + (x \gg (w/2)) - r = 4$
recommended; a odd; b random - Can be implemented entirely in CPU registers - 2-10x faster than a single random hash table probe

Wee hash function (variable-length inputs)

WEE(k, a, b, t, r, m): (1) $u = \lceil t/w \rceil$ (2) $k_0, k_1, \dots, k_u = \text{chop}(k)$
(3) $q = b$ (4) for $i = 1$ to u : $q = f_{\{a+2^t\}}^{+r}(q + k_i)$ (5)
return $q \bmod m$ - chop(k) breaks key into w -bit words, padding with zeros - Consistent with single-word version - Approximately 5-independent based on CBC-MAC security assumptions

Data Structures

Direct-Address Table

- **Properties:** Array $T[0:m-1]$; slot k points to element with key k (or NIL); universe $U = \{0, \dots, m-1\}$
- **Operations:** SEARCH $O(1)$, INSERT $O(1)$, DELETE $O(1)$
- **When to use:** Universe of keys is reasonably small
- **Space optimization:** Store elements directly in slots (key = index)

Hash Table with Chaining

- **Properties:** Array $T[0:m-1]$; each slot points to linked list of elements with that hash value; load factor $\alpha = n/m$
- **Operations:** INSERT $O(1)$, SEARCH $\Theta(1+\alpha)$ avg / $\Theta(n)$ worst, DELETE $O(1)$ (doubly linked lists)
- **When to use:** Dictionary operations with average $O(1)$; many keys possible but few stored

Hash Table with Open Addressing

- **Properties:** All elements in table itself; $\alpha \leq 1$; no pointers; uses probe sequences
- **Operations:** INSERT expected $\leq 1/(1-\alpha)$ probes; SEARCH expected $\leq (1/\alpha)\ln(1/(1-\alpha))$ successful, $\leq 1/(1-\alpha)$ unsuccessful
- **When to use:** Want to avoid pointers; load factor can be kept low; deletions infrequent

Classifications & Hierarchies

Collision resolution methods: - Chaining (lists outside table) -
Open addressing (all in table): - Linear probing ($h_1(k)=1$) -
Double hashing (general h_1) - Quadratic probing, etc.

Hash function approaches: - Static hashing: division method, multiplication method - Random hashing: universal families, multiply-shift, number-theoretic, cryptographic - Wee hash function family

Hash function property hierarchy: - Uniform $\rightarrow$ Universal $\rightarrow$ λ -universal $\rightarrow$ d-independent

Comparisons & Trade-offs

Dimension	Chaining	Open Addressing
Storage	Extra memory for pointers/lists	No extra storage
Load factor	α can be > 1	$\alpha \leq 1$
Deletion	Easy ($O(1)$ with doubly linked lists)	Tricky (need DELETED or special method)
Cache performance	Poor (pointer chasing)	Better (especially linear probing)
Worst-case search	$\Theta(n)$ (all keys same slot)	Must probe full table if $\alpha=1$

Dimension	Linear Probing	Double Hashing
Distinct probe sequences	m	$\Theta(m^2)$
Primary clustering	Yes	No
Cache performance	Excellent	Poor
Deletion	Possible without DELETED	Usually needs DELETED

Dimension	Division Method	Multiplication Method	Universal Hashing
Speed	Fast (1 division)	Moderate	Varies
Provable avg-case	No	No	Yes
Vulnerable to adversary	Yes	Yes	No (randomized)

Formulas & Equations

Load factor

$$\alpha = n/m$$

Division method hash

$$h(k) = k \bmod m$$

Multiplication method hash

$$h(k) = \lfloor m(kA \bmod 1) \rfloor, \text{ where } kA \bmod 1 = kA - \lfloor kA \rfloor, 0 < A < 1$$

Multiply-shift hash

$$h_\ell(k) = ((ka) \bmod 2^w) \gg (w-\ell), \text{ where } m = 2^\ell$$

Double hashing probe sequence

$$h(k, i) = (h_\ell(k) + i \cdot h_\ell(k)) \bmod m$$

Linear probing hash

$$h(k, i) = (h_\ell(k) + i) \bmod m$$

Inverse function for linear probing deletion

$$g(k, q) = (q - h_\ell(k)) \bmod m$$

Number-theoretic universal hash

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$$

Wee hash function

$$f_\ell(k) = \text{swap}((2k^2 + ak) \bmod 2^w) \quad \text{swap}(x) = (x \gg (w/2)) + (x \ll (w/2))$$

Probability of collision (universal hashing)

$$\Pr[h(k_1) = h(k_2)] \leq 1/m$$

Rules, Laws & Theorems

Theorem 11.1 (Unsuccessful search, chaining)

- **Statement:** In a hash table with chaining and independent uniform hashing, an unsuccessful search takes $\Theta(1+\alpha)$ time on average.
- **Proof:** Key equally likely to hash to any of m slots; expected list length = α . Expected elements examined = α . Total time = $\Theta(1+\alpha)$.

Theorem 11.2 (Successful search, chaining)

- **Statement:** In a hash table with chaining and independent uniform hashing, a successful search takes $\Theta(1+\alpha)$ time on average.

- **Proof:** Element equally likely to be any of n stored. Expected elements examined = $1 + (\alpha/2 - \alpha/2n)$. Uses indicator random variables. Total time = $\Theta(1+\alpha)$.

Corollary 11.3 (Universal hashing performance)

- **Statement:** Using universal hashing with chaining, any sequence of s INSERT, SEARCH, DELETE operations with $n = O(m)$ INSERTs takes $\Theta(s)$ expected time.
- **Proof:** INSERT/DELETE constant time. $\alpha = O(1)$, SEARCH $O(1)$ expected by Theorem 11.2 proof (depends only on collision prob $\leq 1/m$).

Theorem 11.4 (H_{ab} is universal)

- **Statement:** The family $H_{ab} = \{h_{ab} : h_{ab}(k) = ((ak+b) \bmod p) \bmod m\}$ is universal.
- **Proof:** Distinct $k_1, k_2 \rightarrow r_1 \neq r_2 \bmod p$. $(a,b) \leftrightarrow (r_1, r_2)$ 1-to-1 correspondence. For given r_1 , at most $(p-1)/m$ values r_2 have $r_1 \equiv r_2 \pmod{m}$. So collision prob $\leq 1/m$.

Theorem 11.5 (Multiply-shift $2^w/m$ -universal)

- **Statement:** $H = \{h_a : h_a(k) = ((ka) \bmod 2^w) \ll (w-\ell), a \text{ odd}\}$ is $2^w/m$ -universal.
- **Implication:** $\Pr[\text{collision}] \leq 2/m$.

Theorem 11.6 (Unsuccessful search, open addressing)

- **Statement:** Given open-address hash table with $\alpha = n/m < 1$, expected number of probes in unsuccessful search $\leq 1/(1-\alpha)$, assuming independent uniform permutation hashing and no deletions.
- **Proof:** $\Pr[\geq i \text{ probes}] \leq (n/m)((n-1)/(m-1)) \cdots ((n-i+2)/(m-i+2)) \leq \alpha^{i-1}$. $E[X] = \sum_{i \geq 1} \Pr[X \geq i] \leq \sum_{i \geq 1} \alpha^{i-1} = 1/(1-\alpha)$.
- **Example:** $\alpha=0.5 \rightarrow \leq 2$ probes; $\alpha=0.9 \rightarrow \leq 10$ probes.

Corollary 11.7 (Insertion, open addressing)

- **Statement:** Insertion into open-address hash table with $\alpha < 1$ requires at most $1/(1-\alpha)$ probes on average.

Theorem 11.8 (Successful search, open addressing)

- **Statement:** Given open-address hash table with $\alpha < 1$, expected number of probes in successful search $\leq (1/\alpha) \ln(1/(1-\alpha))$, assuming independent uniform permutation hashing and no deletions.
- **Proof:** Key inserted when load factor was $i/m \rightarrow$ expected probes $\leq m/(m-i)$. Average over all n keys: $(1/n) \sum_{i=0}^{n-1} m/(m-i)$

$$\sum_{i=0}^{n-1} m/(m-i) = (1/\alpha) \sum_{i=0}^{n-1} 1/(m-i) \leq (1/\alpha) \ln(1/(1-\alpha)).$$

- **Example:** $\alpha=0.5 \rightarrow <1.387$ probes; $\alpha=0.9 \rightarrow <2.559$ probes.

Theorem 11.9 (Linear probing with 5-independent hashing)

- **Statement:** If $h_\square$ is 5-independent and $\alpha \leq 2/3$, then expected constant time to search, insert, or delete in a linear-probing hash table.
- **Performance:** $O(1/\epsilon^2)$ for $\alpha = 1-\epsilon$.

Edge Cases & Pitfalls

- **Worst-case chaining:** All n keys hash to same slot $\rightarrow \Theta(n)$ search (worse than direct addressing only when table is small)
- **Open addressing deletion:** Cannot simply store NIL; must use DELETED marker or special linear-probing deletion; search times no longer depend on α when DELETED used
- **No deletions in analysis:** Open addressing analysis assumes no deletions occur
- **Division method constraints:** m should be prime not close to power of 2
- **Double hashing:** $h_\square(k)$ must be relatively prime to m for full coverage; if $\gcd(m, h_\square(k)) = d > 1$, only $1/d$ of table is probed
- **Collision inevitable:** Since $|U| > m$, there must be at least two keys that collide
- **Hash function must be deterministic:** Given input k must always produce same output $h(k)$
- **Random oracle is unachievable ideal:** Can only approximate in practice
- **Cryptographic hash functions:** Complex but can leverage CPU crypto instructions (AES-NI)
- **5-independence needed for linear probing:** Simple hash functions may not give expected constant time

Case Studies & Examples

Direct-address table example (Figure 11.1)

Universe $U = \{0, 1, \dots, 9\}$, actual keys $K = \{2, 3, 5, 8\}$. $T[2]$, $T[3]$, $T[5]$, $T[8]$ point to elements; other slots contain NIL.

Division method example

$$h(k) = k \bmod 12, \quad k = 100 \rightarrow h(k) = 4.$$

Multiplication method example

$A = (\sqrt{5} - 1)/2 \approx 0.6180339887\dots$, $m = 1000$. For keys 61, 62, 63, 64, 65: compute $\lfloor 1000((kA) \bmod 1) \rfloor$ (Exercise 11.3-4).

Multiply-shift example

$k = 123456$, $\ell = 14$, $m = 2^{14} = 16384$, $w = 32$, $a = 2654435769$. $ka = 327706022297664 = (76300 \cdot 2^{32}) + 17612864$. $r_{\ell} = 76300$, $r_{\ell} = 17612864$. 14 most significant bits of $r_{\ell} \rightarrow h_{\ell}(k) = 67$.

Number-theoretic universal hash example

$p = 17$, $m = 6$, $h_{\ell, \ell}(8) = ((3 \cdot 8 + 4) \bmod 17) \bmod 6 = (28 \bmod 17) \bmod 6 = 11 \bmod 6 = 5$.

Double hashing example (Figure 11.5)

Table size $m = 13$, $h_{\ell}(k) = k \bmod 13$, $h_{\ell}(k) = 1 + (k \bmod 11)$. Key 14: $14 \bmod 13 = 1$, $14 \bmod 11 = 3$, so $h_{\ell}(14) = 4$. Probe sequence: slot 1, slot 5, slot 9 (empty after 3 probes). Key 14 inserted at slot 9.

Double hashing example 2

$k = 123456$, $m = 701$, $m' = 700 \rightarrow h_{\ell}(k) = 80$, $h_{\ell}(k) = 257$. Probes: 80, $(80 + 257) \bmod 701 = 337$, $(80 + 2 \cdot 257) \bmod 701 = 594$, etc.

Linear probing insertion example (Figure 11.6)

$h_{\ell}(k) = k \bmod 10$. Keys inserted: 74, 43, 93, 18, 82, 38, 92. After deletion of 43 from slot 3: 93 moves to slot 3, then 92 moves to slot 5 (formerly slot for 93). No more moves needed.

Chaining insertion example (Exercise 11.2-2)

Hash table with 9 slots, $h(k) = k \bmod 9$. Insert: 5, 28, 19, 15, 20, 33, 12, 17, 10. $5 \rightarrow 5$, $28 \rightarrow 1$, $19 \rightarrow 1$ (collision with 28, append to chain), $15 \rightarrow 6$, $20 \rightarrow 2$, $33 \rightarrow 6$ (collision with 15), $12 \rightarrow 3$, $17 \rightarrow 8$, $10 \rightarrow 1$ (collision with 28, 19)

Diagrams & Visuals

Figure 11.1: Direct-address table. Universe $\{0, \dots, 9\}$, keys $\{2, 3, 5, 8\}$. Slots for these keys point to elements; other slots contain NIL.

Figure 11.2: Hash function h mapping keys to slots. Keys k_{ℓ} and k_{ℓ} collide (map to same slot).

Figure 11.3: Collision resolution by chaining. Each nonempty slot $T[j]$ points to linked list of keys whose hash value is j . (Doubly linked shown.)

Figure 11.4: Multiply-shift method. w -bit key $k \times w$ -bit $a \rightarrow 2w$ -bit result $r_{\ell} 2^w + r_{\ell}$. ℓ highest-order bits of r_{ℓ} form hash value $h_{\ell}(k)$.

Figure 11.5: Double hashing insertion. Table size 13, $h_1(k) = k \bmod 13$, $h_2(k) = 1 + (k \bmod 11)$. Key 14 probes slots 1, 5, then inserts at 9.

Figure 11.6: Linear probing deletion. Table size 10, $h_1(k) = k \bmod 10$. After inserting 74, 43, 93, 18, 82, 38, 92: deleting 43 from slot 3 causes $93 \rightarrow$ slot 3, $92 \rightarrow$ slot 5.

End-of-Chapter Material

Exercises 11.1: - **11.1-1:** Find maximum element in direct-address table; worst-case performance? - **11.1-2:** Use bit vector for dynamic set; $O(1)$ dictionary ops. - **11.1-3:** Direct-address table with non-distinct keys and satellite data; $O(1)$ dictionary ops. - **11.1-4** $\square$: Implement dictionary on huge array without initializing entire array; use auxiliary array as stack; $O(1)$ SEARCH/INSERT/DELETE, $O(1)$ initialization.

Exercises 11.2: - **11.2-1:** Expected number of collisions with independent uniform hashing (cardinality of colliding pairs). - **11.2-2:** Insert keys 5, 28, 19, 15, 20, 33, 12, 17, 10 into 9-slot hash table with $h(k) = k \bmod 9$, chaining. - **11.2-3:** Effect of keeping chains in sorted order on successful/unsuccessful search, insertion, deletion. - **11.2-4:** Implement free list for storage allocation within hash table; doubly or singly linked? - **11.2-5:** If $|U| > (n-1)m$, then U has subset of size n all hashing to same slot $\rightarrow \Theta(n)$ worst-case searching. - **11.2-6:** Select key uniformly at random in expected $O(L \cdot (1 + 1/\alpha))$ time knowing chain lengths and longest L .

Exercises 11.3: - **11.3-1:** How to use hash values when searching linked list with long string keys. - **11.3-2:** Division method on radix-128 string using constant extra storage. - **11.3-3:** $h(k) = k \bmod (2^p - 1)$, radix- 2^p strings: permuting characters gives same hash $\rightarrow$ undesirable property. - **11.3-4:** Compute hash locations for keys 61-65 with $m=1000$, $A = (\sqrt{5}-1)/2$. - **11.3-5** $\square$: Show $\square \geq 1/|Q| - 1/|U|$ for any $\square$ -universal family H from U to Q . - **11.3-6** $\square$: Define hash family $H = \{h_b : h_b(\langle a_1, \dots, a_{d-1} \rangle) = a_1 + a_1 b + \dots + a_{d-1} b^{d-1} \bmod p\}$. Show H is $\square$ -universal for $\square = (d-1)/p$.

Exercises 11.4: - **11.4-1:** Insert keys 10, 22, 31, 4, 15, 28, 17, 88, 59 into $m=11$ with linear probing $h(k, i) = (k+i) \bmod 11$ and double hashing $h_1(k) = k$, $h_2(k) = 1 + (k \bmod 10)$. - **11.4-2:** Pseudocode for HASH-DELETE with DELETED marker. - **11.4-3:** Upper bounds for unsuccessful and successful search when $\alpha=3/4$ and $\alpha=7/8$. - **11.4-4:** Expected successful search probes when $\alpha=1$ is H_m (m -th harmonic number). - **11.4-5** $\square$: Double hashing with $\gcd(m, h_2(k)) = d \geq 1$ examines $1/d$ of table; if $d=1$ (relatively prime), may examine entire table. - **11.4-6** $\square$: Find α where unsuccessful search expected probes = $2 \times$ successful search expected probes.

Exercises 11.5: - **11.5-1** $\square$: Prove f_a^{+r} is one-to-one for odd a , $r \geq 0$ (proof by contradiction, modulo 2^w). - **11.5-2** $\square$: Argue

that a random oracle is 5-independent. - **11.5-3** : For r rounds of wee hash function, find least r such that flipping any single bit of input may cause any bit of output to flip.

Problems: - **11-1 Longest-probe bound for hashing:** Shows $E[\text{longest probe sequence}] = O(\lg n)$ for open addressing with $n \leq m/2$. Parts (a)-(d): prove $\Pr[X_i > 2\lg n] = O(1/n^2)$, $\Pr[X > 2\lg n] = O(1/n)$, $E[X] = O(\lg n)$. - **11-2 Searching a static set:** (a) $O(\lg n)$ worst-case search using binary search on sorted array. (b) Extra storage $m-n$ needed for open addressing to match $O(\lg n)$ avg performance. - **11-3 Slot-size bound for chaining:** Prove $E[M] = O(\lg n / \lg \lg n)$ where $M = \max$ keys in any slot. Uses Stirling's approximation, Chernoff-like bounds. - **11-4 Hashing and authentication:** (a) 2-independent $\square$ universal. (b) Universal but not 2-independent family. (c) Modified family is 2-independent. (d) Authentication tag: adversary success prob $\leq 1/p$ with 2-independent family.

Ch. 12 – Binary Search Trees

Named Entities (Terms & Definitions)

- **Binary search tree (BST):** Binary tree where each node has key, left, right, p. Satisfies binary-search-tree property.
- **Binary-search-tree property:** For any node x , if y is in left subtree of x then $y.\text{key} \leq x.\text{key}$; if y is in right subtree of x then $y.\text{key} \geq x.\text{key}$.
- **Inorder tree walk:** Recursive algorithm printing keys in sorted order; prints root between left and right subtrees.
- **Preorder tree walk:** Prints root before left and right subtrees.
- **Postorder tree walk:** Prints root after left and right subtrees.
- **Successor of node x :** Node with smallest key greater than $x.\text{key}$ (if distinct); next node visited in inorder tree walk.
- **Predecessor of node x :** Node with largest key smaller than $x.\text{key}$.
- **Trailing pointer:** In TREE-INSERT, variable y that maintains parent of current node x .
- **TRANSPLANT:** Subroutine that replaces one subtree as a child of its parent with another subtree.
- **Randomly built binary search tree:** Created by inserting keys in random order (each of $n!$ permutations equally likely).
- **Total path length $P(T)$:** Sum over all nodes x in T of depth $d(x, T)$.
- **Radix tree (trie):** Data structure storing bit strings; go left at depth i if $a_i = 0$, right if $a_i = 1$.
- **Catalan number:** $b_n = (1/(n+1))C(2n, n)$, the number of different binary trees with n nodes.

Processes / Algorithms / Pathways

INORDER-TREE-WALK(x)

- **Goal:** Print all keys in sorted order via recursive inorder traversal
- **Steps:** (1) if $x \neq \text{NIL}$: (2) INORDER-TREE-WALK($x.\text{left}$) (3) print $x.\text{key}$ (4) INORDER-TREE-WALK($x.\text{right}$)
- **Complexity:** $\Theta(n)$ for n -node tree (Theorem 12.1)

TREE-SEARCH(x, k)

- **Goal:** Search for key k in subtree rooted at x
- **Input/Output:** Node x , key $k \rightarrow$ pointer to node with key k or NIL
- **Steps:** (1) if $x == \text{NIL}$ or $k == x.\text{key}$: return x (2) if $k < x.\text{key}$: return TREE-SEARCH($x.\text{left}$, k) (3) else: return TREE-SEARCH($x.\text{right}$, k)
- **Complexity:** $O(h)$
- **Example:** Search for 13 in tree (Figure 12.2(a)): path $15 \rightarrow 6 \rightarrow 7 \rightarrow 13$

ITERATIVE-TREE-SEARCH(x, k)

- **Goal:** Iterative version of TREE-SEARCH
- **Steps:** (1) while $x \neq \text{NIL}$ and $k \neq x.\text{key}$: (2) if $k < x.\text{key}$: $x = x.\text{left}$ (3) else: $x = x.\text{right}$ (4) return x
- **Complexity:** $O(h)$ (often more efficient on most computers)

TREE-MINIMUM(x)

- **Goal:** Find minimum key in subtree rooted at x
- **Steps:** (1) while $x.\text{left} \neq \text{NIL}$: $x = x.\text{left}$ (2) return x
- **Complexity:** $O(h)$
- **Example:** In Figure 12.2(b), follows left pointers from root $\rightarrow$ key 2

TREE-MAXIMUM(x)

- **Goal:** Find maximum key in subtree rooted at x
- **Steps:** (1) while $x.\text{right} \neq \text{NIL}$: $x = x.\text{right}$ (2) return x
- **Complexity:** $O(h)$
- **Example:** In Figure 12.2(b), follows right pointers from root $\rightarrow$ key 20

TREE-SUCCESSOR(x)

- **Goal:** Find successor of node x in inorder walk
- **Input/Output:** Node $x \rightarrow$ node or NIL if x is last

- **Steps:** (1) if $x.\text{right} \neq \text{NIL}$: return $\text{TREE-MINIMUM}(x.\text{right})$ (2) $y = x.p$ (3) while $y \neq \text{NIL}$ and $x == y.\text{right}$: $x = y$; $y = y.p$ (4) return y
- **Complexity:** $O(h)$
- **Example 1** (Figure 12.2(c)): Successor of 15 $\rightarrow \text{TREE-MINIMUM}(\text{right subtree}) \rightarrow 17$
- **Example 2** (Figure 12.2(d)): Successor of 13 (no right child) $\rightarrow$ go up: 13 is right child of 7, 7 is right child of 6, 6 is left child of 15 $\rightarrow$ successor is 15

TREE-PREDECESSOR(x)

- **Goal:** Find predecessor of node x (symmetric to TREE-SUCCESSOR)
- **Complexity:** $O(h)$

TREE-INSERT(T, z)

- **Goal:** Insert node z into BST T ($z.\text{key}$ filled in, $z.\text{left} = z.\text{right} = \text{NIL}$)
- **Steps:** (1) $x = T.\text{root}$, $y = \text{NIL}$ (2) while $x \neq \text{NIL}$: $y = x$; if $z.\text{key} < x.\text{key}$: $x = x.\text{left}$ else: $x = x.\text{right}$ (3) $z.p = y$ (4) if $y == \text{NIL}$: $T.\text{root} = z$ (5) elseif $z.\text{key} < y.\text{key}$: $y.\text{left} = z$ else: $y.\text{right} = z$
- **Complexity:** $O(h)$
- **Example** (Figure 12.3): Inserting key 13 into tree. Path from root down: compares 15, goes left to 6, goes right to 7, goes right to NIL $\rightarrow$ inserts as right child of 7.

TRANSPLANT(T, u, v)

- **Goal:** Replace subtree rooted at u with subtree rooted at v
- **Steps:** (1) if $u.p == \text{NIL}$: $T.\text{root} = v$ (2) elseif $u == u.p.\text{left}$: $u.p.\text{left} = v$ (3) else: $u.p.\text{right} = v$ (4) if $v \neq \text{NIL}$: $v.p = u.p$
- **Note:** Does NOT update $v.\text{left}$ and $v.\text{right}$ – caller's responsibility

TREE-DELETE(T, z)

- **Goal:** Delete node z from BST T
- **Steps:**
 - **Case 1** (z has no left child): $\text{TRANSPLANT}(T, z, z.\text{right})$ – replace z by its right child (handles both no children and only right child)
 - **Case 2** (z has left child but no right child): $\text{TRANSPLANT}(T, z, z.\text{left})$ – replace z by its left child
 - **Case 3** (z has two children): (a) $y = \text{TREE-MINIMUM}(z.\text{right})$ – successor, no left child (b) if $y \neq z.\text{right}$: $\text{TRANSPLANT}(T, y, y.\text{right})$; $y.\text{right} = z.\text{right}$; $y.\text{right}.p = y$ (c) $\text{TRANSPLANT}(T, z, y)$; $y.\text{left} = z.\text{left}$; $y.\text{left}.p = y$

- **Complexity:** $O(h)$
- **Four subcases** (Figure 12.4): (a) no left child, (b) left child but no right child, (c) two children and successor is right child, (d) two children and successor is not right child

Data Structures

Binary Search Tree

- **Properties:** Each node has key, left, right, p. Binary-search-tree property holds. Height h .
- **Operations:** SEARCH $O(h)$, MINIMUM $O(h)$, MAXIMUM $O(h)$, SUCCESSOR $O(h)$, PREDECESSOR $O(h)$, INSERT $O(h)$, DELETE $O(h)$
- **When to use:** Dynamic set with ordered operations; sorted order traversal
- **Worst case:** Linear chain $\rightarrow \Theta(n)$ time; balanced $\rightarrow \Theta(\lg n)$

Radix Tree (Trie)

- **Properties:** Stores bit strings; left at depth i if $a_i = 0$, right if $a_i = 1$; keys in nodes determined by path from root
- **Operation:** Sort S of distinct bit strings lexicographically in $\Theta(\text{total length})$ time

Classifications & Hierarchies

Tree walks: - Inorder: left $\rightarrow$ root $\rightarrow$ right (sorted order) -
Preorder: root $\rightarrow$ left $\rightarrow$ right - Postorder: left $\rightarrow$ right $\rightarrow$ root

Deletion cases (TREE-DELETE): - z has no left child $\rightarrow$ replace with right child - z has left child but no right child $\rightarrow$ replace with left child - z has two children $\rightarrow$ replace with successor (z 's right subtree's minimum) - Successor is z 's right child - Successor is not z 's right child (further down the right subtree)

Comparisons & Trade-offs

Operation	BST (height h)	Sorted Array
SEARCH	$O(h)$	$O(\lg n)$ binary search
INSERT	$O(h)$	$\Theta(n)$ (shift elements)
DELETE	$O(h)$	$\Theta(n)$ (shift elements)
MINIMUM/MAXIMUM	$O(h)$	$O(1)$
SUCCESSOR/PREDECESSOR	$O(h)$	$O(1)$ (if index known)
Build method	Time	
Insert n items in arbitrary order	$O(nh)$ worst	
Insert n items in random order	$O(n \lg n)$ expected	
Insert n items in sorted order	$\Theta(n^2)$ (degenerate to chain)	

Formulas & Equations

BST sorting

- Build BST by inserting n numbers, then inorder walk: $O(nh)$ worst-case, $O(n \lg n)$ best-case (balanced tree)

Average depth in randomly built BST

- Expected total path length $P(n) = O(n \lg n)$
- Recurrence: $P(n) = (1/n) \sum_{k=0}^{n-1} (P(k) + P(n-k-1) + n - 1)$

Number of different binary trees (Catalan number)

$$b_n = (1/(n+1)) C(2n, n) \approx 4^n / (n^{3/2} \sqrt{\pi})$$

Recurrence for number of binary trees

$$b_0 = 1, b_n = \sum_{i=0}^{n-1} b_i b_{n-1-i} \text{ for } n \geq 1$$

Generating function

$$B(x) = \sum_{n \geq 0} b_n x^n = x B(x)^2 + 1 = (1 - \sqrt{1-4x}) / (2x)$$

Rules, Laws & Theorems

Binary-Search-Tree Property

- **Statement:** For any node x , all keys in left subtree $\leq x.\text{key} \leq$ all keys in right subtree.
- **Consequence:** Inorder walk prints sorted order.

Theorem 12.1 (Inorder walk time)

- **Statement:** INORDER-TREE-WALK(x) on n -node subtree takes $\Theta(n)$ time.

Theorem 12.2 (Query operations)

- **Statement:** SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR each run in $O(h)$ time on BST of height h .

Theorem 12.3 (Insert and Delete)

- **Statement:** INSERT and DELETE run in $O(h)$ time on BST of height h .

Successor property

- **Statement:** If node x has two children, its successor has no left child and its predecessor has no right child.

Successor when no right subtree

- **Statement:** If right subtree of x is empty and x has successor y , then y is the lowest ancestor of x whose left child is also an ancestor of x .

Edge Cases & Pitfalls

- **Equal keys:** BST property uses $\leq$ and $\geq$; equal keys go to right subtree in TREE-INSERT. With identical keys, tree can degenerate.
- **Successor/predecessor edge cases:** When node is last in inorder walk $\rightarrow$ NIL.
- **TRANSPLANT updates:** Does not update $v.left/v.right$ – caller must do this in TREE-DELETE case 3.
- **TREE-DELETE with two children:** Must first splice out successor, then replace z with y (order matters).
- **Deletion not commutative:** Deleting x then y may not give same tree as deleting y then x . (Exercise 12.3-5)
- **Leaf successor relationship:** For leaf x and its parent y : $y.key$ is either the smallest key larger than $x.key$ or the largest key smaller than $x.key$. (Exercise 12.2-9)
- **Stale pointers:** Alternative deletion approaches (copying key from successor) can cause stale external pointers.

Case Studies & Examples

BST example (Figure 12.1)

1. BST on 6 nodes with height 2: root=6, left child=5 (with left=2, right=5), right child=7 (with right=8). Inorder: 2,5,5,6,7,8. (b) Same keys, height 4 (less efficient).

Search example (Figure 12.2(a))

Searching for 13: start at 15 ($13 < 15 \rightarrow$ left), 6 ($13 > 6 \rightarrow$ right), 7 ($13 > 7 \rightarrow$ right), find 13.

Insertion example (Figure 12.3)

Inserting node with key 13: traverses $15 \rightarrow 6 \rightarrow 7 \rightarrow$ inserts as right child of 7.

Deletion examples (Figure 12.4)

1. z with no left child: replace z by right child r. (b) z with left child but no right child: replace z by left child l. (c) z with two children, successor y is right child: replace z by y, give y left child l. (d) z with two children, successor y not right child: first replace y by its right child x, then replace z by y.

BST construction from sorted input

Inserting keys in sorted order (e.g., 1,2,3,4,5) creates a chain of height $n-1 \rightarrow \Theta(n)$ per operation $\rightarrow \Theta(n^2)$ total for sorting.

Radix tree example (Figure 12.5)

Bit strings: 1011, 10, 011, 100, 0. Sort output: 0, 011, 10, 100, 1011.

Diagrams & Visuals

Figure 12.1: Two BSTs with same keys. (a) Height 2 (balanced). (b) Height 4 (unbalanced). Both show left/right/p attributes.

Figure 12.2: Queries. (a) Search for 13: path 15→6→7→13 (blue). (b) Minimum=2 (follow left), maximum=20 (follow right). (c) Successor of 15=17 (minimum of right subtree). (d) Successor of 13=15 (lowest ancestor where left child is also ancestor).

Figure 12.3: Inserting key 13. Blue path: 15→6→7→NIL. Orange: new node and link.

Figure 12.4: Four deletion cases. (a) No left child. (b) Left child only. (c) Two children, successor is right child. (d) Two children, successor not right child.

Figure 12.5: Radix tree for bit strings 1011, 10, 011, 100, 0.

End-of-Chapter Material

Exercises 12.1: - **12.1-1:** Draw BSTs of heights 2,3,4,5,6 for keys {1,4,5,10,16,17,21}. - **12.1-2:** Difference between BST property and min-heap property? Can min-heap print sorted order in $O(n)$? - **12.1-3:** Nonrecursive inorder tree walk (using stack or pointer comparison). - **12.1-4:** Recursive preorder and postorder tree walks, $\Theta(n)$ time. - **12.1-5:** Any comparison-based BST construction from n elements takes $\Omega(n \lg n)$ worst-case.

Exercises 12.2: - **12.2-1:** Which sequences cannot examine when searching for 363 in BST containing 1-1000? (a) valid, (b) valid, (c) invalid (912 > 911 after going right then left?), (d) valid, (e) invalid (621 > 347 after going right?). - **12.2-2:** Recursive

versions of TREE-MINIMUM and TREE-MAXIMUM. - **12.2-3**: Write TREE-PREDECESSOR procedure. - **12.2-4**: Counterexample to Professor Kilmer's claim about search-path partitions. - **12.2-5**: Node with two children: successor has no left child, predecessor has no right child. - **12.2-6**: If right subtree empty and x has successor y , then y is lowest ancestor whose left child is also ancestor of x . - **12.2-7**: Inorder walk by MINIMUM + $n-1$ SUCCESSOR calls runs in $\Theta(n)$. - **12.2-8**: k successive SUCCESSOR calls take $O(k+h)$ no matter where you start. - **12.2-9**: For leaf x and parent y : $y.key$ is either smallest key $> x.key$ or largest key $< x.key$.

Exercises 12.3: - **12.3-1**: Recursive version of TREE-INSERT. - **12.3-2**: Nodes examined in search = 1 + nodes examined when key was inserted. - **12.3-3**: BST sort worst-case $O(n^2)$ (sorted input), best-case $O(n \lg n)$ (balanced). - **12.3-4**: When can TRANSPLANT parameter v be NIL? - **12.3-5**: Is deletion commutative? Provide counterexample. - **12.3-6**: BST with $x.succ$ instead of $x.p$: pseudocode for SEARCH, INSERT, DELETE in $O(h)$. - **12.3-7**: Deletion using predecessor instead of successor; fair random strategy.

Problems: - **12-1 BST with equal keys**: (a) Insert n identical keys $\rightarrow \Theta(n^2)$ performance (degenerate). Strategies: (b) Boolean flag alternating left/right. (c) List of equal keys at node. (d) Random left/right (expected $O(n \lg n)$, worst $O(n^2)$). - **12-2 Radix trees**: Sort distinct bit strings lexicographically in $\Theta(\text{total length})$ using radix tree. - **12-3 Average node depth in randomly built BST**: Prove $E[\text{depth}] = O(\lg n)$. Uses total path length $P(T) = P(TL) + P(TR) + n - 1$. Recurrence $P(n) = (1/n) \sum (P(k) + P(n-k-1) + n-1)$. Similar to randomized quicksort analysis. - **12-4 Number of different binary trees**: (a) $b_0 = 1$, $b_n = \sum b_i b_{n-1-i}$. (b) $B(x) = xB(x)^2 + 1$. (c) $b_n = (1/(n+1))C(2n, n)$ (Catalan number). (d) $b_n = \Theta(4^{n/n} \{3/2\})$.

Ch. 13 – Red-Black Trees

Named Entities (Terms & Definitions)

- **Red-black tree**: Binary search tree with one extra bit per node (color: RED or BLACK). Ensures no root-to-leaf path is more than twice as long as any other $\rightarrow$ approximately balanced.
- **Red-black properties**:
 1. Every node is either red or black.
 2. The root is black.
 3. Every leaf (NIL) is black.
 4. If a node is red, then both its children are black.
 5. For each node, all simple paths from the node to descendant leaves contain the same number of black nodes.
- **Black-height $bh(x)$** : Number of black nodes on any simple path from (but not including) node x down to a leaf. Black-height of tree = black-height of root.

- **Sentinel T.nil:** Single sentinel representing all NILs in red-black tree; color is BLACK.
- **Rotation:** Local operation in search tree preserving BST property. Two kinds: left rotation and right rotation. $O(1)$ time.
- **Uncle:** Sibling of a node's parent.
- **Relaxed red-black tree:** BST satisfying red-black properties 1, 3, 4, and 5, but root may be either red or black (property 2 may be violated).
- **Doubly black / red-and-black:** Conceptual node states when a black node's blackness is transferred to a child during deletion fixup. "Doubly black" contributes 2 to black count; "red-and-black" contributes 1.
- **AA-trees:** Variant of red-black trees where left children can never be red.
- **Left-leaning red-black tree:** Each node with three children split into two nodes; only left children can be red; simpler code but more rotations.
- **Treap:** Hybrid of binary search tree and heap using random priorities.
- **Splay tree:** Self-adjusting BST with splay operations (rotations) on every access; $O(\lg n)$ amortized cost.
- **AVL tree:** Height-balanced BST where heights of left and right subtrees differ by at most 1 per node.
- **Join operation:** Combines two dynamic sets S_1 , S_2 and element x (with $x_1.\text{key} \leq x.\text{key} \leq x_2.\text{key}$ for all $x_1 \in S_1$, $x_2 \in S_2$) into $S = S_1 \cup \{x\} \cup S_2$.
- **Persistent dynamic set:** A set where past versions are maintained as it is updated.

Processes / Algorithms / Pathways

LEFT-ROTATE(T, x)

- **Goal:** Left rotation on node x ($x.\text{right}$ must be non-sentinel)
- **Steps:** (1) $y = x.\text{right}$ (2) $x.\text{right} = y.\text{left}$ (3) if $y.\text{left} \neq T.\text{nil}$: $y.\text{left}.p = x$ (4) $y.p = x.p$ (5) if $x.p == T.\text{nil}$: $T.\text{root} = y$ (6) elseif $x == x.p.\text{left}$: $x.p.\text{left} = y$ else: $x.p.\text{right} = y$ (7) $y.\text{left} = x$ (8) $x.p = y$
- **Complexity:** $O(1)$
- **Preserves:** BST property (keys in α precede $x.\text{key}$, precede keys in β , precede $y.\text{key}$, precede keys in γ) – see Figure 13.2

RIGHT-ROTATE(T, y)

- **Goal:** Right rotation on node y (symmetric to left rotation)
- **Complexity:** $O(1)$

RB-TRANSPLANT(T, u, v)

- **Goal:** Replace subtree rooted at u with subtree rooted at v in red-black tree
- **Steps:** (1) if $u.p == T.nil$: $T.root = v$ (2) elseif $u == u.p.left$: $u.p.left = v$ (3) else: $u.p.right = v$ (4) $v.p = u.p$ (unconditional, even if v is T.nil)
- **Difference from TRANSPLANT:** Uses T.nil instead of NIL; v.p assignment unconditional

RB-INSERT(T, z)

- **Goal:** Insert node z into red-black tree T, maintaining red-black properties
- **Steps:** Lines 1-16 similar to TREE-INSERT but: (1) Use T.nil instead of NIL (2) Set $z.left = z.right = T.nil$ (3) Color z RED (4) Call RB-INSERT-FIXUP
- **Complexity:** $O(\lg n)$ total
- **Why red?:** If z were black, property 5 (same black count on all paths) would be violated immediately.

RB-INSERT-FIXUP(T, z)

- **Goal:** Restore red-black properties after insertion
- **Invariant:** (a) z is red. (b) If z.p is root, z.p is black. (c) At most one violation: property 2 (z is red root) or property 4 (z and z.p both red).
- **Case 1** (z's uncle y is red): (1) $z.p.color = BLACK$ (2) $y.color = BLACK$ (3) $z.p.p.color = RED$ (4) $z = z.p.p$ (moves up two levels)
- **Case 2** (uncle black, z is right child): (1) $z = z.p$ (2) LEFT-ROTATE(T, z) → transforms to case 3
- **Case 3** (uncle black, z is left child): (1) $z.p.color = BLACK$ (2) $z.p.p.color = RED$ (3) RIGHT-ROTATE(T, z.p.p) → loop terminates
- **Termination:** Loop ends when z.p is black (or z is root). Line 30: $T.root.color = BLACK$ ensures property 2.
- **Complexity:** $O(\lg n)$ time; at most 2 rotations
- **Example** (Figure 13.4): Insert node z (both z and z.p red). Case 1 (uncle red): recolor, z moves up. Case 2 (uncle black, z right child): left rotation → case 3. Case 3: recolor + right rotation. Tree becomes legal.

RB-DELETE(T, z)

- **Goal:** Delete node z from red-black tree T, maintaining red-black properties
- **Structure:** Like TREE-DELETE but tracks y (node removed or moved) and x (node taking y's place). Stores y-original-color. If y was BLACK, calls RB-DELETE-FIXUP.

- **Why only fixup when y was black?:** If y was red: (1) no black-height changes, (2) no red-red adjacency, (3) root remains black.
- **Complexity:** $O(\lg n)$ time; at most 3 rotations

RB-DELETE-FIXUP(T, x)

- **Goal:** Restore red-black properties after deletion (handles "doubly black" node x)
- **Goal of while loop:** Move extra black up the tree until: (1) x is red-and-black $\rightarrow$ color singly black; (2) x is root $\rightarrow$ extra black vanishes; (3) rotations/recolorings resolve it.
- **Case 1** (x's sibling w is red): (1) w.color = BLACK (2) x.p.color = RED (3) LEFT-ROTATE(T, x.p) (4) w = x.p.right $\rightarrow$ transforms into case 2, 3, or 4
- **Case 2** (w is black, both w's children black): (1) w.color = RED (2) x = x.p (moves extra black up)
- **Case 3** (w black, w.left red, w.right black): (1) w.left.color = BLACK (2) w.color = RED (3) RIGHT-ROTATE(T, w) (4) w = x.p.right $\rightarrow$ transforms to case 4
- **Case 4** (w black, w.right red): (1) w.color = x.p.color (2) x.p.color = BLACK (3) w.right.color = BLACK (4) LEFT-ROTATE(T, x.p) (5) x = T.root $\rightarrow$ terminates
- **Termination:** Line 44: x.color = BLACK (handles red-and-black case and ensures root is black)
- **Complexity:** $O(\lg n)$ time; at most 3 rotations total; only case 2 can repeat (moves x up)

Data Structures

Red-Black Tree

- **Properties:** BST + red-black properties; height $\leq 2 \lg(n+1)$ (Lemma 13.1); each node has color, key, left, right, p
- **Operations:** SEARCH $O(\lg n)$, MINIMUM $O(\lg n)$, MAXIMUM $O(\lg n)$, SUCCESSOR $O(\lg n)$, PREDECESSOR $O(\lg n)$, INSERT $O(\lg n)$, DELETE $O(\lg n)$
- **When to use:** Need guaranteed $O(\lg n)$ worst-case for dynamic-set operations

AVL Tree (Problem 13-3)

- **Properties:** Height-balanced: $|\text{left.h} - \text{right.h}| \leq 1$ for each node. Height $O(\lg n)$. At least F_h nodes at height h (Fibonacci numbers).
- **Operations:** INSERT $O(\lg n)$ with $O(\lg n)$ rotations
- **Balance procedure:** Use rotations to fix height differences of 2.

Persistent Binary Search Tree (Problem 13-1)

- **Properties:** Maintains past versions. Copy-on-write: copy nodes on path to root when inserting/deleting.
- **Operations:** INSERT $O(h)$ time and space (copies $O(h)$ nodes)
- **With parent pointers:** $\Omega(n)$ time/space per insertion
- **With red-black trees:** $O(\lg n)$ worst-case per insertion/deletion

Classifications & Hierarchies

Balanced search tree types: - Red-black trees ($O(\lg n)$ guaranteed, ≤ 2 rotations per insert, ≤ 3 per delete) - AVL trees ($O(\lg n)$, stricter balance, more rotations) - 2-3 trees / B-trees (variable node degree) - Left-leaning red-black trees (simpler code, more rotations) - AA-trees (no red left children) - Treaps (random priorities, expected $O(\lg n)$) - Splay trees (self-adjusting, amortized $O(\lg n)$) - Skip lists (probabilistic, not a tree)

RB-INSERT-FIXUP cases: - Case 1: Uncle red $\rightarrow$ recolor, move z up - Case 2: Uncle black, z is right child $\rightarrow$ left rotation $\rightarrow$ case 3 - Case 3: Uncle black, z is left child $\rightarrow$ recolor + right rotation $\rightarrow$ done

RB-DELETE-FIXUP cases (x is left child): - Case 1: Sibling w red $\rightarrow$ recolor + left rotation $\rightarrow$ falls to cases 2/3/4 - Case 2: w black, both children black $\rightarrow$ recolor w red, move x up - Case 3: w black, w.left red, w.right black $\rightarrow$ recolor + right rotation $\rightarrow$ case 4 - Case 4: w black, w.right red $\rightarrow$ recolor + left rotation $\rightarrow$ done

Formulas & Equations

Height bound

$$h \leq 2 \lg(n + 1)$$

Black-height bound

$bh(\text{root}) \geq h/2$ (by property 4, at least half nodes on any root-to-leaf path are black)

Minimum nodes in subtree with black-height $bh(x)$

At least $2^{bh(x)} - 1$ internal nodes

Height inequality from Lemma 13.1 proof

$$n \geq 2^{h/2} - 1 \rightarrow \lg(n+1) \geq h/2 \rightarrow h \leq 2 \lg(n+1)$$

Black-height recurrence

For node x : each child has black-height $bh(x)-1$ (if black) or $bh(x)$ (if red)

Rules, Laws & Theorems

Lemma 13.1 (Red-black tree height)

- **Statement:** A red-black tree with n internal nodes has height at most $2 \lg(n+1)$.
- **Proof:** (1) Prove subtree rooted at x has $\geq 2^{\{bh(x)\}-1}$ internal nodes (induction). (2) By property 4, black-height $\geq h/2$. (3) $n \geq 2^{\{h/2\}-1} \rightarrow h \leq 2 \lg(n+1)$.

Consequences of Lemma 13.1

- SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR: $O(\lg n)$
- INSERT, DELETE (with fixup): $O(\lg n)$

Red-black property 5 implication

All simple paths from any node to descendant leaves have same number of black nodes. Ensures black-height is well-defined.

Property 4 implication

No two red nodes appear consecutively on any path. At least half the nodes on any root-to-leaf path (excluding root) are black.

Rotation properties

- Preserves BST order (inorder walk unchanged)
- $O(1)$ time
- Exactly $n-1$ possible rotations in any n -node BST (Exercise 13.2-2)
- Any n -node BST can transform to any other with $O(n)$ rotations (Exercise 13.2-4)

RB-INSERT invariant (three-part)

1. z is red. (b) If $z.p$ is root, $z.p$ is black. (c) At most one violation of properties 2 or 4.

RB-INSERT termination

Root colored black in line 30; loop terminates because $z.p$ becomes black (either naturally or through cases 2/3 fixing it).

Edge Cases & Pitfalls

- **Sentinel T.nil:** All NIL pointers replaced by single sentinel. Color is BLACK. Root's parent is T.nil.
- **Never set T.nil.color to RED:** RB-INSERT-FIXUP never sets T.nil.color to RED – the sentinel's color remains BLACK always.
- **Deletion when y was black:** Three problems: (1) root violation if red child becomes root (property 2), (2) red-red if x and x.p both red (property 4), (3) property 5 violated (paths through y have one less black). Fixup uses doubly-black concept.
- **Case 2 in DELETE-FIXUP:** Only case that repeats (moves x up the tree). $O(\lg n)$ repetitions max.
- **RB-DELETE-FIXUP relies on x.p:** Even when x is T.nil, x.p must be set correctly. RB-DELETE ensures this with line 16.
- **Red node cannot have exactly one non-NIL child:** Exercise 13.1-8 – would violate property 4 and 5.
- **Black-height of sentinel = 0:** The NIL sentinel has black-height 0.
- **Max internal nodes with black-height k:** $2^{\{2k\}} - 1$ (full tree with alternating red/black). Min: $2^k - 1$ (all black nodes). (Exercise 13.1-6)
- **Max ratio of red to black nodes:** 2:1 (alternating levels of red/black). Min: 0 (all black). (Exercise 13.1-7)

Case Studies & Examples

RB-INSERT-FIXUP example (Figure 13.4)

1. After inserting z: both z and parent are red → violation property 4. Uncle y is red → case 1: recolor parent, uncle, grandparent; z moves up two levels. (b) Again z and parent red. Uncle y is black → case 2 (z is right child): left rotation. (c) Now z is left child → case 3: recolor + right rotation. (d) Legal red-black tree.

Successive insertion example (Exercise 13.3-2)

Insert 41, 38, 31, 12, 19, 8 into empty red-black tree. Result shown in Exercise 13.4-4 for deletion sequence.

Red-black tree of height 3 (Exercise 13.1-1)

Complete BST on $\{1, \dots, 15\}$ with height 3. Color nodes to achieve black-heights 2, 3, and 4.

AVL tree example (Problem 13-3)

Prove AVL tree of height h has at least F_h nodes (Fibonacci). Height $O(\lg n)$. Insert procedure uses BALANCE with rotations.

Persistent BST example (Figure 13.8, Problem 13-1)

1. BST with keys 2,3,4,7,8,10. (b) After inserting key 5: new nodes created for key 5, key 7 (new parent), key 8 (new parent), key 4 (new root). Old nodes (2,3,10) shared with previous version.

Red-black tree join example (Problem 13-2)

RB-JOIN(T_1 , x , T_2): assume $T_1.bh \geq T_2.bh$. Find black node y in T_1 with largest key having black-height = $T_2.bh$. Replace subtree T_y by $T_y \cup \{x\} \cup T_2$. Color x red or black to maintain properties. $O(\lg n)$ time.

Diagrams & Visuals

Figure 13.1: Red-black tree. (a) Full tree with NIL leaves, black-heights marked. (b) Same tree with single sentinel $T.nil$. (c) Simplified drawing (leaves omitted) used in rest of chapter.

Figure 13.2: Rotation operations. LEFT-ROTATE(T, x) transforms right config to left config. RIGHT-ROTATE(T, y) is inverse. α, β, γ are arbitrary subtrees. Keys in $\alpha < x.key < \beta < y.key < \gamma$ preserved.

Figure 13.3: LEFT-ROTATE example on BST. Inorder tree walk of input and modified tree produce same key sequence.

Figure 13.4: RB-INSERT-FIXUP operation sequence: (a) Case 1 (uncle red) $\rightarrow$ (b) Case 2 (z right child) $\rightarrow$ (c) Case 3 (z left child) $\rightarrow$ (d) Legal tree.

Figure 13.5: Case 1 of RB-INSERT-FIXUP. Uncle y red. Blackness of grandparent transfers down to parent and uncle. z moves up to grandparent.

Figure 13.6: Cases 2 and 3. Uncle y black. Case 2 (z is right child) $\rightarrow$ left rotation $\rightarrow$ Case 3 (z is left child) $\rightarrow$ recolor + right rotation $\rightarrow$ done.

Figure 13.7: Four cases of RB-DELETE-FIXUP (x is left child). (a) Case 1: sibling w red $\rightarrow$ recolor + left rotation $\rightarrow$ case 2/3/4. (b) Case 2: w black, both children black $\rightarrow$ recolor w red, x moves up. (c) Case 3: w black, $w.left$ red, $w.right$ black $\rightarrow$ recolor + right rotation $\rightarrow$ case 4. (d) Case 4: w black, $w.right$ red $\rightarrow$ recolor + left rotation $\rightarrow$ done.

Figure 13.8: Persistent BST. (a) Original tree {2,3,4,7,8,10}. (b) After inserting 5: new nodes (blue) copied along path to root; old nodes shared.

End-of-Chapter Material

Exercises 13.1: - **13.1-1:** Draw complete BST of height 3 on $\{1, \dots, 15\}$. Color for black-heights 2, 3, 4. - **13.1-2:** Result of TREE-INSERT 36 into Figure 13.1. Red? Black? - **13.1-3:** Relaxed red-black tree with red root: change root to black $\rightarrow$ legal red-black tree? - **13.1-4:** Absorb red children into black parent: possible degrees? Leaf depths? - **13.1-5:** Longest path $\leq 2 \times$ shortest path from any node x to descendant leaf. - **13.1-6:** Largest internal nodes with black-height k ? Smallest? - **13.1-7:** Largest ratio of red:black nodes (2:1)? Smallest ratio (0)? - **13.1-8:** Red node cannot have exactly one non-NIL child.

Exercises 13.2: - **13.2-1:** Write RIGHT-ROTATE pseudocode. - **13.2-2:** Exactly $n-1$ possible rotations in n -node BST. - **13.2-3:** Depth changes of nodes a, b, c after left rotation on x (Figure 13.2). - **13.2-4:** Any n -node BST can transform to any other with $O(n)$ rotations (right-going chain). - **13.2-5** $\square$: Give example where $T \square$ cannot right-convert to $T \square$. If possible, $O(n^2)$ right rotations suffice.

Exercises 13.3: - **13.3-1:** Why color z red, not black? (Black would violate property 5.) - **13.3-2:** Show RB trees after inserting 41, 38, 31, 12, 19, 8. - **13.3-3:** Label black-heights in Figures 13.5 and 13.6; verify property 5 preserved. - **13.3-4:** Show RB-INSERT-FIXUP never sets $T.nil.color$ to RED. - **13.3-5:** If $n > 1$, tree has at least one red node. - **13.3-6:** Efficient RB-INSERT without parent pointers (store path in stack).

Exercises 13.4: - **13.4-1:** If y is red in RB-DELETE, no black-heights change. - **13.4-2:** Root must be black after RB-DELETE-FIXUP. - **13.4-3:** If both x and $x.p$ are red, RB-DELETE-FIXUP restores property 4. - **13.4-4:** Show RB trees after deleting 8, 12, 19, 31, 38, 41 (from Exercise 13.3-2 tree). - **13.4-5:** Which lines of RB-DELETE-FIXUP examine/modify $T.nil$? - **13.4-6:** Count black nodes in Figure 13.7 before/after each transformation. - **13.4-7:** Show $x.p$ must be black at start of case 1 in RB-DELETE-FIXUP. - **13.4-8:** After RB-INSERT then RB-DELETE of same node: same tree? - **13.4-9** $\square$: Implement RB-ENUMERATE(T, r, a, b) in $O(m + \lg n)$ time ($m = \text{keys output}$, $a \leq k \leq b$).

Problems: - **13-1 Persistent dynamic sets:** (a) Identify nodes changed during insert/delete. (b) Write PERSISTENT-TREE-INSERT using COPY-NODE. (c) $O(h)$ time and space. (d) With parent pointers $\rightarrow \Omega(n)$ time/space. (e) Red-black trees $\rightarrow O(\lg n)$ per op. - **13-2 Join operation on red-black trees:** (a) Maintain black-height attribute bh without extra storage in nodes. (b) Find black node y in $T \square$ with black-height = $T \square.bh$. (c-d) Replace subtree, set colors, fix properties. (e) Symmetric case. (f) $O(\lg n)$ time. - **13-3 AVL trees:** (a) Height $O(\lg n)$. (b) BALANCE(x) procedure using rotations to fix height difference ≤ 2 . (c) AVL-INSERT recursive procedure. (d) $O(\lg n)$ time, $O(\lg n)$ rotations.

Ch. 14 – Dynamic Programming

Named Entities (Terms & Definitions)

- **Dynamic programming:** A method for solving problems by combining solutions to subproblems, where subproblems overlap; “programming” refers to a tabular method.
- **Memoization:** A top-down approach that saves (remembers) the result of each subproblem so it can be looked up later rather than recomputed; from “memo” (not “memorization”).
- **Bottom-up method:** Solves subproblems in order of increasing size, smallest first, storing solutions; typically has better constant factors due to lower overhead.
- **Optimal substructure:** Property where an optimal solution to a problem contains within it optimal solutions to subproblems.
- **Overlapping subproblems:** Property where a recursive algorithm solves the same subproblems repeatedly; the space of subproblems must be “small” (polynomial in input size).
- **Subproblem graph:** A directed graph containing one vertex for each distinct subproblem, with edges from subproblem x to subproblem y if solving x requires a solution to y .
- **Rod-cutting problem:** Given a rod of length n and a price table p_i for $i=1, \dots, n$, determine maximum revenue r_n obtainable by cutting the rod and selling pieces.
- **Matrix-chain multiplication problem:** Given a chain of matrices $\langle A_1, A_2, \dots, A_n \rangle$ with dimensions $p_{i-1} \times p_i$, fully parenthesize the product to minimize scalar multiplications.
- **Fully parenthesized:** A product of matrices is fully parenthesized if it is either a single matrix or the product of two fully parenthesized matrix products surrounded by parentheses.
- **Longest common subsequence (LCS) problem:** Given two sequences X and Y , find a maximum-length subsequence common to both.
- **Subsequence:** A sequence with 0 or more elements left out, preserving original order.
- **Prefix (of a sequence):** For sequence $X = \langle x_1, x_2, \dots, x_m \rangle$, the i -th prefix $X_i = \langle x_1, x_2, \dots, x_i \rangle$.
- **Optimal binary search tree:** A BST that minimizes expected search cost given known probabilities p_i for each key k_i and q_i for each dummy key d_i .
- **Dummy key:** Represents values not in the key set; d_0 for values $< k_1$, d_n for values $> k_n$, d_i for values between k_i and k_{i+1} .
- **Catalan numbers:** The number of ways to parenthesize a matrix chain of n matrices grows as $\Omega(4^n / n^{3/2})$; $P(n)$ follows Catalan recurrence.
- **TD/eD classification:** A dynamic-programming algorithm is TD/eD if its table size is $O(n^t)$ and each entry depends on $O(n^e)$ other entries. Matrix-chain is 2D/1D, LCS is 2D/0D.
- **Edit distance:** The cost of the least expensive sequence of transformation operations (copy, replace, delete, insert,

twiddle, kill) that transforms source string x to target string y.

- **Palindrome**: A nonempty string that reads the same forward and backward.
- **Bitonic tour**: A tour that starts at the leftmost point, goes strictly rightward to the rightmost point, then strictly leftward back to the start.
- **Seam carving**: Removing one pixel per row from an image, where removed pixels in adjacent rows must be in the same or adjacent columns, forming a seam.
- **Sabermetrics**: Statistical analysis of baseball records; WAR = "wins above replacement."

Processes / Algorithms / Pathways

Cut-Rod (naive recursive)

- **Goal**: Compute maximum revenue r_n for rod of length n
- **Input/Output**: Input: array $p[1:n]$ of prices, integer n . Output: maximum revenue q
- **Steps**: (1) If $n=0$, return 0. (2) Set $q = -\text{infinity}$. (3) For $i=1$ to n : $q = \max(q, p[i] + \text{Cut-Rod}(p, n-i))$. (4) Return q .
- **Complexity**: Time $O(2^n)$ – exponential. $T(n) = 1 + \sum_{j=0}^{n-1} T(j)$, and $T(n) = 2^n$.
- **Why it's slow**: Solves same subproblems repeatedly; recursion tree has 2^n nodes and 2^{n-1} leaves.

Memoized-Cut-Rod (top-down DP)

- **Goal**: Compute maximum revenue r_n with memoization
- **Input/Output**: Input: $p[1:n]$, n . Output: maximum revenue.
- **Steps**: (1) Initialize array $r[0:n]$ with $-\text{infinity}$. (2) Call Memoized-Cut-Rod-Aux(p, n, r). Aux: (3) If $r[n] \geq 0$, return $r[n]$. (4) If $n=0$, $q=0$ else $q=-\text{infinity}$. (5) For $i=1$ to n : $q = \max(q, p[i] + \text{Memoized-Cut-Rod-Aux}(p, n-i, r))$. (6) $r[n]=q$, return q .
- **Complexity**: Time $\Theta(n^2)$, Space $\Theta(n)$.

Bottom-Up-Cut-Rod (bottom-up DP)

- **Goal**: Compute maximum revenue r_n using bottom-up approach
- **Input/Output**: Input: $p[1:n]$, n . Output: max revenue $r[n]$.
- **Steps**: (1) Create array $r[0:n]$. (2) $r[0]=0$. (3) For $j=1$ to n : (4) $q=-\text{infinity}$. (5) For $i=1$ to j : $q = \max(q, p[i] + r[j-i])$. (6) $r[j]=q$. (7) Return $r[n]$.
- **Complexity**: Time $\Theta(n^2)$, Space $\Theta(n)$.

Extended-Bottom-Up-Cut-Rod

- **Goal**: Compute both max revenue and optimal first-piece sizes

- **Input/Output:** Input: $p[1:n]$, n . Output: arrays $r[0:n]$ and $s[1:n]$ where $s[j]$ is optimal size of first cut for rod length j .
- **Steps:** (1) Create $r[0:n]$ and $s[1:n]$. (2) $r[0]=0$. (3) For $j=1$ to n : (4) $q=-\text{infinity}$. (5) For $i=1$ to j : (6) if $q < p[i] + r[j-i]$: $q = p[i] + r[j-i]$; $s[j]=i$. (7) $r[j]=q$. (8) Return r and s .
- **Complexity:** Time $\Theta(n^2)$, Space $\Theta(n)$.

Print-Cut-Rod-Solution

- **Goal:** Print optimal decomposition for rod of length n
- **Steps:** (1) $(r, s) = \text{Extended-Bottom-Up-Cut-Rod}(p, n)$. (2) While $n > 0$: print $s[n]$; $n = n - s[n]$.

Rectangular-Matrix-Multiply

- **Goal:** Multiply matrices A ($p \times q$) and B ($q \times r$), accumulate into C ($p \times r$)
- **Steps:** (1) For $i=1$ to p : (2) For $j=1$ to r : (3) For $k=1$ to q : $c_{ij} = c_{ij} + a_{ik} * b_{kj}$.
- **Complexity:** $\Theta(pqr)$ scalar multiplications.

Matrix-Chain-Order (bottom-up DP)

- **Goal:** Compute minimum scalar multiplications for matrix chain product
- **Input/Output:** Input: sequence $p = \langle p_0, \dots, p_n \rangle$ of dimensions, n . Output: tables $m[1:n, 1:n]$ and $s[1:n-1, 2:n]$.
- **Steps:** (1) Create tables m and s . (2) For $i=1$ to n : $m[i, i]=0$. (3) For $l=2$ to n (chain length): (4) For $i=1$ to $n-l+1$: (5) $j=i+l-1$. (6) $m[i, j]=\text{infinity}$. (7) For $k=i$ to $j-1$: (8) $q=m[i, k]+m[k+1, j]+p_{i-1}p_kp_j$. (9) If $q < m[i, j]$: $m[i, j]=q$; $s[i, j]=k$. (10) Return m and s .
- **Complexity:** Time $O(n^3)$ ($\Theta(n^3)$), Space $\Theta(n^2)$.

Print-Optimal-Parens

- **Goal:** Print optimal parenthesization of matrix chain $A_i \dots A_j$
- **Steps:** (1) If $i=j$: print " A_i ". (2) Else: print "("; Print-Optimal-Parens($s, i, s[i, j]$); Print-Optimal-Parens($s, s[i, j]+1, j$); print ")".
- **Complexity:** $O(n)$ time.
- **Example** (Fig 14.5): For $n=6$ with dims $30 \times 35, 35 \times 15, 15 \times 5, 5 \times 10, 10 \times 20, 20 \times 25$: $m[1, 6]=15, 125$; optimal parenthesization is $((A_1(A_2A_3))((A_4A_5)A_6))$.

Recursive-Matrix-Chain (naive exponential)

- **Goal:** Naive recursive computation of $m[i, j]$
- **Steps:** (1) If $i=j$ return 0. (2) $m[i, j]=\text{infinity}$. (3) For $k=i$ to $j-1$: $q=\text{Recursive-Matrix-Chain}(p, i, k)+\text{Recursive-Matrix-}$

- Chain($p, k+1, j$)+ $p_{i-1}p_{kp_j}$; if $q < m[i, j]$, $m[i, j] = q$. (4)
 Return $m[i, j]$.
- **Complexity:** $T(n) \geq 2^{n-1}$ – exponential ($\Omega(2^n)$).

Memoized-Matrix-Chain (top-down DP)

- **Goal:** Compute minimum scalar multiplications with memoization
- **Steps:** (1) Create table $m[1:n, 1:n]$ initialized to infinity. (2) Call Lookup-Chain($m, p, 1, n$). Lookup-Chain: (3) If $m[i, j] < \text{infinity}$ return $m[i, j]$. (4) If $i=j$: $m[i, j]=0$. (5) Else for $k=i$ to $j-1$: $q = \text{Lookup-Chain}(m, p, i, k) + \text{Lookup-Chain}(m, p, k+1, j) + p_{i-1}p_{kp_j}$; if $q < m[i, j]$, $m[i, j] = q$. (6) Return $m[i, j]$.
- **Complexity:** Time $O(n^3)$, Space $\Theta(n^2)$.

LCS-Length

- **Goal:** Compute length of LCS of sequences X and Y
- **Input/Output:** Input: $X[1:m]$, $Y[1:n]$. Output: tables $c[0:m, 0:n]$ and $b[1:m, 1:n]$.
- **Steps:** (1) Create tables b and c. (2) For $i=1$ to m : $c[i, 0]=0$. (3) For $j=0$ to n : $c[0, j]=0$. (4) For $i=1$ to m , for $j=1$ to n : (5) If $x_i == y_j$: $c[i, j] = c[i-1, j-1] + 1$; $b[i, j] = \text{"nw-arrow"}$. (6) Else if $c[i-1, j] \geq c[i, j-1]$: $c[i, j] = c[i-1, j]$; $b[i, j] = \text{"up-arrow"}$. (7) Else: $c[i, j] = c[i, j-1]$; $b[i, j] = \text{"left-arrow"}$. (8) Return c and b.
- **Complexity:** Time $\Theta(mn)$, Space $\Theta(mn)$.

Print-LCS

- **Goal:** Print an LCS of X and Y
- **Steps:** (1) If $i==0$ or $j==0$: return. (2) If $b[i, j] == \text{"nw-arrow"}$: Print-LCS($b, X, i-1, j-1$); print x_i . (3) Else if $b[i, j] == \text{"up-arrow"}$: Print-LCS($b, X, i-1, j$). (4) Else: Print-LCS($b, X, i, j-1$).
- **Complexity:** $O(m+n)$ time.

Optimal-BST

- **Goal:** Compute expected search cost of optimal BST and record roots
- **Input/Output:** Input: probabilities $p_1, \dots, p_n$, $q_0, \dots, q_n$, n . Output: tables $e[1:n+1, 0:n]$, $\text{root}[1:n, 1:n]$.
- **Steps:** (1) Create tables e, w, root. (2) For $i=1$ to $n+1$: $e[i, i-1] = q_{i-1}$; $w[i, i-1] = q_{i-1}$. (3) For $l=1$ to n : (4) For $i=1$ to $n-l+1$: (5) $j=i+l-1$. (6) $e[i, j] = \text{infinity}$. (7) $w[i, j] = w[i, j-1] + p_j + q_j$. (8) For $r=i$ to j : $t = e[i, r-1] + e[r+1, j] + w[i, j]$; if $t < e[i, j]$: $e[i, j] = t$; $\text{root}[i, j] = r$. (9) Return e and root.
- **Complexity:** Time $\Theta(n^3)$, Space $\Theta(n^2)$. (Can be improved to $\Theta(n^2)$ using Knuth's root bounds: $\text{root}[i, j-1] \leq \text{root}[i, j] \leq \text{root}[i+1, j]$.)

Design Paradigm: DP

- **Key idea:** Solve each subproblem once, save its answer (table/memo). Trade time for memory.
- **Optimal substructure:** An optimal solution contains optimal solutions to subproblems. Verify via cut-and-paste proof.
- **Overlapping subproblems:** The space of subproblems must be "small" (polynomial). Recursive algorithm revisits same subproblems repeatedly.
- **When to use:** Optimization problems with optimal substructure and overlapping subproblems. Subproblems must be independent (do not share resources).
- **Four steps:** (1) Characterize structure of optimal solution. (2) Recursively define value of optimal solution. (3) Compute value (bottom-up or top-down memoized). (4) Construct optimal solution from computed info.
- **How to discover optimal substructure:** (1) Show solution involves making a choice. (2) Suppose you are given the optimal choice. (3) Determine which subproblems ensue. (4) Show subproblem solutions must be optimal (cut-and-paste).
- **Template/pseudocode** (bottom-up DP):

Create table(s) for subproblem solutions
Initialize base cases
For each subproblem in increasing size:
 For each choice:
 Use optimal solutions to smaller subproblems + cost of choice
 Keep best
Return final solution (optionally reconstruct using stored decisions)
- **Proof technique:** Cut-and-paste: suppose subproblem solution is not optimal, cut it out and paste in optimal subproblem solution, yielding better overall solution – contradiction.
- **Running time:** Typically (number of subproblems) $\times$ (number of choices per subproblem). Rod cutting: $\Theta(n)$ subproblems, $O(n)$ choices $\Rightarrow \Theta(n^2)$. Matrix-chain: $\Theta(n^2)$ subproblems, $O(n)$ choices $\Rightarrow \Theta(n^3)$.
- **Subproblem graph:** Solving each subproblem once $\Rightarrow$ running time is sum of degrees of vertices. Typically linear in $|V| + |E|$.
- **Bottom-up vs top-down memoized:** Bottom-up often has better constants (less overhead). Memoized may save time if not all subproblems need solving.

Comparisons & Trade-offs

Dimension	Top-down with Memoization	Bottom-up
Approach	Recursive with memo table	Iterative, smallest subproblems first
Overhead	Recursion + table maintenance	Table maintenance only
Constant factors	Higher	Lower
Subproblem solving	Only those actually needed	All subproblems solved
Code clarity	More natural from recurrence	May need care with ordering
When preferred	Not all subproblems needed	All subproblems needed

Dimension	Dynamic Programming	Greedy Algorithms
Choice depends on	Solutions to subproblems	Local best at moment
Order	Bottom-up (or top-down memo)	Top-down first, then subproblem
When subproblems solved	Before making choice	After making choice
Overlapping subproblems	Required	Not required

Formulas & Equations

Rod Cutting Recurrence (two-piece decomposition)

$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$ for $n \geq 1$, $r_0 = 0$ -
 r_n = max revenue for rod of length n [dollars] - p_i = price for rod of length i [dollars]

Rod Cutting Recurrence (first-piece decomposition)

$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i})$ - Same variables; simpler view: first piece of length i , remainder $n-i$ optimally cut.

Number of Ways to Cut Rod

2^{n-1} ways (each of $n-1$ cut locations independently chosen)

$T(n)$ for Cut-Rod

$T(n) = 1 + \sum_{j=0}^{n-1} T(j)$ $T(n) = 2^n - 1$ - $T(n)$ = number of recursive calls made by Cut-Rod(p , n)

Matrix-Chain Multiplication Cost

$m[i,j] = \min_{\{i \leq k < j\}} (m[i,k] + m[k+1,j] + p_{i-1} * p_k * p_j)$ for $i < j$ $m[i,i] = 0$ - $m[i,j]$ = minimum scalar multiplications to compute $A_i..j$ - $p_{i-1} \times p_i$ = dimensions of A_i

Number of Parenthesizations (Catalan numbers)

$P(n) = \{ 1 \text{ if } n=1; \sum_{k=1}^{n-1} P(k)P(n-k) \text{ if } n \geq 2 \}$ - $P(n) = \Omega(4^n / n^{\{3/2\}}) = \Omega(2^n)$

LCS Recurrence

$c[i,j] = \{ 0 \text{ if } i=0 \text{ or } j=0; c[i-1,j-1]+1 \text{ if } i,j>0 \text{ and } x_i=y_j; \max(c[i-1,j], c[i,j-1]) \text{ if } i,j>0 \text{ and } x_i \neq y_j \}$ - $c[i,j]$ = length of LCS of X_i and Y_j

Optimal BST: Expected Search Cost

$E[\text{search cost in } T] = \sum_{i=1}^n (\text{depth}_T(k_i)+1)*p_i + \sum_{i=0}^n (\text{depth}_T(d_i)+1)*q_i = 1 + \sum_{i=1}^n \text{depth}_T(k_i)*p_i + \sum_{i=0}^n \text{depth}_T(d_i)*q_i$

Optimal BST Recurrence

$e[i,j] = \{ q_{i-1} \text{ if } j=i-1; \min_{\{i \leq r \leq j\}} (e[i,r-1] + e[r+1,j] + w(i,j)) \text{ if } i \leq j \}$ $w(i,j) = \sum_{l=i}^j p_l + \sum_{l=i-1}^j q_l$ - $e[i,j]$ = expected cost of optimal BST for keys $k_i..k_j$ - $w(i,j)$ = sum of all probabilities in subtree with keys $k_i..k_j$ - $w[i,j] = w[i,j-1] + p_j + q_j$

Balancing in Probability

$\sum_{i=1}^n p_i + \sum_{i=0}^n q_i = 1$

Rules, Laws & Theorems

Optimal Substructure of LCS (Theorem 14.1)

- **Statement:** Let $X=\langle x_1, \dots, x_m \rangle$ and $Y=\langle y_1, \dots, y_n \rangle$, and let $Z=\langle z_1, \dots, z_k \rangle$ be any LCS of X and Y . (1) If $x_m = y_n$, then $z_k = x_m = y_n$ and $Z_{\{k-1\}}$ is an LCS of $X_{\{m-1\}}$ and $Y_{\{n-1\}}$. (2) If $x_m \neq y_n$ and $z_k \neq x_m$, then Z is an LCS of $X_{\{m-1\}}$ and Y . (3) If $x_m \neq y_n$ and $z_k \neq y_n$, then Z is an LCS of X and $Y_{\{n-1\}}$.
- **Conditions:** X and Y are sequences

LCS Optimal Substructure Proof

- **Statement:** Proof by contradiction: (1) if $z_k \neq x_m$, appending $x_m=y_n$ gives longer common subsequence. (2) If $z_k \neq x_m$, any longer common subsequence of $X_{\{m-1\}}$ and Y would also be longer for X_m and Y .
- **Conditions:** Standard LCS problem

Optimal Substructure for Shortest Path (holds) vs. Longest Simple Path (fails)

- **Statement:** Unweighted shortest path has optimal substructure; unweighted longest simple path does NOT have optimal substructure because subproblems are NOT independent (they share resources/vertices).
- **Conditions:** For longest simple path, combining optimal subproblem solutions may yield a non-simple path.

Running Time DP Rule of Thumb

- **Statement:** Running time = (number of subproblems) $\times$ (number of choices per subproblem). Rod cutting: $\Theta(n)$ subproblems $\times$ $O(n)$ choices = $O(n^2)$. Matrix-chain: $\Theta(n^2)$ subproblems $\times$ $O(n)$ choices = $O(n^3)$.

Five Distinct Parenthesizations for $n=4$

- **Statement:** For $A_1 A_2 A_3 A_4$, the five ways are:
 $(A_1(A_2(A_3 A_4)))$, $(A_1((A_2 A_3)A_4))$, $((A_1 A_2)(A_3 A_4))$, $((A_1(A_2 A_3))A_4)$, $((A_1 A_2)A_3)A_4$.

Edge Cases & Pitfalls

- **Not all problems have optimal substructure:** Longest simple path fails; subproblems are not independent.
- **Independence of subproblems:** Subproblems must not share resources; for shortest path, any vertex other than the splicing point cannot appear in both subpaths (proved by contradiction).
- **Greedy does not work for rod cutting:** Choosing the highest density (p_i/i) piece first does NOT always yield optimal revenue (Exercise 14.1-2).
- **Empty subtrees in OBST:** When selecting k_i as root, left subtree has no keys but contains dummy key $d_{\{i-1\}}$. Table e needs indices up to $n+1$ and 0 to handle empty subtrees with d_n and d_0 .
- **Optimal BST $\neq$ minimum height tree:** Optimal BST may not have the smallest overall height, nor the key with highest probability at root.
- **Subproblem boundaries for OBST:** Unlike matrix-chain where $s[i,j]$ is within a defined range, for OBST you need $e[1:n+1, 0:n]$ so that $e[n+1, n]$ (subtree with only d_n) and $e[1, 0]$ (subtree with only d_0) are defined.

- **Full parenthesization always has exactly $n-1$ pairs of parentheses** for n matrices.
- **Recursive-Matrix-Chain is exponential** ($\Omega(2^n)$) – not better than exhaustive enumeration.
- **Memoization fails for divide-and-conquer** like Merge-Sort because subproblems don't overlap (each subproblem is distinct).
- **Cut-and-paste proof requires independence:** For longest simple path, splicing two optimal subpath solutions produces a non-simple path.

Case Studies & Examples

Rod Cutting: Sample Pricing (Figure 14.1)

- **What:** Price table for rods: length i : 1 2 3 4 5 6 7 8 9 10; price p_i : 1 5 8 9 10 17 17 20 24 30.
- **Results:** Optimal revenues: $r_1=1$, $r_2=5$, $r_3=8$, $r_4=10$ (cut 2+2), $r_5=13$ (2+3), $r_6=17$ (no cut), $r_7=18$ (1+6 or 2+2+3), $r_8=22$ (2+6), $r_9=25$ (3+6), $r_{10}=30$ (no cut).
- **Significance:** Demonstrates optimal substructure and that greedy by density fails.

Rod Cutting: $n=4$ Example (Figure 14.2)

- **What:** 8 possible ways to cut a 4-inch rod.
- **Results:** Optimal is cutting into two 2-inch pieces: $5+5=10$.
- **Significance:** Illustrates exponential possibilities (2^{n-1}).

Matrix-Chain: 3-Matrix Example

- **What:** A_1 : 10×100 , A_2 : 100×5 , A_3 : 5×50 .
- **Results:** $((A_1 A_2) A_3)$ costs $101005 + 10550 = 5000 + 2500 = 7500$. $(A_1(A_2 A_3))$ costs $100550 + 1010050 = 25000 + 50000 = 75000$. Factor of 10 difference.
- **Significance:** Dramatically shows impact of parenthesization.

Matrix-Chain: $n=6$ Example (Figure 14.5)

- **What:** 6 matrices with dimensions: $A_1: 30 \times 35$, $A_2: 35 \times 15$, $A_3: 15 \times 5$, $A_4: 5 \times 10$, $A_5: 10 \times 20$, $A_6: 20 \times 25$.
- **Results:** $m[1,6]=15,125$. Optimal parenthesization: $((A_1(A_2 A_3))((A_4 A_5) A_6))$.
- **Significance:** Shows filled m and s tables.

LCS Example (Figure 14.8)

- **What:** $X = \langle A, B, C, B, D, A, B \rangle$, $Y = \langle B, D, C, A, B, A \rangle$.
- **Results:** LCS length = 4, LCS = $\langle B, C, B, A \rangle$ (also $\langle B, D, A, B \rangle$).
- **Significance:** Illustrates c and b tables filled by LCS-Length.

Optimal BST Example (Figure 14.10)

- **What:** $n=5$ keys with probabilities:
 $p=[0.15, 0.10, 0.05, 0.10, 0.20]$,
 $q=[0.05, 0.10, 0.05, 0.05, 0.05, 0.10]$.
- **Results:** Optimal cost $e[1,5]=2.75$. Root is k_2 (not k_5 which has highest p). Structure: k_2 root, k_1 left child of k_2 , k_5 right child of k_2 , k_4 left child of k_5 , k_3 left child of k_4 .
- **Significance:** Shows optimal BST $\neq$ highest probability at root; expected cost 2.75 vs 2.80 for alternative tree.

Longest Simple Path Counterexample (Figure 14.6)

- **What:** Directed graph with edges $q \rightarrow r$, $q \rightarrow s$, $r \rightarrow t$, $s \rightarrow t$, $t \rightarrow r$.
- **Results:** Longest simple path from q to t is $q \rightarrow r \rightarrow t$ (length 2). Subpath $q \rightarrow r$ is not longest from q to r ($q \rightarrow s \rightarrow t \rightarrow r$ is longer with length 3). Subproblem solutions conflict.
- **Significance:** Demonstrates failure of optimal substructure due to non-independent subproblems.

Diagrams & Visuals

[Rod cutting recursion tree: root labeled 4, children 3,2,1,0; grandchildren continue pattern. Total 2^n nodes.]

[Subproblem graph for rod cutting with $n=4$: vertices 0,1,2,3,4 with directed edges from higher to lower numbers.]

[Matrix-chain m -table: rotated so main diagonal is horizontal. Entries for chain lengths 1..6. $m[1,6]=15125$ in upper-right.]

[LCS c -table and b -table: 7×6 grid with arrows and numbers. LCS traced from lower-right following "nw" arrows.]

[OBST tables e , w , root: rotated with diagonals horizontal. $root[1,5]=2$.]

End-of-Chapter Material

Key Terms

dynamic programming, memoization, bottom-up method, optimal substructure, overlapping subproblems, subproblem graph, rod cutting, matrix-chain multiplication, LCS, optimal BST, edit distance, bitonic tour, palindrome, seam carving

Review Questions (Exercises)

14.1-1: Show that equation (14.4) follows from (14.3) and $T(0)=1$.

14.1-2: Show by counterexample that greedy by density (p_i/i) fails for rod cutting.

14.1-3: Modify rod cutting for fixed cost c per cut. Give DP algorithm.

14.1-4: Modify Cut-Rod and Memoized-Cut-Rod-Aux so loops go up to $\text{floor}(n/2)$. What changes needed? How are running times affected?

14.1-5: Modify Memoized-Cut-Rod to return actual solution (not just value).

14.1-6: Give $O(n)$ -time DP algorithm for n th Fibonacci number. Draw subproblem graph. How many vertices and edges?

14.2-1: Find optimal parenthesization for dimensions $\langle 5, 10, 3, 12, 5, 50, 6 \rangle$.

14.2-2: Give recursive Matrix-Chain-Multiply(A, s, i, j) that actually performs optimal multiplications.

14.2-3: Use substitution method to show recurrence (14.6) is $\Omega(2^n)$.

14.2-4: Describe subproblem graph for matrix-chain; number of vertices and edges?

14.2-5: Let $R(i, j)$ be number of times $m[i, j]$ is referenced. Show total references = $\sum_{i=1}^n \sum_{j=i}^n R(i, j) = (n^3 - n)/3$.

14.2-6: Show full parenthesization of n -element expression has exactly $n-1$ pairs of parentheses.

14.3-1: Which is more efficient: enumerating all parenthesizations or running Recursive-Matrix-Chain? Justify.

14.3-2: Draw recursion tree for Merge-Sort on 16 elements. Why does memoization fail?

14.3-3: Does maximizing (not minimizing) matrix multiplications have optimal substructure?

14.3-4: Find instance where greedy split (minimizing $p_{i-1} p_k p_j$) yields suboptimal solution.

14.3-5: Rod cutting with limit l_i on pieces of length i – does optimal substructure still hold?

14.4-1: Find LCS of $\langle 1, 0, 0, 1, 0, 1, 0, 1 \rangle$ and $\langle 0, 1, 0, 1, 1, 0, 1, 1, 0 \rangle$.

14.4-2: Pseudocode to reconstruct LCS from c table (without b table) in $O(m+n)$ time.

14.4-3: Memoized version of LCS-Length in $O(mn)$ time.

14.4-4: Compute LCS length using only $2 \cdot \min(m,n)$ entries, then $\min(m,n)$ entries.

14.4-5: $O(n^2)$ -time algorithm for longest monotonically increasing subsequence.

14.4-6: $O(n \lg n)$ -time algorithm for longest monotonically increasing subsequence.

14.5-1: Pseudocode for Construct-Optimal-BST(root,n) that outputs tree structure.

14.5-2: Determine cost and structure of optimal BST for $n=7$ keys with given probabilities.

14.5-3: How would computing $w(i,j)$ directly from formula (14.12) affect asymptotic running time?

14.5-4: Use Knuth's root bounds ($\text{root}[i,j-1] \leq \text{root}[i,j] \leq \text{root}[i+1,j]$) to modify Optimal-BST to run in $\Theta(n^2)$.

Problems

14-1 Longest simple path in a DAG: Directed acyclic graph with real-valued edge weights, vertices s, t . Find longest weighted simple path from s to t using DP. Analyze running time.

14-2 Longest palindrome subsequence: Given input string, find longest palindrome that is a subsequence. E.g., "character" $\rightarrow$ "carac". Running time?

14-3 Bitonic euclidean TSP: $O(n^2)$ -time algorithm for shortest bitonic tour. Points on plane, no two have same x-coordinate.

14-4 Printing neatly: DP to print paragraph with monospaced font, M chars per line, minimizing sum of cubes of extra spaces (except last line).

14-5 Edit distance: DP to find edit distance from $x[1:m]$ to $y[1:n]$ with operations copy (cost Q_C), replace (Q_R), delete (Q_D), insert (Q_I), twiddle (Q_T), kill (Q_K). Also relate to DNA sequence alignment (score: +1 match, -1 mismatch, -2 gap).

14-6 Planning a company party: Tree of employees with conviviality ratings. No employee and immediate supervisor both attend. Maximize sum of ratings.

14-7 Viterbi algorithm: (a) Find path in edge-labeled directed graph matching sound sequence. (b) Find most probable path matching sounds, given edge probabilities.

14-8 Image compression by seam carving: (a) Number of possible seams grows exponentially in m . (b) Given disruption measure $d[i,j]$, find seam with lowest disruption measure.

14-9 Breaking a string: String of n chars, break points $L[1:m]$. Find least-cost sequence of breaks (cost n to break string of length n).

14-10 Planning an investment strategy: n investments, return rates r_{ij} , fees f_1 (no switch) and f_2 (switch, $f_2 > f_1$). (a) Prove optimal to invest all in single investment each year. (b) Optimal substructure. (c) Design algorithm. (d) With \$15,000 limit per investment, optimal substructure fails.

14-11 Inventory planning: n months, demand d_i , produce up to m per month with full-time staff, extra at \$ c per machine, inventory cost $h(j)$. Minimize costs.

14-12 Signing free-agent baseball players: Budget \$ X , N positions, P players per position, each player has $p.cost$ and $p.war$. Max total WAR, spend $\leq X$. Output players signed, total WAR, total cost.

Ch. 15 – Greedy Algorithms

Named Entities (Terms & Definitions)

- **Greedy algorithm:** Always makes the choice that looks best at the moment (locally optimal choice) hoping it leads to a globally optimal solution.
- **Activity-selection problem:** Schedule a maximum-size set of mutually compatible activities that require exclusive use of a common resource.
- **Compatible activities:** Activities a_i and a_j are compatible if their half-open time intervals $[s_i, f_i)$ and $[s_j, f_j)$ do not overlap ($s_i \geq f_j$ or $s_j \geq f_i$).
- **Greedy-choice property:** A globally optimal solution can be assembled by making locally optimal (greedy) choices.
- **0-1 knapsack problem:** Thief must choose subset of n items (each with value v_i and weight w_i) to maximize total value without exceeding weight capacity W ; each item taken or left (binary choice). Solvable by DP, not greedy.
- **Fractional knapsack problem:** Thief can take fractions of items; solvable by greedy (take highest value per pound first).
- **Huffman code:** Optimal prefix-free code for data compression; uses character frequencies to build optimal binary tree.
- **Prefix-free code:** No codeword is a prefix of any other codeword; simplifies decoding.
- **Fixed-length code:** Each character represented by $\lceil \lg n \rceil$ bits.
- **Variable-length code:** Frequent characters get short codewords, infrequent get long codewords.
- **Full binary tree:** Every nonleaf node has two children; an optimal prefix-free code is always represented by a full binary tree.

- **Cost of a tree T:** $B(T) = \sum_{c \in C} c.\text{freq} * d_T(c)$, where $d_T(c)$ is depth of c 's leaf.
- **Furthest-in-future:** Optimal offline caching strategy – evict the block whose next access comes furthest in the future.
- **Cache hit:** Requested block is already in cache.
- **Cache miss:** Requested block is not in cache.
- **Compulsory miss:** Cache miss while cache is still being filled (fewer than k blocks).
- **Online caching problem:** Must decide which blocks to evict without knowing future requests.
- **Offline caching problem:** Know entire sequence of requests in advance; furthest-in-future is optimal.
- **LRU (Least-Recently-Used):** Strategy that evicts the block that was least recently requested; commonly used but not always optimal.
- **Interval-graph coloring problem:** Model of scheduling activities in minimum number of lecture halls; equivalent to coloring interval graph.

Processes / Algorithms / Pathways

Recursive-Activity-Selector

- **Goal:** Select maximum-size subset of mutually compatible activities from S_k
- **Input/Output:** Input: arrays $s[1:n]$ (start times), $f[1:n]$ (finish times), index k , size n . Activities sort by finish time. Fictitious a_0 with $f_0=0$. Output: max-size set of compatible activities from S_k .
- **Steps:** (1) $m = k+1$. (2) While $m \leq n$ and $s[m] < f[k]$: $m = m+1$. (3) If $m \leq n$: return $\{a_m\} \cup \text{Recursive-Activity-Selector}(s, f, m, n)$. (4) Else return empty set.
- **Complexity:** $\Theta(n)$ time (each activity examined exactly once across all recursive calls).
- **Initial call:** $\text{Recursive-Activity-Selector}(s, f, 0, n)$.

Greedy-Activity-Selector (iterative)

- **Goal:** Select maximum-size subset of compatible activities
- **Input/Output:** Input: $s[1:n]$, $f[1:n]$ (sorted by finish time). Output: set A of selected activities.
- **Steps:** (1) $A = \{a_1\}$. (2) $k = 1$. (3) For $m = 2$ to n : (4) If $s[m] \geq f[k]$: $A = A \cup \{a_m\}$; $k = m$. (5) Return A .
- **Complexity:** $\Theta(n)$ time assuming sorted input.
- **Example** (Figure 15.1): Activities $a_1(1,4)$, $a_2(3,5)$, $a_3(0,6)$, $a_4(5,7)$, $a_5(3,9)$, $a_6(5,9)$, $a_7(6,10)$, $a_8(8,11)$, $a_9(8,12)$, $a_{10}(2,14)$, $a_{11}(12,16)$. Result: $\{a_1, a_4, a_8, a_{11}\}$.

Huffman (greedy code construction)

- **Goal:** Construct optimal prefix-free binary code

- **Input/Output:** Input: set C of n characters with $c.freq$ attributes. Output: root of optimal Huffman tree.
- **Steps:** (1) $n = |C|$. (2) $Q = C$ (min-priority queue keyed by $freq$). (3) For $i=1$ to $n-1$: (4) Allocate new node z . (5) $x = \text{Extract-Min}(Q)$. (6) $y = \text{Extract-Min}(Q)$. (7) $z.left = x$. (8) $z.right = y$. (9) $z.freq = x.freq + y.freq$. (10) $\text{Insert}(Q, z)$. (11) Return $\text{Extract-Min}(Q)$ (the root).
- **Complexity:** $O(n \lg n)$ with binary heap ($O(n)$ Build-Min-Heap + $(n-1) \cdot O(\lg n)$ for extract/insert).
- **Example** (Figure 15.4-15.6): Frequencies: a:45, b:13, c:12, d:16, e:9, f:5. Fixed-length: 300,000 bits. Variable-length: 224,000 bits. Codes: a=0, b=101, c=100, d=111, e=1101, f=1100.

Furthest-in-Future (offline caching)

- **Goal:** Minimize cache misses given known request sequence
- **Input:** Set C of blocks in cache, cache size k , request sequence $b_1..b_n$, index i of current request.
- **Strategy:** Upon a cache miss when cache is full, evict the block whose next access is furthest in the future (or never again).
- **Optimality:** Proved via greedy-choice property (Theorem 15.5) and optimal substructure.

Design Paradigm: Greedy

- **Key idea:** Make the locally optimal choice at each step, hoping it leads to a globally optimal solution. Usually top-down: make a choice, then solve the remaining subproblem.
- **Optimal substructure:** An optimal solution contains within it optimal solutions to subproblems (shared with DP).
- **Greedy-choice property:** Can assemble globally optimal solution by making locally optimal choices. Unlike DP, choice does not depend on solutions to subproblems.
- **When to use:** When both greedy-choice property and optimal substructure hold. Used for activity selection, Huffman codes, fractional knapsack, minimum spanning trees (Ch. 21), Dijkstra's shortest path (Ch. 22).
- **Six-step process** (from activity selection): (1) Determine optimal substructure. (2) Develop recursive solution. (3) Show greedy choice leaves only one subproblem. (4) Prove greedy choice is always safe. (5) Develop recursive greedy algorithm. (6) Convert to iterative.
- **Simplified process:** (1) Cast as make a choice + one subproblem. (2) Prove greedy choice is always safe. (3) Show optimal substructure: greedy choice + optimal solution to subproblem $\Rightarrow$ optimal to original.

- **Proof technique:** Exchange argument – take any optimal solution, modify it to substitute the greedy choice, show the result is no worse (at least as good).
- **Template/pseudocode:**
Initialize result
While problem instance not solved:
 Make greedy choice (locally optimal)
 Reduce instance to smaller subproblem
Return result
- **Comparison with DP:** DP makes choice based on subproblem solutions (bottom-up); greedy makes choice first, then solves one subproblem (top-down). Greedy is more efficient when applicable but works for fewer problems.

Comparisons & Trade-offs

Dimension	Greedy Algorithms	Dynamic Programming
Choice basis	Looks best at moment	Depends on subproblem solutions
Subproblems solved	After making choice (one subproblem)	Before making choice (many subproblems)
Order	Top-down	Bottom-up (or top-down memoized)
Efficiency	Usually more efficient	Usually less efficient
Overlapping subproblems	Not needed	Needed
Greedy-choice property	Required	Not required
Dimension	0-1 Knapsack	Fractional Knapsack
Item divisibility	Whole items only (0/1)	Fractions allowed
Greedy works?	No	Yes (by value/weight)
Solution method	DP $O(nW)$	Greedy $O(n \lg n)$

Formulas & Equations

Number of ways to parenthesize n matrices (Catalan)

$P(n) = \sum_{k=1}^{n-1} P(k) * P(n-k)$ for $n \geq 2$, $P(1)=1$ - Grows as $\Omega(4^n / n^{\{3/2\}})$

Activity Selection DP Recurrence

$c[i,j] = \max_{k \in S_{ij}} (c[i,k] + c[k,j] + 1)$ - $c[i,j]$ = size of max compatible subset in S_{ij}

Huffman Tree Cost

$B(T) = \sum_{c \in C} c.\text{freq} * d_T(c)$ - $d_T(c)$ = depth of character c 's leaf = codeword length - $B(T)$ = total bits to encode file using code tree T

Huffman Merging Cost Identity

$B(T) = \sum_{\text{internal nodes } v} (v.\text{left}.\text{freq} + v.\text{right}.\text{freq})$ - The sum over all internal nodes of the combined frequencies of the two children equals the total cost.

Total Bits for Variable-Length Code Example

$(45*1 + 13*3 + 12*3 + 16*3 + 9*4 + 5*4) * 1000 = 224,000$ bits - Savings of ~25% over 300,000 bit fixed-length code.

Fractional Knapsack Value per Pound

v_i / w_i for each item i - Items sorted by descending v_i/w_i .

Activity Selection Compatibility Check

$s_i \geq f_j$ or $s_j \geq f_i$ - Activities a_i and a_j are compatible if their intervals do not overlap.

Rules, Laws & Theorems

Theorem 15.1 (Greedy choice for activity selection)

- **Statement:** Consider any nonempty subproblem S_k , and let a_m be an activity in S_k with the earliest finish time. Then a_m is included in some maximum-size subset of mutually compatible activities of S_k .
- **Proof:** Exchange argument: take an optimal solution A_k , let a_j be earliest-finishing in A_k . If $a_j \neq a_m$, substitute a_m for a_j . Since $f_m \leq f_j$, the new set is compatible and same size, thus also optimal and includes a_m .

Lemma 15.2 (Greedy-choice property for Huffman codes)

- **Statement:** Let C be an alphabet, x and y two characters with lowest frequencies. There exists an optimal prefix-free code for C in which codewords for x and y have the same length and differ only in the last bit.
- **Proof:** Take optimal tree T , find deepest sibling leaves a, b . Swap a with x , b with y ; each swap does not increase cost. Resulting tree T'' is optimal with x, y as deepest siblings.

Lemma 15.3 (Optimal substructure for Huffman codes)

- **Statement:** Let C' be C with x, y removed and new character z added with $z.\text{freq} = x.\text{freq} + y.\text{freq}$. If T' is optimal for C' , then T (replace leaf z with internal node having x, y as children) is optimal for C .
- **Proof:** $B(T) = B(T') + x.\text{freq} + y.\text{freq}$. By contradiction: if T not optimal, then exists T'' with x, y as siblings s.t. $B(T'') < B(T)$. Replace x, y parent with leaf $z \Rightarrow$ contradicts T' optimal.

Theorem 15.4 (Huffman produces optimal code)

- **Statement:** Procedure Huffman produces an optimal prefix-free code.
- **Proof:** Immediate from Lemmas 15.2 and 15.3.

Theorem 15.5 (Greedy-choice property for offline caching)

- **Statement:** Consider subproblem (C, i) when cache is full and a cache miss occurs. Let z be the block in C whose next access is furthest in the future. Evicting z upon request for b_i is included in some optimal solution.
- **Proof:** Exchange argument: take optimal solution S that evicts some $x \neq z$. Construct S' that evicts z instead. Show via induction that S' induces at most as many cache misses and has identical behavior after b_m (where $b_m = z$).

Edge Cases & Pitfalls

- **Greedy does not always work:** Activity selection greedy works, but selecting by least duration, fewest overlapping remaining activities, or earliest start time fails (Ex 15.1-3).
- **0-1 knapsack fails with greedy:** Greedy by value/weight fails for 0-1 knapsack (e.g., items: \$60/10lb, \$100/20lb, \$120/30lb, capacity 50lb. Greedy picks item 1, but optimal is items 2 and 3 with value \$220).
- **Huffman tree must be full:** Non-full binary tree cannot be optimal (Exercise 15.3-2). Full tree = every nonleaf has exactly two children.
- **No compression scheme shrinks every file:** For any lossless compression scheme, there exist files that it compresses, but by pigeonhole principle, some files must get longer (Ex 15.3-8).
- **Huffman with equal frequencies:** If all 256 ASCII chars are about equally common ($\max < 2 \cdot \min \text{freq}$), Huffman coding is no better than 8-bit fixed-length (Ex 15.3-7).
- **LRU vs Furthest-in-Future:** LRU can induce more cache misses when past access pattern differs from future (Ex 15.4-2).
- **Contracting cache on half-full causes thrashing:** If halving size when $< 1/2$ full, sequence of insert/delete pairs causes

Theta(n) expansions and contractions, each costing Theta(n), total Theta(n²). Solution: contract at 1/4 full.

- **Activity-sort must be by finish time:** Greedy-Activity-Selector and Recursive-Activity-Selector assume activities sorted by monotonically increasing finish time; otherwise O(n lg n) sort first.

Case Studies & Examples

Activity Selection Example (Figure 15.1)

- **What:** 11 activities with start/finish times.
- **Results:** Optimal solution: {a₁, a₄, a₈, a₁₁} (also {a₂, a₄, a₉, a₁₁}).
- **Significance:** Illustrates greedy choice (earliest finish = a₁) leads to optimal.

0-1 vs. Fractional Knapsack (Figure 15.3)

- **What:** 3 items: item 1 \$60/10lb (\$6/lb), item 2 \$100/20lb (\$5/lb), item 3 \$120/30lb (\$4/lb). Capacity 50.
- **Results:** 0-1 optimal = items 2+3 = \$220. Greedy picks item 1 first => suboptimal. Fractional optimal = item 1 (10lb) + item 2 (20lb) + 2/3 of item 3 (20lb) = \$60+\$100+\$80 = \$240.
- **Significance:** Shows greedy works for fractional but not 0-1 knapsack.

Huffman Coding Example (Figures 15.4-15.6)

- **What:** 100,000 char file, 6 chars a-f, frequencies: a:45000, b:13000, c:12000, d:16000, e:9000, f:5000.
- **Results:** Fixed-length: 300,000 bits (3 per char). Huffman: 224,000 bits (a:0, b:101, c:100, d:111, e:1101, f:1100). Savings ~25%.
- **Significance:** Huffman merges lowest-frequency nodes repeatedly, building tree from leaves up.

Offline Caching Example

- **What:** Request sequence s, q, s, q, q, s, p, p, r, s, s, q, p, r, q. Cache size k.
- **Significance:** Furthest-in-future yields optimal misses; proofs establish greedy-choice and optimal-substructure properties.

Diagrams & Visuals

[Activity intervals: horizontal bars for 11 activities with time on x-axis. Selected activities a₁, a₄, a₈, a₁₁ highlighted.]

[Huffman merge steps: (a) 6 singleton nodes (f:5, e:9, c:12, b:13, d:16, a:45). (b) merge 5,9 into 14. (c) merge 12,13 into 25.

(d) merge 14,16 into 30. (e) merge 25,30 into 55. (f) merge 45,55 into 100 (root). Each internal node labeled with sum of children frequencies.]

End-of-Chapter Material

Key Terms

greedy algorithm, greedy-choice property, optimal substructure, activity selection, compatible activities, 0-1 knapsack, fractional knapsack, Huffman code, prefix-free code, full binary tree, codeword, furthest-in-future, cache hit, cache miss, compulsory miss, offline caching, LRU

Review Questions (Exercises)

15.1-1: DP algorithm for activity selection based on recurrence (15.2). Compare running time with Greedy-Activity-Selector.

15.1-2: Select last activity to start that is compatible (prove optimal).

15.1-3: Show selecting by least duration fails; selecting by fewest overlaps fails; selecting by earliest start fails.

15.1-4: Interval-graph coloring: schedule all activities in minimum lecture halls. Give efficient greedy algorithm.

15.1-5: Weighted activity selection: each activity a_i has value v_i . Maximize total value of selected compatible activities (polynomial time).

15.2-1: Prove fractional knapsack has greedy-choice property.

15.2-2: Give $O(nW)$ DP for 0-1 knapsack.

15.2-3: If sorted by increasing weight = sorted by decreasing value, give efficient algorithm for 0-1 knapsack.

15.2-4: Professor Gekko: minimize number of water stops. Two liters, skate m miles per liter. Give algorithm, prove optimal.

15.2-5: Smallest set of unit-length closed intervals containing all given points on real line.

15.2-6: Solve fractional knapsack in $O(n)$ time.

15.2-7: Maximize product of sums from reordered sets A and B .

15.3-1: In Lemma 15.2 proof, if $x.\text{freq} = b.\text{freq}$, then $a.\text{freq} = b.\text{freq} = x.\text{freq} = y.\text{freq}$.

15.3-2: Prove non-full binary tree cannot be optimal prefix-free code.

15.3-3: Optimal Huffman code for frequencies: a:1, b:1, c:2, d:3, e:5, f:8, g:13, h:21 (first 8 Fibonacci numbers). Generalize to first n Fibonacci numbers.

15.3-4: Prove $B(T) = \text{sum over internal nodes of } (\text{left.freq} + \text{right.freq})$.

15.3-5: Represent optimal prefix-free code using $2n-1 + \text{ncel}(\lg n)$ bits ($2n-1$ for tree structure via walk, plus $\text{ncel}(\lg n)$ for leaf frequencies).

15.3-6: Generalize Huffman to ternary codewords (0,1,2). Prove optimal.

15.3-7: If all 256 chars about equally common ($\max < 2 \cdot \min \text{freq}$), Huffman no better than 8-bit fixed-length. Prove.

15.3-8: Prove no lossless compression scheme can guarantee shorter output for every input (pigeonhole principle).

15.4-1: Pseudocode for cache manager using furthest-in-future.

15.4-2: Show LRU is not optimal (example where LRU induces more misses).

15.4-3: In Theorem 15.5 proof, show why requiring $y = x$ (block evicted by S) breaks the proof.

15.4-4: Show that for any solution allowing multiple blocks to enter cache per request, there is a solution bringing in only one block per request that is at least as good.

Problems

15-1 Coin changing: (a) Greedy algorithm for quarters, dimes, nickels, pennies (prove optimal). (b) Denominations $c^0, c^1, \dots, c^k$ (powers of $c > 1$) – greedy optimal. (c) Set of denominations where greedy fails (must include penny). (d) $O(nk)$ DP for any k denominations with a penny.

15-2 Scheduling to minimize average completion time: (a) Nonpreemptive tasks: schedule shortest-processing-time first. Prove optimal, analyze running time. (b) Preemptive tasks with release times: algorithm to minimize average completion time.

Ch. 16 – Amortized Analysis

Named Entities (Terms & Definitions)

- **Amortized analysis:** Averaging the time required to perform a sequence of data-structure operations over all operations performed; guarantees average performance of each operation in the worst case (no probability involved).

- **Aggregate analysis:** Show that a sequence of n operations takes $T(n)$ worst-case time total; amortized cost per operation = $T(n)/n$ (all operations have same amortized cost).
- **Accounting method:** Assign differing amortized costs to different operations; overcharge some operations early, storing credit on specific objects; later undercharged operations use stored credit. Credit must never be negative.
- **Potential method:** Represent prepaid work as “potential energy” of the data structure as a whole. A potential function Φ maps each data structure D_i to a real number. Amortized cost = actual cost + change in potential. If $\Phi(D_i) \geq \Phi(D_0)$, total amortized cost upper-bounds total actual cost.
- **Credit:** In accounting method, the difference between amortized cost and actual cost, stored on specific objects; must remain nonnegative.
- **Potential function:** $\Phi(D)$ that maps data structure D to a real number (potential); used in potential method.
- **Multipop:** Stack operation that pops k top objects (or entire stack if fewer than k).
- **Increment (binary counter):** Operation that increments a k -bit binary counter by 1, flipping bits as needed.
- **Dynamic table:** Table that can expand (when full) or contract (when too empty) to accommodate varying numbers of items.
- **Load factor:** $\alpha(T) = \text{num_items} / \text{size of table}$. For empty table, defined as 1.
- **Table expansion:** When inserting into a full table, allocate new table with $2x$ slots, copy all items, then insert.
- **Table contraction:** When deleting causes load factor to drop below threshold, allocate smaller table, copy items.
- **Binary reflected Gray code:** Sequence of 2^k integers where each consecutive pair differs in exactly one bit. Constructed recursively: for $k \geq 2$, take Gray code for $k-1$, reflect it, add 2^{k-1} to reflected part, concatenate.

Processes / Algorithms / Pathways

Multipop (stack operation)

- **Goal:** Pop k objects from stack S (or all if fewer than k)
- **Steps:** (1) While not Stack-Empty(S) and $k > 0$: Pop(S); $k = k-1$.
- **Complexity:** Actual cost = $\min(s, k)$ where s = stack size. $O(\min(s, k))$ time.

Increment (binary counter)

- **Goal:** Increment k -bit binary counter by 1
- **Input/Output:** Input: array $A[0:k-1]$ ($A[0]$ = least significant bit). Output: incremented counter.
- **Steps:** (1) $i = 0$. (2) While $i < k$ and $A[i] == 1$: $A[i] = 0$; $i = i+1$. (3) If $i < k$: $A[i] = 1$.

- **Complexity:** Actual cost = number of bits flipped. Worst-case $\Theta(k)$ if all bits are 1.

Table-Insert (dynamic table)

- **Goal:** Insert item x into dynamic table T , expanding if full
- **Steps:** (1) If $T.size == 0$: allocate $T.table$ with 1 slot; $T.size=1$. (2) If $T.num == T.size$: (3) allocate new-table with $2T.size$ slots. (4) Insert all items from $T.table$ into new-table. (5) Free $T.table$. (6) $T.table = new-table$; $T.size = 2T.size$. (7) Insert x into $T.table$. (8) $T.num = T.num + 1$.
- **Complexity:** Actual cost $c_i = 1$ if no expansion, i if expansion occurs ($i-1$ copies + 1 insertion).
- **Expansion occurs** when $i-1$ is an exact power of 2.

Table-Delete (dynamic table)

- **Goal:** Delete item from table, contracting when load factor $< 1/4$
- **Strategy:** Halve table size when $T.num < T.size/4$ (not at $1/2$ to avoid thrashing).
- **Contraction cost:** $size/4$ to copy items to new smaller table.
- **Complexity:** Amortized cost $O(1)$ per operation.

Design Paradigm: Amortized Analysis

- **Key idea:** Average the cost of operations over a sequence; show that expensive operations are rare enough that the average cost per operation is small.
- **Three methods:** Aggregate ($T(n)/n$ for all ops), Accounting (different amortized costs per op type), Potential (potential function on data structure).
- **When to use:** When analyzing data structures where occasional expensive operations are offset by many cheap ones (e.g., dynamic tables, binary counters, stacks with multipop).
- **Template – Aggregate:** (1) Bound total cost $T(n)$ of sequence of n operations in worst case. (2) Amortized cost = $T(n)/n$.
- **Template – Accounting:** (1) Assign amortized cost to each operation type. (2) Ensure total amortized cost $\geq$ total actual cost for all sequences. (3) Credit = amortized - actual; must never be negative.
- **Template – Potential:** (1) Define potential function $\Phi(D)$ mapping data structure to real. (2) Amortized cost $c_i^{\wedge} = c_i + \Phi(D_i) - \Phi(D_{i-1})$. (3) Ensure $\Phi(D_i) \geq \Phi(D_0)$ for all i (or $\Phi(D_i) \geq 0$ if $\Phi(D_0)=0$). (4) Total amortized cost = $\sum c_i + \Phi(D_n) - \Phi(D_0) \geq \sum c_i$.
- **Proof technique:** Show total amortized cost bounds total actual cost using invariant (credit ≥ 0 in accounting, potential $\geq$ initial in potential method).

Comparisons & Trade-offs

Dimension	Aggregate	Accounting	Potential
Per-operation cost	Same for all operations	May differ by operation type	May differ by operation
Mechanism	Average $T(n)/n$	Credit on specific objects	Potential function on whole DS
Difficulty	Simple	Moderate	Requires clever potential function
Flexibility	Least	Medium	Most
Credit/ potential tracking	None	Per-object	Global

Formulas & Equations

Aggregate Analysis: Stack

Total cost of n PUSH, POP, MULTIPOP operations = $O(n)$ - Each object popped must have been pushed; #POP calls $\leq$ #PUSH calls $\leq n$. - Amortized cost per operation = $O(1)$.

Aggregate Analysis: Binary Counter

Total flips for n INCREMENT operations = $\sum_{i=0}^{k-1} \text{floor}(n/2^i) < 2n$ - Bit $A[i]$ flips $\text{floor}(n/2^i)$ times. - Total cost = $O(n)$. Amortized cost per operation = $O(1)$.

Aggregate Analysis: Dynamic Table (insertions only)

$c_i = \{ i \text{ if } i-1 \text{ is power of } 2; 1 \text{ otherwise} \}$ Total cost of n TABLE-INSERT = $n + \sum_{j=0}^{\text{floor}(\lg(n-1))} 2^j < n + 2n = 3n$ - Amortized cost per insertion ≤ 3 .

Accounting Method: Stack

- Amortized costs: PUSH=2, POP=0, MULTIPOP=0.
- \$1 for actual push, \$1 stored as credit on the plate. POP uses stored credit.
- Total amortized = $O(n) \Rightarrow$ total actual = $O(n)$.

Accounting Method: Binary Counter

- Amortized cost to set a bit to 1 is \$2: \$1 for actual setting, \$1 stored as credit on the bit.
- Resetting a bit to 0 costs \$0 (paid by stored credit).
- Each INCREMENT sets at most one bit to 1 $\Rightarrow$ amortized cost $\leq$ \$2.

- Total for n operations = $O(n)$.

Accounting Method: Dynamic Table

- Each TABLE-INSERT charged \$3: \$1 for immediate insertion, \$1 on item as prepayment for reinsertion, \$1 on an existing item as prepayment for reinsertion.
- Total = $3n$ for n operations.

Potential Method: General Definition

$c_i^{\wedge} = c_i + \Phi(D_i) - \Phi(D_{i-1})$ $\sum_{i=1}^n c_i^{\wedge} = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$ - If $\Phi(D_n) \geq \Phi(D_0)$, $\sum c_i^{\wedge} \geq \sum c_i$ (upper bound).

Potential Method: Stack

$\Phi(D)$ = number of objects in stack - PUSH: $\Phi(D_i) - \Phi(D_{i-1}) = 1 \Rightarrow c_i^{\wedge} = 1 + 1 = 2$. - MULTIPOP(k): pops $k' = \min(s, k)$. Φ diff = $-k'$. $c_i^{\wedge} = k' - k' = 0$. - POP: $c_i^{\wedge} = 0$. - Total: $O(n)$.

Potential Method: Binary Counter

$\Phi(D_i) = b_i$ = number of 1-bits after i -th operation - For INCREMENT: t_i bits reset to 0, at most 1 set to 1. - $c_i \leq t_i + 1$. Φ diff $\leq (b_{i-1} - t_i + 1) - b_{i-1} = 1 - t_i$. - $c_i^{\wedge} \leq (t_i + 1) + (1 - t_i) = 2$. - Total actual cost for n operations: $\sum c_i = \sum c_i^{\wedge} + b_0 - b_n \leq 2n + b_0 \leq 2n + k$. - If $n = \Omega(k)$, total = $O(n)$.

Potential Method: Dynamic Table (insertions only)

$\Phi(T) = 2 * (T.\text{num} - T.\text{size}/2)$ - When $\alpha = 1/2$ (after expansion), $\Phi = 0$. - When $\alpha = 1$ (full), $\Phi = T.\text{size}$. - No expansion: $c_i = 1$, $\Delta \Phi = 2 \Rightarrow c_i^{\wedge} = 3$. - Expansion: $c_i = i$, $\Delta \Phi = 3 - i \Rightarrow c_i^{\wedge} = 3$. - Total amortized = $3n$ for n insertions.

Potential Method: Dynamic Table (insert and delete)

$\Phi(T) = \{ 2*(T.\text{num} - T.\text{size}/2) \text{ if } \alpha \geq 1/2; T.\text{size}/2 - T.\text{num} \text{ if } 1/4 \leq \alpha < 1/2 \}$ - When $\alpha \geq 1/2$: each insertion $\Delta \Phi = +2$ if no expansion; each deletion $\Delta \Phi = -2$ (if no contraction). - When $\alpha < 1/2$: each deletion $\Delta \Phi = +1$ (if no contraction); each insertion $\Delta \Phi = -1$. - All cases give constant amortized cost.

Aggregate Cost of Table-Insert (figure 16.3-16.4)

$c_i = i$ when $i-1$ is exact power of 2, else $c_i = 1$ Total $\leq n + (1+2+4+\dots+2^{\lfloor \lg(n-1) \rfloor}) = n + (2^{\lfloor \lg(n-1) \rfloor + 1} - 1) < 3n$

Rules, Laws & Theorems

Amortized Cost Upper Bound (Accounting)

- **Statement:** If total amortized cost $\geq$ total actual cost for all sequences of operations, then total amortized cost is an upper bound on total actual cost. Requires total credit (difference) to be nonnegative at all times.

Amortized Cost Upper Bound (Potential)

- **Statement:** The total amortized cost $\sum c_i = \sum c_i + \Phi(D_n) - \Phi(D_0)$. If $\Phi(D_n) \geq \Phi(D_0)$, then $\sum c_i \geq$ total actual cost, giving an upper bound.

Potential Method for Lower Bounds

- **Statement:** Potential functions can also prove lower bounds. For a problem, define $\Phi(\text{config})$. Then number of steps $\geq |\Phi_{\text{final}} - \Phi_{\text{init}}| / |\Delta \Phi_{\text{max}}|$.

Table Expansion/Contraction Thresholds

- **Statement:** Double size at load factor 1; halve size at load factor 1/4. This keeps load factor between 1/4 and 1, and guarantees $O(1)$ amortized cost per operation.
- **If halving at 1/2:** Thrashing occurs with $\Theta(n^2)$ total cost for n alternating insert/delete operations.

Edge Cases & Pitfalls

- **Amortized $\neq$ average-case:** Amortized analysis guarantees worst-case bound averaged over operations (no probability). Average-case analysis uses probability distributions.
- **Accounting method credit must never be negative:** Undercharging early operations makes total amortized cost invalid as an upper bound.
- **Binary counter with DECREMENT:** If DECREMENT is added as an operation, n operations can cost $\Theta(nk)$ time (Ex 16.1-2).
- **Stack with MULTIPUSH:** If MULTIPUSH(k) pushes k items onto stack, the $O(1)$ amortized bound fails because a single operation can push many items without corresponding pops (Ex 16.1-1).

- **Counter with RESET:** To achieve $O(n)$ for mixed INCREMENT and RESET operations, keep a pointer to the high-order 1 (Ex 16.2-3).
- **Table contraction at $1/2$:** Causes thrashing: expansion costs $\Theta(n)$, then contraction costs $\Theta(n)$, repeated $\Theta(n)$ times $\Rightarrow \Theta(n^2)$. Solution: contract at $1/4$.
- **Empty table:** Has size 0 and load factor defined as 1 (convention).
- **Credits in code?:** Assigned amortized costs are analysis-only, not stored in actual code.
- **Implementation-independent potential method:** Potential function applies to the data structure as a whole, not to specific objects. Different potential functions may yield different amortized costs.

Case Studies & Examples

Stack with MULTIPOP (Figure 16.1)

- **What:** Stack initially has objects. MULTIPOP(S,4) pops top 4. MULTIPOP(S,7) empties remaining stack.
- **Results:** Though MULTIPOP can cost $O(n)$ in worst case, any sequence of n operations costs $O(n)$ total (each popped object must have been pushed).
- **Significance:** Aggregate, accounting, and potential methods all yield $O(1)$ amortized cost per operation.

Binary Counter (Figure 16.2)

- **What:** 8-bit counter goes from 0 to 16. Shows flips per increment.
- **Results:** Bit $A[0]$ flips each time, $A[1]$ every other time, $A[2]$ every 4th time, etc. Total flips $< 2n$ for n increments.
- **Significance:** All three methods show $O(1)$ amortized cost per INCREMENT, though worst-case individual cost is $\Theta(k)$.

Dynamic Table – Insertions Only (Figures 16.3, 16.4)

- **What:** Table doubles when full. After expansion, items each have \$1 credit to pay for reinsertion at next expansion.
- **Results:** With potential function $\Phi = 2 \cdot (\text{num} - \text{size}/2)$, amortized cost of each insertion = 3.
- **Significance:** Total cost for n insertions $\leq 3n$, $O(1)$ amortized.

Dynamic Table – Insertions and Deletions (Figures 16.5, 16.6)

- **What:** Double at load factor 1, halve at load factor $1/4$.
- **Results:** Potential increases as load factor deviates from $1/2$, providing credit for expansion/contraction. Amortized cost = 3 for insertion, 2 for deletion (load factor crossing $1/2$), or 1 (contraction).

- **Significance:** Avoids thrashing that occurs with contraction at $1/2$.

Binary Reflected Gray Code (Problem 16-1)

- **What:** Sequence of integers $0..2^k-1$ where exactly one bit changes between consecutive values.
- **Construction:** For $k=1$: $\langle 0,1 \rangle$. For $k \geq 2$: take code for $k-1$, reflect it, add 2^{k-1} to reflected values, concatenate. E.g., $k=2$: $\langle 0,1,3,2 \rangle$ (binary $00,01,11,10$). $k=3$: $\langle 0,1,3,2,6,7,5,4 \rangle$.
- **Significance:** Determine which bit flips given index i . Can compute entire sequence in $\Theta(2^k)$ time.

Diagrams & Visuals

[MULTIPOP on stack: (a) initial stack, (b) after MULTIPOP($S,4$) – top 4 removed, (c) after MULTIPOP($S,7$) – emptied.]

[Binary counter flips: 8-bit counter $0 \rightarrow 16$, bits shown flipping per increment. Cost per increment shown at right; total cost $< 2n$.]

[Dynamic table expansion via accounting: \$3 per insertion. \$1 for immediate insertion, \$1 on new item as credit, \$1 on existing item as credit. When table full, each item has \$1 for reinsertion. After expansion, credit reset to 0.]

[Potential over insertions: num_i (brown line), size_i (blue line), Phi_i (red line). Potential builds up before each expansion, drops to 0 after expansion.]

[Potential for insert+delete: num_i , size_i , Phi_i . Load factor stays between $1/4$ and 1 . Potential builds as load factor deviates from $1/2$.]

End-of-Chapter Material

Key Terms

amortized analysis, aggregate analysis, accounting method, credit, potential method, potential function, MULTIPOP, binary counter, dynamic table, table expansion, table contraction, load factor, binary reflected Gray code

Review Questions (Exercises)

16.1-1: If MULTIPUSH(k) is included, does $O(1)$ amortized bound for stack hold?

16.1-2: Show DECREMENT in binary counter can cause $\Theta(nk)$ time for n operations.

16.1-3: Aggregate analysis: sequence of n operations, i -th costs i if i is exact power of 2, else 1. Determine amortized cost per operation.

16.2-1: Stack with backup copy every k operations. Show $O(n)$ cost by assigning amortized costs.

16.2-2: Redo 16.1-3 using accounting method.

16.2-3: Implement counter with INCREMENT and RESET using array of bits, $O(n)$ time for any sequence. Hint: keep pointer to high-order 1.

16.3-1: Given Φ with $\Phi(D_i) \geq \Phi(D_0) \neq 0$, construct Φ' with $\Phi'(D_0)=0$ and same amortized costs.

16.3-2: Redo 16.1-3 using potential method.

16.3-3: Binary min-heap with INSERT and EXTRACT-MIN. Give potential function so INSERT has $O(\lg n)$ amortized and EXTRACT-MIN has $O(1)$ amortized. n is current number of items.

16.3-4: Total cost of n PUSH, POP, MULTIPOP starting with s_0 objects, ending with s_n objects.

16.3-5: Implement queue with two stacks so ENQUEUE and DEQUEUE have $O(1)$ amortized cost.

16.3-6: Dynamic multiset with INSERT and DELETE-LARGER-HALF. Any sequence of m operations runs in $O(m)$ time. Also output elements in $O(|S|)$ time.

16.4-1: Analyze first table insertion using potential method.

16.4-2: Dynamic open-address hash table: why consider table full at $\alpha < 1$? How to make expected amortized cost $O(1)$?

16.4-3: Accounting method for insertion and deletion with doubling at load factor 1, halving at $1/4$.

16.4-4: Contract by multiplying size by $2/3$ when load factor $< 1/3$. Using $\Phi = |2*(\text{num} - \text{size}/2)|$, show amortized cost bounded by constant.

Problems

16-1 Binary reflected Gray code: (a) Given index i , determine which bit flips when going from $(i-1)$ st to i -th Gray code integer. (b) Show how to compute entire 2^k -length Gray code sequence in $\Theta(2^k)$ time given constant-time bit flip.

16-2 Making binary search dynamic: Maintain k sorted arrays $A_0..A_{k-1}$ where length of $A_i = 2^i$ and array is full or empty (binary representation of n). (a) SEARCH operation and worst-case

time. (b) INSERT operation, worst-case and amortized time. (c) DELETE operation, worst-case and amortized time.

16-3 Amortized weight-balanced trees: Node x is α -balanced if $\text{left.size} \leq \alpha \text{ x.size}$ and $\text{right.size} \leq \alpha \text{ x.size}$ ($1/2 \leq \alpha < 1$). (a) Rebuild subtree to be $1/2$ -balanced in $\Theta(\text{x.size})$ time. (b) Search in α -balanced tree takes $O(\lg n)$. (c)-(e) Potential method analysis: define $\Delta(x) = |\text{left.size} - \text{right.size}|$, $\Phi(T) = c * \sum_x \Delta(x)$. Show rebuilding takes $O(1)$ amortized time; INSERT/DELETE cost $O(\lg n)$ amortized.

16-4 Cost of restructuring red-black trees: (a) RB-INSERT can cause $\Omega(\lg n)$ color changes; RB-DELETE can also. (b) Identify terminating/nonterminating cases of RB-INSERT-FIXUP and RB-DELETE-FIXUP. (c)-(h) Define potential $\Phi(T)$ = number of red nodes (for insertions) or refined potential for insert+delete. Show each nonterminating case decreases potential by at least 1, so amortized structural modifications = $O(1)$ per operation. Any sequence of m operations causes $O(m)$ structural modifications.

Ch. 17 – Augmenting Data Structures

Named Entities (Terms & Definitions)

- **Order-statistic tree:** A red-black tree augmented with a size attribute in each node, where $x.\text{size}$ = number of internal nodes in the subtree rooted at x (including x , excluding sentinels). Used to support fast order-statistic operations.
- **size attribute** ($x.\text{size}$): Contains $x.\text{left.size} + x.\text{right.size} + 1$. The sentinel $T.\text{nil}$ has size 0.
- **Interval:** Represented as object i with attributes $i.\text{low}$ (low endpoint) and $i.\text{high}$ (high endpoint).
- **Interval trichotomy:** For any two closed intervals i and i' , exactly one holds: (a) they overlap ($i.\text{low} \leq i'.\text{high}$ and $i'.\text{low} \leq i.\text{high}$), (b) i is left of i' ($i.\text{high} < i'.\text{low}$), (c) i is right of i' ($i'.\text{high} < i.\text{low}$).
- **Interval tree:** A red-black tree where each node x contains an interval $x.\text{int}$. Key is the low endpoint $x.\text{int.low}$. Each node also stores $x.\text{max}$ = maximum value of any interval endpoint in the subtree rooted at x .
- **Overlap:** Intervals i and i' overlap if $i \cap i' \neq \emptyset$, i.e., $i.\text{low} \leq i'.\text{high}$ and $i'.\text{low} \leq i.\text{high}$.
- **Closed interval:** Includes both endpoints.
- **Augmenting a data structure:** Adding additional information to a standard data structure to support new operations efficiently.

Processes / Algorithms / Pathways

OS-SELECT(x, i)

- **Goal:** Returns pointer to node containing the i -th smallest key in the subtree rooted at x .
- **Input/Output:** Input: node x , integer i ($1 \leq i \leq \text{size of subtree}$). Output: pointer to node containing i -th smallest key.
- **Steps:** (1) Compute $r = x.\text{left.size} + 1$ (rank of x within subtree rooted at x). (2) If $i == r$, return x . (3) If $i < r$, return OS-SELECT($x.\text{left}$, i). (4) Else return OS-SELECT($x.\text{right}$, $i - r$).
- **Complexity:** Time $O(\lg n)$, Space $O(\text{height})$ due to recursion.
- **Example:** In Figure 17.1, OS-SELECT(root , 17): $\text{root key}=26$, $r=13$, go right with $i=4$; node $\text{key}=41$, $r=6$, go left with $i=4$; node $\text{key}=30$, $r=2$, go right with $i=2$; node $\text{key}=38$, $r=2$, return node $\text{key}=38$.

OS-RANK(T, x)

- **Goal:** Returns the rank (position in linear order) of node x in the order-statistic tree T .
- **Input/Output:** Input: tree T , node x . Output: integer rank of x in T .
- **Steps:** (1) $r = x.\text{left.size} + 1$. (2) $y = x$. (3) While $y \neq T.\text{root}$: if $y == y.p.\text{right}$, $r = r + y.p.\text{left.size} + 1$; $y = y.p.$ (4) Return r .
- **Loop invariant:** At start of each iteration, r is rank of $x.\text{key}$ in subtree rooted at y .
- **Complexity:** Time $O(\lg n)$, Space $O(1)$.
- **Example:** For node with key 38 in Figure 17.1: iterations: $y=38, r=2$; $y=30, r=4$; $y=41, r=4$; $y=26, r=17$. Returns 17.

Left-Rotate (augmented for size)

- **Goal:** Update size attributes during rotation in $O(1)$ time.
- **Steps:** (13) $y.\text{size} = x.\text{size}$; (14) $x.\text{size} = x.\text{left.size} + x.\text{right.size} + 1$.
- **Complexity:** $O(1)$ per rotation.

INTERVAL-SEARCH(T, i)

- **Goal:** Return pointer to a node whose interval overlaps i , or $T.\text{nil}$ if none exists.
- **Input/Output:** Input: interval tree T , interval i . Output: node x with $x.\text{int}$ overlapping i , or $T.\text{nil}$.
- **Steps:** (1) $x = T.\text{root}$. (2) While $x \neq T.\text{nil}$ and i does not overlap $x.\text{int}$: (3) If $x.\text{left} \neq T.\text{nil}$ and $x.\text{left.max} \geq i.\text{low}$ then $x = x.\text{left}$ (4) else $x = x.\text{right}$. (5) Return x .
- **Complexity:** Time $O(\lg n)$, Space $O(1)$.

- **Example (successful):** Search $i=[22,25]$ on Figure 17.4. Start root $[16,21]$, no overlap, $x.\text{left}.\text{max}=23 \geq 22$, go left to $[8,9]$. No overlap, $x.\text{left}.\text{max}=10 < 22$, go right to $[15,23]$. Overlaps i , return node.
- **Example (unsuccessful):** Search $i=[11,14]$. Root $[16,21]$, no overlap, $x.\text{left}.\text{max}=23 \geq 11$, go left to $[8,9]$. No overlap, $x.\text{left}.\text{max}=10 < 11$, go right to $[15,23]$. No overlap, $x.\text{left}=T.\text{nil}$, go right, $x=T.\text{nil}$, return $T.\text{nil}$.

Maintaining subtree sizes during insertion

- **Phase 1 (downward):** Increment $x.\text{size}$ for each node x on path from root to insertion point. $O(\lg n)$ time.
- **Phase 2 (rotations):** At most 2 rotations, each updating size in $O(1)$ time via augmented LEFT-ROTATE/RIGHT-ROTATE. Total $O(\lg n)$.

Maintaining subtree sizes during deletion

- **Phase 1:** Remove node, traverse path from lowest moved node up to root, decrementing size on each node. $O(\lg n)$.
- **Phase 2:** Handle $O(1)$ rotations as in insertion. Total $O(\lg n)$.

Data Structures

Order-Statistic Tree

- **Properties:** Red-black tree augmented with $x.\text{size}$ attribute. Keys need not be distinct. Rank defined by inorder walk position.
- **Operations:** OS-SELECT $O(\lg n)$, OS-RANK $O(\lg n)$, plus all standard red-black tree operations (INSERT, DELETE, SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR).
- **Complexities:** All operations $O(\lg n)$.

Interval Tree

- **Properties:** Red-black tree keyed by low endpoint. Each node x stores interval $x.\text{int}$ and $x.\text{max} = \max(x.\text{int}.\text{high}, x.\text{left}.\text{max}, x.\text{right}.\text{max})$.
- **Operations:** INTERVAL-INSERT $O(\lg n)$, INTERVAL-DELETE $O(\lg n)$, INTERVAL-SEARCH $O(\lg n)$.
- **Complexities:** All $O(\lg n)$.

Design Paradigm: Augmenting

- **Four-step methodology:** (1) Choose an underlying data structure. (2) Determine additional information to maintain. (3) Verify that the additional information can be maintained efficiently by basic modifying operations. (4) Develop new operations.

- **Theorem 17.1 (Augmenting a red-black tree):** Let f be an attribute that depends only on information in x , $x.left$, $x.right$ (including $x.left.f$ and $x.right.f$), computable in $O(1)$ time. Then insertion and deletion can maintain f values without asymptotically affecting $O(\lg n)$ running time.
- **Proof sketch:** A change to $x.f$ propagates only to ancestors of x ($O(\lg n)$ nodes). Insertion Phase 1: compute $x.f$ in $O(1)$, propagate up $O(\lg n)$. Phase 2: rotations affect $O(1)$ nodes, each requiring $O(\lg n)$ propagation worst-case, but at most 2 rotations total. Deletion analogous with at most 3 rotations.

Comparisons & Trade-offs

Dimension	Order-Statistic Tree (size attr)	Storing rank directly
OS-SELECT time	$O(\lg n)$	$O(\lg n)$
OS-RANK time	$O(\lg n)$	$O(1)$
Insert update cost	$O(\lg n)$	$O(n)$ (inserting min changes all ranks)
Delete update cost	$O(\lg n)$	$O(n)$

Rules, Laws & Theorems

Theorem 17.1 (Augmenting a Red-Black Tree)

- **Statement:** Let f be an attribute augmenting a red-black tree T of n nodes. Suppose f for each node x depends only on x , $x.left$, $x.right$ (possibly including $x.left.f$ and $x.right.f$), and $x.f$ is computable in $O(1)$ time. Then insertion and deletion can maintain f values without asymptotically affecting $O(\lg n)$ running time.
- **Proof sketch:** Changes to f propagate only up to ancestors. Insertion Phase 1: $O(\lg n)$ propagation. Phase 2: rotations affect $O(1)$ nodes, each propagated up in $O(\lg n)$. At most 2 rotations. Deletion: Phase 1 propagates $O(\lg n)$; Phase 2 at most 3 rotations.

Theorem 17.2

- **Statement:** Any execution of $\text{INTERVAL-SEARCH}(T, i)$ either returns a node whose interval overlaps i , or returns $T.nil$ and T contains no node whose interval overlaps i .
- **Proof sketch:** Two cases for direction. (1) If search goes right: $x.left = T.nil$ or $x.left.max < i.low$. Any interval i' in left subtree has $i'.high \leq x.left.max < i.low$, so no overlap by interval trichotomy. (2) If search goes left: left subtree nonempty and $x.left.max \geq i.low$. Either left subtree contains overlapping interval (done), or it doesn't. If not, some i' in left subtree has $i'.high = x.left.max \geq i.low$. Since i' does not overlap i , trichotomy gives $i.high <$

$i'.low$. Since $i'.low \leq x.int.low \leq \text{any } i''.low$ in right subtree, $i.high < i''.low$, so no overlap in right subtree.

Interval Trichotomy

- **Statement:** For any two closed intervals i and i' , exactly one of: (a) they overlap, (b) i is left of i' ($i.high < i'.low$), (c) i is right of i' ($i'.high < i.low$).

Formulas & Equations

Subtree size identity

$x.size = x.left.size + x.right.size + 1$ (with $T.nil.size = 0$)

Rank of x within subtree rooted at x

$r = x.left.size + 1$

Max attribute update

$x.max = \max\{x.int.high, x.left.max, x.right.max\}$

Edge Cases & Pitfalls

- When equal keys exist, rank is defined as the position in an inorder walk (not uniquely determined by key value alone).
- Sentinel $T.nil$ must have $size=0$ for the identity to work.
- Theorem 17.1 requires that rotations are constant-bounded. Balanced-tree schemes with $\Theta(\lg n)$ rotations per operation could degrade to $\Theta(\lg^2 n)$ if each rotation traverses to root.
- INTERVAL-SEARCH returns a single overlapping interval, not all.
- Interval tree is keyed on low endpoint; this property is crucial for correctness proof of INTERVAL-SEARCH.

Case Studies & Examples

- **Order-statistic tree:** Figure 17.1 shows a tree with keys 26 (root, $size=20$), various sizes computed as subtree size.
- **Rotation update:** Figure 17.2 shows that updating size during LEFT-ROTATE is local (only x and y change).
- **Interval tree:** Figure 17.4 shows 10 intervals with low/high endpoints and max values.
- **Interval search examples:** Successful search $i=[22,25]$ finds $[15,23]$; unsuccessful search $i=[11,14]$ returns $T.nil$.

End-of-Chapter Material

Key terms: order-statistic tree, size attribute, interval trichotomy, interval tree, augmenting, max attribute.

Exercises: - **17.1-1:** Show OS-SELECT($T.root$, 10) on Figure 17.1. - **17.1-2:** Show OS-RANK(T , x) with $x.key=35$ on Figure 17.1. - **17.1-3:** Write nonrecursive OS-SELECT. - **17.1-4:** Write OS-KEY-RANK(T , k) returning rank of key k (distinct keys assumed). - **17.1-5:** Find i -th successor of x in $O(\lg n)$ time by combining OS-RANK and OS-SELECT. - **17.1-6:** Maintain per-node rank instead of size; show how to handle rotations. - **17.1-7:** Count inversions in $O(n \lg n)$ using order-statistic tree: insert elements right-to-left, use OS-RANK to count smaller elements already in tree. - **17.1-8:** $O(n \lg n)$ algorithm to count intersecting chord pairs on a circle using order-statistic tree. - **17.2-1:** Add pointers for MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR in $O(1)$ on augmented order-statistic tree. - **17.2-2:** Black-heights can be maintained; depths cannot without affecting asymptotic performance (updating all descendants). - **17.2-3:** For associative binary operator $\square$, $x.f = x \square .a \square \dots \square x \square .a$ (inorder), update f in $O(1)$ after rotation. Applies to size attribute. - **17.3-1:** Write LEFT-ROTATE for interval tree updating max in $O(1)$. - **17.3-2:** Efficient algorithm returning interval overlapping i with minimum low endpoint (or $T.nil$). - **17.3-3:** List all overlapping intervals in $O(\min\{n, k \lg n\})$ time, k = number in output. - **17.3-4:** Modify for INTERVAL-SEARCH-EXACTLY returning node with matching low and high, $O(\lg n)$. - **17.3-5:** Maintain MIN-GAP on dynamic set using augmented red-black tree, with $O(\lg n)$ operations. - **17.3-6:** $O(n \lg n)$ algorithm to detect overlapping rectangles using sweep line and interval tree.

Problems: - **17-1 Point of Maximum Overlap:** (a) Max overlap point is always an endpoint. (b) Data structure using red-black tree of endpoints with $+1/-1$ values, augmented to track max overlap. - **17-2 Josephus Permutation:** (a) $O(n)$ algorithm when m is constant using circular linked list. (b) $O(n \lg n)$ algorithm using order-statistic tree: repeatedly select and delete (i -th remaining element).

Ch. 18 – B-Trees

Named Entities (Terms & Definitions)

- **B-tree:** A balanced search tree designed for disk storage. Generalizes binary search trees; nodes may have many children. Every n -node B-tree has height $O(\lg n)$, but with larger base (t), giving lower height than red-black trees.
- **Minimum degree t :** Fixed integer $t \geq 2$. Every node (except root) has at least $t-1$ keys and at most $2t-1$ keys. Root has at least 1 key.
- **Full node:** A node containing exactly $2t-1$ keys.
- **Disk block:** Unit of disk I/O. B-tree nodes typically sized to match disk blocks.
- **Platter:** Rotating magnetic surface in a disk drive.
- **Head:** Read/write element at end of an arm.
- **Track:** Surface passing under stationary head.

- **Latency**: Time waiting for mechanical movements (rotation + arm movement).
- **2-3-4 tree**: B-tree with $t=2$. Internal nodes have 2, 3, or 4 children.
- **B+-tree**: Variant storing satellite info only in leaves; internal nodes store keys and child pointers only.
- ****B*-tree****: Variant requiring each internal node to be at least $2/3$ full.
- **DISK-READ(x)**: Reads block containing object x into main memory (no-op if already in memory).
- **DISK-WRITE(x)**: Writes block containing x to disk.
- **Split**: Operation dividing a full node ($2t-1$ keys) around its median key into two nodes of $t-1$ keys each. Median key moves to parent.
- **Merge**: Combining a key and two sibling nodes (each with $t-1$ keys) into one node with $2t-1$ keys (used in deletion).

Processes / Algorithms / Pathways

B-TREE-SEARCH(x, k)

- **Goal**: Search for key k in subtree rooted at x .
- **Input/Output**: Input: node x , key k . Output: ordered pair (y, i) with $y.key_i = k$, or NIL if not found.
- **Steps**: (1) $i = 1$. (2) While $i \leq x.n$ and $k > x.key_i$: $i = i + 1$. (3) If $i \leq x.n$ and $k == x.key_i$: return (x, i) . (4) Else if $x.leaf$: return NIL. (5) Else DISK-READ($x.ci$), return B-TREE-SEARCH($x.ci, k$).
- **Complexity**: Disk accesses $O(h) = O(\log_t n)$. CPU time $O(t h) = O(t \log_t n)$.

B-TREE-CREATE(T)

- **Goal**: Create an empty B-tree.
- **Steps**: (1) $x = \text{ALLOCATE-NODE}()$. (2) $x.leaf = \text{TRUE}$. (3) $x.n = 0$. (4) DISK-WRITE(x). (5) $T.root = x$.
- **Complexity**: $O(1)$ disk operations, $O(1)$ CPU time.

B-TREE-SPLIT-CHILD(x, i)

- **Goal**: Split full child $y = x.ci$ (which has $2t-1$ keys) into two nodes, moving median key up to x .
- **Input/Output**: Input: nonfull internal node x , index i such that $x.ci$ is full. Output: modified x with extra child, y reduced to $t-1$ keys, new node z created.
- **Steps**: (1) $y = x.ci$. (2) $z = \text{ALLOCATE-NODE}()$, $z.leaf = y.leaf$, $z.n = t-1$. (3) Copy y 's largest $t-1$ keys ($y.key[j+t]$ for $j=1..t-1$) to z . (4) If not $y.leaf$, copy corresponding t children ($y.c[j+t]$ for $j=1..t$) to z . (5) $y.n = t-1$. (6) Shift x 's children right to make room ($x.c[j+1] = x.c[j]$ for $j = x.n+1$ down to $i+1$). (7) $x.ci+1 = z$. (8) Shift x 's keys right ($x.key[j+1] = x.key[j]$ for $j = x.n$ down to i). (9) $x.key_i =$

y.keyt. (10) $x.n = x.n + 1$. (11) DISK-WRITE(y), DISK-WRITE(z), DISK-WRITE(x).

- **Complexity:** CPU time $\Theta(t)$, disk operations $O(1)$.

B-TREE-SPLIT-ROOT(T)

- **Goal:** Split the root when it is full. Tree height increases by 1.
- **Steps:** (1) $s = \text{ALLOCATE-NODE}()$, $s.\text{leaf} = \text{FALSE}$, $s.n = 0$, $s.ci = T.\text{root}$. (2) $T.\text{root} = s$. (3) B-TREE-SPLIT-CHILD($s, 1$). (4) Return s .
- **Complexity:** $O(1)$ disk ops, $\Theta(t)$ CPU.

B-TREE-INSERT(T, k)

- **Goal:** Insert key k into B-tree T in a single pass down the tree.
- **Steps:** (1) $r = T.\text{root}$. (2) If r is full ($r.n == 2t-1$): $s = \text{B-TREE-SPLIT-ROOT}(T)$, B-TREE-INSERT-NONFULL(s, k). (3) Else B-TREE-INSERT-NONFULL(r, k).
- **Complexity:** Disk accesses $O(h)$, CPU time $O(t h) = O(t \log_t n)$.

B-TREE-INSERT-NONFULL(x, k)

- **Goal:** Insert key k into nonfull node x , recursing if needed.
- **Steps:** (1) $i = x.n$. (2) If $x.\text{leaf}$: shift keys $> k$ right, insert k , increment $x.n$, DISK-WRITE(x). (3) Else: find child index i where k belongs, DISK-READ($x.ci$). (4) If $x.ci$ is full: B-TREE-SPLIT-CHILD(x, i); if $k > x.\text{key}_i$, $i = i+1$. (5) B-TREE-INSERT-NONFULL($x.ci, k$).
- **Complexity:** Disk $O(h)$, CPU $O(t h)$.

B-TREE-DELETE(T, k)

- **Goal:** Delete key k from B-tree T in a single downward pass (no backing up). Combines search and deletion. Guarantees each non-root node visited has at least t keys (not just $t-1$) at the time of visitation.
- **Cases:**
 - **Case 1** (leaf): If x contains k , delete k from x . If x does not contain k , key not in tree.
 - **Case 2** (internal node containing $k = x.\text{key}_i$):
 - **Case 2a:** child $x.ci$ has $\geq t$ keys. Find predecessor k' in subtree rooted at $x.ci$, recursively delete k' , replace k with k' in x .
 - **Case 2b:** $x.ci$ has $t-1$ keys but $x.ci+1$ has $\geq t$ keys. Symmetric: replace k with successor k' from $x.ci+1$.
 - **Case 2c:** Both $x.ci$ and $x.ci+1$ have $t-1$ keys. Merge k and $x.ci+1$ into $x.ci$ (now $2t-1$ keys), free $x.ci+1$, recursively delete k from $x.ci$.

- **Case 3** (internal node not containing k): Determine child $x.ci$ where k would be. If $x.ci$ has only $t-1$ keys, ensure it gets $\geq t$ keys before recursing:
 - **Case 3a**: $x.ci$ has $t-1$ keys but an immediate sibling has $\geq t$ keys. Move key from x down to $x.ci$, move key from sibling up to x , move appropriate child pointer from sibling to $x.ci$.
 - **Case 3b**: $x.ci$ and both siblings have $t-1$ keys. Merge $x.ci$ with one sibling, moving a key from x down to become median key.
- If root ends up with 0 keys (cases 2c, 3b), delete root and its only child becomes new root; height decreases by 1.
- **Complexity**: Disk $O(h)$, CPU $O(t h) = O(t \log_t n)$.

Data Structures

B-Tree Node

- **Properties**: $x.n$ (number of keys), $x.key_1, \dots, x.key_{x.n}$ (monotonically increasing), $x.leaf$ (bool), $x.c_1, \dots, x.c_{x.n+1}$ (child pointers, internal nodes only). All leaves at same depth.
- **Bounds**: $t-1 \leq x.n \leq 2t-1$ (root: $1 \leq x.n \leq 2t-1$). Minimum degree $t \geq 2$.
- **Disk layout**: Node typically as large as a disk block.

B-Tree (B-tree definition)

- **Properties**: Rooted tree with root $T.root$. Properties: (1) Every node has $x.n$, $x.key_i$ sorted, $x.leaf$. (2) Internal nodes have $x.n+1$ child pointers. (3) Keys separate subtree ranges: $k_1 \leq key_1 \leq k_2 \leq key_2 \leq \dots \leq key_{x.n} \leq k_{x.n+1}$. (4) All leaves same depth. (5) Bounds by t . Height $h \leq \log_t((n+1)/2)$.
- **Operations**: SEARCH, INSERT, DELETE, plus standard dynamic-set operations.
- **Complexities**: Height $O(\log_t n)$. Disk accesses $O(\log_t n)$. CPU time $O(t \log_t n)$.

Formulas & Equations

Height bound

$$h \leq \log_t ((n+1)/2)$$

Proof: Root has ≥ 1 key; all other nodes have $\geq t-1$ keys. At depth 1: ≥ 2 nodes. At depth 2: $\geq 2t$ nodes. At depth h : $\geq 2t^{h-1}$ nodes. Total keys $n \geq 1 + (t-1) * (2 + 2t + 2t^2 + \dots + 2t^{h-1}) = 1 + 2(t-1)(t^h - 1)/(t - 1) = 2t^h - 1$. So $t^h \leq (n+1)/2$, hence $h \leq \log_t((n+1)/2)$.

Comparisons & Trade-offs

Dimension	B-tree	Red-black tree
Height	$O(\log_t n)$	$O(\lg n)$
Base of log	$t (\geq 2, \text{ may be } 50\text{-}2000)$	2
Branching factor	Many children (t to $2t$)	2
Disk accesses	$O(\log_t n)$ – lower	$O(\lg n)$ – higher
Node size	Large (disk block sized)	Small
Main use	Disk-based storage	In-memory

Edge Cases & Pitfalls

- $t = 1$ is not allowed because then $t-1 = 0$ (node could have 0 keys).
- Root is allowed to have fewer than $t-1$ keys (as few as 1).
- Splitting is the only way the tree grows in height (height increases at top, not bottom).
- After case 2c and 3b deletion, the root may become empty (0 keys) and is deleted, reducing height.
- B-TREE-INSERT splits every full node encountered while going down, ensuring parent is never full when a split is needed.
- B-TREE-DELETE ensures each non-root node visited has at least t keys (not $t-1$) to avoid needing to back up.
- The procedures assume root is always in main memory (no DISK-READ on root needed).
- B-trees are balanced search trees, not binary; the branching decision is $(x.n+1)$ -way.

Case Studies & Examples

- **Figure 18.1:** B-tree of English consonants; search for R examines nodes along path.
- **Figure 18.3:** B-tree with $t=1001$, height 2, storing >1 billion keys. Only 2 disk accesses needed (root in memory).
- **Figure 18.5:** Splitting a node ($t=4$). $y=x.ci$ splits into y and z ; median key S moves to parent.
- **Figure 18.6:** Splitting root ($t=4$). Root r splits into two; new root s created; height increases by 1.
- **Figure 18.7:** Insertion sequence ($t=3$, max 5 keys per node): (a) initial tree; (b) insert B – simple leaf insert; (c) insert Q – splits RSTUV; (d) insert L – splits full root, height grows; (e) insert F – splits ABCDE first.
- **Figure 18.8:** Deletion examples ($t=3$): (b) delete F – case 1 (leaf); (c) delete M – case 2a (predecessor L replaces M); (d) delete G – case 2c (push G down, merge, delete from leaf); (e) delete D – case 3b (merge), root becomes empty, height shrinks; (f) delete B – case 3a (borrow from sibling).

End-of-Chapter Material

Exercises: - **18.1-1:** $t=1$ not allowed because nodes could have 0 keys. - **18.1-2:** Figure 18.1 must satisfy B-tree properties for some t . - **18.1-3:** Show all legal B-trees of minimum degree 2 storing keys 1-5. - **18.1-4:** Max keys in B-tree of height h with min degree t : $2t^{h+1} - 1$. - **18.1-5:** Red-black tree with each black node absorbing red children gives a 2-3-4 tree. - **18.2-1:** Insert sequence F, S, Q, K, C, L, H, T, V, W, M, R, N, P, A, B, X, Y, D, Z, E into B-tree with $t=2$. Show configurations before splits and final. - **18.2-2:** Redundant DISK-READ/WRITE conditions. - **18.2-3:** Prove for $t=2$, keys 1-15, no insertion sequence achieves minimum possible height. - **18.2-4:** For keys $1..n$ inserted into B-tree with $t=2$, count final number of nodes. - **18.2-5:** Different t for leaf vs internal nodes – modify create/insert. - **18.2-6:** Binary search within node makes CPU time $O(\lg n)$ independent of t . - **18.2-7:** Choose t to minimize $a + bt$ given disk read time, with optimal t analysis. - **18.3-1:** Delete C, P, V in order from Figure 18.8(f). - **18.3-2:** Write pseudocode for B-TREE-DELETE.

Problems: - **18-1 Stacks on Secondary Storage:** (a) $O(n)$ disk accesses, $O(nm)$ CPU for simple implementation. (b) One block in memory: worst-case $O(n)$ disk accesses for n PUSHes, $O(nm)$ CPU. (c) Same for mixed operations. (d) Two blocks: amortized $O(1/m)$ disk accesses and $O(1)$ CPU per operation. - **18-2 Joining and Splitting 2-3-4 Trees:** (a) Maintain x .height attribute. (b) Join T' and T'' with key k in $O(1 + |h' - h''|)$ time. (c) Path from root to key breaks S' into trees and keys with decreasing heights. (d) Split in $O(\lg n)$ using telescoping join costs.

Ch. 19 – Data Structures for Disjoint Sets

Named Entities (Terms & Definitions)

- **Disjoint-set data structure:** Maintains a collection $S = \{S_1, S_2, \dots, S_k\}$ of disjoint dynamic sets. Supports MAKE-SET, UNION, FIND-SET.
- **Representative:** A distinguished member of each set. Used to identify the set.
- **MAKE-SET(x):** Creates a new singleton set $\{x\}$, with x as representative. x must not already belong to another set.
- **UNION(x, y):** Unites the sets containing x and y into one set. Destroys the original two sets.
- **FIND-SET(x):** Returns pointer to representative of the unique set containing x .
- **n :** Number of MAKE-SET operations.
- **m :** Total number of MAKE-SET, UNION, and FIND-SET operations ($m \geq n$).
- **Weighted-union heuristic:** Always append the shorter list to the longer list during UNION on linked-list representation.
- **Union by rank:** Make root with smaller rank point to root with larger rank. Rank is upper bound on node height.

- **Path compression:** During FIND-SET, make each node on the find path point directly to the root.
- **Disjoint-set forest:** Tree representation where each set is a rooted tree, each node points to its parent, root is representative.
- **Find path:** The simple path from a node to its root during FIND-SET.
- **Rank:** Integer $x.rank$ for each node, upper bound on its height (number of edges in longest simple path from descendant leaf to x). Initialized to 0.
- **Level of function A_k :** Parameter k in the Ackermann-like function.
- **$\alpha(n)$:** Inverse Ackermann function; lowest level k such that $A_k(1) \geq n$. $\alpha(n) \leq 4$ for all practical values of n .
- **Functional iteration:** $f^{\{i\}}(n)$ defined as applying f repeatedly: $f^{\{1\}}(n) = f(n)$, $f^{\{i\}}(n) = f(f^{\{i-1\}}(n))$.
- **Potential function $\phi_q(x)$:** Used for amortized analysis. For root or rank 0: $\alpha(n) \cdot x.rank$. For nonroot with rank ≥ 1 : $(\alpha(n) - level(x)) \cdot x.rank - iter(x)$.
- **level(x):** Greatest level k for which $A_k(x.rank) \leq x.p.rank$. Bounded by $0 \leq level(x) < \alpha(n)$.
- **iter(x):** Largest number of times to iteratively apply $A_{level(x)}$ to $x.rank$ before exceeding $x.p.rank$. Bounded by $1 \leq iter(x) \leq x.rank$.
- **Ackermann function (variant used):** $A_k(j)$ defined: $A_0(j) = j + 1$; $A_k^{\{1\}}(j) = A_{k-1}^{\{j+1\}}(j)$ for $k \geq 1$; $A_k(j) = A_k^{\{1\}}(j)$.
- **LINK(x, y):** Subroutine called by UNION; takes two roots and links them based on ranks.

Processes / Algorithms / Pathways

MAKE-SET(x) – Forest with union by rank

- **Goal:** Create a new singleton set.
- **Steps:** (1) $x.p = x$. (2) $x.rank = 0$.
- **Complexity:** $O(1)$ time.

FIND-SET(x) – With path compression

- **Goal:** Return representative (root) of the set containing x . Compresses path.
- **Steps:** (1) If $x \neq x.p$: $x.p = \text{FIND-SET}(x.p)$. (2) Return $x.p$.
- **Properties:** Two-pass method: first pass finds root (going up), second pass (unwinding) updates each node to point directly to root.
- **Complexity:** Nearly $O(\alpha(n))$ amortized per operation.

UNION(x, y) – With union by rank

- **Goal:** Unite sets containing x and y .
- **Steps:** (1) $\text{LINK}(\text{FIND-SET}(x), \text{FIND-SET}(y))$.

- **Complexity:** Amortized $O(\alpha(n))$.

LINK(x, y) – Union by rank

- **Goal:** Link root of one tree to another based on ranks.
- **Steps:** (1) If $x.rank > y.rank$: $y.p = x$. (2) Else: $x.p = y$. (3) If $x.rank == y.rank$: $y.rank = y.rank + 1$.
- **Complexity:** $O(1)$ actual time.

CONNECTED-COMPONENTS(G)

- **Goal:** Compute connected components of undirected graph G.
- **Steps:** (1) For each vertex v in $G.V$: MAKE-SET(v). (2) For each edge (u, v) in $G.E$: if FIND-SET(u) $\neq$ FIND-SET(v): UNION(u, v).
- **Complexity:** $O((V+E) \alpha(V))$ when using forest with union by rank and path compression.

SAME-COMPONENT(u, v)

- **Goal:** Test if two vertices are in the same connected component.
- **Steps:** (1) Return TRUE if FIND-SET(u) == FIND-SET(v), else FALSE.

Linked-list UNION with weighted-union heuristic

- **Goal:** Append shorter list to longer list. Maintain list length.
- **Steps:** (1) Determine which list is shorter. (2) Append shorter list to longer list. (3) Update set pointer for each element of appended list. (4) Update tail and length of resulting list. (5) Destroy set object of appended list.
- **Complexity:** $O(n \lg n)$ total across all UNION operations for sequence of m operations, n MAKE-SET.

Data Structures

Linked-List Disjoint Set

- **Properties:** Each set = linked list. Set object has head, tail pointers. Each list object has set member, next pointer, pointer back to set object. Representative = first element of list. Objects may appear in any order.
- **Operations:** MAKE-SET $O(1)$, FIND-SET $O(1)$, UNION $O(\text{length of appended list})$ without heuristic, $O(n \lg n)$ total with weighted-union heuristic.
- **Complexities with weighted-union:** $O(m + n \lg n)$ for sequence of m operations, n MAKE-SET.

Disjoint-Set Forest

- **Properties:** Each set = rooted tree. Each node x : $x.p$ (parent), $x.rank$ (upper bound on height). Root is its own parent and serves as representative.
- **Operations:** MAKE-SET $O(1)$, FIND-SET nearly $O(1)$ amortized, UNION nearly $O(1)$ amortized.
- **Complexities:** $O(m \alpha(n))$ for sequence of m operations with union by rank + path compression.

Comparisons & Trade-offs

Dimension	Linked-list (simple)	Linked-list (weighted union)	Forest (union by rank + path compression)
MAKE-SET	$O(1)$	$O(1)$	$O(1)$
FIND-SET	$O(1)$	$O(1)$	$O(\alpha(n))$ amortized
UNION	$O(n)$ worst	$O(\lg n)$ amortized	$O(\alpha(n))$ amortized
Sequence of m ops	$O(m + n^2)$	$O(m + n \lg n)$	$O(m \alpha(n))$

Formulas & Equations

$A_k(j)$ – Ackermann-like function

For integers $j, k \geq 0$: - $A_0(j) = j + 1$ - $A_k^{(1)}(j) = A_{k-1}^{(j+1)}(j)$ for $k \geq 1$ (using functional iteration $A_{k-1}^{(j+1)}(j)$) - $A_k(j) = A_k^{(1)}(j)$

Closed forms: - $A_1(j) = 2j + 1$ (Lemma 19.2) - $A_2(j) = 2^{j+1}(j+1) - 1$ (Lemma 19.3)

Values: - $A_0(1) = 2$ - $A_1(1) = 3$ - $A_2(1) = 7$ - $A_3(1) = 2047$ - $A_4(1) = 2^{2059} - 1 > 16^{514} \gg 10^{80}$ (atoms in observable universe)

$\alpha(n)$ – Inverse Ackermann

$$\alpha(n) = \min\{k : A_k(1) \geq n\}$$

$\alpha(n) \leq 4$ for all practical values of n . Only when n exceeds $A_4(1)$ (astronomically huge) does $\alpha(n) > 4$.

Potential function

For node x after q operations: - If x is root or $x.rank = 0$: $\Phi_q(x) = \alpha(n) \cdot x.rank$ - If x is not root and $x.rank \geq 1$: $\Phi_q(x) = (\alpha(n) - level(x)) \cdot x.rank - iter(x) - level(x) = \max\{k : A_k(x.rank) \leq x.p.rank\}$ with $0 \leq level(x) < \alpha(n) - iter(x) =$

$\max\{i : A_{\text{level}(x)}^{\{i\}}(x.\text{rank}) \leq x.p.\text{rank}\}$ with $1 \leq \text{iter}(x) \leq x.\text{rank}$

Total potential: $\Phi_q = \sum_x \phi_q(x)$

Rules, Laws & Theorems

Lemma 19.2

- **Statement:** For any integer $j \geq 1$, $A^{\{j\}}(j) = 2j + 1$.
- **Proof:** Induction on i to show $A^{\{i\}}(j) = j + i$. Then $A^{\{j\}}(j) = A^{\{1\}}(j) = A^{\{j+1\}}(j) = j + (j+1) = 2j + 1$.

Lemma 19.3

- **Statement:** For any integer $j \geq 1$, $A^{\{j\}}(j) = 2^{j+1}(j+1) - 1$.
- **Proof:** Induction to show $A^{\{i\}}(j) = 2^{i+1}(j+1) - 1$. Then $A^{\{j\}}(j) = A^{\{1\}}(j) = A^{\{j+1\}}(j) = 2^{j+1}(j+1) - 1$.

Lemma 19.4

- **Statement:** For all nodes x : $x.\text{rank} \leq x.p.\text{rank}$, strict if $x \neq x.p$. $x.\text{rank}$ initially 0, increases until x becomes nonroot, then never changes. $x.p.\text{rank}$ monotonically increases.
- **Proof:** Induction on number of operations.

Corollary 19.5

- **Statement:** On the simple path from any node up to its root, node ranks strictly increase.

Lemma 19.6

- **Statement:** Every node has rank at most $n - 1$.
- **Proof:** At most $n-1$ LINK operations, each either leaves ranks alone or increments some node's rank by 1.

Lemma 19.7

- **Statement:** Converting each UNION into two FIND-SET + one LINK yields $m = \Theta(m')$, preserving asymptotic time bound $O(m \alpha(n))$.

Lemma 19.8

- **Statement:** For every node x and all q : $0 \leq \phi_q(x) \leq \alpha(n) \cdot x.\text{rank}$.

Corollary 19.9

- **Statement:** If x is not a root and $x.\text{rank} > 0$, then $\phi_q(x) < \alpha(n) \cdot x.\text{rank}$.

Lemma 19.10

- **Statement:** For nonroot x , after LINK or FIND-SET: $\phi_q(x) \leq \phi_{q-1}(x)$. If $x.\text{rank} \geq 1$ and $\text{level}(x)$ or $\text{iter}(x)$ changes, $\phi_q(x) \leq \phi_{q-1}(x) - 1$.
- **Proof:** $x.\text{rank}$ unchanged, $\alpha(n)$ unchanged. If level unchanged and iter increases, potential drops by ≥ 1 . If level increases, $(\alpha(n) - \text{level}) \cdot x.\text{rank}$ term drops by $\geq x.\text{rank}$ while iter drop is at most $x.\text{rank} - 1$, net drop ≥ 1 .

Lemma 19.11

- **Statement:** Amortized cost of MAKE-SET is $O(1)$. (New node has rank 0, potential 0.)

Lemma 19.12

- **Statement:** Amortized cost of LINK is $O(\alpha(n))$.
- **Proof:** Actual $O(1)$. Only y 's potential can increase (by at most $\alpha(n)$). Children's potentials don't increase. x 's potential decreases or stays. So amortized = $O(1) + \alpha(n) = O(\alpha(n))$.

Lemma 19.13

- **Statement:** Amortized cost of FIND-SET is $O(\alpha(n))$.
- **Proof:** Actual cost $O(s)$ where s is find path length. At most $\alpha(n)+2$ nodes on path don't decrease potential. At least $s - (\alpha(n)+2)$ nodes decrease potential by ≥ 1 . So amortized $\leq O(s) - (s - (\alpha(n)+2)) = O(\alpha(n))$.

Theorem 19.1 (Linked-list with weighted union)

- **Statement:** Sequence of m MAKE-SET, UNION, FIND-SET operations, n MAKE-SET, takes $O(m + n \lg n)$ time.
- **Proof:** Each object's pointer updated at most $\lg n$ times (set size doubles each time). Total $O(n \lg n)$ for pointer updates. Each UNION $O(1)$ for tail/length. MAKE-SET and FIND-SET $O(1)$ each.

Theorem 19.14 (Forest with union by rank + path compression)

- **Statement:** A sequence of m MAKE-SET, UNION, and FIND-SET operations, n of which are MAKE-SET, on a disjoint-set forest with union by rank and path compression runs in $O(m \alpha(n))$ time.
- **Proof:** From Lemmas 19.7, 19.11, 19.12, 19.13.

Edge Cases & Pitfalls

- After $n-1$ UNION operations, only one set remains; at most $n-1$ UNION operations can occur.
- The first n operations must be MAKE-SET operations.
- In FIND-SET, path compression does not change any ranks – ranks are upper bounds, not exact heights.
- Union by rank alone gives $O(m \lg n)$ bound, which is tight (Exercise 19.3-3 gives $\Omega(m \lg n)$ sequence).
- Path compression alone (without union by rank) gives $\Theta(n + f \cdot (1 + \log_{2+f/n} n))$ worst-case time.
- The potential function analysis uses $\alpha(n)$ which is constant (≤ 4) for all practical n , but the asymptotic bound is technically superlinear.
- For linked-list with simple UNION (no weighted union), sequence of $2n-1$ operations takes $\Theta(n^2)$ time – amortized $\Theta(n)$ per operation.
- In the linked-list representation, when lists are appended, the set object of the appended list is destroyed. Each element of the appended list has its pointer to the set object updated.
- Rank can be stored in $\lceil \lg \lg n \rceil$ bits (since $\text{rank} \leq \lceil \lg n \rceil$ per Exercise 19.4-2/19.4-3).

Case Studies & Examples

- **Connected components:** Graph with vertices $\{a..j\}$ and edges processed in specific order (Figure 19.1). After processing all edges, 4 connected components remain.
- **Linked-list weighted union:** Figure 19.2 – UNION(g, e) appends list $\{b, c, e, h\}$ to list $\{d, f, g\}$. Representative f . Four pointers updated.
- **Bad sequence for simple UNION:** Figure 19.3 – Sequence of MAKE-SET(x_1)...MAKE-SET(x_n) then UNION(x_1, x_2), UNION(x_2, x_3), ..., UNION(x_{n-1}, x_n). Total updates: $\sum_{i=1}^{n-1} i = \Theta(n^2)$.
- **Forest with union by rank:** Figure 19.4 – Two trees with roots c and f ; UNION(e, g) makes root f point to root c (or vice versa based on ranks).
- **Path compression:** Figure 19.5 – FIND-SET(a) compresses path so $\{a, b, c\}$ all point directly to root.

End-of-Chapter Material

Exercises: - **19.1-1:** Compute connected components after each edge in given sequence. - **19.1-2:** Prove two vertices belong to same connected component iff they belong to same set after CONNECTED-COMPONENTS. - **19.1-3:** FIND-SET called $2|E|$ times; UNION called $|V| - k$ times (where k = number of connected components). - **19.2-1:** Write pseudocode for linked-list with weighted-union heuristic. - **19.2-2:** Show data structure and FIND-SET results for given sequence of 16 elements. - **19.2-3:** Adapt aggregate proof to

get $O(1)$ for MAKE-SET/FIND-SET, $O(\lg n)$ for UNION. - **19.2-4:** Tight bound for Figure 19.3 with weighted-union heuristic: $O(n \lg n)$. - **19.2-5:** Single pointer in set object: use tail as representative (circular list approach). - **19.2-6:** Remove tail pointer by splicing (circular linked list) rather than appending. - **19.3-1:** Redo 19.2-2 using forest with union by rank and path compression. - **19.3-2:** Write nonrecursive FIND-SET with path compression (two-pass iterative). - **19.3-3:** Sequence showing $\Omega(m \lg n)$ with union by rank only: create tall trees by alternating UNIONS. - **19.3-4:** Add circular linked list member to each node for PRINT-SET in $O(\text{set size})$. - **19.3-5:** Sequence where all LINKS precede FIND-SETs runs in $O(m)$ with union by rank + path compression; without union by rank, still $O(m)$ because all LINKS create no path structure that later FIND-SETs cannot compress efficiently. - **19.4-1:** Prove Lemma 19.4 by induction. - **19.4-2:** Prove every node has rank $\leq \lg n$. - **19.4-3:** $\lg \lg n$ bits needed for $x.\text{rank}$. - **19.4-4:** Union by rank only: $O(m \lg n)$ using rank $\leq \lg n$ bound. - **19.4-5:** Is Professor Dante correct that $\text{level}(x) \leq \text{level}(x.p)$? Not always; need to check definition. - **19.4-6:** Scale potential function by constant c to handle hidden constant in $O(s)$. - **19.4-7:** $\alpha'(n) = \min\{k: A_k(1) \geq \lg(n+1)\} \leq 3$ for all practical n ; modify potential to prove $O(m \alpha'(n))$ bound.

Problems: - **19-1 Offline Minimum:** (a) Given insertion/extract-min sequence, fill extracted array. (b) Prove OFFLINE-MINIMUM correct. (c) Implement with disjoint-set data structure efficiently; union sets K_j and K_l when extraction processed. - **19-2 Depth Determination:** (a) Simple tree representation gives $\Theta(m^2)$ worst-case. (b)-(d) Use disjoint-set forest with pseudodistance $v.d$; FIND-DEPTH sums d along path; GRAFT implemented via UNION/LINK with rank-based linking and pseudodistance updates. (e) $O(m \alpha(n))$ bound. - **19-3 Tarjan's Offline LCA:** Given rooted tree T and set P of unordered node pairs, computes all lowest common ancestors. LCA(u) does: MAKE-SET(u), set ancestor, recurse on children, UNION(u, v), update ancestor, mark u BLACK, then for each $\{u, v\}$ in P where v is BLACK, print $\text{LCA} = \text{FIND-SET}(v).\text{ancestor}$. (a) Line 10 executes once per pair. (b) Number of sets = depth of u . (c) Correctness proof by induction. (d) $O((n+|P|) \alpha(n))$ using disjoint-set forest.

Graph Algorithms – Comprehensive Study Notes

Ch. 20 – Elementary Graph Algorithms

Named Entities (Terms & Definitions)

- **Graph $G = (V, E)$:** A mathematical structure consisting of a set V of vertices and a set E of edges.
- **Adjacency-list representation:** An array Adj of $|V|$ lists, one per vertex. For each $u \in V$, $Adj[u]$ contains all vertices v such that $(u,v) \in E$. Memory: $\Theta(V+E)$. For undirected graphs, sum of lengths is $2|E|$; for directed, it is $|E|$.
- **Adjacency-matrix representation:** A $|V| \times |V|$ matrix $A = (a_{ij})$ where $a_{ij} = 1$ if $(i,j) \in E$, 0 otherwise. Memory: $\Theta(V^2)$. Undirected graphs: $A = A^T$ (symmetric).
- **Sparse graph:** $|E|$ is much less than $|V|^2$. Adjacency lists preferred.
- **Dense graph:** $|E|$ is close to $|V|^2$. Adjacency matrix preferred.
- **Weighted graph:** Each edge has an associated weight given by $w: E \rightarrow \mathbb{R}$.
- **Source vertex:** Distinguished vertex s from which search begins.
- **Shortest-path distance $\delta(s,v)$:** Minimum number of edges in any path from s to v (in unweighted graphs). $\delta(s,v) = \infty$ if no path exists.
- **Breadth-first tree:** A tree rooted at s produced by BFS, containing all reachable vertices, where the unique simple path from s to v is a shortest path.
- **Predecessor (parent) $v.\pi$:** The vertex discovered immediately before v in the search.
- **Depth-first forest:** The predecessor subgraph $G_\pi = (V, E_\pi)$ where $E_\pi = \{(v.\pi, v) : v \in V \text{ and } v.\pi \neq \text{NIL}\}$. Forms a forest of trees.
- **Discovery time $u.d$:** Timestamp when vertex u is first discovered (grayed) during DFS.
- **Finish time $u.f$:** Timestamp when u 's adjacency list is fully examined (blackened). $u.d < u.f$ for all u .
- **Parenthesis structure:** Discovery and finish times obey nesting; if DFS prints " u " on discovery and " u " on finish, the expression is properly parenthesized.
- **Tree edge:** Edge in the depth-first forest G_π . Formed when v is first discovered by exploring (u,v) .
- **Back edge:** Edge (u,v) connecting a vertex u to an ancestor v in a depth-first tree. Self-loops are back edges.
- **Forward edge:** Nontree edge (u,v) connecting u to a proper descendant v in a depth-first tree.
- **Cross edge:** All other edges. Can go between vertices in the same tree where neither is an ancestor of the other, or between different trees.
- **Topological sort:** A linear ordering of all vertices of a DAG such that if $(u,v) \in E$, then u appears before v in the ordering.
- **Directed acyclic graph (DAG):** A directed graph with no cycles.

- **Strongly connected component (SCC):** A maximal set of vertices $C \subseteq V$ such that for every pair $u, v \in C$, $u \rightarrow v$ and $v \rightarrow u$ (mutually reachable).
- **Transpose graph G^T :** G with all edges reversed: $E^T = \{(v, u) : (u, v) \in E\}$.
- **Component graph G_{SCC} :** Each SCC becomes a vertex; edge (C_i, C_j) exists if G has an edge from any vertex in C_i to any vertex in C_j . G_{SCC} is always a DAG.
- **Cut $(S, V-S)$:** A partition of V .
- **Light edge crossing a cut:** An edge crossing the cut with minimum weight.
- **Safe edge:** An edge (u, v) that can be added to A while maintaining $A \subseteq$ some MST.
- **Spanning tree:** An acyclic subset $T \subseteq E$ that connects all vertices of G .

Processes / Algorithms / Pathways

BFS (Breadth-First Search)

- **Goal:** Systematically explore graph G from source s , compute shortest-path distances (in number of edges) to all reachable vertices, and build a breadth-first tree.
- **Input:** Graph $G = (V, E)$, source vertex s . G can be directed or undirected.
- **Output:** For each vertex v : $v.d$ = distance from s , $v.\pi$ = predecessor in breadth-first tree.
- **Data structures:** Queue Q (FIFO), color attributes (WHITE/GRAY/BLACK), distance d , predecessor π .
- **Algorithm:**
 1. Initialize: set all vertices white, $d = \infty$, $\pi = \text{NIL}$ except s : gray, $d = 0$, $\pi = \text{NIL}$.
 2. Initialize $Q = \emptyset$, $\text{ENQUEUE}(Q, s)$.
 3. While $Q \neq \emptyset$:
 1. $u = \text{DEQUEUE}(Q)$
 2. For each v in $G.\text{Adj}[u]$: if v is WHITE: $v.\text{color} = \text{GRAY}$, $v.d = u.d + 1$, $v.\pi = u$, $\text{ENQUEUE}(Q, v)$
 3. $u.\text{color} = \text{BLACK}$
- **Complexity:** Time $O(V+E)$, Space $O(V)$ (queue + attributes).
- **Correctness:** Theorem 20.5 – BFS discovers every vertex reachable from s ; upon termination, $v.d = \delta(s, v)$ for all $v \in V$.
- **Properties:**
 - Queue contains gray vertices; the d values in Q are either all k or a sequence of k 's followed by $k+1$'s (Lemma 20.3).
 - d values monotonically increase as vertices are enqueued (Corollary 20.4).
- **Example:** Figure 20.3 – BFS on an undirected graph. Starting from source, vertices are discovered in waves: distance 0 (source), then distance 1 neighbors, then distance 2, etc. The queue holds portions of two consecutive waves simultaneously.

DFS (Depth-First Search)

- **Goal:** Explore graph by going as deep as possible, backtracking, and continuing. Timestamps provide structural information.
- **Input:** Graph $G = (V, E)$, directed or undirected.
- **Output:** Discovery times $u.d$, finish times $u.f$, predecessor π , edge classifications.
- **Algorithm (DFS):**
 1. For each $u \in G.V$: $u.color = WHITE$, $u.\pi = NIL$
 2. $time = 0$
 3. For each $u \in G.V$: if $u.color == WHITE$, call $DFS-VISIT(G, u)$
- **DFS-VISIT(G, u):**
 1. $time = time + 1$, $u.d = time$, $u.color = GRAY$
 2. For each v in $G.Adj[u]$: if $v.color == WHITE$: $v.\pi = u$, $DFS-VISIT(G, v)$
 3. $time = time + 1$, $u.f = time$, $u.color = BLACK$
- **Complexity:** $\Theta(V+E)$.
- **Edge classification during DFS:** When edge (u, v) is first explored:
 - $WHITE\ v \rightarrow$ tree edge
 - $GRAY\ v \rightarrow$ back edge
 - $BLACK\ v \rightarrow$ forward edge (if $u.d < v.d$) or cross edge (if $u.d > v.d$)
- **In undirected graphs:** Every edge is either a tree edge or a back edge (Theorem 20.10).
- **Example:** Figure 20.4 – DFS on a directed graph. Vertices timestamped with discovery/finish times. Tree (T), back (B), forward (F), and cross (C) edges labeled.

Topological Sort

- **Goal:** Linear ordering of DAG vertices respecting edge direction.
- **Input:** Directed acyclic graph $G = (V, E)$.
- **Output:** Linked list of vertices in topologically sorted order.
- **Algorithm:** Run $DFS(G)$; as each vertex is finished, insert it onto front of a linked list. Return linked list.
- **Complexity:** $\Theta(V+E)$.
- **Correctness** (Theorem 20.12): For any edge (u, v) in a DAG, $v.f < u.f$, so listing vertices in decreasing finish time gives topological order.
- **Lemma 20.11:** A directed graph is acyclic iff DFS yields no back edges.
- **Example:** Figure 20.7 – Professor Bumstead's clothing DAG. Vertices are garments; edges indicate precedence (socks before shoes). Topological sort gives a valid dressing order.

Strongly Connected Components (Kosaraju's Algorithm)

- **Goal:** Decompose directed graph into SCCs.

- **Input:** Directed graph $G = (V, E)$.
- **Output:** SCCs (sets of vertices mutually reachable).
- **Algorithm:**
 1. Call DFS(G) to compute finish times $u.f$ for all u .
 2. Create G^T (transpose of G).
 3. Call DFS(G^T), but in main loop, consider vertices in order of decreasing $u.f$ (from step 1).
 4. Output each DFS tree in step 3 as a separate SCC.
- **Complexity:** $\Theta(V+E)$.
- **Correctness** (Theorem 20.16): Uses Lemma 20.13 (no path between distinct SCCs in both directions), Lemma 20.14 (if edge from C' to C , then $f(C') > f(C)$), and Corollary 20.15 (in G^T , edges from C to C' only if $f(C) > f(C')$). Second DFS on G^T in decreasing finish order visits SCCs in reverse topological order of G_{SCC} .
- **Example:** Figure 20.9 – Graph with 4 SCCs. First DFS computes finish times. G^T is built. Second DFS on G^T processes vertices in decreasing finish time; each tree in the forest corresponds to one SCC.

Print-Path

- **Goal:** Print vertices on a shortest path from s to v .
- **Algorithm:** If $v == s$, print s . Else if $v.\pi == \text{NIL}$, print “no path”. Else recursively print path from s to $v.\pi$, then print v .
- **Complexity:** $O(\text{length of path})$.

Comparisons & Trade-offs

Dimension	Adjacency List	Adjacency Matrix
Memory	$\Theta(V+E)$	$\Theta(V^2)$
Edge lookup	$O(\text{degree}(u))$ worst-case	$O(1)$
Iterate all edges	$\Theta(V+E)$	$\Theta(V^2)$
Best for	Sparse graphs	Dense graphs
Weighted graphs	Store weight with neighbor	Store weight in matrix cell
Unweighted graphs	Pointers/objects	1 bit per entry possible

Dimension	BFS	DFS
Data structure	Queue (FIFO)	Stack (recursion)
Source	Single source	Multiple sources possible
Predecessor subgraph	Single tree	Forest
Distances	Shortest path (unweighted)	Not computed
Edge types		

Dimension	BFS	DFS
	Tree, cross, back (directed)	Tree, back, forward, cross
Application	Shortest paths, Prim's, Dijkstra's	Topological sort, SCC, cycle detection

Rules, Laws & Theorems

Lemma 20.1 (Shortest-path inequality)

- **Statement:** For any edge $(u,v) \in E$, $\delta(s,v) \leq \delta(s,u) + 1$.
- **Proof:** If u reachable from s , shortest path to v is at most shortest path to u plus the edge (u,v) . If u not reachable, $\delta(s,u) = \infty$, inequality holds trivially.

Lemma 20.2 (Distance lower bound)

- **Statement:** BFS-computed $v.d$ satisfies $v.d \geq \delta(s,v)$ at all times.
- **Proof:** Induction on number of ENQUEUE operations. Base: $s.d = 0 = \delta(s,s)$, others $\infty \geq \delta(s,v)$. Step: $v.d = u.d+1 \geq \delta(s,u)+1 \geq \delta(s,v)$.

Lemma 20.3 (Queue monotonicity)

- **Statement:** If queue Q contains $v_1, v_2, \dots, v_k$, then $v_1.d \leq v_2.d+1$ and $v_k.d \leq v_{k-1}.d$.
- **Proof:** Induction on queue operations. Dequeueing preserves property. Enqueueing: v is enqueued with $v.d = u.d+1 \leq v_1.d+1$ and $v_1.d \leq u.d+1 = v.d$.

Corollary 20.4

- **Statement:** If v_1 enqueued before v_2 , then $v_1.d \leq v_2.d$.

Theorem 20.5 (Correctness of BFS)

- **Statement:** BFS discovers every vertex reachable from s , terminates with $v.d = \delta(s,v)$ for all $v \in V$. For $v \neq s$ reachable, shortest path from s to v is shortest path from s to $v.\pi$ followed by $(v.\pi, v)$.
- **Proof:** By contradiction. Let v be vertex with minimum $\delta(s,v)$ where $v.d \neq \delta(s,v)$. Since $v.d \geq \delta(s,v)$, we must have $v.d > \delta(s,v)$. Let u precede v on some shortest path. Then $\delta(s,v) = \delta(s,u)+1$ and by minimality $u.d = \delta(s,u)$. At time u dequeued, v must be white, gray, or black – each case contradicts $v.d > \delta(s,v) = u.d+1$.

Lemma 20.6 (BFS predecessor subgraph)

- **Statement:** BFS constructs π so that $G_\pi = (V_\pi, E_\pi)$ is a breadth-first tree.
- **Proof:** V_π = vertices reachable from s . G_π is connected and $|E_\pi| = |V_\pi| - 1$, so it forms a tree. By Theorem 20.5, paths are shortest paths.

Theorem 20.7 (Parenthesis theorem)

- **Statement:** For any two vertices u, v in any DFS, exactly one holds: (1) intervals $[u.d, u.f]$ and $[v.d, v.f]$ disjoint, neither descendant; (2) $[u.d, u.f] \sqsubset [v.d, v.f]$ and u descendant of v ; (3) $[v.d, v.f] \sqsubset [u.d, u.f]$ and v descendant of u .
- **Proof:** If $u.d < v.d$: if $v.d < u.f$, then v discovered while u gray $\rightarrow v$ descendant of u , interval nested. If $u.f < v.d$, intervals disjoint, neither descendant.

Corollary 20.8 (Nesting of descendants' intervals)

- **Statement:** v is proper descendant of u iff $u.d < v.d < v.f < u.f$.

Theorem 20.9 (White-path theorem)

- **Statement:** v is descendant of u in DFS forest iff at time $u.d$, there is a path from u to v consisting entirely of white vertices.
- **Proof:** $\Rightarrow$: If v descendant, all vertices on unique path from u to v in DFS tree are white at $u.d$. $\Leftarrow$: If such a white path exists but v not descendant, let w be predecessor of v on path closest to u . Then w is descendant of u , so $w.f \leq u.f$, and $u.d < v.d < w.f \leq u.f$, implying by Theorem 20.7 that v is descendant after all.

Theorem 20.10 (DFS of undirected graph)

- **Statement:** In DFS of undirected graph, every edge is either a tree edge or a back edge.
- **Proof:** For (u, v) with $u.d < v.d$. If search explores $u \rightarrow v$ first, v is white $\rightarrow$ tree edge. If $v \rightarrow u$ first, then u already gray $\rightarrow$ back edge.

Lemma 20.11 (DAG characterization)

- **Statement:** G is acyclic iff DFS yields no back edges.
- **Proof:** $\Rightarrow$: Back edge (u, v) with v ancestor of u creates cycle. $\Leftarrow$: If cycle c exists, let v be first vertex discovered in c . The preceding edge on c goes to v , and the remaining vertices are white, so by white-path theorem that edge becomes a back edge.

Theorem 20.12 (Correctness of topological sort)

- **Statement:** TOPOLOGICAL-SORT produces a topological sort of the input DAG.
- **Proof:** For any edge (u,v) in a DAG, v cannot be gray (would be back edge, contradicting Lemma 20.11). If v is white, becomes descendant of $u \rightarrow v.f < u.f$. If v is black, $v.f$ already set and $u.f > v.f$. So $v.f < u.f$ always, and listing in decreasing finish time yields topological order.

Lemma 20.13 (SCC lemma)

- **Statement:** If C, C' are distinct SCCs and G has path $u \rightarrow u'$ ($u \in C, u' \in C'$), then G cannot have path $v' \rightarrow v$ ($v' \in C', v \in C$).
- **Proof:** If both paths existed, u and v' would be mutually reachable, contradicting distinctness.

Lemma 20.14 (SCC finish times)

- **Statement:** If edge $(u,v) \in E$ with $u \in C', v \in C$, then $f(C') > f(C)$.
- **Proof:** Two cases. (1) If $d(C') < d(C)$: first vertex x in C' has all C and C' white; white-path theorem makes all vertices in C and C' descendants of x , so $x.f = f(C') > f(C)$. (2) If $d(C') > d(C)$: first vertex y in C has all C as descendants, so $y.f = f(C)$. At $y.f$, C' still white, so for any $w \in C', w.f > y.f$, hence $f(C') > f(C)$.

Corollary 20.15

- **Statement:** If $f(C) > f(C')$, then G contains no edge from C to C' .
- **Proof:** Contrapositive of Lemma 20.14: if $f(C') < f(C)$, no edge from C' to C in G , so no edge from C to C' in G .

Theorem 20.16 (Correctness of SCC algorithm)

- **Statement:** STRONGLY-CONNECTED-COMPONENTS correctly computes SCCs.
- **Proof:** Induction on number of DFS trees in second DFS. Root u of $(k+1)$ st tree is in some SCC C . By finish-time maximality and induction, all other vertices of C are white. By Corollary 20.15, no edges in G leave C to unvisited components. So the tree contains exactly C .

Edge Cases & Pitfalls

- BFS for unreachable vertices: $v.d = \infty, v.\pi = \text{NIL}$.
- DFS from multiple sources: produces a forest, not a single tree.
- In DFS, forward/cross edges possible in directed graphs but NOT in undirected graphs.

- Topological sort requires DAG; presence of a cycle makes it impossible.
- SCC algorithm uses finish times from first DFS; the ordering matters in second DFS.
- DFS results (timestamps, tree structure) depend on vertex order in loops and adjacency list order; BFS distances do not, but BFS tree may.
- Self-loops are classified as back edges in DFS.

Diagrams & Visuals

[BFS Waves]: Source s (distance 0) $\rightarrow$ all neighbors of s (distance 1) $\rightarrow$ distance 2 $\rightarrow$... Queue contains frontier (gray vertices), the "wavefront" expanding uniformly.

[DFS Timestamps]: Each vertex gets interval $[d, f]$. These intervals are either disjoint or nested (never interleaved). Nested intervals = ancestor-descendant.

[Edge Classification by Color]: - Exploring $(u,v) \rightarrow v$ is WHITE $\rightarrow$ tree edge - Exploring $(u,v) \rightarrow v$ is GRAY $\rightarrow$ back edge - Exploring $(u,v) \rightarrow v$ is BLACK $\rightarrow$ forward edge ($u.d < v.d$) or cross edge ($u.d > v.d$)

Step 2: Build G^T (reverse edges) Step 3: DFS on G^T in decreasing finish time order Step 4: Each DFS tree = one SCC

End-of-Chapter Material

Key Terms: adjacency list, adjacency matrix, sparse/dense graph, BFS, DFS, discovery time, finish time, tree/back/forward/cross edge, topological sort, DAG, strongly connected component, transpose, component graph.

Review Questions and Exercises:

20.1-1: Out-degree: $O(V+E)$ by scanning all adjacency lists. In-degree: initialize array of size V to 0, scan all lists and increment for each neighbor: $O(V+E)$.

20.1-2: Complete binary tree with 7 vertices: edges $(1,2), (1,3), (2,4), (2,5), (3,6), (3,7)$. Adjacency lists: standard; adjacency matrix: 7×7 symmetric.

20.1-3: Transpose from adjacency list: create new lists, for each (u,v) in original, add u to $\text{Adj}'[v]$. $O(V+E)$. From adjacency matrix: transpose matrix by swapping indices: $O(V^2)$.

20.1-4: Use boolean array per vertex or sorting to detect duplicates. $O(V+E)$.

20.1-5: G^2 : adjacency list – for each u , for each v in $\text{Adj}[u]$, for each w in $\text{Adj}[v]$, add w to $\text{Adj}^2[u]$ (use boolean array to

dedup). $O(V(V+E))$ or $O(V^3)$ worst-case. Adjacency matrix: $G^2 = A^2$ (boolean multiplication).

20.1-6: Universal sink: Start at (1,1), if $A[i][j]=1$ move right ($i++$), if 0 move down ($j++$). $O(V)$ time. Check candidate row.

20.1-7: BB : entry (i,j) = number of edges from i to j minus number of edges from j to i going both ways? Actually: B is incidence matrix. BB^T gives: diagonal entry = $\text{out-degree}(i) + \text{in-degree}(i)$, off-diagonal (i,j) = $-(\text{number of edges between } i \text{ and } j \text{ in both directions})$.

20.1-8: Hash table: expected $O(1)$ edge lookup. Disadvantage: extra space, no order. Alternate: balanced BST ($O(\log \text{degree})$). Disadvantage: $O(\log n)$ lookup.

20.2-1: BFS on directed graph of Fig 20.2(a) from vertex 3. [Student should compute d and π values.]

20.2-2: BFS on undirected graph of Fig 20.3 from u . [Student exercise.]

20.2-3: Single-bit color suffices: remove line 18, BLACK can be treated as GRAY since dequeue distinguishes. Obviate color: use $d \neq \infty$ to detect discovered vertices.

20.2-4: BFS with adjacency matrix: $O(V^2)$ because scanning each row costs $O(V)$ per vertex.

20.2-5: d values independent of adjacency list order because distance is minimum edges from s . Tree depends on order.

20.2-6: Directed graph where BFS cannot produce a particular shortest-path tree. [Construction needed.]

20.2-7: Wrestler problem: 2-coloring (bipartite check). BFS from each uncolored vertex, assign alternating colors, check no same-color rivalry. $O(n+r)$.

□ 20.2-8: Diameter of tree: BFS from arbitrary vertex $\rightarrow$ farthest vertex a . BFS from $a \rightarrow$ distance to farthest vertex = diameter. $O(V)$.

20.3-1: Directed graph color chart: To: WHITE GRAY BLACK From WHITE: — — — GRAY: tree back fwd/cross BLACK: — — —

Undirected: only tree edges (gray $\rightarrow$ white) and back edges (gray $\rightarrow$ gray).

20.3-2: DFS on Fig 20.6 with alphabetical order. [Student should compute timestamps and edge classifications.]

20.3-3: Parenthesis structure for Fig 20.4.

20.3-4: Single-bit color: remove line 10 (blackening). BLACK vertices are those finished; can use finish time > 0 to distinguish.

20.3-5: Edge classification by timestamps: a. Tree/forward: $u.d < v.d < v.f < u.f$ b. Back: $v.d \leq u.d < u.f \leq v.f$ c. Cross: $v.d < v.f < u.d < u.f$

20.3-6: DFS with explicit stack (iterative).

20.3-7: Counterexample: G with $u \rightarrow v$ and edge (v,u) also. DFS from another source may give $u.d < v.d$ but v not descendant.

20.3-8: Counterexample: path $u \rightarrow v$, but DFS finishes other vertices first, so $v.f > u.f$ (not $v.d \leq u.f$).

20.3-9: Modify DFS-VISIT to print edge type when exploring (u,v) . Undirected: skip duplicate print.

20.3-10: A vertex u can be in its own DFS tree if it is first discovered as a new source (in DFS outer loop) and both its incoming/outgoing neighbors are already discovered/visited in other trees.

20.3-11: Euler tour in undirected graph: DFS that follows each edge in both directions (traverse edge twice, once each way). Maze with pennies: drop pennies to mark visited passages.

20.3-12: Connected components: modify DFS to increment component counter when starting new source; assign $v.cc = \text{counter}$ for all vertices in that DFS tree.

□ 20.3-13: Singly connected test: Run DFS; for each cross/forward edge, check if it violates. $O(V \cdot E)$ or $O(V+E)$ using SCC decomposition.

20.4-1: Topological sort of Fig 20.8 with alphabetical order.

20.4-2: Number of paths from a to b in DAG: DP with topological order. $O(V+E)$.

20.4-3: Undirected graph cycle detection in $O(V)$: if $|E| \geq |V|$, there must be a cycle (connected case). For disconnected, run DFS on each component; count edges vs vertices.

20.4-4: False. The topological sort does not necessarily minimize "bad" edges.

20.4-5: Kahn's algorithm: maintain in-degree array, queue of in-degree-0 vertices. $O(V+E)$. With cycles: some vertices never reach in-degree 0, left in queue.

20.5-1: Adding an edge: can reduce number of SCCs (merge components) or keep same. Cannot increase number of SCCs.

20.5-2: SCC on Fig 20.6. [Student should compute finish times and trees.]

20.5-3: Modified algorithm (original graph, increasing finish time) does NOT always produce correct results.

20.5-4: $((G^T)_{\text{SCC}})^T = G_{\text{SCC}}$.

20.5-5: Compute SCCs, build component graph by checking edges between components, avoid duplicates. $O(V+E)$.

20.5-6: Minimum-edge graph with same SCCs and component graph: within each SCC, form a directed cycle; between components, use one edge per component graph edge.

20.5-7: Semiconnected check: Compute SCCs, topologically sort component graph (which must have a Hamiltonian path). Check that the component graph's topological order has edges from each component to the next.

20.5-8: Maximum difference $\Delta(s,t)$: compute $\min(u)$ (minimum label reachable from u) via DFS on reversed graph with labels; then find max label difference.

Problems:

20-1: BFS edge classification. Undirected: no back/forward edges. Directed: no forward edges. Tree: $v.d = u.d + 1$. Cross: $v.d \leq u.d + 1$. Back: $0 \leq v.d \leq u.d$.

20-2: Articulation points, bridges, biconnected components via DFS. Root is articulation point iff ≥ 2 children. Nonroot v is articulation point iff $\exists$ child s with no back edge from s or its descendants to proper ancestor of v . Compute $v.\text{low} = \min(v.d, \min(w.d \text{ for back edge } (v,w)), \min(\text{child.low}))$. Bridge: edge not on any simple cycle. Biconnected components use stack of edges.

20-3: Euler tour in directed graph: exists iff $\text{in-degree}(v) = \text{out-degree}(v)$ for all v . Algorithm: merge edge-disjoint cycles.

20-4: Reachability with labels: process vertices in decreasing label order; propagate min label through reverse graph. $O(V+E)$.

20-5: Planar graph insertion/query: maintain for each vertex its most recently inserted neighbor.

Ch. 21 – Minimum Spanning Trees

Named Entities (Terms & Definitions)

- **Minimum spanning tree (MST):** An acyclic subset $T \subseteq E$ that connects all vertices of a connected, undirected, weighted

- graph $G = (V, E)$ and minimizes total weight $w(T) = \sum_{(u,v) \in T} w(u,v)$.
- **Cut $(S, V-S)$:** A partition of the vertex set V into two sets.
 - **Edge crosses a cut:** Edge (u,v) with one endpoint in S , other in $V-S$.
 - **Cut respects A :** No edge in A crosses the cut.
 - **Light edge:** An edge crossing a cut with minimum weight (ties allowed).
 - **Safe edge:** An edge (u,v) such that $A \cup \{(u,v)\}$ is a subset of some MST.
 - **Generic MST algorithm:** Grows MST one edge at a time, maintaining invariant that A is subset of some MST. At each step, find and add a safe edge.
 - **Forest $G_A = (V, A)$:** A is always acyclic; initially $A = \emptyset$ gives $|V|$ trees.
 - **Second-best minimum spanning tree:** A spanning tree T' with second-smallest total weight.

Processes / Algorithms / Pathways

Generic MST

- **Goal:** Grow MST by adding safe edges.
- **Algorithm:**
 1. $A = \emptyset$
 2. While A does not form a spanning tree:
 1. Find a safe edge (u,v) for A
 2. $A = A \cup \{(u,v)\}$
 3. Return A
- **Loop invariant:** A is a subset of some MST.
- **Complexity:** $|V|-1$ iterations; depends on finding safe edges efficiently.

Kruskal's Algorithm

- **Goal:** Find MST by processing edges in increasing weight order.
- **Input:** Connected, undirected graph $G = (V, E)$ with weight function w .
- **Output:** Set A of edges forming an MST.
- **Algorithm:**
 1. $A = \emptyset$
 2. For each $v \in G.V$: MAKE-SET(v)
 3. Sort edges of $G.E$ into monotonically increasing order by weight
 4. For each edge (u,v) in sorted order: if FIND-SET(u) $\neq$ FIND-SET(v): $A = A \cup \{(u,v)\}$ UNION(u,v)
 5. Return A
- **Complexity:** $O(E \lg V)$. Breakdown: sorting $O(E \lg E) = O(E \lg V)$; disjoint-set operations $O((V+E)\alpha(V)) = O(E \alpha(V))$; total $O(E \lg V)$.

- **Correctness:** Corollary 21.2 – (u,v) connecting two distinct trees is a light edge, hence safe.
- **Example:** Figure 21.4 – Process edges in sorted weight order; add edge if it connects two different trees in the forest. Edges that would create a cycle are skipped.

Prim's Algorithm

- **Goal:** Find MST by growing a single tree from a root.
- **Input:** Connected graph $G = (V,E)$, weight function w , root r .
- **Output:** MST edges in $A = \{(v, v.\pi) : v \in V - \{r\}\}$.
- **Data structures:** Min-priority queue Q of vertices not in tree, keyed by $v.key = \text{minimum weight of edge connecting } v \text{ to tree}$.
- **Algorithm:**
 1. For each $u \in G.V$: $u.key = \infty$, $u.\pi = \text{NIL}$
 2. $r.key = 0$
 3. $Q = \text{all vertices (INSERT each)}$
 4. While $Q \neq \emptyset$:
 1. $u = \text{EXTRACT-MIN}(Q)$
 2. For each $v \in G.\text{Adj}[u]$: if $v \in Q$ and $w(u,v) < v.key$:
 $v.\pi = u$ $v.key = w(u,v)$ $\text{DECREASE-KEY}(Q, v, v.key)$
 5. At termination, $A = \{(v, v.\pi) : v \in V - \{r\}\}$ is the MST.
- **Complexity:**
 - Binary heap: $O(V \lg V + E \lg V) = O(E \lg V)$
 - Fibonacci heap: $O(E + V \lg V)$
 - Array (dense graph): $O(V^2)$
- **Correctness:** By Corollary 21.2, each added light edge is safe for A .
- **Loop invariant:**
 1. $A = \{(v, v.\pi) : v \in V - \{r\} - Q\}$
 2. Vertices in $V - Q$ are in the MST
 3. For $v \in Q$, $v.key = \text{weight of a light edge } (v, v.\pi) \text{ connecting } v \text{ to some vertex in MST}$
- **Example:** Figure 21.5 – Start from root a . At each step, extract minimum-key vertex from Q , add it to tree, update neighbors' keys. Continue until all vertices are in tree.

MST-REDUCE (Borůvka-style preprocessing)

- **Goal:** Reduce graph size for faster MST computation.
- **Steps:** For each vertex u , select minimum-weight incident edge. Union the vertices of these edges. Contract the graph, keeping minimum weight between contracted vertices.
- **Complexity:** $O(E)$ per phase.
- **Property:** After each phase, $|V'| \leq |V|/2$.

Comparisons & Trade-offs

Dimension	Kruskal	Prim (binary heap)	Prim (Fibonacci heap)
Time	$O(E \lg V)$	$O(E \lg V)$	$O(E + V \lg V)$

Dimension	Kruskal	Prim (binary heap)	Prim (Fibonacci heap)
Data structure	Disjoint sets	Binary heap	Fibonacci heap
Edge processing	Sorted list	Priority queue	Priority queue
Strategy	Process edges globally	Grow tree from root	Grow tree from root
Best for	Sparse graphs	Sparse graphs	Dense graphs
Dimension Adjacency List Adjacency Matrix			
Prim $O(V^2)$ –	Good for dense		
Kruskal	Good for sparse	Bad (sorting edges anyway)	

Rules, Laws & Theorems

Theorem 21.1 (Cut property – recognizing safe edges)

- **Statement:** Let $G = (V, E)$ be connected, undirected with weight function w . Let $A \subseteq E$ be included in some MST. Let $(S, V-S)$ be any cut that respects A , and let (u, v) be a light edge crossing this cut. Then (u, v) is safe for A .
- **Proof (cut-and-paste):** Let T be an MST containing A . Assume T does not contain (u, v) . Adding (u, v) to T creates a cycle. Some edge (x, y) on the cycle crosses the cut $(S, V-S)$; (x, y) is not in A . Remove (x, y) and add (u, v) to get $T' = T - \{(x, y)\} \cup \{(u, v)\}$. Since $w(u, v) \leq w(x, y)$, $w(T') \leq w(T)$. Since T is MST, $w(T') = w(T)$, so T' is also MST. Since $A \subseteq T$ and $(x, y) \notin A$, we have $A \subseteq \{(u, v)\} \cup T'$, so (u, v) is safe.
- **Key insight:** The cut property is the central theorem that makes the greedy approach work for MST.

Corollary 21.2 (Safe edge for a component)

- **Statement:** Let $A \subseteq E$ be included in some MST, and let $C = (V_C, E_C)$ be a connected component (tree) in forest $G_A = (V, A)$. If (u, v) is a light edge connecting C to some other component in G_A , then (u, v) is safe for A .
- **Proof:** The cut $(V_C, V-V_C)$ respects A , and (u, v) is a light edge for this cut. Apply Theorem 21.1.

Cycle Property

- **Statement (Exercise 21.1-5):** Let e be a maximum-weight edge on some cycle of connected graph G . Then there is an MST of G that does not include e .
- **Proof:** If e is removed, the cycle is broken; by the cut property, a lighter edge on the cycle can replace e .

Lemma (Unique MST)

- **Statement (Exercise 21.1-6):** If for every cut of the graph there is a unique light edge crossing it, then the MST is unique. The converse is false.

Lemma (MST edge weight list)

- **Statement (Exercise 21.1-8):** If T and T' are two MSTs of G , the sorted list of edge weights of T equals the sorted list of edge weights of T' .

Lemma (MST induced subgraph)

- **Statement (Exercise 21.1-9):** Let T be MST of G , $V' \subseteq V$, $T' = T$ induced by V' , $G' = G$ induced by V' . If T' is connected, then T' is MST of G' .

Lemma (Edge weight decrease)

- **Statement (Exercise 21.1-10):** If the weight of an edge in T decreases, T remains an MST.

Edge Cases & Pitfalls

- The MST is not unique if multiple edges have the same weight (ties).
- If graph is disconnected, MST does not exist (minimum spanning forest does).
- Kruskal's algorithm can produce different MSTs depending on tie-breaking.
- Prim's algorithm can produce different MSTs if there are tie weights.
- All edge weights positive $\rightarrow$ any minimum-weight connected subgraph is a tree (Exercise 21.1-7).
- Zero or negative weights: connected minimum-weight subgraph may not be a tree (could include cycles to reduce weight).
- MST-REDUCE produces a contracted graph with $|V'| \leq |V|/2$ per phase.

Diagrams & Visuals

[Cut Property]: A cut $(S, V-S)$ partitions vertices. Light edge (u,v) crosses the cut. Adding (u,v) to tree T and removing the heavier crossing edge (x,y) yields another MST T' .

Kruskal's Algorithm: Forest of trees. Edges examined in sorted order. If an edge connects two different trees, merge them. If it connects vertices in the same tree, skip (would create cycle).

End-of-Chapter Material

Key Terms: minimum spanning tree, cut, light edge, safe edge, cut property, cycle property, greedy algorithm, Kruskal, Prim, Borůvka, Fibonacci heap, disjoint set, contraction.

Exercises:

21.1-1: Minimum-weight edge (u,v) in connected graph G belongs to some MST. Proof: Take any cut separating u from v ; (u,v) is a light edge for this cut (or tied), so Theorem 21.1 applies.

21.1-2: Counterexample to converse of Theorem 21.1: safe edge may not be light for a given cut.

21.1-3: If (u,v) is in some MST, then there is a cut for which (u,v) is light: the cut created by removing (u,v) from the MST.

21.1-4: Example where set of light edges (for all cuts) does not form an MST.

21.1-5: Max-weight edge on a cycle is not in some MST. Proof by contradiction using cut property.

21.1-6: Unique light edge per cut $\rightarrow$ unique MST. Converse false (e.g., all edges unique weight $\rightarrow$ MST unique, but some cuts may have multiple light edges if weights are not unique? Actually if all weights are unique, each cut has a unique light edge. So the condition is sufficient but not necessary.)

21.1-7: All positive weights $\rightarrow$ minimum-weight connected subgraph is a tree. If weights nonpositive, could be cheaper to include extra edges (with negative weight).

21.1-8: Sorted edge weight list same for all MSTs.

21.1-9: MST induced subgraph property.

21.1-10: Decreasing weight of edge in T keeps T as MST.

□ 21.1-11: Edge not in T decreases weight: add the edge to T (creates cycle), remove maximum-weight edge on the cycle. $O(V)$ time with preprocessing of max-edge between all pairs.

21.2-1: For each MST T , sort edges so that all edges in T come before edges not in T (with appropriate tie-breaking).

21.2-2: Prim $O(V^2)$ with adjacency matrix: array for key, no heap. Each iteration scans all V vertices to find minimum key. $O(V^2)$ total.

21.2-3: Sparse ($|E| = \Theta(V)$): Fibonacci heap $O(V \lg V + V) = O(V \lg V)$, binary heap $O(E \lg V) = O(V \lg V)$ – same asymptotic. Dense ($|E| = \Theta(V^2)$): Fibonacci $O(V^2)$, binary $O(V^2 \lg V)$ – Fibonacci faster. Fibonacci faster when $|E| = \omega(V)$ but $|E| = o(V \lg V / \lg \lg V)$.

V?) Actually Fibonacci gives $O(E+V \lg V)$, binary gives $O(E \lg V + V \lg V)$. Fibonacci faster when $E = \omega(V)$ but $\lg V$ is not the dominating factor. More precisely, Fibonacci is faster when $|E| = \omega(V)$ or when V is very large.

21.2-4: Edge weights in $[1, |V|]$: use counting sort for edges $\rightarrow O(E + V)$ Kruskal. Range $[1, W]$ with constant W : still counting sort $O(E+V)$.

21.2-5: Prim with integer weights $[1, |V|]$: can use van Emde Boas tree or Dial's algorithm (buckets) to achieve $O(E + VW)$ or $O(E + V \lg V)$. For bounded W : $O(E + VW)$ with buckets.

21.2-6: Professor Borden's divide-and-conquer algorithm fails – counterexample needed.

□ 21.2-7: Edge weights uniform $[0,1]$: expected edge weights are random. Kruskal with sorting $O(E \lg V)$ vs Prim $O(E+V \lg V)$. In practice, Prim with binary heap is faster for dense, Kruskal for sparse. The uniform distribution might allow bucket sort for Kruskal: $O(E+V)$ expected.

□ 21.2-8: Adding new vertex and incident edges: run Prim from new vertex on all old vertices, $O(V)$ using array-based Prim.

Problems:

21-1: Second-best MST. (a) MST unique with distinct weights; second-best may not be unique. (b) There exists $(u,v) \in T$, $(x,y) \notin T$ such that swapping yields second-best MST. (c) Compute $\max[u,v]$ on path in T for all pairs in $O(V^2)$. (d) For each edge not in T , compute weight if added minus max edge on cycle; choose minimum over all non-tree edges.

21-2: MST in sparse graphs. Borůvka-style preprocessing: MST-REDUCE runs in $O(E)$ and reduces vertices by at least half. With k phases: $O(kE)$. Run Prim after: $O(E + V \lg V)$. Choose $k = \lg \lg V$: $O(E \lg \lg V)$. Asymptotically beats Prim without preprocessing when $|E| = o(V \lg V)$.

21-3: Alternative MST algorithms. MAYBE-MST-A (reverse-delete): correctly finds MST. MAYBE-MST-B (arbitrary order, add if no cycle): returns any spanning tree, not necessarily minimum. MAYBE-MST-C (add edge, remove max-weight on cycle): correctly finds MST (reverse-delete variant).

21-4: Bottleneck spanning tree. (a) MST is a bottleneck spanning tree. (b) Determine if bottleneck value $\leq b$: remove all edges with weight $> b$, check connectivity. $O(E)$. (c) Binary search on b using connectivity test: $O(E \lg V)$. Alternatively, median-finding + contraction gives $O(E)$ expected.

Ch. 22 – Single-Source Shortest Paths

Named Entities (Terms & Definitions)

- **Shortest-path problem:** Given weighted, directed graph $G = (V, E)$ with weight function $w: E \rightarrow \mathbb{R}$, find shortest path from source s to all vertices $v \in V$.
- **Path weight $w(p)$:** Sum of weights of edges on path $p = \langle v_0, v_1, \dots, v_k \rangle$: $w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$.
- **Shortest-path weight $\delta(u, v)$:** $\delta(u, v) = \min\{w(p) : p \text{ is a path from } u \text{ to } v\}$, or ∞ if no path, $-\infty$ if negative-weight cycle reachable.
- **Shortest path:** Any path p with weight $w(p) = \delta(u, v)$.
- **Optimal substructure (Lemma 22.1):** Subpaths of shortest paths are shortest paths.
- **Negative-weight edge:** Edge with weight < 0 . Allowed in Bellman-Ford; not in Dijkstra.
- **Negative-weight cycle:** A cycle with total weight < 0 . Reachable from s makes $\delta(s, v) = -\infty$ for vertices on and reachable from the cycle.
- **0-weight cycle:** Can be removed from shortest path without changing weight; assume shortest paths are simple (no cycles).
- **Shortest-paths tree:** Rooted tree at s containing unique shortest path to every reachable vertex.
- **Relaxation:** The process of testing whether going through u improves the shortest path to v found so far. If $v.d > u.d + w(u, v)$, update $v.d = u.d + w(u, v)$ and $v.\pi = u$.
- **Shortest-path estimate $v.d$:** Upper bound on $\delta(s, v)$. Initialized to ∞ (except $s.d = 0$).
- **Predecessor $v.\pi$:** Predecessor on current shortest path.
- **Constraint graph:** Graph constructed from system of difference constraints $x_i - x_j \leq b_k$. Vertices = variables + v_i ; edges: (v_i, v_j) with weight b_k for each constraint, plus (v_i, v_i) with weight 0.
- **Difference constraints:** Constraints of the form $x_i - x_j \leq b_k$.
- **Feasible solution:** Vector x satisfying $Ax \leq b$.
- **Critical path:** Longest path through a PERT chart DAG; gives lower bound on project completion time.
- **Arbitrage:** Using discrepancies in exchange rates to make profit by finding a cycle with product of exchange rates > 1 (equivalent to negative-weight cycle in log-transformed graph).

Processes / Algorithms / Pathways

Initialize-Single-Source

- **Goal:** Initialize shortest-path estimates.
- **Algorithm:** For each $v \in G.V$: $v.d = \infty$, $v.\pi = \text{NIL}$. Set $s.d = 0$.
- **Complexity:** $\Theta(V)$.

Relax

- **Goal:** Relax edge (u,v) , potentially improving shortest-path estimate to v .
- **Algorithm:** If $v.d > u.d + w(u,v)$: $v.d = u.d + w(u,v)$, $v.\pi = u$.
- **Complexity:** $O(1)$.

Bellman-Ford Algorithm

- **Goal:** Solve single-source shortest paths with possibly negative edge weights. Detect negative-weight cycles.
- **Input:** Graph $G = (V,E)$, weight $w: E \rightarrow \mathbb{R}$, source s .
- **Output:** TRUE if no negative-weight cycle reachable from s , else FALSE. On TRUE: $v.d = \delta(s,v)$ for all v , G_π is shortest-paths tree.
- **Algorithm:**
 1. INITIALIZE-SINGLE-SOURCE(G, s)
 2. For $i = 1$ to $|V|-1$: For each edge $(u,v) \in G.E$: RELAX(u, v, w)
 3. For each edge $(u,v) \in G.E$: If $v.d > u.d + w(u,v)$: return FALSE
 4. Return TRUE
- **Complexity:** $O(VE)$. With adjacency list: $O(V^2+VE) = O(VE)$ when $|E| = \Omega(V)$. Can be $O(VE)$ always with edge list.
- **Correctness** (Theorem 22.4):
 - No negative-weight cycles reachable $\rightarrow$ returns TRUE, $v.d = \delta(s,v)$ for all v , G_π is shortest-paths tree.
 - Negative-weight cycle reachable $\rightarrow$ returns FALSE.
 - **Lemma 22.2:** After $|V|-1$ iterations, $v.d = \delta(s,v)$ for all reachable v (by path-relaxation property).
 - **Proof of cycle detection:** If negative-weight cycle exists, summing inequalities $v.d \leq u.d + w(u,v)$ around the cycle gives $0 \leq w(c) < 0$, contradiction.
- **Example:** Figure 22.4 – Graph with 5 vertices. Source s . Four passes over edges ($|V|-1 = 4$). After each pass, estimates improve. Final pass: all distances correct. No negative cycle $\rightarrow$ returns TRUE.

DAG Shortest Paths

- **Goal:** Compute single-source shortest paths in DAG in linear time.
- **Input:** Weighted DAG $G = (V,E)$, source s .
- **Output:** $v.d = \delta(s,v)$ for all v (including negative-weight edges; no cycles possible).
- **Algorithm:**
 1. Topologically sort G
 2. INITIALIZE-SINGLE-SOURCE(G, s)
 3. For each vertex u in topologically sorted order: For each v in $G.Adj[u]$: RELAX(u, v, w)
- **Complexity:** $\Theta(V+E)$.

- **Correctness** (Theorem 22.5): Topological order ensures edges on any shortest path are relaxed in path order. Path-relaxation property gives correctness.
- **Application**: Critical path in PERT chart – longest path through DAG. Compute by negating edge weights or reversing comparison.
- **Example**: Figure 22.5 – DAG with 6 vertices. Topologically sorted. Process vertices left to right, relaxing outgoing edges. Each vertex's distance becomes final after its outgoing edges are relaxed.

Dijkstra's Algorithm

- **Goal**: Single-source shortest paths with nonnegative edge weights.
- **Input**: Graph $G = (V, E)$ with weight $w: E \rightarrow \mathbb{R}$, $w(u, v) \geq 0$ for all edges, source s .
- **Output**: $v.d = \delta(s, v)$ for all v ; G_π is shortest-paths tree.
- **Data structures**: Set S (vertices whose final distances known), min-priority queue $Q = V - S$ keyed by d values.
- **Algorithm**:
 1. INITIALIZE-SINGLE-SOURCE(G, s)
 2. $S = \emptyset$
 3. $Q =$ all vertices (INSERT each)
 4. While $Q \neq \emptyset$:
 1. $u = \text{EXTRACT-MIN}(Q)$
 2. $S = S \cup \{u\}$
 3. For each v in $G.\text{Adj}[u]$: RELAX(u, v, w) If $v.d$ decreased: DECREASE-KEY($Q, v, v.d$)
- **Complexity**:
 - Array: $O(V^2)$
 - Binary heap: $O((V+E) \lg V) = O(E \lg V)$
 - Fibonacci heap: $O(V \lg V + E)$
- **Correctness** (Theorem 22.6): Inductive proof. At start of each iteration, $v.d = \delta(s, v)$ for all $v \in S$. When $u = \text{EXTRACT-MIN}$, show $u.d = \delta(s, u)$. Uses convergence property: let y be first vertex on shortest path to u not in S , $x =$ predecessor of y in S . By induction $x.d = \delta(s, x)$; edge (x, y) was relaxed when x added to S , so $y.d = \delta(s, y)$ by convergence property. Since all edge weights nonnegative, $\delta(s, y) \leq \delta(s, u)$. Also u has min d in $V - S$, so $u.d \leq y.d = \delta(s, y) \leq \delta(s, u) \leq u.d$. Hence $u.d = \delta(s, u)$.
- **Corollary 22.7**: G_π is a shortest-paths tree.
- **Example**: Figure 22.6 – Graph with 6 vertices, source s at left. Vertices extracted from priority queue in order of increasing shortest-path weight. S grows; d values are final when vertex joins S .
- **Relation to BFS and Prim**: BFS-like in that $S =$ black vertices in BFS. Prim-like in that both use min-priority queue.

Solving Difference Constraints (via Bellman-Ford)

- **Goal:** Find feasible solution x to $Ax \leq b$ (difference constraints) or determine infeasibility.
- **Method:**
 1. Build constraint graph $G = (V, E)$: $V = \{v_1, v_2, \dots, v_n\}$; edge (v_i, v_j) with weight b_k for constraint $x_i - x_j \leq b_k$; edges (v_i, v_i) with weight 0.
 2. Run Bellman-Ford on G with source v_1 .
 3. If Bellman-Ford returns TRUE: $x_i = \delta(v_1, v_i)$ is a feasible solution. If FALSE: no feasible solution.
- **Complexity:** $O((n+1)(n+m)) = O(n^2 + nm)$. With optimization: $O(nm)$.
- **Theorem 22.9:** If G has no negative-weight cycles, $x = (\delta(v_1, v_1), \dots, \delta(v_1, v_n))$ is feasible. If G has negative-weight cycle, no feasible solution exists.
- **Example:** Figure 22.8 – System of 8 difference constraints in 5 variables. Constraint graph has 6 vertices (v_1 through v_6). Bellman-Ford gives $\delta(v_1, v_i) = (-5, -3, 0, -1, -4)$, which satisfies all constraints.

Comparisons & Trade-offs

Dimension	Bellman-Ford	DAG Shortest Paths	Dijkstra	
Edge weights	Any (neg allowed)	Any (neg allowed; no cycles)	Nonnegative only	
Time	$O(VE)$	$O(V+E)$	$O(V^2)$ or $O(E \lg V)$ or $O(V \lg V+E)$	
Negative cycles	Detects	No cycles exist	Not applicable	
Relaxations per edge		V	-1	
Order of relaxation	Any order	Topological order	Min-priority queue	
Correctness basis	Path-relaxation property	Topological + path-relaxation	Greedy + nonnegative weights	
Dimension	Adjacency List	Array	Binary Heap	Fibonacci Heap
Dijkstra	$O(V^2)$	$O(E \lg V)$	$O(E + V \lg V)$	
Best for	Dense ($E = \Omega(V^2)$)	Sparse		Very sparse

Key Properties of Shortest Paths and Relaxation

Lemma 22.1 (Optimal substructure)

- **Statement:** Subpaths of shortest paths are shortest paths.

- **Proof:** If subpath p_{ii} from v_i to v_i were not shortest, replace it with shorter path to get shorter overall path, contradiction.

Lemma 22.10 (Triangle inequality)

- **Statement:** For any edge $(u,v) \in E$, $\delta(s,v) \leq \delta(s,u) + w(u,v)$.
- **Proof:** Shortest path to v is no longer than shortest path to u plus edge (u,v) .

Lemma 22.11 (Upper-bound property)

- **Statement:** $v.d \geq \delta(s,v)$ always, and once $v.d = \delta(s,v)$, it never changes.
- **Proof:** Induction on number of relaxation steps. Base: $\infty \geq \delta(s,v)$, $s.d = 0 \geq \delta(s,s)$. Step: $v.d = u.d + w(u,v) \geq \delta(s,u) + w(u,v) \geq \delta(s,v)$. Once at lower bound, cannot decrease (would violate upper-bound) nor increase.

Corollary 22.12 (No-path property)

- **Statement:** If no path from s to v , then $v.d = \delta(s,v) = \infty$ always.
- **Proof:** From upper-bound property, $\infty = \delta(s,v) \leq v.d$, so $v.d = \infty$.

Lemma 22.13 (Relaxation inequality)

- **Statement:** Immediately after relaxing (u,v) , $v.d \leq u.d + w(u,v)$.
- **Proof:** If $v.d > u.d + w(u,v)$ before, set to $u.d + w(u,v)$. If $v.d \leq u.d + w(u,v)$ already, unchanged.

Lemma 22.14 (Convergence property)

- **Statement:** If $s \rightarrow u \rightarrow v$ is a shortest path and $u.d = \delta(s,u)$ before relaxing (u,v) , then $v.d = \delta(s,v)$ after and forever.
- **Proof:** After relax: $v.d \leq u.d + w(u,v) = \delta(s,u) + w(u,v) = \delta(s,v)$. Upper-bound gives $v.d \geq \delta(s,v)$. So $v.d = \delta(s,v)$.

Lemma 22.15 (Path-relaxation property)

- **Statement:** If edges of shortest path $p = \langle v_1, v_2, \dots, v_k \rangle$ are relaxed in order, then $v_k.d = \delta(s, v_k)$ after those relaxations and forever after, regardless of intermixed relaxations.
- **Proof:** Induction on i . Base: $v_1.d = s.d = 0 = \delta(s, s)$. Step: after (v_{i-1}, v_i) relaxed, convergence property gives $v_i.d = \delta(s, v_i)$.

Lemma 22.16 (Predecessor subgraph is rooted tree)

- **Statement:** G_π forms a rooted tree with root s , maintained as invariant over any relaxation sequence, assuming no reachable negative-weight cycles.
- **Proof:** G_π is acyclic (otherwise a negative-weight cycle would exist). For each $v \in V_\pi$, there is a unique simple path from s to v (otherwise predecessor would have two values).

Lemma 22.17 (Predecessor-subgraph property)

- **Statement:** If $v.d = \delta(s,v)$ for all $v \in V$ after relaxation steps, then G_π is a shortest-paths tree rooted at s .
- **Proof:** $V_\pi =$ vertices reachable from s (finite d iff $v.\pi \neq \text{NIL}$). G_π forms rooted tree (Lemma 22.16). For each v , the unique path in G_π from s to v is a shortest path (by summing edge inequalities).

Rules, Laws & Theorems

Theorem 22.4 (Correctness of Bellman-Ford)

- **Statement:** Run Bellman-Ford on $G = (V,E)$ with source s and weight function w . If no negative-weight cycles reachable from s : returns TRUE, $v.d = \delta(s,v)$ for all v , G_π is shortest-paths tree. If negative-weight cycle reachable: returns FALSE.
- **Proof:** No negative cycle $\rightarrow$ Lemma 22.2 gives $v.d = \delta(s,v)$ for reachable v ; no-path property gives for unreachable; triangle inequality gives $v.d \leq u.d + w(u,v)$ for all edges, so no edge fails test; returns TRUE. Negative cycle $\rightarrow$ summing inequalities around cycle gives $0 \leq w(c) < 0$, contradiction, so some edge fails; returns FALSE.

Theorem 22.5 (Correctness of DAG shortest paths)

- **Statement:** DAG-SHORTEST-PATHS terminates with $v.d = \delta(s,v)$ for all v ; G_π is shortest-paths tree.
- **Proof:** If reachable, topological order ensures edges of shortest path relaxed in order; path-relaxation property gives distances. No-path property for unreachable. Predecessor-subgraph property gives tree.

Theorem 22.6 (Correctness of Dijkstra)

- **Statement:** Dijkstra's algorithm terminates with $u.d = \delta(s,u)$ for all $u \in V$ (nonnegative weights).
- **Proof:** Induction on $|S|$. Base: $|S| = 0$ trivial; $|S| = 1$, $S = \{s\}$, $s.d = 0 = \delta(s,s)$. Step: maintain $v.d = \delta(s,v)$ for all $v \in S$. Extract u from $V-S$. Let y be first vertex on shortest path to u not in S , $x = y$'s predecessor in S . By convergence, $y.d = \delta(s,y)$. Since nonnegative weights: $\delta(s,y) \leq \delta(s,u)$.

Since u has min d in $V-S$: $u.d \leq y.d = \delta(s, y)$. So $\delta(s, u) \leq u.d \leq \delta(s, y) \leq \delta(s, u)$, thus $u.d = \delta(s, u)$.

Theorem 22.9 (Difference constraints and shortest paths)

- **Statement:** Given system $Ax \leq b$ of difference constraints, let G be the constraint graph. If G has no negative-weight cycles, then $x_i = \delta(v_i, v_0)$ is feasible. If G has a negative-weight cycle, no feasible solution exists.
- **Proof:** No negative cycle: triangle inequality gives $\delta(v_i, v_j) - \delta(v_j, v_i) \leq w(v_i, v_j)$, so constraints satisfied. Negative cycle: summing constraints around cycle gives $0 \leq w(c) < 0$, contradiction.

Lemma 22.8 (Translation invariance of difference constraints)

- **Statement:** If x is a solution to $Ax \leq b$, then $x + d = (x_1 + d, \dots, x_n + d)$ is also a solution.
- **Proof:** $(x_i + d) - (x_j + d) = x_i - x_j$.

Lemma (Cycle exclusion in shortest paths)

- **Statement:** Shortest paths cannot contain positive-weight cycles (would get shorter by removing). Zero-weight cycles can be removed. Hence, assume shortest paths are simple with at most $|V|-1$ edges.

Edge Cases & Pitfalls

- Negative-weight cycles reachable from s make $\delta(s, v) = -\infty$ for vertices on and beyond the cycle.
- Bellman-Ford returns FALSE but does not identify which vertices are affected by negative cycles (Exercise 22.1-4: modify to set $v.d = -\infty$).
- Dijkstra fails with negative edge weights even without negative cycles – the greedy selection breaks.
- DAG shortest paths works with negative weights (no cycles $\rightarrow$ no negative cycles).
- Difference constraints: the additional vertex v_0 ensures all vertices reachable from source; $w(v_0, v_i) = 0$.
- Shortest paths may not be unique; multiple shortest-paths trees may exist (Figure 22.2).
- When weights are integers in bounded range, special techniques (Dial's algorithm, radix heaps) improve Dijkstra's runtime.
- PERT chart: critical path = longest path in DAG; use negation of weights.

Diagrams & Visuals

[Relaxation (Figure 22.3)]: Edge (u,v) with weight 2. Before: $u.d = 5$, $v.d = 9$. After relax: $v.d = 7$ (since $5+2 < 9$). If $v.d$ had been 6, no change ($6 \leq 5+2 = 7$).

[Bellman-Ford (Figure 22.4)]: Source s . Four passes. After each pass, distances propagate one more edge along shortest paths. Edges relaxed in fixed order each pass. Pass 1: immediate neighbors corrected. Pass 2: two-edge paths. Pass 3: three-edge paths. Pass 4: four-edge paths (max edges in shortest path).

[Dijkstra (Figure 22.6)]: Source s leftmost. Vertices extracted in order of final shortest-path weight ($s \rightarrow y \rightarrow t \rightarrow x \rightarrow z$). S grows by one vertex per iteration. Once extracted, distance never changes.

[Constraint Graph (Figure 22.8)]: Variables $x_1 \dots x_n$ as vertices $v_1 \dots v_n$. Additional vertex v_0 . Edge (v_0, v_i) with weight b_i for constraint $x_i - x_0 \leq b_i$. Edge (v_i, v_j) with weight 0 . Bellman-Ford from v_0 gives $\delta(v_0, v_i)$ as feasible solution.

End-of-Chapter Material

Key Terms: shortest path, shortest-path weight, relaxation, triangle inequality, upper-bound property, convergence property, path-relaxation property, predecessor-subgraph property, Bellman-Ford, DAG shortest paths, Dijkstra, difference constraints, constraint graph, feasible solution, critical path, PERT chart, negative-weight cycle, optimal substructure.

Exercises:

22.1-1: Bellman-Ford on Fig 22.4 with source z . Then with edge (z,x) changed to 4.

22.1-2: Corollary 22.3: $v.d < \infty$ iff there is a path from s to v . Proof: If path exists, $\delta(s,v) < \infty$, by upper-bound $v.d \geq \delta(s,v)$, but also $v.d$ is finite because Bellman-Ford sets $v.d = \delta(s,v)$. If no path, no-path property says $v.d = \infty$.

22.1-3: Stop early if no relaxation made any change in a pass (early termination). Maximum $m+1$ passes where m = maximum number of edges in a shortest path.

22.1-4: Modify to set $v.d = -\infty$ for vertices affected by negative-weight cycles. Run Bellman-Ford normally, then repeat relaxations; any vertex whose d decreases gets $-\infty$.

22.1-5: With edge list representation: $O(VE)$ always. With adjacency lists: to achieve $O(VE)$, explicitly iterate over edges rather than scanning adjacency lists.

22.1-6: $\delta^*(v) = \min_u \delta(u,v)$. Run Bellman-Ford from each vertex? Better: add super-source with 0-weight edges to all vertices, run Bellman-Ford. $O(VE)$.

22.1-7: Finding negative-weight cycle: after Bellman-Ford, if any edge (u,v) satisfies $v.d > u.d + w(u,v)$, follow π back from v to find cycle.

22.2-1: DAG shortest paths on Fig 22.5 with source r .

22.2-2: Changing to first $|V|-1$ vertices: still correct because unreachable vertices remain with $d=\infty$ and topological order handles all within first $|V|-1$.

22.2-3: PERT with vertex weights: redefine edge weight = weight of source vertex. Longest path in DAG: negate weights or modify RELAX.

□ 22.2-4: Number of paths in DAG: DP with topological order. For each pair, or use DP accumulating from sources. $O(V+E)$ for total count if using topological DP with clever counting.

22.3-1: Dijkstra on Fig 22.2 with source s , then with source z .

22.3-2: Graph with negative edge where Dijkstra fails: triangle with $s \rightarrow a$ (weight 1), $s \rightarrow b$ (weight 2), $a \rightarrow b$ (weight -3). Dijkstra picks a first, sets $b.d = 2$. Later relaxes $a \rightarrow b$ setting $b.d = -2$ but b already in S . Correct answer: $b.d = -2$.

22.3-3: While $|Q| > 1$ ($|V|-1$ iterations): correct if we finalize last vertex separately? Yes, because the last vertex in Q has no outgoing edges that could improve others. So algorithm remains correct.

22.3-4: Modified Dijkstra with Q containing only reached vertices: initialize $Q = \{s\}$. Extract min, add newly reached vertices to Q .

22.3-5: Verifying Dijkstra output $O(V+E)$: check $s.d = 0$, $v.d = v.\pi.d + w(v.\pi, v)$ for all $v \neq s$, d values nonnegative and satisfy triangle inequality.

22.3-6: Counterexample: shortest path edges relaxed out of order because vertices are processed in order of d values, not path order.

22.3-7: Most reliable path: maximize product of probabilities. Transform by taking logs: maximize $\sum \log r(u,v) \rightarrow$ minimize $\sum -\log r(u,v)$ (all nonnegative since $0 \leq r \leq 1 \rightarrow -\log r \geq 0$). Run Dijkstra with weight $w'(u,v) = -\log r(u,v)$.

22.3-8: G' has $V + \sum w(u,v)$ vertices (expand each edge into $w(u,v)$ unit-weight edges). BFS on G' processes vertices in same order as Dijkstra extracts them.

22.3-9: Dial's algorithm: maintain array of buckets of size $W+1$, bucket i holds vertices with $d \bmod (W+1) = i$. Each vertex moves to later bucket when d increases. $O(WV+E)$.

22.3-10: $O((V+E) \lg W)$: use binary heap on distinct d values. Number of distinct d values in $V-S$ is at most $W+1$ (bounded range).

22.3-11: Only edges from s may be negative. After first extract-min(s), all vertices have finite d values, and subsequent extracts behave normally.

22.3-12: Edge weights in $[C, 2C]$. Use buckets (Dial's algorithm) with C -sized buckets. The d values are monotonic and bounded, so bucket-based priority queue gives $O(V+E)$.

22.4-1 through 22.4-12: Various difference constraint exercises. Build constraint graph, run Bellman-Ford. Key: to handle equality constraints $x_i = x_j + b$, replace with two inequalities $x_i - x_j \leq b$ and $x_j - x_i \leq -b$.

22.4-5: $O(nm)$ modification: do not scan all $n+1$ adjacency lists each pass; instead process the m edges directly from an edge list.

22.4-8: Bellman-Ford maximizes $\sum x_i$ subject to $Ax \leq b$ and $x_i \leq 0$.

22.4-9: Bellman-Ford minimizes $\max\{x_i\} - \min\{x_i\}$ subject to $Ax \leq b$.

22.5-1: Two additional shortest-paths trees for Fig 22.2.

22.5-2: Graph where every edge appears in some shortest-paths tree and is absent from another.

22.5-3: Modify triangle inequality proof for ∞ and $-\infty$ cases.

22.5-4: If $s.\pi$ becomes non-NIL, a relaxation changed $s.d$ from 0 to a lower value (since only $s.d = 0$ initially). This implies a negative-weight cycle reachable from s .

22.5-5: π assignment producing cycle in G_π without relaxation (e.g., arbitrary predecessors forming a directed cycle).

22.5-6: Induction: base $s \in V_\pi$ has path of length 0. When $v.\pi = u$ and u has path to s in G_π , extend with (u,v) .

22.5-7: Sequence of $|V|-1$ relaxations that produces correct distances: relax edges in order of a shortest-paths tree BFS (bottom-up or by increasing distance).

22.5-8: With negative-weight cycle reachable from s , infinite decreasing sequence: repeatedly relax edges around the cycle, each time reducing d values.

Problems:

22-1: Yen's improvement. (a) G_f and G_b are both acyclic with opposite topological orders. (b) After $\lceil |V|/2 \rceil$ passes, distances correct: forward edges propagate in first half of pass, backward edges in second half. (c) No asymptotic improvement (still $O(VE)$), but constant factor halved.

22-2: Nesting boxes. (a) Nesting is transitive. (b) Sort dimensions of each box, then compare sorted vectors. (c) Build DAG of nesting relationships, find longest path (DP). Time: $O(n^2 + n d \log d)$ or $O(n \lg n + n d)$ with sorting.

22-3: Arbitrage. (a) Transform weights: $w'(u,v) = -\log R[u,v]$. Negative-weight cycle in transformed graph = arbitrage opportunity. Run Bellman-Ford. $O(n^3)$. (b) If negative cycle detected, follow predecessors to output cycle.

22-4: Gabow's scaling algorithm. (a) If $\delta(s,v) \leq |E|$, run Dijkstra with edge weights = distances (bucket-based). $O(E)$. (b) Use unweighted BFS-like approach since $\delta(s,v) \leq 1$. (c) Relationship between w_i and w_{i-1} : double or double+1. δ_i bounded between $2\delta_{i-1}$ and $2\delta_{i-1} + |V| - 1$. (d) Reweighted edges $\hat{w}_i$ are nonnegative. (e) Use Dijkstra with reweighted edges to compute δ_i from δ_{i-1} . (f) Each step $O(E)$, total $O(E \lg W)$.

22-5: Karp's minimum mean-weight cycle. (a) $\mu^* = 0$ if no negative cycles. (b) $\delta(s,v) = \min_{0 \leq k \leq n-1} \delta_k(s,v)$. (c) On 0-weight cycle, distances along cycle are consistent. (d) On minimum mean-weight cycle, there is a vertex v where $\delta_n(v) - \delta_{n-k}(v) = k \cdot \mu^*$ for appropriate k . (e) $\mu^* = \min_v \max_{0 \leq k \leq n-1} (\delta_n(v) - \delta_k(v)) / (n-k)$. (f) Adding constant t to all edges increases μ^* by t . (g) Compute $\delta_k(s,v)$ for $k = 0, \dots, n$ using Bellman-Ford-like DP. $O(VE)$ total.

22-6: Bitonic shortest paths. Since weights on any shortest path from s to v form a bitonic sequence, one can use modified relaxation that processes edges in order. Since bitonic = first decreasing then increasing (or vice versa), we can run two passes. The specific algorithm depends on whether bitonic means increasing then decreasing.

Chapter Notes

The shortest-path problem has a long history. Ford [148] credited with general idea of edge relaxation. Dijkstra's algorithm [116] from 1959 (no priority queue in original). Bellman-Ford from Bellman [45] and Ford [149], also attributed to Moore [334]. Lawler [276] gives linear-time DAG algorithm as folklore. Thorup [435, 436] gives $O(E+V)$ for undirected graphs and $O(E \lg \lg V)$ for directed with integer weights. Cherkassky, Goldberg, Radzik [89] conducted extensive experiments comparing shortest-path algorithms.

Ch. 23 – All-Pairs Shortest Paths

Named Entities (Terms & Definitions)

- **All-pairs shortest-paths problem:** Find shortest paths between every pair of vertices (u, v) in a weighted, directed graph $G = (V, E)$ with weight function $w: E \rightarrow \mathbb{R}$. Output is an $n \times n$ matrix where entry (i, j) is $\delta(i, j)$, the shortest-path weight from i to j .
- **Diameter of a network:** The longest of all shortest paths in a graph.
- **Predecessor matrix $\Pi = (\pi_{ij})$:** π_{ij} is NIL if $i = j$ or no path exists; otherwise π_{ij} is the predecessor of j on a shortest path from i . The i -th row defines a shortest-paths tree rooted at i .
- **Predecessor subgraph $G_{\pi, i}$:** $G_{\pi, i} = (V_{\pi, i}, E_{\pi, i})$ where $V_{\pi, i} = \{j \in V : \pi_{ij} \neq \text{NIL}\} \cup \{i\}$ and $E_{\pi, i} = \{(\pi_{ij}, j) : j \in V_{\pi, i} - \{i\}\}$.
- **Intermediate vertex:** Any vertex of a simple path $p = \langle v_1, v_2, \dots, v_l \rangle$ other than v_1 or v_l .
- **Transitive closure:** Graph $G^* = (V, E)$ where $E = \{(i, j) : \text{there is a path from } i \text{ to } j \text{ in } G\}$.
- **Reweighting:** Technique to transform edge weights so they become nonnegative while preserving shortest-path relationships.
- **Tropical semiring:** Algebraic structure with $\min$ for $\oplus$, $+$ for $\otimes$, ∞ for identity of $\oplus$, 0 for identity of $\otimes$.
- **ϵ -dense graph:** $|E| = \Theta(V^{1+\epsilon})$ for some constant $0 < \epsilon \leq 1$.
- **Closed semiring:** Algebraic framework for solving path problems in directed graphs.

Processes / Algorithms / Pathways

Slow All-Pairs Shortest Paths (SLOW-APSP)

- **Goal:** Compute all-pairs shortest-path weights using DP with matrix multiplication extending paths one edge at a time.
- **Input/Output:** Input: $n \times n$ weight matrix W and $L^{(0)}$. Output: $L^{(n-1)} = W^{(n-1)}$ containing shortest-path weights.
- **Steps:** (1) Initialize $L = L^{(0)}$ (0 on diagonal, ∞ elsewhere). (2) For $r = 1$ to $n-1$: (a) Initialize $M = \infty$; (b) Call EXTEND-SHORTEST-PATHS(L, W, M, n) to compute $M = L \cdot W$; (c) Set $L = M$. (3) Return L .
- **Complexity:** Time $\Theta(n^4)$, Space $\Theta(n^2)$.
- **Example:** Figure 23.1 shows sequence $L^{(r)}$ for a 5-vertex graph. $L^{(0)}$ has 0 s on diagonal, edge weights, ∞ elsewhere. Each iteration extends paths by one edge. $L^{(5)} = L^{(4)}$ so algorithm terminates.

EXTEND-SHORTEST-PATHS

- **Goal:** Extend shortest paths by one more edge, computing $L^r = L^{r-1} \cdot W$ using min-plus matrix multiplication.
- **Steps:** For $i = 1$ to n : For $j = 1$ to n : For $k = 1$ to n : $l_{ij}^r = \min(l_{ij}^{r-1}, l_{ik}^{r-1} + w_{kj})$.
- **Complexity:** Time $\Theta(n^3)$, Space $\Theta(n^2)$.
- **Relation to matrix multiplication:** Replace $+$ with $\min$, $\cdot$ with $+$. Then $l_{ij}^r = \min_k (l_{ik}^{r-1} + w_{kj})$ is analogous to $c_{ij} = \sum_k a_{ik} \cdot b_{kj}$.

Faster-APSP (Repeated Squaring)

- **Goal:** Compute L^{n-1} using only $\lceil \lg(n-1) \rceil$ matrix multiplications via repeated squaring.
- **Input/Output:** Input: $n \times n$ weight matrix W , size n . Output: matrix of shortest-path weights.
- **Steps:** (1) Initialize $L = W$, $r = 1$. (2) While $r < n-1$: (a) Initialize $M = \infty$; (b) $M = \text{EXTEND-SHORTEST-PATHS}(L, L, M, n) \rightarrow M = L^2$; (c) $r = 2r$; (d) $L = M$. (3) Return L .
- **Complexity:** Time $\Theta(n^3 \lg n)$, Space $\Theta(n^2)$.
- **Example:** For $n = 5$, need $\lceil \lg 4 \rceil = 2$ squarings: $L = W^2$, then $L = W^4 = W^{n-1}$ since $n-1 = 4$.

Floyd-Warshall Algorithm

- **Goal:** Compute all-pairs shortest-path weights in $\Theta(n^3)$ time using intermediate-vertex characterization.
- **Input/Output:** Input: $n \times n$ weight matrix W . Output: $D^{(n)}$ matrix of shortest-path weights.
- **Steps:** (1) $D^{(0)} = W$. (2) For $k = 1$ to n : For $i = 1$ to n : For $j = 1$ to n : $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$. (3) Return $D^{(n)}$.
- **Complexity:** Time $\Theta(n^3)$, Space $\Theta(n^2)$ (in-place version drops superscripts).
- **Recurrence:** $d_{ij}^{(k)}$ = weight of shortest path from i to j with intermediate vertices in $\{1, \dots, k\}$. $d_{ij}^{(0)} = w_{ij}$. For $k \geq 1$: $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$.
- **Example:** Figure 23.4 shows $D^{(k)}$ and $\Pi^{(k)}$ matrices for a 5-vertex graph.

Predecessor Matrix for Floyd-Warshall

- **Recurrence:** $\pi_{ij}^{(0)} = \text{NIL}$ if $i = j$ or $w_{ij} = \infty$; else $\pi_{ij}^{(0)} = i$. For $k \geq 1$: if $d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$ then $\pi_{ij}^{(k)} = \pi_{ij}^{(k-1)}$; else $\pi_{ij}^{(k)} = \pi_{kj}^{(k-1)}$.
- **Usage:** $\text{PRINT-ALL-PAIRS-SHORTEST-PATH}(\Pi, i, j)$: if $i = j$ print i ; else if $\pi_{ij} = \text{NIL}$ print "no path"; else recursively print path from i to π_{ij} then print j .

Transitive Closure Algorithm

- **Goal:** Compute whether a path exists between each pair of vertices (boolean matrix).
- **Steps:** Use logical OR ($\vee$) and AND ($\wedge$) instead of min and +.
 $t_{ij}^{(0)} = 1$ if $i = j$ or $(i, j) \in E$, else 0. For $k \geq 1$:
 $t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee (t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)})$.
- **Complexity:** Time $\Theta(n^3)$, Space $\Theta(n^2)$. Uses only boolean values so space is word-factor smaller than Floyd-Warshall.

Johnson's Algorithm

- **Goal:** All-pairs shortest paths for sparse graphs; handles negative weights (no negative cycles).
- **Input/Output:** Input: graph $G = (V, E)$ with weight function w . Output: $n \times n$ matrix D of shortest-path weights, or reports negative-weight cycle.
- **Steps:** (1) Create $G' = (V \cup \{s\}, E \cup \{(s, v) : v \in V\})$ with $w(s, v) = 0$. (2) Run Bellman-Ford on G' with source s to compute $h(v) = \delta(s, v)$. If negative cycle detected, report and exit. (3) Reweight: $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ for all edges (now all ≥ 0). (4) For each vertex $u \in V$, run Dijkstra on G with $\hat{w}$ to compute $\delta(u, v)$. (5) For each v : $d_{uv} = \delta(u, v) - h(u) + h(v)$. (6) Return D .
- **Complexity:** Time $O(V^2 \lg V + VE)$ with Fibonacci heap; $O(VE \lg V)$ with binary heap. Space $O(V^2)$.
- **Example:** Figure 23.6 shows execution on the graph from Figure 23.1. Part (a): G' with h values. Part (b): Reweighted edges. Parts (c)-(g): Dijkstra runs from each source.

PRINT-ALL-PAIRS-SHORTEST-PATH(Π , i , j)

- **Steps:** (1) If $i == j$: print i . (2) Else if $\pi_{ij} == \text{NIL}$: print "no path from i to j exists". (3) Else: PRINT-ALL-PAIRS-SHORTEST-PATH(Π , i , π_{ij}); print j .

Comparisons & Trade-offs

Dimension	Repeated Squaring	Floyd-Warshall	Johnson's
Time	$\Theta(V^3 \lg V)$	$\Theta(V^3)$	$O(V^2 \lg V + VE)$
Graph type	Dense (adjacency matrix)	Dense (adjacency matrix)	Sparse (adjacency list)
Negative edges	Yes	Yes	Yes
Negative cycles	Detected	Detected	Detected
Approach	DP by path length Matrix	DP by intermediate vertices Matrix	Reweightings + Dijkstra

Dimension Data structure	Repeated Squaring	Floyd-Warshall	Johnson's Adjacency lists
Best for	Moderate-sized graphs	Moderate-sized graphs	Large, sparse graphs
Dimension	Min-Plus Multiplication	Standard Matrix Multiplication	
Operation $\square$	min	+	
Operation $\square$	+	$\times$	
Identity for $\square$	∞	0	
Identity for $\square$	0	1	

Formulas & Equations

Shortest path with at most r edges

$$l_{ij}^{(r)} = \min(l_{ij}^{(r-1)}, \min_{1 \leq k \leq n} (l_{ik}^{(r-1)} + w_{kj}))$$

$$l_{ij}^{(0)} = 0 \text{ if } i = j, \infty \text{ otherwise } \delta(i, j) = l_{ij}^{(n-1)}$$

Floyd-Warshall recurrence

$$d_{ij}^{(0)} = w_{ij} \quad d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) \quad \delta(i, j) = d_{ij}^{(n)}$$

Transitive closure recurrence

$$t_{ij}^{(0)} = 1 \text{ if } i = j \text{ or } (i, j) \in E, \text{ else } 0 \quad t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee (t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)})$$

Reweighting (Johnson's)

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v) \quad \hat{\delta}(u, v) = \delta(u, v) + h(u) - h(v)$$

$$\delta(u, v) = \hat{\delta}(u, v) - h(u) + h(v)$$

Triangle inequality (Lemma 22.10)

$$\delta(s, v) \leq \delta(s, u) + w(u, v)$$

Rules, Laws & Theorems

Lemma 23.1 (Reweighting does not change shortest paths)

- **Statement:** Given weighted directed graph $G = (V, E)$ with weight w , let $h: V \rightarrow \mathbb{R}$ be any function. Define $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$. Then: (1) A path p is a shortest path from

- v_0 to v_k under w iff it is a shortest path under $\hat{w}$. (2) G has a negative-weight cycle under w iff it has one under $\hat{w}$.
- **Proof sketch:** $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$. Since $h(v_0)$ and $h(v_k)$ are constant for given endpoints, ordering of path weights is preserved. For cycles: $\hat{w}(c) = w(c) + h(v_0) - h(v_k) = w(c)$ because $v_0 = v_k$.

Triangle inequality for reweighting

- **Statement:** If $h(v) = \delta(s, v)$, then $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$.
- **Proof:** By triangle inequality, $\delta(s, v) \leq \delta(s, u) + w(u, v)$, so $h(v) \leq h(u) + w(u, v)$, hence $\hat{w}(u, v) \geq 0$.

Min-plus matrix multiplication associativity

- **Statement:** Matrix multiplication defined by EXTEND-SHORTEST-PATHS (min for $+$, $+$ for $\times$) is associative (Exercise 23.1-4).
- **Proof sketch:** $(L^{(1)} \cdot L^{(2)}) \cdot L^{(3)} = L^{(1)} \cdot (L^{(2)} \cdot L^{(3)})$ because min and $+$ are associative and $+$ distributes over min.

Edge Cases & Pitfalls

- **Negative-weight cycles:** If present, shortest paths are undefined (can go arbitrarily negative). Repeated squaring computes $L^{(n-1)}$ but may not detect the problem; $L^{(r)}$ may keep decreasing for $r > n-1$. Floyd-Warshall: $d_{ii} < 0$ after iteration k indicates vertex i is on or reachable from a negative cycle.
- **Integer vs irrational capacities:** Ford-Fulkerson may never terminate with irrational capacities (Ch 24); all-pairs algorithms assume no negative cycles.
- **$\infty - \infty$ undefined:** In Johnson's algorithm, if $h(u)$ or $h(v)$ is ∞ (unreachable), reweighting is undefined. Adding source s ensures all vertices reachable from s .
- **Self-loops:** $w_{ii} = 0$ by convention for adjacency matrix representation.
- **Sparse graphs:** Floyd-Warshall is $\Theta(V^3)$ regardless; Johnson's is better for sparse graphs.

Diagrams & Visuals

[Figure 23.1: Directed graph and sequence of matrices $L^{(r)}$ computed by SLOW-APSP. Shows a 5-vertex graph with edges: $1 \rightarrow 2$ (3), $2 \rightarrow 3$ (8), $3 \rightarrow 4$ (2), $4 \rightarrow 5$ (-4), $5 \rightarrow 2$ (7), $3 \rightarrow 5$ (-5), $1 \rightarrow 5$ (-4), $4 \rightarrow 1$ (6), $2 \rightarrow 1$ (1).]

[Figure 23.2: Weighted directed graph for exercises with 5 vertices, specific edge weights.]

[Figure 23.3: Optimal substructure in Floyd-Warshall – path p from i to j with highest intermediate vertex k decomposes into p_1 (i to k) and p_2 (k to j).]

[Figure 23.4: $D^{(k)}$ and $\Pi^{(k)}$ matrices for Floyd-Warshall on graph from Figure 23.1.]

[Figure 23.5: Directed graph and $T^{(k)}$ matrices for transitive closure algorithm.]

[Figure 23.6: Johnson's algorithm execution on graph from Figure 23.1 – parts (a) through (g).]

End-of-Chapter Material

Key Terms: All-pairs shortest paths, predecessor matrix, repeated squaring, Floyd-Warshall algorithm, transitive closure, Johnson's algorithm, reweighting, $\mathbb{R}$ -dense graph, closed semiring, tropical semiring.

Exercises 23.1: - 23.1-1: Run SLOW-APSP and FASTER-APSP on Figure 23.2. - 23.1-2: Why is $w_{ii} = 0$ convenient? Because $w_{ii} = 0$ ensures path with one more edge doesn't get shorter by adding a self-loop. - 23.1-3: What does $L^{(r)}$ correspond to in regular matrix multiplication? Powers of the weight matrix W^r in min-plus algebra. - 23.1-4: Show min-plus matrix multiplication is associative. - 23.1-5: Express single-source shortest paths as product of matrices and a vector, analogous to Bellman-Ford. - 23.1-6: Argue that matrix M in SLOW-APSP is unnecessary (in-place computation works). In FASTER-APSP, M is needed because both inputs are L . - 23.1-7: Compute predecessor matrix from completed L matrix in $O(n^3)$ time. - 23.1-8: Modify EXTEND-SHORTEST-PATHS and SLOW-APSP to compute Π matrices. - 23.1-9: Modify FASTER-APSP to detect negative-weight cycles (check diagonal for negative values after squaring). - 23.1-10: Efficient algorithm to find minimum-length negative-weight cycle.

Exercises 23.2: - 23.2-1: Run Floyd-Warshall on Figure 23.2, show $D^{(k)}$. - 23.2-2: Compute transitive closure using Section 23.1 matrix multiplication approach. - 23.2-3: Modify Floyd-Warshall to compute $\Pi^{(k)}$ matrices; prove predecessor subgraph is shortest-paths tree. - 23.2-4: Show Floyd-Warshall' (in-place, dropping superscripts) is correct, requires $O(n^2)$ space. - 23.2-5: Analyze alternative definition for predecessor matrix when equality in recurrence. - 23.2-6: Detect negative-weight cycle using Floyd-Warshall output (check $d_{ii} < 0$). - 23.2-7: Reconstruct shortest paths using Φ matrix (highest-numbered intermediate vertex on path). - 23.2-8: $O(VE)$ -time algorithm for transitive closure using BFS from each vertex. - 23.2-9: Transitive closure of general digraph from DAG transitive closure in $f(|V|, |E|) + O(V+E^*)$.

Exercises 23.3: - 23.3-1: Use Johnson's on Figure 23.2, show h and $\hat{w}$ values. - 23.3-2: Purpose of adding new vertex s – ensures

all vertices reachable from s , so $h(v) = \delta(s,v)$ is finite. - 23.3-3: If all $w(u,v) \geq 0$, relationship between w and $\hat{w}$ when $h = 0$. - 23.3-4: Professor Greenstreet's simpler reweighting subtracts min weight – doesn't preserve shortest paths. - 23.3-5: Show if G has 0-weight cycle c , then $\hat{w}(u,v) = 0$ for every edge in c . - 23.3-6: Professor Michener's suggestion (skip adding s , use any vertex) – counterexample when graph not strongly connected; correct if strongly connected.

Problems: - **23-1 Transitive closure of a dynamic graph:** (a) Update in $O(V^2)$ per edge insertion. (b) Show $\Omega(V^2)$ lower bound. (c) Algorithm for sequence of r insertions. - **23-2 Shortest paths in d -dense graphs:** (a) d -ary heap operations. (b) Single-source in $O(E)$. (c) All-pairs in $O(VE)$. (d) All-pairs with negative edges in $O(VE)$.

Ch. 24 – Maximum Flow

Named Entities (Terms & Definitions)

- **Flow network $G = (V, E)$:** Directed graph where each edge $(u,v) \in E$ has nonnegative capacity $c(u,v) \geq 0$. No antiparallel edges allowed. Two distinguished vertices: source s and sink t . Each vertex lies on some path $s \rightarrow v \rightarrow t$.
- **Flow f :** Real-valued function $f: V \times V \rightarrow \mathbb{R}$ satisfying: (1) Capacity constraint: $0 \leq f(u,v) \leq c(u,v)$. (2) Flow conservation: $\sum_v f(v,u) = \sum_v f(u,v)$ for all $u \in V - \{s,t\}$.
- **Flow value $|f|$:** $|f| = \sum_v f(s,v) - \sum_v f(v,s)$. Typically source has no incoming edges, so $|f| = \sum_v f(s,v)$.
- **Maximum-flow problem:** Given flow network G with source s and sink t , find a flow of maximum value.
- **Antiparallel edges:** Edges (u,v) and (v,u) both in E . Can be eliminated by splitting one edge with a new vertex.
- **Supersource/Supersink:** For multiple-source/multiple-sink problems, add supersource s with ∞ capacity edges to each source, and supersink t with ∞ capacity edges from each sink.
- **Residual network G_f :** $G_f = (V, E_f)$ where $E_f = \{(u,v) \in V \times V : c_f(u,v) > 0\}$. Residual capacity: $c_f(u,v) = c(u,v) - f(u,v)$ if $(u,v) \in E$; $c_f(u,v) = f(v,u)$ if $(v,u) \in E$; else 0.
- **Augmenting path p :** Simple path from s to t in residual network G_f .
- **Residual capacity of a path:** $c_f(p) = \min\{c_f(u,v) : (u,v) \text{ is in } p\}$.
- **Cancellation:** Decreasing flow on an edge by pushing flow on the reverse edge in the residual network.
- **Augmentation:** $f \uparrow f'$ where $(f \uparrow f')(u,v) = f(u,v) + f'(u,v) - f'(v,u)$.
- **Cut (S,T) of flow network:** Partition of V into S and $T = V - S$ with $s \in S$ and $t \in T$.
- **Net flow across cut:** $f(S,T) = \sum_{u \in S} \sum_{v \in T} f(u,v) - \sum_{u \in S} \sum_{v \in T} f(v,u)$.
- **Capacity of cut:** $c(S,T) = \sum_{u \in S} \sum_{v \in T} c(u,v)$.

- **Minimum cut:** Cut with minimum capacity over all cuts of the network.
- **Maximum bipartite matching:** Maximum set of edges $M \subseteq E$ in undirected bipartite graph $G = (L \cup R, E)$ such that no two edges share a vertex.
- **Integer-valued flow:** Flow where $f(u,v)$ is integer for all (u,v) .
- **Edge connectivity:** Minimum number of edges that must be removed to disconnect an undirected graph.
- **Push-relabel algorithms:** Allow violation of flow conservation (preflow); assign heights; push flow from higher to lower vertices.
- **Preflow:** Flow where flow into a vertex may exceed flow out (overflowing vertices allowed).
- **Global minimum cut:** Partition of V into two nonempty sets, minimizing total crossing edges; no distinguished source/sink.
- **Contraction algorithm:** Randomly contract edges until 2 vertices remain; returns a cut.

Processes / Algorithms / Pathways

Ford-Fulkerson Method

- **Goal:** Find maximum flow in a flow network.
- **Input/Output:** Input: flow network $G = (V, E)$ with source s , sink t , capacities c . Output: maximum flow f .
- **Steps:** (1) Initialize flow f to 0 for all edges. (2) While there exists an augmenting path p in residual network G_f : (a) Compute $c_f(p) = \min\{c_f(u,v) : (u,v) \text{ in } p\}$. (b) For each edge (u,v) in p : if $(u,v) \in E$ then $(u,v).f += c_f(p)$; else $(v,u).f -= c_f(p)$. (3) Return f .
- **Complexity:** Time $O(E \cdot |f^*|)$ for integer capacities (each augmentation increases flow by ≥ 1). Could be exponential in worst case (Figure 24.7 shows 2,000,000 iterations possible). With irrational capacities, may never terminate.

Edmonds-Karp Algorithm

- **Goal:** Polynomial-time implementation of Ford-Fulkerson using BFS to find shortest augmenting path.
- **Steps:** Same as Ford-Fulkerson but always finds shortest augmenting path (fewest edges) in G_f using BFS.
- **Complexity:** Time $O(VE^2)$. Space $O(V + E)$.
- **Key proof:** Each edge can become critical at most $|V|/2$ times $\rightarrow O(VE)$ critical edges $\rightarrow$ each augmentation takes $O(E) \rightarrow O(VE^2)$.

Maximum-Flow by Scaling (MAX-FLOW-BY-SCALING)

- **Goal:** Compute maximum flow using capacity scaling technique.

- **Steps:** (1) $K = 2^{\lceil \lg C \rceil}$ where $C = \max \text{ capacity}$. (2) While $K \geq 1$: while exists augmenting path of capacity $\geq K$, augment flow along it; $K = K/2$. (3) Return f .
- **Complexity:** Time $O(E^2 \lg C)$.

Widest Augmenting Path

- **Goal:** Choose augmenting path with greatest residual capacity.
- **Complexity:** At most $|E| \ln |f^*|$ augmentations to find max flow.
- **Key inequality:** $c_f(p) \geq (|f^*| - |f|)/|E|$.

Maximum Bipartite Matching via Flow

- **Goal:** Find maximum cardinality matching in undirected bipartite graph $G = (L \sqcup R, E)$.
- **Construction:** Create flow network $G' = (V', E')$: $V' = V \sqcup \{s, t\}$; $E' = \{(s, u) : u \in L\} \sqcup \{(u, v) : u \in L, v \in R, (u, v) \in E\} \sqcup \{(v, t) : v \in R\}$. Assign unit capacity to each edge.
- **Steps:** Run Ford-Fulkerson on G' . Max flow value = size of max matching. Extract matching $M = \{(u, v) : f(u, v) > 0\}$.
- **Complexity:** Time $O(VE)$ since $|f^*| = O(V)$ and each augmentation takes $O(E)$.
- **Proof:** Lemma 24.9: Matching $\leftrightarrow$ integer-valued flow bijection. Corollary 24.11: max matching value = max flow value. Theorem 24.10 (Integrality): Ford-Fulkerson on integer capacities yields integer flow.

Comparisons & Trade-offs

Dimension	Ford-Fulkerson	Edmonds-Karp	Scaling	Widest Path
Time	$O(E \cdot f^*)$	f^*	)	$O(VE^2)$
Path selection	Any	BFS (shortest)	Capacity $\geq K$	Max residual capacity
Termination	May not (irrational)	Always polynomial	Always polynomial	Always polynomial
Special case	Good when	f^*	small	General purpose
Approach	Max Flow		Bipartite Matching	
Algorithm	Ford-Fulkerson / Edmonds-Karp		Ford-Fulkerson on constructed network	
Time	$O(VE^2)$		$O(VE)$	
Special structure	None required		Constructed network has unit capacities	

Formulas & Equations

Residual capacity

$$c_f(u,v) = c(u,v) - f(u,v) \text{ if } (u,v) \in E \quad c_f(u,v) = f(v,u) \text{ if } (v,u) \in E \quad c_f(u,v) = 0 \text{ otherwise}$$

Flow augmentation

$$(f \uparrow f')(u,v) = f(u,v) + f'(u,v) - f'(v,u)$$

Net flow across cut

$$f(S,T) = \sum_{\{u \in S\}} \sum_{\{v \in T\}} f(u,v) - \sum_{\{u \in S\}} \sum_{\{v \in T\}} f(v,u)$$

Capacity of cut

$$c(S,T) = \sum_{\{u \in S\}} \sum_{\{v \in T\}} c(u,v)$$

Rules, Laws & Theorems

Lemma 24.1 (Augmented flow)

- **Statement:** Let f be a flow in G , f' be a flow in G_f . Then $f \uparrow f'$ is a flow in G with value $|f \uparrow f'| = |f| + |f'|$.
- **Proof sketch:** Capacity constraint: $(f \uparrow f')(u,v) = f(u,v) + f'(u,v) - f'(v,u) \leq c(u,v)$ and ≥ 0 . Flow conservation follows from summing over edges. Flow value adds because $\sum_v (f \uparrow f')(s,v) - (f \uparrow f')(v,s) = |f| + |f'|$.

Lemma 24.2 (Flow in augmenting path)

- **Statement:** Define $f_p(u,v) = c_f(p)$ if $(u,v) \in p$; $= -c_f(p)$ if $(v,u) \in p$; $= 0$ otherwise. Then f_p is a flow in G_f with value $|f_p| = c_f(p) > 0$.

Corollary 24.3

- **Statement:** Augmenting f by f_p gives flow with value $|f| + c_f(p) > |f|$.

Lemma 24.4 (Net flow across cut equals flow value)

- **Statement:** For any flow f and any cut (S,T) , $f(S,T) = |f|$.

Corollary 24.5 (Cut capacity bounds flow)

- **Statement:** For any flow f and any cut (S,T) , $|f| \leq c(S,T)$.

Theorem 24.6 (Max-flow min-cut theorem)

- **Statement:** If f is a flow in flow network G , then the following are equivalent:
 1. f is a maximum flow in G .
 2. The residual network G_f contains no augmenting paths.
 3. $|f| = c(S, T)$ for some cut (S, T) of G .
- **Proof sketch:** (1) $\Rightarrow$ (2): If augmenting path existed, could increase flow. (2) $\Rightarrow$ (3): Define $S = \{v : \text{path } s \rightarrow v \text{ in } G_f\}$, $T = V - S$. Then $|f| = c(S, T)$. (3) $\Rightarrow$ (1): By Corollary 24.5, $|f| \leq c(S, T)$. Equality implies maximum.

Lemma 24.7 (Monotonicity of shortest-path distances)

- **Statement:** For Edmonds-Karp, for all $v \in V - \{s, t\}$, $\delta_f(s, v)$ in G_f increases monotonically with each flow augmentation.
- **Proof sketch:** Contradiction: assume δ decreases for some v , use triangle inequality.

Theorem 24.8 (Number of augmentations in Edmonds-Karp)

- **Statement:** Edmonds-Karp performs $O(VE)$ flow augmentations.
- **Proof sketch:** Edge (u, v) is critical on path p if $c_f(p) = c_f(u, v)$. After augmentation, (u, v) disappears. It reappears only when flow decreases on (u, v) , which increases distance of u from source by ≥ 2 . Each edge can be critical at most $\lfloor V/2 \rfloor$ times $\rightarrow O(VE)$ critical edges $\rightarrow O(VE)$ augmentations.

Theorem 24.10 (Integrality theorem)

- **Statement:** If all capacities are integers, then Ford-Fulkerson produces a flow f where $|f|$ is integer and $f(u, v)$ is integer for all vertices.
- **Proof sketch:** Induction on number of iterations. Initially all flows 0. Each augmentation adds/subtracts integer $c_f(p)$ (minimum of integer residual capacities).

Lemma 24.9 (Matching-flow correspondence)

- **Statement:** For bipartite G with corresponding flow network G' : (1) If M is matching in G , there is integer-valued flow f in G' with $|f| = |M|$. (2) If f is integer-valued flow in G' , there is matching M in G with $|M| = |f|$ consisting of edges where $f(u, v) > 0$.

Corollary 24.11

- **Statement:** Cardinality of maximum matching in bipartite G equals value of maximum flow in G' .

Edge Cases & Pitfalls

- **Antiparallel edges:** If both (u,v) and (v,u) exist, the residual network definition fails. Split one edge by adding a new vertex.
- **Multiple sources/sinks:** Add supersource with ∞ edges to all sources, supersink with ∞ edges from all sinks.
- **Vertex capacities:** Replace each vertex v with capacity $l(v)$ by two vertices v_{in} , v_{out} connected by edge with capacity $l(v)$. All incoming edges go to v_{in} , outgoing from v_{out} .
- **Irrational capacities:** Ford-Fulkerson may never terminate. Use Edmonds-Karp for guaranteed polynomial time.
- **Integer vs rational capacities:** Scale rational capacities to integers by multiplying by LCM.
- **Flow into source:** Normally 0, but residual networks may include flow returning to source.
- **Edges entering source:** If present, flow into source may be nonzero; can be transformed to eliminate it.
- **Self-loops:** Disallowed in flow networks.

Diagrams & Visuals

[Figure 24.1: (a) Flow network example – Lucky Puck Company trucking problem. Source s (Vancouver), sink t (Winnipeg). Capacities shown on edges. (b) Flow f in G with value 19, shown as f/c per edge.]

[Figure 24.2: Converting antiparallel edges. (a) Network with (v_1,v_2) and (v_2,v_1) . (b) Equivalent network with new vertex v' , edge (v_1,v') and (v',v_2) replacing (v_1,v_2) .]

[Figure 24.3: Multiple sources/sinks conversion. (a) Three sources s_1,s_2,s_3 and two sinks t_1,t_2 . (b) Supersource $s \rightarrow$ each s_i , each $t_i \rightarrow$ supersink t , all ∞ capacity.]

[Figure 24.4: Residual network example. (a) G and flow f . (b) G_f with augmenting path (residual capacity 4). (c) Flow after augmentation. (d) New residual network.]

[Figure 24.5: Cut $(\{s,v_1,v_2\}, \{v_3,v_4,t\})$ with net flow 19 and capacity 26.]

[Figure 24.6: Ford-Fulkerson execution showing residual networks and flows (a)–(f), maximum flow = 23.]

[Figure 24.7: Bad case for Ford-Fulkerson requiring 2,000,000 augmentations. Alternating paths $s \rightarrow u \rightarrow v \rightarrow t$ and $s \rightarrow v \rightarrow u \rightarrow t$.]

[Figure 24.8: Bipartite matching. (a) Matching size 2. (b) Maximum matching size 3. (c) Corresponding flow network with unit capacities.]

End-of-Chapter Material

Key Terms: Flow network, capacity, flow, flow conservation, flow value, maximum flow, residual network, residual capacity, augmenting path, cancellation, augmentation, cut, net flow across cut, capacity of cut, minimum cut, max-flow min-cut theorem, Edmonds-Karp algorithm, integrality theorem, bipartite matching, corresponding flow network, integer-valued flow, push-relabel algorithm, preflow, global minimum cut, edge connectivity.

Exercises 24.1: - 24.1-1: Splitting edge yields equivalent network. - 24.1-2: Multiple sources/sinks reduces to single source/sink. - 24.1-3: Vertex not on path $s \rightarrow v \rightarrow t$ can have zero flow in some max flow. - 24.1-4: Flows form a convex set: $\alpha f_1 + (1-\alpha)f_2$ is a flow. - 24.1-5: State max flow as linear program. - 24.1-6: Two children avoiding each other's blocks $\rightarrow$ max flow problem (vertex-disjoint paths). - 24.1-7: Vertex capacities $\rightarrow$ edge capacities: split vertex into v_{in} , v_{out} with capacity $l(v)$.

Exercises 24.2: - 24.2-1: Complete proof of equation (24.6) = equation (24.5). - 24.2-2: Compute net flow and capacity for cut $(\{s, v_2, v_4\}, \{v_1, v_3, t\})$. - 24.2-3: Show Edmonds-Karp execution on Figure 24.1(a). - 24.2-4: Minimum cut for Figure 24.6 max flow; which augmenting path cancels flow? - 24.2-5: Finite capacity original edges $\rightarrow$ finite flow in multiple source/sink network. - 24.2-6: Supply/demand constraints $\rightarrow$ single-source/sink max flow. - 24.2-7: Prove Lemma 24.2. - 24.2-8: Redefining residual to disallow edges into s still works. - 24.2-9: Does $f \uparrow f'$ satisfy flow conservation and capacity constraint? - 24.2-10: Find max flow with at most $|E|$ augmenting paths. - 24.2-11: Edge connectivity via at most $|V|$ max-flow computations. - 24.2-12: Given flow with edge (v, s) having $f(v, s)=1$, find another flow f' with $f'(v, s)=0$ and same value. - 24.2-13: Among minimum cuts, find one with fewest edges by modifying capacities.

Exercises 24.3: - 24.3-1: Run Ford-Fulkerson on Figure 24.8(c). - 24.3-2: Prove Theorem 24.10 by induction. - 24.3-3: Upper bound on augmenting path length in bipartite matching flow network.

Problems: - **24-1 Escape problem:** (a) Vertex capacities $\rightarrow$ ordinary max flow. (b) Determine if m vertex-disjoint boundary paths exist. - **24-2 Minimum path cover:** (a) Efficient algorithm for DAG via flow network. (b) Does it work for cyclic graphs? - **24-3 Hiring consulting experts:** (a) Show job in T implies required categories in T . (b) Max net revenue from min cut. (c) Efficient algorithm. - **24-4 Updating maximum flow:** (a) Edge capacity increases by 1 $- O(V+E)$ update. (b) Edge capacity decreases by 1 $- O(V+E)$ update. - **24-5 Maximum flow by scaling:** (a) Min cut $\leq C|E|$. (b) Find augmenting path of capacity $\geq K$ in $O(E)$. (c) MAX-FLOW-BY-SCALING is correct. (d) Residual min cut $< 2K|E|$. (e) Inner while loops $O(E)$ per K . (f) Total $O(E^2 \lg C)$. - **24-6 Widest augmenting path:** (a) Modify Dijkstra. (b) At most $|E|$

paths suffice. (c) $c_f(p) \geq (|f| - |f_i|)/|E|$. (d) $|f| - |f_i| \leq |f| (1 - 1/|E|)^i$. (e) $|f| - |f_i| < |f| e^{-i/|E|}$. (f) At most $|E| \ln |f|$ augmentations. - **24-7 Global minimum cut:** (a) Via solving max flow for all source/sink pairs. (b) $\Theta(V)$ max flows suffice. (c) $\mu(G/(u,v)) \leq \mu(G)$. (d) $\mu(G) \leq 2|E|/|V|$. (e) Random edge in min cut with prob $\leq 2/|V|$. (f) Contraction algorithm probability bound. (g)-(i) Repeated contraction yields high-probability min cut.

Ch. 25 – Matchings in Bipartite Graphs

Named Entities (Terms & Definitions)

- **Matching M :** Subset of edges $M \subseteq E$ in undirected graph $G = (V, E)$ such that each vertex has at most one incident edge in M .
- **Matched/unmatched vertex:** A vertex v is matched if some edge in M is incident on v ; otherwise unmatched.
- **Maximal matching:** Matching to which no additional edge can be added without violating the matching property. Every maximum matching is maximal, but not vice versa.
- **Maximum matching:** Matching of maximum cardinality.
- **Perfect matching:** Matching under which every vertex is matched. Requires $|L| = |R|$ in bipartite graphs.
- **M -alternating path:** Simple path whose edges alternate between being in M and in $E - M$.
- **M -augmenting path:** M -alternating path whose first and last edges belong to $E - M$ (hence starts and ends at unmatched vertices). Contains one more edge not in M than in M , so consists of an odd number of edges.
- **Symmetric difference $X \oplus Y$:** $(X - Y) \cup (Y - X)$. Commutative and associative. $X \oplus X = \emptyset$, $X \oplus \emptyset = X$.
- **d -regular graph:** Every vertex has degree d .
- **Hall's theorem:** Condition for existence of perfect matching: $|A| \leq |N(A)|$ for all $A \subseteq L$, where $N(A)$ is neighborhood of A .
- **Stable-marriage problem:** Match n women and n men in a complete bipartite graph where each person ranks all members of the opposite sex. Goal: stable matching (no blocking pair).
- **Blocking pair:** A woman and man not matched to each other who both prefer each other over their assigned partners.
- **Stable matching:** Matching with no blocking pairs.
- **Gale-Shapley algorithm:** Algorithm that always finds a stable matching. Women-proposing version gives women best possible partner, men worst possible.
- **Weak Pareto optimality:** In the stable matching produced by Gale-Shapley with women proposing, no matching (stable or unstable) gives every woman a partner she prefers.
- **Stable-roommates problem:** Stable matching on complete non-bipartite graph with even number of vertices. May have no stable matching.
- **Assignment problem:** Find a maximum-weight perfect matching in a complete bipartite graph with weighted edges.

- **Feasible vertex labeling h :** Assignment of labels to vertices such that $l.h + r.h \geq w(l,r)$ for all $l \in L, r \in R$.
- **Default vertex labeling:** $l.h = \max\{w(l,r) : r \in R\}$ for $l \in L$; $r.h = 0$ for $r \in R$.
- **Equality subgraph G_h :** Subgraph of G where edge $(l,r) \in E_h$ iff $l.h + r.h = w(l,r)$.
- **Hungarian algorithm:** Algorithm for assignment problem using feasible labels and equality subgraphs.
- **Directed equality subgraph $G_{\{M,h\}}$:** Directed graph from G_h where unmatched edges are directed $L \rightarrow R$, matched edges directed $R \rightarrow L$.
- **Growth step:** Execution of lines 10–22 in FIND-AUGMENTING-PATH (relabeling when search fails).
- **Fractional matching:** Function $x: E \rightarrow [0,1]$ such that for each vertex, sum of incident fractional edges ≤ 1 .
- **Cycle cover:** Set of edge-disjoint directed cycles covering each vertex at most once.

Processes / Algorithms / Pathways

Hopcroft-Karp Algorithm

- **Goal:** Find maximum matching in undirected bipartite graph $G = (L \sqcup R, E)$.
- **Input/Output:** Input: undirected bipartite graph G . Output: maximum matching M .
- **Steps:**
 1. $M = \emptyset$.
 2. Repeat:
 1. Find a maximal set P of vertex-disjoint shortest M -augmenting paths.
 2. $M = M \oplus (P_1 \oplus P_2 \oplus \dots \oplus P_k)$.
 1. Until $P = \emptyset$. Return M .
- **Finding maximal set of vertex-disjoint shortest augmenting paths ($O(E)$ per phase):**
 - Phase 1: Create directed graph G_M : direct edges $E-M$ from $L \rightarrow R$, edges M from $R \rightarrow L$.
 - Phase 2: Create DAG H via BFS from all unmatched vertices in L . Include only vertices within distance $\leq q$ (where q = shortest distance to any unmatched vertex in R). Keep edges that go between consecutive layers.
 - Phase 3: Create transpose H^T . For each unmatched r in layer q , run DFS to find path to layer 0. Mark discovered vertices so they aren't reused. Each successful path is an M -augmenting path.
- **Matching update:** $M = M \oplus (P_1 \oplus P_2 \oplus \dots \oplus P_k)$ (swap edges on all found paths).
- **Complexity:** Time $O(\sqrt{|V|} \cdot E)$, Space $O(V + E)$.
- **Example:** Figure 25.1–25.3. Initial matching size 4, two augmenting paths found (length 3 paths). Each path augments by 1.

Gale-Shapley Algorithm (Stable Marriage)

- **Goal:** Find a stable matching in a complete bipartite graph with ranked preferences.
- **Input/Output:** Input: men, women, preference rankings. Output: stable matching.
- **Steps (woman-oriented):**
 1. All women and men are free.
 2. While some woman w is free:
 1. Let m be the first man on w 's list she hasn't proposed to.
 2. If m is free: w and m become engaged.
 3. Else if m ranks w higher than his current fiancée w' : m breaks engagement to w' , becomes engaged to w ; w' becomes free.
 4. Else: m rejects w , w remains free.
 1. Return engaged pairs.
- **Complexity:** Time $O(n^2)$. Space $O(n^2)$.
- **Properties:** Always terminates, always returns stable matching. All executions return the same matching. Women get best possible partner; men get worst possible partner.
- **Example:** 4 women and 4 men with detailed preferences. Walk-through of 9 proposal steps leading to stable matching: (Lacey, Brent), (Wanda, Hank), (Karen, Davis), (Emma, Oscar).

Greedy Bipartite Matching

- **Goal:** Find a maximal (but not necessarily maximum) matching.
- **Steps:** $M = \emptyset$. For each $l \in L$: if l has an unmatched neighbor $r \in R$, add (l, r) to M .
- **Performance:** Returns matching at least half the size of maximum matching.

Hungarian Algorithm (Assignment Problem)

- **Goal:** Find maximum-weight perfect matching in complete bipartite graph $G = (L \cup R, E)$.
- **Input/Output:** Input: complete bipartite graph with edge weights $w(l, r)$. Output: maximum-weight perfect matching M .
- **Steps:**
 1. Initialize feasible vertex labels: $l.h = \max\{w(l, r)\}$ for $l \in L$; $r.h = 0$ for $r \in R$.
 2. Find initial matching M in equality subgraph G_h (greedy).
 3. Form equality subgraph G_h and directed equality subgraph $G_{\{M, h\}}$.
 4. While M is not a perfect matching in G_h :
 1. $P = \text{FIND-AUGMENTING-PATH}(G_{\{M, h\}})$.
 2. $M = M \oplus P$.
 3. Update G_h and $G_{\{M, h\}}$.
 1. Return M .
- **Complexity:** Time $O(n^4)$. Can be improved to $O(n^3)$ (Problem 25-2). Space $O(n^2)$.

FIND-AUGMENTING-PATH (subroutine of Hungarian)

- **Steps:**
 1. Initialize BFS queue Q with all unmatched vertices in L (roots). Initialize F_L, F_R.
 2. Repeat:
 1. If Q empty: compute $\delta = \min\{l.h + r.h - w(l,r) : l \in F_L, r \in R - F_R\}$. Update labels: $l.h -= \delta$ for $l \in F_L$; $r.h += \delta$ for $r \in F_R$. Recompute $G_{\{M,h\}}$.
 2. For each new edge (l,r) in $G_{\{M,h\}}$: if $r \in F_R$, set $r.\pi = l$, add r to F_R and Q; if r is unmatched, path found.
 3. Dequeue u. For each neighbor v of u in $G_{\{M,h\}}$: if $v \in L$, set $v.\pi = u$, add to F_L and Q; if $v \in R$ and $v \notin F_R$, set $v.\pi = u$, add to F_R, if unmatched path found.
 1. Trace back from unmatched r in R using π to construct P. Return P.
- **Uses δ update to bring new edges into equality subgraph when BFS stalls.**

Comparisons & Trade-offs

Dimension	Max Flow (Ch 24)	Hopcroft-Karp (Ch 25)
Time for bipartite matching	$O(VE)$	$O(\sqrt{V} \cdot E)$
Approach	Reduce to max flow	Direct augmenting paths
Augmentations	$O(V)$	$O(\sqrt{V})$ phases
Per augmentation	$O(E)$ BFS	$O(E)$ per phase

Dimension	Woman-oriented Gale-Shapley	Man-oriented Gale-Shapley
Proposer	Women	Men
Optimal for	Women (best partner)	Men (best partner)
Worst for	Men	Women

Dimension	Hopcroft-Karp	Hungarian
Problem	Max cardinality matching	Max-weight perfect matching
Edge weights	Unweighted	Weighted
Input graph	Any bipartite	Complete bipartite
Time	$O(\sqrt{V} \cdot E)$	$O(n^4)$ or $O(n^3)$
Approach	Shortest augmenting paths	Feasible labels + equality subgraph

Formulas & Equations

Symmetric difference properties

$$X \oplus Y = (X - Y) \cup (Y - X) \quad X \oplus X = \emptyset \quad X \oplus \emptyset = X \quad X \oplus Y = (X \cup Y) - (X \cap Y)$$

Feasible vertex labeling

$l.h + r.h \geq w(l,r)$ for all $l \in L, r \in R$

Default vertex labeling

$l.h = \max\{w(l,r) : r \in R\}$ for $l \in L$ $r.h = 0$ for $r \in R$

Equality subgraph

$E_h = \{(l,r) \in E : l.h + r.h = w(l,r)\}$

δ computation (when BFS stalls)

$\delta = \min\{l.h + r.h - w(l,r) : l \in F_L, r \in R - F_R\}$

Label update

$l'.h = l.h - \delta$ for $l \in F_L$ $r'.h = r.h + \delta$ for $r \in F_R$

Weight bound for any matching

$w(M) \leq \sum_{v \in L \cup R} v.h$

Maximum matching weight

$w(M^*) = \sum_{v \in L \cup R} v.h$ (when perfect matching exists in equality subgraph)

Rules, Laws & Theorems

Lemma 25.1 (Augmenting path increases matching size)

- **Statement:** If M is a matching and P is an M -augmenting path, then $M' = M \oplus P$ is a matching with $|M'| = |M| + 1$.
- **Proof sketch:** P has $\lceil q/2 \rceil$ edges in $E-M$ and $\lfloor q/2 \rfloor$ in M . Symmetric difference swaps which edges are in the matching, adding one edge overall. First and last vertices were unmatched, become matched.

Corollary 25.2 (Multiple vertex-disjoint augmenting paths)

- **Statement:** If M is a matching and $P_1, \dots, P_k$ are vertex-disjoint M -augmenting paths, then $M' = M \oplus (P_1 \oplus \dots \oplus P_k)$ is a matching with $|M'| = |M| + k$.

Lemma 25.3 (Symmetric difference of two matchings)

- **Statement:** Let M and M^* be matchings in G . The graph $G' = (V, M \oplus M^*)$ is a disjoint union of simple paths, simple cycles, and

isolated vertices, with edges alternating between M and M . If $|M| > |M|$, then G' contains at least $|M| - |M|$ vertex-disjoint M -augmenting paths.

- **Proof sketch:** Each vertex has degree ≤ 2 (at most one edge from each matching). Components are either even-length cycles (equal edges from M and M), *paths with edges alternating. Paths with more M edges than M edges are M -augmenting paths.*

Corollary 25.4 (Augmenting path characterization of maximum matching)

- **Statement:** M is a maximum matching iff G contains no M -augmenting path.
- **Proof sketch:** Forward: if augmenting path exists, can increase M . Backward: if M not maximum, let M^* be maximum; by Lemma 25.3, there exist M -augmenting paths.

Lemma 25.5 (Shortest augmenting path length increases)

- **Statement:** Let M be a matching in bipartite G , and let q be length of shortest M -augmenting path. After augmenting by a maximal set of vertex-disjoint shortest M -augmenting paths, any shortest augmenting path in the new matching has length $> q$.

Lemma 25.6 (Bound via shortest path length)

- **Statement:** If shortest M -augmenting path has q edges, then maximum matching size $\leq |M| + |V|/(q+1)$.

Lemma 25.7 (Hopcroft-Karp iterates $O(\sqrt{V})$ times)

- **Statement:** The repeat loop of Hopcroft-Karp iterates $O(\sqrt{|V|})$ times.
- **Proof sketch:** After $\sqrt{|V|}$ iterations, $q \geq \sqrt{|V|}$. Then remaining augmentations $\leq |V|/(\sqrt{|V|}+1) = O(\sqrt{|V|})$.

Theorem 25.8 (Hopcroft-Karp running time)

- **Statement:** Hopcroft-Karp runs in $O(\sqrt{|V|} \cdot |E|)$ time on undirected bipartite graph.
- **Proof sketch:** $O(\sqrt{V})$ iterations $\times O(E)$ per iteration.

Theorem 25.9 (Gale-Shapley correctness)

- **Statement:** Gale-Shapley always terminates and returns a stable matching.
- **Proof sketch:** Termination: if some woman remains free, she has proposed to all men; all men would be engaged; with equal numbers, all women engaged – contradiction. Total iterations $\leq n^2$. No blocking pairs: if woman w prefers m' to her partner m , she proposed to m' earlier; m' rejected her for

someone he prefers, so m' doesn't prefer w to his current partner.

Corollary 25.10

- **Statement:** Gale-Shapley runs in $O(n^2)$ time.

Theorem 25.11 (Gale-Shapley produces same result regardless of choices)

- **Statement:** Regardless of how free women are chosen, Gale-Shapley always returns the same stable matching, and each woman has the best partner possible in any stable matching (woman-optimal).
- **Proof sketch:** By contradiction. If some woman w has better partner m' in another stable matching, then m' must have rejected w in favor of some w' . This leads to a blocking pair in the other matching.

Corollary 25.12

- **Statement:** There exist stable matchings that Gale-Shapley does not return (since multiple stable matchings can exist but Gale-Shapley returns only one).

Corollary 25.13

- **Statement:** In the stable matching returned by Gale-Shapley (woman-oriented), each man has the worst partner possible in any stable matching.
- **Proof sketch:** If a man m preferred his partner in another stable matching, that would create a blocking pair with the woman who has m as her best possible partner.

Hall's Theorem (Exercise 25.1-5)

- **Statement:** A bipartite graph $G = (L \sqcup R, E)$ with $|L| = |R|$ has a perfect matching iff $|A| \leq |N(A)|$ for every $A \subseteq L$, where $N(A)$ is the neighborhood of A .

d-regular bipartite graph (Exercise 25.1-6)

- **Statement:** Every d -regular bipartite graph contains a perfect matching, and in fact contains d disjoint perfect matchings.

Theorem 25.14 (Hungarian optimality)

- **Statement:** Let G be complete bipartite with weights $w(l, r)$. Let h be a feasible vertex labeling and G_h the equality subgraph. If G_h contains a perfect matching M , then M is an optimal solution to the assignment problem on G .

- **Proof sketch:** $w(M) = \sum_{l \in L} l.h + \sum_{r \in R} r.h$ because each edge in M satisfies $l.h + r.h = w(l,r)$ and each vertex appears exactly once. For any perfect matching M , $w(M) \leq \sum l.h + \sum r.h \leq w(M)$. Thus M is maximum-weight.

Lemma 25.15 (Label update properties)

- **Statement:** After updating labels h' as in equation (25.5): (1) Edges in BFS forest F remain in $E_{\{M,h'\}}$. (2) Edges in matching M remain in $E_{\{M,h'\}}$. (3) At least one new edge (l,r) with $l \in F_L$, $r \in R - F_R$ enters $E_{\{M,h'\}}$.
- **Proof sketch:** Feasibility: for $l \in F_L$, $r \in R - F_R$, $l.h' + r.h' = l.h - \delta + r.h \geq w(l,r)$ by δ definition. Property 1: $l \in F_L$, $r \in F_R \rightarrow l.h' + r.h' = l.h + r.h$. Property 2: matched edges have both endpoints in or both outside F_L/F_R . Property 3: δ defined as minimum, so some edge achieves the minimum and enters equality subgraph.

Edge Cases & Pitfalls

- **Maximal matching $\neq$ maximum matching:** A maximal matching cannot be extended by any edge but may be much smaller than maximum.
- **Bipartite-only algorithms:** Stable-roommates problem on general graphs may have no solution. Hopcroft-Karp and Hungarian require bipartite graphs.
- **Complete bipartite graph for stable marriage:** If not complete, Gale-Shapley still works; just limit proposals to existing edges.
- **$|L| \neq |R|$ for Hungarian:** Add dummy vertices with zero-weight edges to balance.
- **Converting minimization to maximization:** Negate edge weights or use large constant minus weight.
- **Incomplete preference lists (stable marriage):** Gale-Shapley still works – men reject if not on their list or not preferred.
- **Hospitals with multiple slots (NRMP):** Replace each hospital with as many copies as its capacity; each copy has the same preference list.

Diagrams & Visuals

[Figure 25.1: Bipartite graph $L=\{l_1, \dots, l_7\}$, $R=\{r_1, \dots, r_8\}$. (a) Matching M size 4 (blue). (b) M -augmenting path P (orange) from l_6 to r_8 . (c) $M' = M \oplus P$ size 5 (blue).]

[Figure 25.2: (a) Directed graph G_M for graph in 25.1(a) with BFS distances. (b) DAG H created in second phase; vertices with distance $> q=3$ omitted.]

[Figure 25.3: Transpose H^T of DAG H . DFS from r_1 finds path (orange); DFS from r_4 finds path (yellow); DFS from r_6 fails.]

[Figure 25.4: Hungarian algorithm start – (a) 7×7 weight matrix with default labels and red edges in G_h . (b) Equality subgraph G_h with greedy matching (blue). (c) Directed equality subgraph $G_{\{M,h\}}$.]

[Figure 25.5: BFS finding M-augmenting path in $G_{\{M,h\}}$, parts (a)–(g), culminating in path $\square (l_4, r_2), (r_2, l_1), (l_1, r_3), (r_3, l_6), (l_6, r_5) \square$.]

[Figure 25.6: After updating matching, new BFS stalls (queue empty).]

[Figure 25.7: $\delta=1$ label update adds edge (l_5, r_3) . BFS continues but stalls again.]

[Figure 25.8: $\delta=1$ label update, edges $(l_1, r_6), (l_5, r_6), (l_7, r_6)$ enter. BFS finds path $\square (l_5, r_3), (r_3, l_1), (l_1, r_6) \square$.]

[Figure 25.9: After updating matching, BFS with root l_7 stalls with $\delta=2$.]

[Figure 25.10: $\delta=2$ label update; edge (l_3, r_1) enters. BFS finds path $\square (l_7, r_7), (r_7, l_3), (l_3, r_1) \square$.]

[Figure 25.11: Final perfect matching in G_h , sum of weights = 65 (maximum).]

End-of-Chapter Material

Key Terms: Matching, maximal matching, maximum matching, perfect matching, M-alternating path, M-augmenting path, symmetric difference, Hopcroft-Karp algorithm, stable-marriage problem, stable matching, blocking pair, Gale-Shapley algorithm, woman-optimal, man-optimal, weak Pareto optimality, stable-roommates problem, assignment problem, feasible vertex labeling, equality subgraph, Hungarian algorithm, directed equality subgraph, growth step, fractional matching, cycle cover.

Exercises 25.1: - 25.1-1: Run Hopcroft-Karp on Figure 25.1 to find max matching. - 25.1-2: Compare M-augmenting paths with augmenting paths in flow networks. - 25.1-3: Advantage of searching in $H^A T$ from layer q to layer 0 vs. H from layer 0 to q . - 25.1-4: Bound iterations of Hopcroft-Karp by $O(\sqrt{V})$. - 25.1-5: $\square$ Prove Hall's theorem. - 25.1-6: d -regular bipartite has perfect matching; contains d disjoint perfect matchings.

Exercises 25.2: - 25.2-1: Implement Gale-Shapley in $O(n^2)$ time. - 25.2-2: Is unstable matching possible with 2 women and 2 men? Yes – provide example. - 25.2-3: Modify Gale-Shapley for NRMP (hospitals with multiple slots, unequal numbers). - 25.2-4: Prove weak Pareto optimality for Gale-Shapley. - 25.2-5: Find input for stable-roommates with no stable matching.

Exercises 25.3: - 25.3-1: Rewrite FIND-AUGMENTING-PATH to check for unmatched R vertex in one place. - 25.3-2: Greedy Bipartite Matching returns matching at least half of maximum. - 25.3-3: If edge in $G_{\{M,h\}}$ but not in $G_{\{M,h'\}}$, then $l \leq L - F_L$ and $r \leq F_R$. - 25.3-4: Why not check v already discovered when $v \in L$? Because vertices in L are only reached from R , and each R vertex is discovered only once. - 25.3-5: Show how to check edge membership in $E_{\{M,h\}}$ without building $G_{\{M,h\}}$. - 25.3-6: Minimize rather than maximize: negate weights or subtract all weights from a large constant. - 25.3-7: Handle $|L| \neq |R|$ by adding dummy vertices with zero-weight edges.

Problems: - **25-1 Perfect matchings in a regular bipartite graph:** (a) Euler tour iff all degrees even. (b) $O(E)$ Euler tour algorithm. (c) Find d disjoint perfect matchings in $O(E \lg d)$ time when d is power of 2. - **25-2 Improving Hungarian algorithm to $O(n^3)$:** (a) Compute δ in $O(n)$ using σ attribute ($r.\sigma = \min\{l.h + r.h - w(l,r) : l \in F_L\}$). (b) Update σ in $O(n)$ after δ computed. (c) Update σ in $O(n^2)$ when F_L changes. (d) Conclude $O(n^3)$ overall. - **25-3 Other matching problems via Hungarian:** (a) Max-weight matching in incomplete bipartite graph with positive weights. (b) Same with non-positive weights. (c) Maximum-weight cycle cover in directed graph. - **25-4 Fractional matchings:** (a) Fractional matching value $\geq |M|$. (b) Fractional matching value $\leq |M|$ for bipartite graphs. (c) Weighted fractional matching value = max weighted matching in bipartite graphs. (d) Counterexample in non-bipartite graph. - **25-5 Computing vertex labels:** Given max-weight perfect matching M , compute feasible vertex labeling h such that M is a perfect matching in equality subgraph G_h .

Group 8: Chapters 26–28 Study Guide

Ch. 26 – Parallel Algorithms

Named Entities (Terms & Definitions)

- **Parallel algorithm:** An algorithm where multiple instructions can execute simultaneously.
- **Serial algorithm:** An algorithm suitable for a uniprocessor executing one instruction at a time.
- **Task-parallel algorithm:** Parallel algorithm where programmer identifies what tasks may run in parallel but not which processor executes them; uses a scheduler for load-balancing.
- **Fork-join parallelism:** Parallelism expressed via spawn and sync keywords (and parallel loops). The programmer specifies which tasks *may* run in parallel, not which *must*.
- **Multicore computer (multicore):** A computer containing multiple processing cores sharing a common shared memory.

- **Cluster:** Aggregation of multicores connected via a network, usually with distributed memory.
- **Supercomputer:** The most powerful clusters, comprising many thousands of multicores.
- **Thread:** A software abstraction of a “virtual processor”; each thread has its own program counter and can execute independently; threads share a common memory.
- **Scheduler:** Part of the task-parallel platform that automatically load-balances tasks across processors.
- **Serial projection:** The serial algorithm obtained by deleting parallel, spawn, and sync keywords from the parallel pseudocode.
- **Logical parallelism:** Specification of which parts of a computation *may* proceed in parallel; the runtime scheduler decides actual parallelism.
- **Trace (parallel trace / computation dag):** Directed acyclic graph $G=(V,E)$ where V is strands (instructions) and E represents dependencies between strands.
- **Strand:** A chain of instructions with no parallel or procedural control (no spawn, sync, call, or return). Grouped instructions form a single vertex in the trace.
- **Invocation tree:** Tree of procedure instances showing caller/callee relationships.
- **Ideal parallel computer:** A set of processors with a sequentially consistent shared memory; each processor has equal computing power; scheduling cost is ignored.
- **Sequential consistency:** The shared memory behaves as if exactly one instruction from one processor executes at a time, preserving each processor’s individual instruction order.
- **Load instruction:** Copies data from memory location to a processor register.
- **Store instruction:** Copies data from a processor register to a memory location.
- **Work (T_1):** Total time to execute the entire computation on one processor; sum of times taken by each strand.
- **Span (T_∞):** Fastest possible time on an unlimited number of processors; weight of the longest (critical) path in the trace.
- **Critical path:** A longest weighted path in the trace.
- **Speedup:** T_1 / T_P – how many times faster on P processors than on one.
- **Linear speedup:** Speedup is $\Theta(P)$.
- **Perfect linear speedup:** Speedup equals P exactly ($T_1 / T_P = P$).
- **Parallelism:** Ratio T_1 / T_∞ . Denotes average amount of work per step along the critical path; also the maximum possible speedup.
- **Parallel slackness:** $(T_1 / T_\infty) / P = T_1 / (P \cdot T_\infty)$. Factor by which parallelism exceeds number of processors.
- **Work law:** $T_P \geq T_1 / P$.
- **Span law:** $T_P \geq T_\infty$.

- **Greedy scheduler:** Assigns as many strands to processors as possible each step; never leaves a processor idle if work exists.
- **Complete step:** $\geq P$ strands are ready; all P processors are utilized.
- **Incomplete step:** $< P$ strands are ready; each ready strand gets its own processor, some idle.
- **Deterministic algorithm:** Always does the same thing on the same input regardless of scheduling.
- **Nondeterministic algorithm:** Behavior may vary run-to-run on same input.
- **Determinacy race:** Occurs when two logically parallel instructions access the same memory location and at least one modifies it.
- **Mutually noninterfering:** Parallel strands only read (do not modify) shared memory locations.
- **Parallel loop:** A loop whose iterations can execute simultaneously; expressed with parallel for.
- **Coarsening:** Executing several iterations in a single leaf of the recursion tree to reduce overhead, at the expense of reduced parallelism.

Processes / Algorithms / Pathways

Serial Fibonacci (FIB)

- **Goal:** Compute n th Fibonacci number recursively.
- **Input/Output:** $n \rightarrow F_n$.
- **Steps:** (1) if $n \leq 1$ return n ; (2) $x = \text{FIB}(n-1)$; (3) $y = \text{FIB}(n-2)$; (4) return $x+y$.
- **Complexity:** Time $\Theta(F_n) = \Theta(\phi^n)$ where $\phi = (1+\sqrt{5})/2$ (golden ratio).
- **Analysis:** Recurrence $T(n) = T(n-1) + T(n-2) + \Theta(1)$. Solved by substitution: $T(n) \leq aF_n - b$, choose b large enough to dominate $\Theta(1)$, a large enough for base cases.
- **Note:** Inefficient due to repeated work (no memoization).

Parallel Fibonacci (P-FIB)

- **Goal:** Compute n th Fibonacci using fork-join parallelism.
- **Input/Output:** $n \rightarrow F_n$.
- **Steps:** (1) if $n \leq 1$ return n ; (2) $x = \text{spawn P-FIB}(n-1)$; (3) $y = \text{P-FIB}(n-2)$; (4) sync; (5) return $x+y$.
- **Complexity:** Work $T_1(n) = \Theta(\phi^n)$; Span $T_\infty(n) = \Theta(n)$; Parallelism $= \Theta(\phi^n/n)$.
- **Trace:** See Figure 26.2. For P-FIB(4): 17 strands (work=17), critical path=8 strands (span=8), parallelism=17/8=2.125.

Greedy Scheduling (Theorem 26.1)

- **Goal:** Bound running time of a greedy scheduler.

- **Input/Output:** P processors, work T_1 , span $T_\infty \rightarrow T_P \leq T_1/P + T_\infty$.
- **Proof:** Complete steps $\leq T_1/P$ (each does P work); Incomplete steps $\leq T_\infty$ (each reduces remaining span by 1).
- **Corollary 26.2:** Greedy scheduler is within factor 2 of optimal: $T_P \leq 2 \cdot T^*_P$.
- **Corollary 26.3:** If $P \gg T_1/T_\infty$ (slackness $\gg 1$), then $T_P \approx T_1/P$ (near-perfect linear speedup). Rule of thumb: slackness ≥ 10 suffices.

Parallel Matrix-Vector Multiplication (P-MAT-VEC)

- **Goal:** Compute $y = y + Ax$ where A is $n \times n$, x is n -vector.
- **Steps:** (1) parallel for $i = 1$ to n ; (2) for $j = 1$ to n ; (3) $y_i = y_i + a_{ij} \cdot x_j$.
- **Complexity:** Work $T_1 = \Theta(n^2)$; Span $T_\infty = \Theta(n)$ ($\Theta(\lg n)$ for parallel loop control + $\Theta(n)$ for inner serial loop); Parallelism = $\Theta(n)$.

Recursive Parallel Loop Implementation (P-MAT-VEC-RECURSIVE)

- **Idea:** Compiler converts parallel for into recursive spawning (binary tree of parallel execution).
- **Steps:** Base case: single iteration $\rightarrow$ execute serial loop. Recursive case: spawn first half, call second half, sync.
- **Work:** asymptotically same as serial projection (overhead amortized by leaf work).
- **Span:** $\Theta(\lg n) + \max\{\text{iter}_\infty(i)\}$.

Race Example (RACE-EXAMPLE)

- **Problem:** Two parallel strands both increment shared variable x . Load/increment/store sequence interleaving can lose updates.
- **Result:** May print 1 instead of 2 (serial projection always prints 2).
- **Mechanism:** Sequential consistency interleaves instructions; if one load+increment occurs, then the other does, then the first stores, the second's store is lost.

Faulty Parallelization (P-MAT-VEC-WRONG)

- **Problem:** Parallelizing inner loop causes determinacy races on y_i updates.
- **Lesson:** Parallel loop index variables are independent per iteration (no race from indices), but shared variable updates cause races.

Chess Lesson (Work/Span Analysis Story)

- **Situation:** 32-processor benchmark optimized from $T_1 = 65s$ to $T_1 = 40s$. Original: $T_1 = 2048$, $T_\infty = 1$. Optimized: $T_1 = 1024$, $T_\infty = 8$.

- **Extrapolation on 512 cores:** Original $T_{\infty} = 2048/512+1 = 5s$. Optimized $T_{\infty} = 1024/512+8 = 10s$.
- **Moral:** Optimization that speeds on few processors can hurt on many because span becomes dominant term. Work/span analysis is superior to measured times alone for extrapolating scalability.

Parallel Matrix Multiplication with Parallel Loops (P-MATRIX-MULTIPLY)

- **Steps:** (1) parallel for $i = 1$ to n ; (2) parallel for $j = 1$ to n ; (3) for $k = 1$ to n ; (4) $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$.
- **Complexity:** Work $T_1 = \Theta(n^3)$; Span $T_{\infty} = \Theta(n) (\Theta(\lg n) + \Theta(\lg n) + \Theta(n) = \Theta(n))$; Parallelism $= \Theta(n^2)$.

Parallel Divide-and-Conquer Matrix Multiplication (P-MATRIX-MULTIPLY-RECURSIVE)

- **Goal:** Compute $C = C + A \cdot B$ using 8 parallel recursive multiplications.
- **Steps:** (1) Base case $n=1$: single multiply-add; (2) Allocate temporary D ; (3) Zero D via parallel loops; (4) Partition into $n/2 \times n/2$ submatrices; (5) Spawn 8 recursive calls for submatrix products (4 into C , 4 into D); (6) Sync; (7) Add D into C via parallel loops.
- **Complexity:** Work $M_1(n) = 8M_1(n/2) + \Theta(n^2) = \Theta(n^3)$. Span $M_{\infty}(n) = M_{\infty}(n/2) + \Theta(\lg n) = \Theta(\lg^2 n)$. Parallelism $= \Theta(n^3/\lg^2 n)$.

Parallel Strassen's Method

- **Steps:** (1) If $n=1$: scalar multiply-add; (2) Create 10 sum/difference matrices $S_1, \dots, S_{10}$ and 7 product matrices $P_1, \dots, P_7$ ($\Theta(n^2)$ work, $\Theta(\lg n)$ span); (3) Recursively spawn 7 parallel multiplications (7 $T_1(n/2)$ work, $T_{\infty}(n/2)$ span); (4) Add/subtract P_i matrices into C submatrices ($\Theta(n^2)$ work, $\Theta(\lg n)$ span).
- **Complexity:** Work $T_1 = \Theta(n^{\lg 7}) = \Theta(n^{2.81})$. Span $T_{\infty} = \Theta(\lg^2 n)$. Parallelism $= \Theta(n^{\lg 7}/\lg^2 n)$.

Parallel Merge Sort (P-MERGE-SORT)

- **Goal:** Sort array in parallel using fork-join parallelism.
- **Steps:** (1) if $p \geq r$ return; (2) $q = \lfloor (p+r)/2 \rfloor$; (3) spawn P-MERGE-SORT(A, p, q); (4) spawn P-MERGE-SORT($A, q+1, r$); (5) sync; (6) P-MERGE(A, p, q, r).
- **With serial merge (P-NAIVE-MERGE-SORT):** Span $T_{\infty}(n) = T_{\infty}(n/2) + \Theta(n) = \Theta(n)$; Parallelism $= \Theta(\lg n)$ (poor).
- **With P-MERGE:** Span $T_{\infty}(n) = T_{\infty}(n/2) + \Theta(\lg^2 n) = \Theta(\lg^3 n)$; Work $T_1(n) = \Theta(n \lg n)$; Parallelism $= \Theta(n/\lg^2 n)$.

FIND-SPLIT-POINT

- **Goal:** Binary search for split point in sorted subarray around key x .
- **Input:** Sorted subarray $A[p:r]$, key x .
- **Output:** Index q ($p \leq q \leq r+1$) where all $A[p:q-1] \leq x$ and all $A[q:r] \geq x$.
- **Complexity:** Work = Span = $\Theta(\lg n)$.

P-MERGE-AUX

- **Goal:** Merge two sorted subarrays into output array B in parallel.
- **Steps:** (1) If both empty, return; (2) Ensure first subarray is the larger one (swap roles if needed); (3) $q_l = \text{midpoint}$ of larger subarray, pivot $x = A[q_l]$; (4) Find split point q_r in smaller subarray via FIND-SPLIT-POINT; (5) Compute $q_l = p_l + (q_l - p_l) + (q_r - p_r)$, place x at $B[q_l]$; (6) Spawn two recursive merges: $(A[p_l:q_l-1], A[p_r:q_r-1] \rightarrow B[p_l:q_l-1])$ and $(A[q_l+1:r_l], A[q_r:r_r] \rightarrow B[q_l+1:r_l])$; (7) Sync.
- **Complexity:** Work $T_l = \Theta(n)$; Span $T_\infty = \Theta(\lg^2 n)$.

P-MERGE (wrapper)

- **Steps:** (1) Allocate scratch array B ; (2) Call P-MERGE-AUX; (3) Parallel copy from B back to A .
- **Complexity:** Work $\Theta(n)$, Span $\Theta(\lg^2 n)$.

Design Paradigms

- **Fork-join parallelism:** Extends serial model with spawn, sync, parallel for. Serial projection yields the corresponding serial algorithm.
- **Work/span analysis:** Two-metric analysis for parallel algorithms. Work = serial time. Span = critical path length. Parallelism = work/span.
- **Greedy scheduling:** Complete steps (P strands ready) vs. incomplete steps ($< P$ ready). Guarantees $T_P \leq T_l/P + T_\infty$.
- **Task-parallel programming:** Programmer identifies parallelism; scheduler handles load-balancing. "Processor-oblivious" approach.
- **Divide-and-conquer parallelism:** Natural for fork-join model (e.g., P-FIB, P-MERGE-SORT, P-MATRIX-MULTIPLY-RECURSIVE).
- **Recursive spawning for parallel loops:** Compiler converts parallel for into binary tree of recursive spawns.
- **Coarsening:** Trade parallelism for reduced overhead by executing multiple iterations per leaf.

Comparisons & Trade-offs

Dimension	P-MAT-VEC (parallel loops)	P-MATRIX-MULTIPLY-RECURSIVE
Work	$\Theta(n^2)$	$\Theta(n^3)$
Span	$\Theta(n)$	$\Theta(\lg^2 n)$
Parallelism	$\Theta(n)$	$\Theta(n^3/\lg^2 n)$
Dimension	P-NAIVE-MERGE-SORT	P-MERGE-SORT
Merge span	$\Theta(n)$ (serial merge)	$\Theta(\lg^2 n)$ (parallel merge)
Total span	$\Theta(n)$	$\Theta(\lg^3 n)$
Parallelism	$\Theta(\lg n)$	$\Theta(n/\lg^2 n)$
Dimension	Original Chess	Optimized Chess
Work T_1	2048s	1024s
Span T_∞	1s	8s
T_2	65s	40s
T_4	5s	10s

Formulas & Equations

Work Law

$$T_P \geq T_1 / P$$

Span Law

$$T_P \geq T_\infty$$

Greedy Scheduling Bound

$$T_P \leq T_1 / P + T_\infty$$

Corollary 26.2 (within factor 2)

$$T_P \leq 2 \cdot T^*_P$$

Series Composition

- Work: $T_1(A;B) = T_1(A) + T_1(B)$
- Span: $T_\infty(A;B) = T_\infty(A) + T_\infty(B)$

Parallel Composition

- Work: $T_1(A||B) = T_1(A) + T_1(B)$
- Span: $T_\infty(A||B) = \max(T_\infty(A), T_\infty(B))$

Parallel Loop Span

$$T_\infty(n) = \Theta(\lg n) + \max\{\text{iter}_\infty(i) : 1 \leq i \leq n\}$$

Fibonacci Numbers (for reference)

$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$ $F_n = \Theta(\phi^n)$ where $\phi = (1+\sqrt{5})/2 \approx 1.618$

P-FIB Span Recurrence

$$T_\infty(n) = \max(T_\infty(n-1), T_\infty(n-2)) + \Theta(1) = T_\infty(n-1) + \Theta(1) = \Theta(n)$$

P-MERGE-AUX Span Recurrence

$$T_\infty(n) = T_\infty(3n/4) + \Theta(\lg n) = \Theta(\lg^2 n)$$

P-MERGE-AUX Work Recurrence

$T_\parallel(n) = T_\parallel(\alpha n) + T_\parallel((1-\alpha)n) + \Theta(\lg n)$ where $1/4 \leq \alpha \leq 3/4$.
Solution: $\Theta(n)$.

P-MERGE-SORT Span Recurrence

$$T_\infty(n) = T_\infty(n/2) + \Theta(\lg^2 n) = \Theta(\lg^3 n)$$

P-MERGE-SORT Work Recurrence

$$T_\parallel(n) = 2T_\parallel(n/2) + \Theta(n) = \Theta(n \lg n)$$

P-MATRIX-MULTIPLY-RECURSIVE Span Recurrence

$$M_\infty(n) = M_\infty(n/2) + \Theta(\lg n) = \Theta(\lg^2 n)$$

Work Recurrence for Recursive MM

$$M_\parallel(n) = 8M_\parallel(n/2) + \Theta(n^2) = \Theta(n^3)$$

Rules, Laws & Theorems

Theorem 26.1 (Greedy Scheduling)

- **Statement:** On an ideal parallel computer with P processors, a greedy scheduler executes a task-parallel computation with work $T_\parallel$ and span T_∞ in time $T_P \leq T_\parallel/P + T_\infty$.

Corollary 26.2 (Near Optimality)

- **Statement:** Greedy scheduler is within factor 2 of optimal:
 $T_P \leq 2 \cdot T^*_P$.

Corollary 26.3 (Near-Perfect Speedup)

- **Statement:** If $P \ll T_\infty / T_\parallel$ (parallel slackness $\gg 1$), then $T_P \approx T_\parallel / P$ (speedup $\approx P$).

Work Law

- **Statement:** $T_P \geq T_\parallel / P$. (P processors can do at most P work per step.)

Span Law

- **Statement:** $T_P \geq T_\infty$. (Cannot run faster than on unlimited processors.)

Sequential Consistency Property

- **Statement:** The shared memory behaves as if exactly one instruction from one processor executes at a time, preserving individual processor orders.

Series-Parallel Composition Rules

- **Statement:** Series: spans add. Parallel: span = maximum of spans. Work always adds.

Edge Cases & Pitfalls

- **Determinacy races:** Two parallel strands accessing same memory with at least one write $\rightarrow$ nondeterministic behavior. Hard to reproduce and debug. Examples: Therac-25, Northeast Blackout 2003.
- **Benign races:** Sometimes acceptable (e.g., parallel hash tables) but frowned upon when deterministic code is an option.
- **Mutual noninterference requirement:** For deterministic algorithms, parallel strands must only read, not modify, shared memory locations.
- **Parallel loop index variables:** Conceptually independent per iteration $\rightarrow$ no race from indices themselves.
- **Coarsening trade-off:** Reduces overhead but reduces parallelism. Acceptable if sufficient slackness exists.
- **Optimization scaling pitfall (chess story):** Reducing work but increasing span can make algorithm slower on many processors. Span dominates when P is large.
- **Fibonacci inefficiency:** $T(n) = \Theta(\phi^n)$ due to repeated recomputation; need memoization for efficiency.
- **Parallelizing inner loop of mat-vec:** P-MAT-VEC-WRONG creates races on y_i updates when parallelizing inner loop.
- **PRAM vs. fork-join:** This chapter uses fork-join model, not PRAM. Previous editions covered PRAM and sorting networks.

- **Divide-by-zero in LU decomposition:** Pivoting needed unless matrix is symmetric positive-definite.

End-of-Chapter Material

Key Terms

work, span, parallelism, speedup, linear speedup, critical path, work law, span law, greedy scheduler, complete step, incomplete step, fork-join parallelism, spawn, sync, parallel for, serial projection, trace (computation dag), strand, determinacy race, sequential consistency, thread, multicore, ideal parallel computer, parallel slackness, coarsening.

Exercises

- **26.1-1:** What does a trace for a serial algorithm look like? (Answer: a single chain – no parallelism.)
- **26.1-2:** If line 4 of P-FIB also spawns P-FIB($n-2$), trace changes: more parallelism? Asymptotic work/span unchanged (still $\Theta(\phi^n)$ work, $\Theta(n)$ span).
- **26.1-3:** Draw trace for P-FIB(5). Work = ?, Span = ?, schedule on 3 processors.
- **26.1-4:** Prove stronger bound: $T_P \leq (T_1 - T_\infty)/P + T_\infty$.
- **26.1-5:** Construct trace where greedy scheduler can take nearly $2\times$ time vs. another greedy execution on same P .
- **26.1-6:** Professor Karan's claims $T_1=80$, $T_2=42$, $T_3=10$. Impossible: check work/span law consistency.
- **26.1-7:** Algorithm for $n \times n$ matrix $\times$ n -vector with $\Theta(n^2/\lg n)$ parallelism and $\Theta(n^2)$ work.
- **26.1-8:** Analyze P-TRANPOSE (parallel for j , parallel for i , exchange). Work $\Theta(n^2)$, Span $\Theta(\lg n)$, Parallelism $\Theta(n^2/\lg n)$.
- **26.1-9:** Same but inner loop is serial for. Work $\Theta(n^2)$, Span $\Theta(n)$, Parallelism $\Theta(n)$.
- **26.1-10:** For what P do chess versions run equally fast? Solve $2048/P+1 = 1024/P+8 \rightarrow P \approx 146$.
- **26.2-1:** Trace for P-MATRIX-MULTIPLY on 2×2 matrices.
- **26.2-2:** Repeat for P-MATRIX-MULTIPLY-RECURSIVE.
- **26.2-3:** Pseudocode for MM with $\Theta(n^3)$ work, $\Theta(\lg n)$ span. Hint: parallelize inner loop using divide-and-conquer (three parallel loops).
- **26.2-4:** Efficient parallel $p \times q$ by $q \times r$ matrix multiplication.
- **26.2-5:** Parallel Floyd-Warshall algorithm.
- **26.3-1:** How to coarsen base case of P-MERGE.
- **26.3-2:** Parallel merge using median-of-two-subarrays.
- **26.3-3:** Parallel PARTITION (not in-place; auxiliary array).
- **26.3-4:** Parallel FFT.
- **26.3-5** □: Parallelize SELECT from Section 9.3.

Problem 26-1: Implementing Parallel Loops Using Recursive Spawning

- **SUM-ARRAYS:** parallel for loop for pairwise addition.

- 1. Rewrite using recursive spawning. Analyze parallelism.
- 1. SUM-ARRAYS' with grain-size=1: what parallelism?
- 1. Span formula in terms of n and grain-size. Optimal grain-size for max parallelism.

Problem 26-2: Avoiding Temporary Matrix in Recursive MM

- 1. Parallelize without temporary D (insert sync to avoid races).
- 1. Recurrences for work and span.
- 1. Compare parallelism with original. 1000×1000 : original parallelism $\approx 10^3$; modified parallelism $\approx ?$

Problem 26-3: Parallel Matrix Algorithms

- 1. Parallel LU-DECOMPOSITION.
- 1. Parallel LUP-DECOMPOSITION.
- 1. Parallel LUP-SOLVE.
- 1. Parallel symmetric positive-definite matrix inversion (using equation (28.14)).

Problem 26-4: Parallel Reductions and Scan (Prefix)

- 1. P-REDUCE: $\Theta(n)$ work, $\Theta(\lg n)$ span.
- 1. P-SCAN-1: work $\Theta(n^2)$, span $\Theta(\lg n)$, parallelism $\Theta(n^2/\lg n)$.
- 1. P-SCAN-2: work $\Theta(n \lg n)$, span $\Theta(\lg^2 n)$, parallelism $\Theta(n/\lg n)$.
- 1. Fill in blanks for P-SCAN-UP/P-SCAN-DOWN (two-pass scan).
- 1. Analyze P-SCAN-3: work $\Theta(n)$, span $\Theta(\lg n)$, parallelism $\Theta(n/\lg n)$.
- 1. Rewrite without temporary array t .
- 1. In-place P-SCAN-4 with $O(1)$ auxiliary.
- 1. Parentheses matching using +-scan: $(\rightarrow 1,) \rightarrow -1$; well-formed if scan never negative and total sum = 0.

Problem 26-5: Parallelizing a Simple Stencil

- 1. 4-way divide ($A_{ii}, A_{i+1,i}, A_{i,i+1}, A_{i+1,i+1}$). Recurrences and parallelism.
- 1. 9-way divide (3×3 subarrays).
- 1. General b^2 -way divide: parallelism $\Theta(n)$ for any $b \geq 2$.
- 1. Stencil achieving $\Theta(n/\lg n)$ parallelism. Inherent parallelism is $\Theta(n)$.

Problem 26-6: Randomized Parallel Algorithms

- 1. Modify work law, span law, greedy bound for expectations ($E[T_\infty]$, $E[T_\infty]$, $E[T_P]$).
- 1. Speedup = $E[T_\infty]/E[T_P]$ (not $E[T_\infty]/T_P$).
- 1. Parallelism = $E[T_\infty]/E[T_P]$.
- 1. P-RANDOMIZED-QUICKSORT.

- 1. Analyze P-RANDOMIZED-QUICKSORT.
 - 1. Parallelize RANDOMIZED-SELECT.
-

Ch. 27 – Online Algorithms

Named Entities (Terms & Definitions)

- **Online algorithm:** Receives input progressively over time; decisions made without knowledge of future input.
- **Offline algorithm:** Has the entire input available at the start.
- **Competitive analysis:** Worst-case comparison of an online algorithm with an optimal algorithm that knows the future.
- **Competitive ratio:** For minimization problems, $\max\{ A(I)/F(I) : I \in U \}$ where A is online algorithm, F is optimal future-knowing algorithm.
- **c-competitive:** An online algorithm with competitive ratio c .
- **Seer (or FORESEE):** An optimal offline algorithm that knows the entire future input sequence.
- **Inversion:** A pair (a,b) where a appears before b in one list but b appears before a in another list.
- **Inversion count $I(L,L')$:** Number of pairs of elements whose order differs between two lists.
- **Potential function Φ :** Used in amortized analysis to bound competitive ratio. Here $\Phi = 2 \cdot (\text{inversion count between MOVE-TO-FRONT and FORESEE lists})$.
- **Cache hit:** When a requested block is already in the cache.
- **Cache miss:** When a requested block is not in the cache.
- **Evict:** Remove a block from the cache to make room for a new block.
- **Epoch (in caching analysis):** A maximal sequence of requests containing at most k distinct blocks.
- **Unbounded competitive ratio:** One that grows with the input size n (e.g., $\Theta(n/k)$).
- **Adversary:** Entity that generates worst-case input to test the online algorithm.
- **Oblivious adversary:** Does not know the random choices of the online algorithm.
- **Nonoblivious adversary:** Knows the random choices (stronger adversary).
- **Expected competitive ratio:** c such that $E[A(I)] \leq c \cdot F(I)$ for all inputs I , expectation over algorithm's random choices.
- **FIFO (First-In, First-Out):** Evict the block that has been in the cache the longest.
- **LIFO (Last-In, First-Out):** Evict the block that has been in the cache the shortest time.
- **LRU (Least Recently Used):** Evict the block whose last use is furthest in the past.
- **LFU (Least Frequently Used):** Evict the block accessed the fewest times (break ties by longest in cache).

- **MARKING**: Deterministic caching algorithm. Maintains a mark bit per block. On miss: if all marked, unmark all; evict an arbitrary unmarked block.
- **RANDOMIZED-MARKING**: Randomized variant: on miss, evict a uniformly random unmarked block.

Processes / Algorithms / Pathways

Stairs vs. Elevator (Hedging)

- **Goal**: Decide whether to wait for elevator or take stairs, minimizing worst-case time.
- **Input**: k floors up, elevator takes 1 min for k floors, stairs take k min. Elevator arrives in m minutes ($0 \leq m \leq B-1$). Know k, B ; don't know m .
- **Seer's optimal**: $t(m) = \min(k, m+1)$. If $m \leq k-1$, wait (time $m+1$); else take stairs (time k).
- **Algorithm "always stairs"**: competitive ratio = k (worst when elevator arrives immediately: $k/1 = k$).
- **Algorithm "always elevator"**: competitive ratio = B/k (worst when elevator takes $B-1$ min: B/k).
- **Hedging strategy**: Wait k minutes, then take stairs. $h(m) = (m+1)$ if $m < k$, else $2k$. Competitive ratio = $\max((k+1)/1, 2k/(k+1), 2) = 2$.
- **Exercise 27.1-1**: Wait p minutes instead of k . Competitive ratio depends on p, k . Optimal $p = k$ (ratio = 2).
- **Exercise 27.1-2 (Ski rental)**: Rent skis at $\$r/\text{day}$, buy at $\$b$. Rent until you've rented $\lceil b/r \rceil$ days, then buy. Competitive ratio = 2.
- **Exercise 27.1-3 (Concentration solitaire)**: 2-competitive algorithm: on each turn, if you know where a matching card is, turn it; otherwise turn two unknown cards.

MOVE-TO-FRONT (List Maintenance)

- **Goal**: Maintain ordered linked list to minimize search cost.
- **Input**: Doubly linked list L of n elements; sequence of search requests.
- **Operation**: Search for x (cost = position r). Then swap x forward to front with $r-1$ swaps (total cost = $2r-1$).
- **Complexity**: Competitive ratio = 4 (Theorem 27.1).
- **Proof**: Use potential function $\Phi = 2 \cdot I(L_M, L_F)$. Show amortized cost $\hat{c}_i \leq 4 \cdot f_i$ where f_i is FORESEE's cost.
 - Break elements into sets BB (before x in both lists), BA (before in M , after in F), AB (after in M , before in F).
 - Position of x in M : $r_M(x) = |BB| + |BA| + 1$.
 - Position of x in F : $r_F(x) = |BB| + |AB| + 1$.
 - Swap with $y \in BB$: inversion count +1. Swap with $z \in BA$: inversion count -1.
 - Potential change from MOVE-TO-FRONT: $+2(|BB| - |BA|)$.
 - FORESEE's t_i swaps change potential by at most $2t_i$.

- Amortized cost $\hat{c}_i = (\text{actual cost}) + (\Phi_i - \Phi_{i-1}) \leq 4 \cdot f_i$.
- Summing: total MOVE-TO-FRONT cost $\leq 4 \cdot \text{total FORESEE cost}$.
- **Exercise 27.2-1:** Optimal static list: sort by decreasing probability. Expected cost $= m \cdot \sum i \cdot p(x_i)$ where p sorted decreasing.
- **Exercise 27.2-2:** Counterexample: FORESEE may pay more in a single step than MOVE-TO-FRONT (but less overall).
- **Exercise 27.2-3:** Frequency count algorithm: maintain sorted-by-frequency list. Is it $O(1)$ -competitive? (Likely not bounded.)
- **Exercise 27.2-4:** In model where moving forward is free, MOVE-TO-FRONT is 2-competitive using potential $\Phi = I(L_M, L_F)$.

LIFO Caching (Theorem 27.2)

- **Goal:** Analyze competitive ratio of LIFO.
- **Input:** Request sequence $1, 2, \dots, k, k+1, k, k+1, k, \dots$ (alternating k and $k+1$ after first $k+1$).
- **LIFO behavior:** Every request causes a miss (n misses).
- **Optimal:** Evict any block except k on first $k+1$ request; one eviction total; $k+1$ misses.
- **Competitive ratio:** $\Theta(n/k)$ (unbounded – grows with input size).

LFU Caching

- **Exercise 27.3-2:** Also has $\Theta(n/k)$ competitive ratio (unbounded).

LRU Caching (Theorem 27.3)

- **Goal:** Show LRU has $O(k)$ competitive ratio.
- **Analysis via epochs:** Epoch i begins upon $(k+1)$ st distinct request since epoch $i-1$ began.
- **LRU in an epoch:** Only first request for each block can cause a miss; at most k misses per epoch.
- **Optimal in an epoch:** First request of each epoch must cause a miss (block was not among k most recent distinct).
- **Competitive ratio:** $\leq k/1 = O(k)$.
- **Exercise 27.3-1:** Show cache contents after each request for example sequence.

FIFO Caching

- **Exercise 27.3-3:** Also $O(k)$ competitive ratio.

Deterministic Caching Lower Bound (Theorem 27.4)

- **Statement:** Any deterministic online caching algorithm has competitive ratio $\Omega(k)$.

- **Proof:** Adversary uses $k+1$ distinct blocks. After first k requests, cache full on blocks $1..k$. Next request = $k+1$, algorithm evicts block b_1 . Adversary next requests b_1 , algorithm evicts b_1 , etc. Online incurs n misses. Optimal (furthest-in-future) evicts block whose next request is furthest away; after first k misses, at most one miss per k requests. Total optimal misses $\leq k + n/k$. Ratio $\geq n/(k + n/k) \geq k/2$ for $n \geq k^2$.

RANDOMIZED-MARKING

- **Goal:** $O(\lg k)$ expected competitive ratio against oblivious adversary.
- **Steps:** (1) If b in cache, mark it. (2) Else (cache miss): if all blocks marked, unmark all. (3) Select unmarked block uniformly at random. (4) Evict it, place b , mark b .
- **Epochs:** A new epoch begins after each time all blocks are unmarked. Each epoch has k requests for k distinct blocks.
- **Analysis:** New requests (blocks not requested in previous epoch): always cause miss. Old requests (blocks from previous epoch): miss probability = $n_{ij}/(k-o_{ij})$ where n_{ij} = new requests before j th old request, o_{ij} = old requests before j th old request.
- **Expected misses per epoch:** $E[X_i] = r_i + r_i \cdot H_k$ (where r_i = number of new requests, H_k = k th harmonic number).
- **Optimal offline bound:** Over two consecutive epochs, at least r_i misses ($k + r_i$ distinct requests in two epochs).
- **Expected competitive ratio:** $O(\lg k)$ (since $H_k = O(\lg k)$).
- **Lemma 27.6 (Ball lemma):** Bag with $x-1$ blue, y white, 1 red balls. Draw without replacement until m blue/red drawn. Probability red is drawn = m/x . (White balls irrelevant.)

Modified MOVE-TO-FRONT (free moves)

- **Exercise 27.2-4:** In cost model where moving forward is free, competitive ratio = 2. Potential function $\Phi = I(L_M, L_F)$.

Design Paradigms

- **Competitive analysis:** Compare online decisions against optimal offline algorithm that knows future. Minimax: algorithm vs. adversary.
- **Hedging (wait-and-see):** Wait for a threshold time, then fall back to alternative. Achieves constant competitive ratio.
- **Potential function method:** Use Φ to bound difference between online and optimal costs over sequence.
- **Inversion counting:** Track how far online list diverges from optimal list.
- **Epoch analysis:** Partition request sequence into epochs of k distinct requests. Bound misses per epoch.

- **Randomization against oblivious adversary:** Random choices prevent adversary from knowing cache contents, leading to $O(\lg k)$ vs. $\Omega(k)$ ratio.
- **Adversary models:** Oblivious vs. nonoblivious. Randomized algorithms need oblivious adversary for benefit.

Comparisons & Trade-offs

Algorithm	Competitive Ratio	Type
Always Stairs	k (function of instance)	Deterministic
Always Elevator	B/k (function of parameters)	Deterministic
Hedging (wait k min)	2	Deterministic
MOVE-TO-FRONT	4	Deterministic
MOVE-TO-FRONT (free moves)	2	Deterministic
LIFO	$\Theta(n/k)$ (unbounded)	Deterministic
LFU	$\Theta(n/k)$ (unbounded)	Deterministic
LRU	$O(k), \Omega(k)$	Deterministic
FIFO	$O(k), \Omega(k)$	Deterministic
Any Deterministic Caching	$\Omega(k)$ (lower bound)	Deterministic
RANDOMIZED-MARKING	$O(\lg k)$	Randomized
Ski Rental (rent until $\lceil b/r \rceil$, then buy)	2	Deterministic
Concentration Solitaire	2	Deterministic

Strategy	Best Case	Worst Case	Competitive Ratio
Always stairs	$m=0$, time= k , opt= 1	k	
Always elevator	$m=0$, time= 1 , opt= 1	B/k	
Hedging (wait k)	$m=0$, time= 1 , opt= 1	2	

Formulas & Equations

Competitive Ratio (Minimization)

$$c = \max \{ A(I) / F(I) : I \in U \}$$

Seer's Time (Stairs vs. Elevator)

$$t(m) = \{ m+1 \text{ if } m < k; k \text{ if } m \geq k \}$$

Hedging Strategy Time

$$h(m) = \{ m+1 \text{ if } m < k; 2k \text{ if } m \geq k \}$$

MOVE-TO-FRONT Cost

$$\text{cost} = 2 \cdot r - 1 \quad (r = \text{position}; r \text{ for search} + r-1 \text{ for swaps})$$

MOVE-TO-FRONT Inversion Change

$\Delta I = |BB| - |BA|$ after MOVE-TO-FRONT swaps

MOVE-TO-FRONT Potential Function

$$\Phi_i = 2 \cdot I(L_M^{\{(i)\}}, L_F^{\{(i)\}})$$

Amortized Cost Bound

$$\hat{c}_i = c_i + \Phi_i - \Phi_{i-1} \leq 4 \cdot f_i$$

Overall Bound

$$\sum c_i \leq \sum \hat{c}_i \leq 4 \cdot \sum f_i$$

RANDOMIZED-MARKING Expected Misses per Epoch

$E[X_i] = r_i + r_i \cdot H_k$ where r_i = new requests, $H_k = \sum_{j=1}^k \frac{1}{j}$

Harmonic Number

$$H_k \approx \ln k + \gamma \approx \Theta(\lg k)$$

Expected Competitive Ratio of RANDOMIZED-MARKING

$$E[A(I)] / F(I) \leq 2 \cdot H_k + (H_k \cdot H_{k-1}) / (\text{any positive}) = O(\lg k)$$

Optimal Offline Misses Bound

Over epochs $i-1$ and i : $m_i + m_{i-1} \geq r_i$

Miss Probability for j th Old Request

$\Pr[\text{miss}] = n_{ij} / (k - j + 1)$ where n_{ij} = new requests before j th old request

Deterministic Caching Lower Bound

$$\text{competitive ratio} \geq n / (k + n/k) \geq k/2 \text{ for } n \geq k^2$$

Rules, Laws & Theorems

Theorem 27.1 (MOVE-TO-FRONT Competitive Ratio)

- **Statement:** MOVE-TO-FRONT has competitive ratio 4.
- **Proof:** Potential function $\Phi = 2 \cdot I(L_M, L_F)$. Amortized analysis.

Theorem 27.2 (LIFO Competitive Ratio)

- **Statement:** LIFO has competitive ratio $\Theta(n/k)$ for caching.
- **Proof:** Lower bound $\Omega(n/k)$ via alternating $k/k+1$ sequence. Upper bound $O(n/k)$ since any algorithm $\leq n$ misses, optimal $\geq k$.

Theorem 27.3 (LRU Competitive Ratio)

- **Statement:** LRU has $O(k)$ competitive ratio.
- **Proof:** Epoch analysis – each epoch starts with $(k+1)$ st distinct request; LRU $\leq k$ misses/epoch; optimal ≥ 1 miss/epoch.

Theorem 27.4 (Deterministic Caching Lower Bound)

- **Statement:** Any deterministic online caching algorithm has competitive ratio $\Omega(k)$.
- **Proof:** Adversary with $k+1$ blocks forces n misses on deterministic algorithm; optimal (furthest-in-future) has $\leq k + n/k$ misses.

Theorem 27.5 (RANDOMIZED-MARKING Competitive Ratio)

- **Statement:** RANDOMIZED-MARKING has expected competitive ratio $O(\lg k)$ against oblivious adversary.
- **Proof:** Epoch analysis + ball lemma + harmonic numbers.

Lemma 27.6 (Ball/Urn Lemma)

- **Statement:** Given $x-1$ blue, y white, 1 red balls. Draw m blue/red balls without replacement. Probability red is among them $= m/x$.

Competitive Ratio Properties

- Always ≥ 1 (for minimization problems).
- Closer to 1 is better.

Edge Cases & Pitfalls

- **Deterministic caching lower bound $\Omega(k)$:** No deterministic algorithm can beat this; randomization is necessary for $O(\lg k)$ ratio.
- **LIFO/LFU unbounded ratio:** Competitive ratio grows with n (input size), while LRU/FIFO ratio depends only on k . Prefer algorithms whose ratio does not depend on n .
- **Furthest-in-future:** Optimal offline caching algorithm (Section 15.4), but not implementable online since it requires future knowledge.

- **Nonoblivious adversary:** Knows random choices; can make randomized algorithms perform no better than deterministic. We assume oblivious adversary.
- **Epoch analysis for RANDOMIZED-MARKING:** Cannot bound optimal offline in a single epoch (might start with ideal cache); need two consecutive epochs.
- **MOVE-TO-FRONT potential function:** Factor of 2 captures "1 for searching + 1 for swapping" per inversion.
- **Ski rental:** Competitive ratio of 2 is optimal for this problem.
- **Hedging threshold:** Optimal wait time = k (the stairs time), giving competitive ratio 2.
- **Concentration solitaire:** Need memory of seen cards. Algorithm that never turns a known card and turns two unknown cards when no match known is 2-competitive.

End-of-Chapter Material

Key Terms

online algorithm, offline algorithm, competitive analysis, competitive ratio, c -competitive, seer (FORESEE), MOVE-TO-FRONT, inversion count, potential function, caching, cache hit, cache miss, eviction, FIFO, LIFO, LRU, LFU, MARKING, RANDOMIZED-MARKING, epoch, adversary, oblivious adversary, nonoblivious adversary, expected competitive ratio, unbounded competitive ratio.

Exercises

- **27.1-1:** Hedging with wait p minutes. Find competitive ratio $f(p,k)$. Optimal $p = k$.
- **27.1-2 (Ski rental):** Rent until $\lceil b/r \rceil$, then buy. Competitive ratio = 2.
- **27.1-3 (Concentration solitaire):** 2-competitive algorithm.
- **27.2-1:** Optimal static list: sort by decreasing probability. Expected cost = $m \cdot \sum_i p(x_i)$.
- **27.2-2:** Counterexample where FORESEE pays more than MOVE-TO-FRONT in one step.
- **27.2-3:** Frequency count algorithm: is it $O(1)$ -competitive? (Open-ended.)
- **27.2-4:** MOVE-TO-FRONT is 2-competitive when moving forward is free. Use $\Phi = I(L_M, L_F)$.
- **27.3-1:** Show cache contents for example sequence; count misses per epoch.
- **27.3-2:** LFU competitive ratio $\Theta(n/k)$.
- **27.3-3:** FIFO competitive ratio $O(k)$.
- **27.3-4:** Deterministic MARKING competitive ratio $O(k)$.
- **27.3-5:** Any deterministic 1-lookahead algorithm has competitive ratio $\Omega(k)$ for any constant l .

Problem 27-1: Cow-Path Problem

- **Scenario:** Hiker left book somewhere on trail of unknown length. Need to find it with bounded competitive ratio (distance walked / optimal distance).
- **Solution:** Zigzag: go forward 1 unit, back 2 units, forward 4 units, back 8 units, etc. (doubling). Competitive ratio = 9 (or constant).

Problem 27-2: Online Scheduling to Minimize Average Completion Time

- **Setup:** Tasks with release times r_i , processing times p_i , nonpreemptive.
 - 1. Shortest processing time (SPT) is not d -competitive for any constant d .
 - 1. SRPT (shortest remaining processing time) as online preemptive algorithm.
 - 1. SRPT completion times C^{SRPT}_i satisfy $C^{\text{SRPT}}_i \leq 2 \cdot C^*_i$ (optimal nonpreemptive completion times).
 - (d)-(f) COMPLETION-TIME-SCHEDULE: use SRPT to get order, then schedule nonpreemptively in that order.
 - 1. Online version is 2-competitive.
-

Ch. 28 – Matrix Operations

Named Entities (Terms & Definitions)

- **Linear system:** Set of n equations in n unknowns: $Ax = b$.
- **Nonsingular matrix:** Has an inverse; rank equals n .
- **Underdetermined system:** Fewer equations than unknowns (or rank $< n$); typically infinitely many solutions.
- **Overdetermined system:** More equations than unknowns; may have no exact solution.
- **LUP decomposition:** $PA = LU$ where L is unit lower-triangular, U is upper-triangular, P is permutation matrix.
- **LU decomposition:** $A = LU$ ($P = I$, no pivoting). Requires no 0s on diagonal; works for symmetric positive-definite matrices.
- **Unit lower-triangular matrix:** Lower-triangular with 1s on diagonal.
- **Upper-triangular matrix:** All entries below diagonal are 0.
- **Permutation matrix:** Matrix with exactly one 1 in each row and column, 0s elsewhere.
- **Forward substitution:** Solving $Ly = b$ for y (y_1 first, then y_2 , etc.). $\Theta(n^2)$ time.
- **Back substitution:** Solving $Ux = y$ for x (x_n first, then x_{n-1} , etc.). $\Theta(n^2)$ time.
- **Gaussian elimination:** Subtract multiples of equations to eliminate variables, producing upper-triangular form.
- **Pivot:** Diagonal element used as denominator in Gaussian elimination. In LUP, choose element with largest absolute value for numerical stability.

- **Pivoting:** Permuting rows to avoid division by 0 or small numbers.
- **Schur complement:** $A' = v w^T / a_{ii}$ (for LU). General form: $S = C - B A_k^{-1} B$ (for SPD matrices).
- **Numerical stability:** How much round-off errors are amplified during computation.
- **Numerically unstable:** Algorithm where round-off errors grow unacceptably.
- **Symmetric positive-definite (SPD) matrix:** $A = A^T$ and $x^T A x > 0$ for all $x \neq 0$.
- **Leading submatrix A_k :** Intersection of first k rows and first k columns of A .
- **Least-squares solution:** Minimizes $\| \eta \|^2 = \| A c - y \|^2$ for overdetermined system.
- **Normal equation:** $A^T A c = A^T y$.
- **Pseudoinverse $A^\dagger$:** $(A^T A)^{-1} A^T$. Generalizes inverse to non-square matrices.
- **Basis functions:** Functions $f_j(x)$ used to form $F(x) = \sum c_j f_j(x)$. Common: $f_j(x) = x^{j-1}$ (polynomial).
- **Approximation error:** $\eta = A c - y$ (m-vector of errors).
- **Hermitian matrix:** $A = A^*$ (A^* = conjugate transpose). Needed for complex matrix inversion.
- **SVD (Singular Value Decomposition):** $A = Q \Sigma Q^T$ where Σ is diagonal, Q, Q^T have orthonormal columns.
- **Orthonormal vectors:** Inner product = 0; each has norm 1.

Processes / Algorithms / Pathways

LUP-SOLVE (Forward and Back Substitution)

- **Goal:** Solve $Ax = b$ given LUP decomposition $PA = LU$.
- **Input:** L (unit lower-triangular), U (upper-triangular), permutation array π , vector b , size n .
- **Steps:**
 1. Forward substitution: for $i=1..n$: $y_i = b_{\pi[i]} - \sum_{j=1}^{i-1} l_{ij} \cdot y_j$.
 2. Back substitution: for $i=n..1$: $x_i = (y_i - \sum_{j=i+1}^n u_{ij} \cdot x_j) / u_{ii}$.
- **Complexity:** $\Theta(n^2)$ time.
- **Example:** $A = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 5 & 4 \\ 5 & 6 & 3 \end{bmatrix}$, $b = [3, 0, 1]^T$. LUP: $L = \begin{bmatrix} 1 & 0 & 0 \\ 0.2 & 1 & 0 \\ 0.6 & 0.5 & 1 \end{bmatrix}$, $U = \begin{bmatrix} 5 & 6 & 3 \\ 0 & 0.8 & -0.6 \\ 0 & 0 & 2.5 \end{bmatrix}$, P swaps rows 1 $\leftrightarrow$ 3. Forward substitution: $y_\pi = b_\pi$, $y_\pi[1]=1$, $y_\pi = b_\pi[2] - 0.2 \cdot 1 = 2.8$, $y_\pi = b_\pi[3] - 0.6 \cdot 1 - 0.5 \cdot 2.8 = -1$. Back substitution: $x_\pi = -1/2.5 = -0.4$, $x_\pi = (2.8 - (-0.6)) / 0.8 = 3.2$, $x_\pi = (3 - 6 \cdot 3.2 - 3 \cdot (-0.4)) / 5 = \dots$ $x = [?, 3.2, -0.4]^T$.

LU-DECOMPOSITION

- **Goal:** Factor $A = LU$ (no pivoting). Requires A nonsingular with nonzero pivots.
- **Input:** $n \times n$ nonsingular matrix A .

- **Steps:**
 1. Initialize L (1s on diagonal, 0s above) and U (0s below diagonal).
 2. For $k = 1$ to n :
 - Set $u_{kk} = a_{kk}$ (pivot).
 - For $i = k+1$ to n : $l_{ik} = a_{ik} / a_{kk}$; $u_{ki} = a_{ki}$.
 - For $i = k+1$ to n : For $j = k+1$ to n : $a_{ij} = a_{ij} - l_{ik} \cdot u_{kj}$ (Schur complement).
 1. Return L, U.
- **Complexity:** $\Theta(n^3)$.
- **Merging L and U in-place:** a_{ij} stores l_{ij} (if $i > j$) or u_{ij} (if $i \leq j$).
- **Recursive formulation:** Partition A into $\begin{bmatrix} a_{11} & w \\ v & A' \end{bmatrix}$. Then $A = \begin{bmatrix} 1 & 0 \\ v/a_{11} & I \end{bmatrix} \cdot \begin{bmatrix} a_{11} & w \\ 0 & A' - vw/a_{11} \end{bmatrix}$.

LUP-DECOMPOSITION

- **Goal:** Factor $PA = LU$ with pivoting for numerical stability.
- **Input:** $n \times n$ nonsingular matrix A.
- **Steps:**
 1. Initialize permutation array $\pi[i] = i$.
 2. For $k = 1$ to n :
 - Find k' ($i \geq k$) with $\max |a_{ik}|$ (the pivot).
 - If pivot = 0, matrix is singular.
 - Exchange $\pi[k] \leftrightarrow \pi[k']$; exchange rows k and k' of A.
 - For $i = k+1$ to n : $a_{ik} = a_{ik} / a_{kk}$.
 - For $i = k+1$ to n : For $j = k+1$ to n : $a_{ij} = a_{ij} - a_{ik} \cdot a_{kj}$.
- **Complexity:** $\Theta(n^3)$ (pivoting costs at most constant factor).
- **In-place:** L and U stored in A (L below diagonal, U above and on diagonal).

Matrix Inversion from LUP Decomposition

- **Goal:** Compute A^{-1} given LUP decomposition.
- **Steps:** Solve $A \cdot X_i = e_i$ (where e_i is i th unit vector) for each column X_i of X . Use LUP-SOLVE. Complexity: $\Theta(n^3)$ total (n columns $\times \Theta(n^2)$ each + $\Theta(n^3)$ for decomposition).

Theorem 28.1: Multiplication is No Harder Than Inversion

- **Statement:** If $I(n) = \Omega(n^2)$ and $I(3n) = O(I(n))$, then $M(n) = O(I(n))$.
- **Proof:** Embed A and B into $3n \times 3n$ matrix $D = \begin{bmatrix} I & A & 0 \\ 0 & I & B \\ 0 & 0 & I \end{bmatrix}$. Then $D^{-1} = \begin{bmatrix} I & -A & AB \\ 0 & I & -B \\ 0 & 0 & I \end{bmatrix}$. Extract AB from upper-right $n \times n$ submatrix.

Theorem 28.2: Inversion is No Harder Than Multiplication

- **Statement:** If $M(n) = \Omega(n^2)$, $M(n+k) = O(M(n))$ for $0 \leq k < n$, and $M(n/2) \leq cM(n)$ for $c < 1/2$, then $I(n) = O(M(n))$.

- **Proof (SPD case):** Partition $A = \begin{bmatrix} B & C \\ C^T & D \end{bmatrix}$. Schur complement $S = D - CB^{-1}C^T$.
 - Steps: (1) Form submatrices; (2) Recursively invert B^{-1} ; (3) Compute $W = CB^{-1}$; (4) Compute $X = WC^T$, $S = D - X$; (5) Recursively invert S^{-1} ; (6) Compute $Y = S^{-1}W$; (7) $Z = W^T Y = B^{-1}C^T S^{-1}CB^{-1}$; (8) $R = B^{-1} + Z$; (9) $T = -Y^T$; (10) $U = -Y$; (11) $V = S^{-1}$.
 - Recurrence: $I(n) = 2I(n/2) + 4M(n/2) + O(n^2) = O(M(n))$ by Master Theorem.
- **General nonsingular A:** $A^{-1} = (A^T A)^{-1} A^T$. Compute $A^T A$ (SPD), invert via divide-and-conquer, multiply by A^T .
- **If n not power of 2:** Embed A in $(n+k) \times (n+k)$ matrix $A' = \begin{bmatrix} A & 0 \\ 0 & I \end{bmatrix}$, where $n+k$ is power of 2.

Least-Squares Approximation

- **Goal:** Find coefficients c for $F(x) = \sum c_j f_j(x)$ that minimize $\|A^T c - y\|^2$.
- **Input:** m data points (x_i, y_i) , n basis functions f_j ($n < m$).
- **Steps:**
 1. Form $m \times n$ matrix A where $a_{ij} = f_j(x_i)$.
 2. Form normal equation: $A^T A c = A^T y$.
 3. Since $A^T A$ is symmetric positive-definite (if A has full column rank), solve via LU decomposition.
 4. $c = (A^T A)^{-1} A^T y = A^{-T} y$.
- **Example:** Data points $(-1,2), (1,1), (2,1), (3,0), (5,3)$. Fit quadratic $F(x) = c_0 + c_1 x + c_2 x^2$. $A = \begin{bmatrix} 1 & -1 & 1 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \\ 1 & 5 & 25 \end{bmatrix}$. Pseudoinverse A^{-T} computed. $c = [1.200, -0.757, 0.214]^T$. $F(x) = 1.200 - 0.757x + 0.214x^2$.
- **Keeling curve example:** CO₂ concentrations from Mauna Loa, 1990–2019. $F(x) = c_0 + c_1 x + c_2 x^2 + c_3 \sin(2\pi x) + c_4 \cos(2\pi x)$. Result: $352.83 + 1.39x + 0.02x^2 + 2.83\sin(2\pi x) - 0.94\cos(2\pi x)$.

Natural Cubic Splines (Problem 28-2)

- **Goal:** Interpolate $n+1$ points with piecewise-cubic curve that is continuous in value, first derivative, and second derivative.
- **Assumptions:** $x_i = i$ for simplicity. Second derivative = 0 at endpoints (natural spline).
- **Matrix equation:** Tridiagonal system in unknown derivatives D_i . Can solve in $O(n)$ time (Problem 28-1).

Tridiagonal Systems (Problem 28-1)

- **Goal:** Solve $Ax = b$ for tridiagonal A (nonzeros only on main, super-, and sub-diagonals).
- **Key insight:** LU decomposition of tridiagonal matrix is $O(n)$ (each step eliminates one subdiagonal).
- **Same for LUP decomposition:** $O(n)$.

- **Inverse is dense:** Forming A^{-1} is asymptotically more expensive ($\Omega(n^2)$).

Design Paradigms

- **Decomposition methods:** Break matrix problem into triangular factors (LU, LUP) for efficient solving.
- **Elimination (Gaussian elimination):** Systematically eliminate variables by subtracting multiples of equations.
- **Divide-and-conquer for matrix inversion:** Partition into 2×2 block structure; recursively invert submatrices.
- **Schur complement technique:** Reduce matrix to smaller subproblem by "completing the square."
- **Least-squares via normal equations:** Transform overdetermined system into square SPD system $A^T A c = A^T y$.
- **Pivoting for stability:** Permute rows to bring largest element to pivot position.
- **In-place computation:** Store L and U in the same matrix A to save space.

Comparisons & Trade-offs

Method	Time	Use Case
$x = A^{-1}b$	$\Theta(n^3)$ inv + $\Theta(n^2)$ multiply	Simple but numerically unstable
LUP decomposition + solve	$\Theta(n^3)$ decomp + $\Theta(n^2)$ solve	Numerically stable, preferred
LU (SPD only)	$\Theta(n^3)$	No pivoting needed for SPD
Normal eqns (least squares)	$O(M(n))$	Overdetermined systems
	LU	LUP
Pivoting	None	Row permutation for max
Valid for	Matrices with nonzero pivots (e.g., SPD)	All nonsingular matrices
Numerical stability	Poor if small pivots	Good (largest pivot chosen)
Time	$\Theta(n^3)$	$\Theta(n^3)$

Formulas & Equations

Linear System

$$Ax = b$$

Solution via Inverse

$$x = A^{-1}b$$

LUP Decomposition

$$PA = LU$$

LU Decomposition (Recursive)

$$A = \begin{bmatrix} [a_{11}, w], & [v, A'] \end{bmatrix} \\ = \begin{bmatrix} [1, 0], & [v/a_{11}, I] \end{bmatrix} \cdot \begin{bmatrix} [a_{11}, w], & [0, A' - vw/a_{11}] \end{bmatrix}$$

Schur Complement (General)

$$S = A' - vw / a_{11}$$

Schur Complement (SPD, block form)

$$S = C - B^T A_k^{-1} B$$

Forward Substitution

$$y_i = b_{\{\pi[i]\}} - \sum_{j=1}^{i-1} l_{\{ij\}} y_j$$

Back Substitution

$$x_i = (y_i - \sum_{j=i+1}^n u_{\{ij\}} x_j) / u_{\{ii\}}$$

Matrix Inversion Block Formula (Block 2x2)

$$A = \begin{bmatrix} [B, C], & [C, D] \end{bmatrix} \\ A^{-1} = \begin{bmatrix} [B^{-1} + B^{-1}C S^{-1}CB^{-1}, & -B^{-1}C S^{-1}], \\ [-S^{-1}CB^{-1}, & S^{-1}] \end{bmatrix}$$

where $S = D - CB^{-1}C$ (Schur complement).

Normal Equation

$$A^T A c = A^T y$$

Least-Squares Solution

$$c = (A^T A)^{-1} A^T y = A^+ y$$

Pseudoinverse

$$A^+ = (A^T A)^{-1} A^T$$

Error Norm

$$||\eta||^2 = \sum_{i=1}^m (F(x_i) - y_i)^2 = \sum_{i=1}^m (\sum_{j=1}^n a_{\{ij\}} c_j - y_i)^2$$

Derivative Condition for Least Squares

$$\partial/\partial c_k (||\eta||^2) = 0 \text{ for } k = 1, \dots, n \rightarrow (Ac - y)^\top A = 0$$

Determinant Relation for Pivots (SPD)

$$k\text{th pivot} = \det(A_k) / \det(A_{\{k-1\}})$$

Inverse via $A^\top A$ (general nonsingular A)

$$A^{-1} = (A^\top A)^{-1} A^\top$$

Embedding for Power-of-2 Matrix Inversion

$$A' = \begin{bmatrix} A & 0 \\ 0 & I \end{bmatrix}$$
$$A'^{-1} = \begin{bmatrix} A^{-1} & 0 \\ 0 & I \end{bmatrix}$$

Product Embedding for Theorem 28.1

$$D = \begin{bmatrix} I & A & 0 \\ 0 & I & B \\ 0 & 0 & I \end{bmatrix}$$
$$D^{-1} = \begin{bmatrix} I & -A & AB \\ 0 & I & -B \\ 0 & 0 & I \end{bmatrix}$$

Rules, Laws & Theorems

Lemma 28.3 (SPD $\rightarrow$ Nonsingular)

- **Statement:** Any positive-definite matrix is nonsingular.
- **Proof:** If singular, $\exists x \neq 0$ with $Ax=0 \rightarrow x^\top Ax=0$, contradicting positive-definiteness.

Lemma 28.4 (Leading Submatrices of SPD)

- **Statement:** If A is SPD, then every leading submatrix A_k is also symmetric and positive-definite.
- **Proof:** Symmetry inherits. If A_k not PD, $\exists x_k \neq 0$ with $x_k^\top A_k x_k \leq 0$. Extend with zeros to n -vector; $x^\top Ax = x_k^\top A_k x_k \leq 0$, contradiction.

Lemma 28.5 (Schur Complement Lemma)

- **Statement:** If A is SPD and A_k is leading submatrix, then Schur complement $S = C - B^\top A_k^{-1} B$ is symmetric and positive-definite.
- **Proof:** Symmetry via properties of transpose. For PD: choose $z \neq 0$, set $y = A_k^{-1} Bz$. Then $x^\top Ax = y^\top A_k y + z^\top Sz > 0 \rightarrow z^\top Sz > 0$.

Corollary 28.6 (LU of SPD Never Divides by Zero)

- **Statement:** LU decomposition of an SPD matrix never divides by 0. All pivots are strictly positive.

- **Proof:** First pivot $a_{11} = e^T A e > 0$. By Lemma 28.5, all Schur complements are SPD, so all subsequent pivots positive.

Theorem D.6 ($A^T A$ Positive-Definite)

- **Statement:** If A has full column rank, then $A^T A$ is symmetric positive-definite.
- **Reference:** Used for least-squares normal equations.

Theorem 28.1 (Multiplication $\leq$ Inversion)

- **Statement:** $M(n) = O(I(n))$ under regularity: $I(n) = \Omega(n^2)$, $I(3n) = O(I(n))$.

Theorem 28.2 (Inversion $\leq$ Multiplication)

- **Statement:** $I(n) = O(M(n))$ under regularity: $M(n) = \Omega(n^2)$, $M(n+k) = O(M(n))$, $M(n/2) \leq cM(n)$ for $c < 1/2$.

Edge Cases & Pitfalls

- **Singular matrix:** Determinant = 0, no inverse. LUP-DECOMPOSITION detects when column k has all zeros $\rightarrow$ "singular matrix" error.
- **Underdetermined vs. overdetermined:** Underdetermined $\rightarrow$ infinitely many solutions or inconsistent. Overdetermined $\rightarrow$ no exact solution, use least squares.
- **Numerical instability:** Computing $A^{-1}b$ directly is less stable than LUP decomposition. Small pivots magnify round-off errors. LUP avoids by choosing max pivot.
- **LU decomposition vs. LUP:** LU fails if any $a_{kk} = 0$. Even if nonzero, small a_{kk} causes numerical instability. LUP always pivots.
- **LU for SPD matrices:** No pivoting needed, and all pivots are strictly positive (Corollary 28.6).
- **Matrix inversion via $A^T A$:** Works theoretically but LUP decomposition is preferred (fewer operations, better numerical properties).
- **Power-of-2 assumption:** When n not power of 2, embed in larger matrix with identity block (adds at most constant factor to time).
- **Pseudoinverse:** Requires A to have full column rank. If not, $A^T A$ is singular.
- **Overfitting (least squares):** Choosing $n = m$ fits noise exactly; better to choose $n \ll m$ to capture patterns without overfitting.
- **Integer modulo 2:** Theorem 28.2 inversion algorithm does not work over $GF(2)$ because $1/2$ doesn't exist and $2I(n/2) > I(n)$ violates regularity.
- **Complex matrices:** Need conjugate transpose A^* instead of A^T ; Hermitian matrices instead of symmetric.

- **Determinant as stability measure:** $\det(A)$ is not a good indicator of stability; condition number $\|A\|_\infty \cdot \|A^{-1}\|_\infty$ is better.

End-of-Chapter Material

Key Terms

LUP decomposition, LU decomposition, forward substitution, back substitution, Gaussian elimination, pivot, pivoting, Schur complement, numerical stability, symmetric positive-definite matrix, leading submatrix, normal equation, least-squares approximation, pseudoinverse, basis functions, overdetermined system, underdetermined system, nonsingular, Hermitian matrix, SVD.

Exercises

- **28.1-1:** Solve given system using forward substitution.
- **28.1-2:** Find LU decomposition of a given 3×3 matrix.
- **28.1-3:** Solve using LUP decomposition.
- **28.1-4:** LUP decomposition of a diagonal matrix: $P = I$, $L = I$, $U = D$.
- **28.1-5:** LUP decomposition of a permutation matrix: unique (P rearranges to identity).
- **28.1-6:** Show that for all $n \geq 1$, there exists a singular $n \times n$ matrix with an LU decomposition.
- **28.1-7:** Is iteration $k=n$ needed in LU-DECOMPOSITION? In LUP-DECOMPOSITION?
- **28.2-1:** Squaring and multiplication have same difficulty: $M(n) \rightarrow O(M(n))$ squaring; $S(n) \rightarrow O(S(n))$ multiplication (use $(A+B)^2 = A^2 + AB + BA + B^2$).
- **28.2-2:** $M(n)$ -time multiplication implies $O(M(n))$ -time LUP decomposition.
- **28.2-3:** Boolean matrix multiplication $\rightarrow$ transitive closure in $O(M(n) \lg n)$; transitive closure $\rightarrow$ Boolean MM in $O(T(n))$.
- **28.2-4:** Theorem 28.2 fails over $GF(2)$ because $2I(n/2) \nmid I(n)$ when characteristic is 2; also can't divide by 2.
- **28.2-5:** Generalize to complex matrices: use conjugate transpose A^* ; Hermitian matrices; Schur complement lemma still holds.
- **28.3-1:** Prove every diagonal element of an SPD matrix is positive: $e_i^T A e_i = a_{ii} > 0$.
- **28.3-2:** For 2×2 SPD matrix $\begin{bmatrix} a & b \\ b & c \end{bmatrix}$, prove $ac - b^2 > 0$ by completing the square.
- **28.3-3:** Prove max element of SPD matrix is on diagonal.
- **28.3-4:** Prove determinant of each leading submatrix of SPD matrix is positive.
- **28.3-5:** Prove k th pivot $= \det(A_k) / \det(A_{k-1})$ during LU decomposition of SPD matrix.
- **28.3-6:** Find best least-squares fit of $F(x) = c_0 + c_1 x \lg x + c_2 x^2$ to points $(1,1)$, $(2,1)$, $(3,3)$, $(4,8)$.

- **28.3-7:** Show pseudoinverse $A^\dagger$ satisfies: $AA^\dagger A = A$, $A^\dagger AA^\dagger = A^\dagger$, $(AA^\dagger)^\dagger = AA^\dagger$, $(A^\dagger A)^\dagger = A^\dagger A$.

Problem 28-1: Tridiagonal Systems

- Given tridiagonal matrix A .
- 1. Find LU decomposition.
- 1. Solve $Ax = [1, 1, 1, 1, 1]^\top$.
- 1. Find A^{-1} .
- 1. Solve SPD tridiagonal in $O(n)$ via LU decomposition. Forming A^{-1} is asymptotically more expensive ($\Omega(n^2)$).
- 1. Solve general nonsingular tridiagonal in $O(n)$ via LUP decomposition.

Problem 28-2: Splines

- **Setup:** $n+1$ points (x_i, y_i) with $x_0 < \dots < x_n$. Fit natural cubic spline (piecewise cubic, C^2 continuous).
- **Assumption:** $x_i = i$ for simplicity.
- 1. Given $D_i = f'(x_i)$, express coefficients a_i, b_i, c_i, d_i in terms of $y_i, y_{i+1}, D_i, D_{i+1}$. $4n$ coefficients computable in $O(n)$.
- 1. Using second-derivative continuity: $D_{i-1} + 4D_i + D_{i+1} = 3(y_{i+1} - y_{i-1})$ for $i=1, \dots, n-1$.
- 1. At endpoints: $2D_0 + D_1 = 3(y_1 - y_0)$; $D_{n-1} + 2D_n = 3(y_n - y_{n-1})$.
- 1. Matrix equation for $D = (D_0, \dots, D_n)^\top$ is tridiagonal, symmetric positive-definite (strictly diagonally dominant).
- 1. Natural cubic spline interpolation in $O(n)$ time (tridiagonal solver).
- 1. General x_i (not equally spaced): similar tridiagonal system but with $h_i = x_{i+1} - x_i$. Still $O(n)$.

Chapter Notes

- LUP decomposition based on Gaussian elimination (C.F. Gauss, 1777-1855).
- Strassen's $O(n^{\lg 7})$ matrix inversion.
- Winograd: multiplication no harder than inversion; Aho, Hopcroft, Ullman: converse.
- SVD factors $A = Q\Sigma Q^\top$. References: Strang, Golub & Van Loan, Higham.
- Condition number $\|A\|_\infty \cdot \|A^{-1}\|_\infty$ is better stability indicator than determinant.

Ch. 29 – Linear Programming

Named Entities (Terms & Definitions)

- **Linear-programming problem:** problem of minimizing or maximizing a linear function subject to a finite set of linear constraints
- **Linear function:** $(f(x_1, x_2, \dots, x_n) = a_1 x_1 + a_2 x_2 + \dots + a_n x_n)$
- **Linear equality:** $(f(x_1, \dots, x_n) = b)$
- **Linear inequality:** $(f(x_1, \dots, x_n) \leq b)$ or $(f(x_1, \dots, x_n) \geq b)$
- **Linear constraints:** either linear equalities or linear inequalities
- **Objective function:** the linear function to be optimized (maximized or minimized)
- **Nonnegativity constraints:** constraints $(x_j \geq 0)$ for all (j)
- **Decision variables:** variables whose values are to be chosen
- **Standard form:** maximize $(c^T x)$ subject to $(Ax \leq b)$, $(x \geq 0)$, where (A) is $(m \times n)$, (b) is (m) -vector, (c) is (n) -vector, (x) is (n) -vector
- **Feasible solution:** a setting of variables $(\{x\})$ satisfying all constraints
- **Infeasible solution:** fails at least one constraint
- **Optimal solution:** feasible solution with maximum (or minimum) objective value over all feasible solutions
- **Optimal objective value:** objective value of an optimal solution
- **Feasible region:** set of all points satisfying all constraints
- **Infeasible LP:** LP with no feasible solutions
- **Unbounded LP:** LP with feasible solutions but no finite optimal objective value
- **Simplex:** the feasible region formed by intersection of half-spaces in (n) -dimensional space
- **Integer linear program:** LP with additional requirement that all variables take integer values (NP-hard to solve)
- **Primal:** the original linear program in a primal-dual pair
- **Dual:** the related minimization LP formed from a primal maximization LP
- **Linear-programming duality:** for a maximization primal, a related minimization dual has the same optimal objective value
- **Weak duality:** any feasible primal solution has value $(\leq)$ any feasible dual solution
- **Complementary slackness:** necessary and sufficient conditions for optimality relating primal variables to dual constraints and vice versa
- **Ellipsoid algorithm:** first polynomial-time algorithm for LP (Khachian, 1979)
- **Interior-point methods:** class of polynomial-time LP algorithms that move through interior of feasible region
- **Simplex algorithm:** widely used LP algorithm; moves along edges of simplex from vertex to vertex; exponential worst-case but good in practice

- **Farkas's lemma:** given $(M^{(m+1)n})$ and (g^{m+1}) , exactly one of (1) $(v: Mv \leq g)$, or (2) $(w: w^T M = 0, w^T g < 0)$ holds
- **Fundamental theorem of linear programming:** any LP either (1) has optimal solution with finite value, (2) is infeasible, or (3) is unbounded
- **Minimum-cost-flow problem:** send (d) units of flow from (s) to (t) minimizing total cost $(\sum a_{uv} f_{uv})$
- **Multicommodity-flow problem:** (k) commodities with sources, sinks, demands sharing a network with capacities; determine whether feasible flow exists
- **Linear-inequality feasibility problem:** determine whether a set of linear inequalities has a simultaneous solution

Processes / Algorithms / Pathways

Simplex Algorithm

- **Goal:** find optimal solution to a linear program
- **Input/Output:** takes LP, returns optimal solution
- **Steps:** (1) Start at a vertex of the simplex (feasible region) (2) In each iteration, move along an edge to a neighboring vertex with objective value no smaller (usually larger) (3) Terminate at a local maximum – because feasible region is convex and objective is linear, this local optimum is global optimum
- **Complexity:** exponential worst-case time, but fast in practice; not polynomial-time
- **Example:** For the 2-variable LP maximize $(x_1 + x_2)$ subject to constraints, the simplex starts at $(0,0)$ and moves along edges to $(2,6)$ which has objective value 8 (optimal)

Ellipsoid Algorithm

- **Goal:** solve LP in polynomial time
- **Complexity:** polynomial-time but slow in practice

Interior-Point Methods

- **Goal:** solve LP in polynomial time
- **Steps:** move through interior of feasible region (not along vertices); intermediate solutions are feasible but not necessarily vertices; final solution is a vertex
- **Complexity:** polynomial-time; can be as fast as or faster than simplex for large inputs

Formulating a Problem as an LP

- **Steps:** (1) Identify decision variables (2) Specify constraints (linear equalities/inequalities) (3) Formulate objective function (linear) (4) Enforce nonnegativity if needed

Converting to Standard Form

- **Equality to inequalities:** $(f(x) = b)$ becomes $(f(x) \leq b)$ and $(-f(x) \leq -b)$
- **Inequality to equality:** $(f(x) \leq b)$ becomes $(f(x) + s = b)$ where (s) is a slack variable
- **Minimization to maximization:** minimize $(c^T x)$ is equivalent to maximize $(-c^T x)$

Shortest Path LP Formulation

- **Goal:** compute shortest-path weight (d_t) from source (s) to destination (t)
- **Variables:** (d_v) for each vertex $(v \in V)$
- **Constraints:** $(d_v \leq d_u + w(u,v))$ for each edge $((u,v) \in E)$; $(d_s = 0)$
- **Objective:** maximize (d_t) (maximizing gives tightest upper bound = shortest path)
- **Size:** $(|V|)$ variables, $(|E| + 1)$ constraints

Maximum Flow LP Formulation

- **Variables:** (f_{uv}) for each pair of vertices
- **Constraints:** (1) Capacity: $(f_{uv} \leq c(u,v))$ (2) Flow conservation: $(\sum_{v \in V} f_{vu} - \sum_{v \in V} f_{uv} = 0)$ for all $(u \in V, u \neq s, t)$ (3) Nonnegativity: $(f_{uv} \geq 0)$
- **Objective:** maximize $(\sum_{v \in V} f_{sv} - \sum_{v \in V} f_{vs})$
- **Size:** $(|V|^2)$ variables, $(2|V| + |E|)$ constraints; can be reduced to $(O(V+E))$

Minimum-Cost Flow LP Formulation

- **Goal:** send exactly (d) units of flow from (s) to (t) at minimum cost
- **Variables:** (f_{uv})
- **Constraints:** capacity constraints, flow conservation, plus $(\sum_{v \in V} f_{sv} - \sum_{v \in V} f_{vs} = d)$
- **Objective:** minimize $(\sum_{(u,v) \in E} a(u,v) f_{uv})$

Multicommodity Flow LP Formulation

- **Variables:** (f_{iuv}) for each commodity (i) and each edge
- **Constraints:** flow conservation per commodity + aggregate flow $(\sum_{i \in I} f_{iuv} \leq c(u,v))$
- **Objective:** null (feasibility only)
- **Note:** only known polynomial-time algorithm is via LP

Forming the Dual LP

- **Input:** primal maximization LP: maximize $(c^T x)$ s.t. $(Ax \leq b), (x \geq 0)$

- **Output:** dual minimization LP: minimize ($b^T y$) s.t. ($A^T y \leq c$), (y)
- **Mechanics:** change max to min, swap roles of (c) and (b), transpose (A), reverse inequality direction
- **Interpretation:** each primal constraint corresponds to a dual variable (y_i); each dual constraint corresponds to a primal variable (x_j)

Formulas & Equations

General LP (Standard Form)

[
]

Compact Standard Form

[$c^T x \leq b, x \geq 0$]

Dual LP (from Primal Standard Form)

[$b^T y \leq c^T x, y \geq 0$]

Weak Duality

[$c^T x \leq b^T y$] for any feasible primal ($\{x\}$) and dual ($\{y\}$)

Strong Duality (Theorem 29.4)

If both primal and dual are feasible and bounded, then optimal values are equal: [$c^T x^* = b^T y^*$]

Complementary Slackness Conditions

[$\{y\}_i (b_i - \sum_{j=1}^n a_{ij} \{x\}_j) = 0 \quad i = 1, \dots, m$] [$\{x\}_j (\sum_{i=1}^m a_{ij} \{y\}_i - c_j) = 0 \quad j = 1, \dots, n$]

Rules, Laws & Theorems

Theorem 29.1 (Weak Linear-Programming Duality)

- **Statement:** For any feasible primal ($\{x\}$) and dual ($\{y\}$),
($c^T \{x\} \leq b^T \{y\}$)

Corollary 29.2

- **Statement:** If feasible primal ($\{x\}$) and dual ($\{y\}$) have equal objective values, then both are optimal

Lemma 29.3 (Farkas's Lemma)

- **Statement:** Given $(M \in \mathbb{R}^{(m+1) \times n})$ and $(g \in \mathbb{R}^{m+1})$, exactly one holds: (1) $(\exists v: Mv \leq g)$, or (2) $(\exists w: w^T M = 0, w^T g < 0)$

Theorem 29.4 (Linear-Programming Duality / Strong Duality)

- **Statement:** If primal and dual are both feasible and bounded, then $(c^T x^* = b^T y^*)$ for optimal solutions

Theorem 29.5 (Fundamental Theorem of Linear Programming)

- **Statement:** Any LP in standard form either (1) has an optimal solution with finite objective value, (2) is infeasible, or (3) is unbounded

Comparisons & Trade-offs

Dimension	Simplex	Ellipsoid	Interior-Point
Time complexity	Exponential worst-case, fast in practice	Polynomial, slow in practice	Polynomial, competitive in practice
Path through feasible region	Along exterior (vertices)	N/A	Through interior
Intermediate solutions	Always vertices	N/A	Feasible but not necessarily vertices
Practical use	Most common historically	Rarely used	Common for large problems
Dimension	Coefficient Form		Point-Value Form
Evaluation at single point	$(O(n))$ (Horner)		Inefficient directly
Addition	$(O(n))$		$(O(n))$
Multiplication	$(O(n^2))$ naive		$(O(n))$ (pointwise)
Conversion cost	—		$(n \log n)$ via FFT

Edge Cases & Pitfalls

- Strict inequalities are NOT allowed in linear programming
- An LP can be infeasible (no solution exists) or unbounded (no finite optimal value)
- A feasible region can be unbounded while still having a finite optimal objective value (Exercise 29.1-5)
- Integer LP is NP-hard (no known polynomial-time algorithm)
- The simplex algorithm can require exponential time on contrived inputs (Klee-Minty example: $(2^n - 1)$ iterations)
- When formulating LPs, variables must be linear; products of variables are not allowed

- Nonnegativity constraints are part of standard form but real problems may need variables that can be negative (can be handled by variable substitution)
- For maximum-flow LP, the formulation with $(|V|^2)$ variables can be made more efficient by using only $(O(V+E))$ constraints (Exercise 29.2-4)

End-of-Chapter Material

Exercises 29.1-1 through 29.1-8 (in-chapter exercises on feasible solutions, infeasibility, unboundedness, conversions, political problem constraints)

Exercises 29.2-1 through 29.2-7 (shortest-path LP, single-source LP, max-flow LP, bipartite matching LP, path-based max-flow formulation, minimum-cost multicommodity flow)

Exercises 29.3-1 through 29.3-8 (forming duals, direct dual of arbitrary LP, dual of max-flow as min-cut, dual of min-cost flow, dual of dual is primal, weak duality for max-flow, 1-variable primal/dual conditions, prove fundamental theorem)

Problems: - **29-1 Linear-inequality feasibility:** (a) use LP to solve feasibility; (b) use feasibility to solve LP - **29-2**

Complementary slackness: verify, prove, characterize optimality - **29-3 Integer linear programming:** weak duality holds but strong duality does not always; $(IP \ P = D \ ID)$ - **29-4 Farkas's lemma:** prove Lemma 29.3 - **29-5 Minimum-cost circulation:** LP formulation, optimal solution when all costs positive, reduce max-flow and shortest-path to min-cost circulation

Ch. 30 – Polynomials and the FFT

Named Entities (Terms & Definitions)

- **Polynomial:** function $(A(x) = \sum_{j=0}^{n-1} a_j x^j)$ over a field (F)
- **Coefficient:** the values $(a_0, a_1, \dots, a_{n-1})$
- **Degree:** highest (k) such that $(a_k \neq 0)$; denoted $\text{degree}(A)$
- **Degree-bound:** any integer strictly greater than the degree; a polynomial of degree-bound (n) has degree $(\leq n-1)$
- **Coefficient representation:** vector $(a = (a_0, a_1, \dots, a_{n-1}))$
- **Point-value representation:** set of (n) point-value pairs $(\{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\})$ with distinct (x_k) and $(y_k = A(x_k))$
- **Convolution:** the coefficient vector $(c = a \cdot b)$ of the product polynomial, where $(c_j = \sum_{k=0}^j a_k b_{j-k})$
- **Evaluation:** computing $(A(x_0))$ given coefficient form
- **Interpolation:** determining coefficient form from point-value representation

- **Vandermonde matrix:** ($V(x_0, \dots, x_{n-1})$) with ((k, j)) entry (x_k^j); determinant is ($\prod_{0 \leq j < k \leq n-1} (x_k - x_j)$)
- **Lagrange's formula:** ($A(x) = \sum_{k=0}^{n-1} y_k \prod_{j \neq k} \frac{x - x_j}{x_k - x_j}$)
- **Horner's rule:** evaluate polynomial in ((n)) time: ($A(x) = a_0 + x(a_1 + x(a_2 + \dots + x(a_{n-1})))$)
- **Complex nth root of unity:** (ω) such that ($\omega^n = 1$); there are exactly (n): (ω^k) for ($k=0, 1, \dots, n-1$)
- **Principal nth root of unity:** ($\omega = e^{2\pi i / n}$)
- **Discrete Fourier Transform (DFT):** vector ($y = (y_0, \dots, y_{n-1})$) where ($y_k = A(\omega^k) = \sum_{j=0}^{n-1} a_j \omega^{kj}$)
- **Fast Fourier Transform (FFT):** computes DFT in ($(n \log n)$) time using divide-and-conquer
- **Twiddle factors:** the factors (ω^k) used in butterfly operations
- **Butterfly operation:** computes ($t = \omega^k y_{\text{odd}}[k]$), then ($y[k] = y_{\text{even}}[k] + t$), ($y[k+n/2] = y_{\text{even}}[k] - t$)
- **Bit-reversal permutation:** rearranges input vector so that element (a_k) moves to position ((k)), where ((k)) is the (n)-bit reversal of (k)
- **Chirp transform:** ($y_k = \sum_{j=0}^{n-1} a_j z^{kj}$) for any complex (z); DFT is special case with ($z = \omega$)

Processes / Algorithms / Pathways

Polynomial Addition (Coefficient Form)

- **Goal:** compute ($C(x) = A(x) + B(x)$)
- **Input:** vectors ($a = (a_0, \dots, a_{n-1})$), ($b = (b_0, \dots, b_{n-1})$)
- **Output:** vector ($c = (c_0, \dots, c_{n-1})$) where ($c_j = a_j + b_j$)
- **Complexity:** ((n)) time

Polynomial Multiplication (Naive, Coefficient Form)

- **Goal:** compute ($C(x) = A(x)B(x)$)
- **Steps:** ($c_j = \sum_{k=0}^j a_k b_{j-k}$) for ($j = 0, \dots, 2n-2$)
- **Complexity:** ((n^2)) time

Fast Polynomial Multiplication via FFT

- **Goal:** multiply two degree-bound (n) polynomials in ($(n \log n)$) time
- **Steps:** (1) Double degree-bound: add (n) high-order zero coefficients to each polynomial (degree-bound ($2n$)) (2) Evaluate: compute point-value representations at ($(2n)$)th roots of unity via FFT of order ($2n$) (3) Pointwise multiply: ($C(x_k) = A(x_k)B(x_k)$) for each root (4) Interpolate: apply inverse FFT to get coefficient form of (C)
- **Complexity:** ((n)) for steps 1 and 3, ($(n \log n)$) for steps 2 and 4 – total ($(n \log n)$)

Horner's Rule (Single-Point Evaluation)

- **Goal:** evaluate ($A(x_0)$) in ((n)) time
- **Steps:** ($y = a_{n-1}$); for ($i = n-2$) down to 0: ($y = a_i + x_0 y$)

Lagrange Interpolation

- **Goal:** interpolate polynomial from (n) point-value pairs
- **Formula:** ($A(x) = \sum_{k=0}^{n-1} y_k$)
- **Complexity:** ((n^2)) time (Exercise 30.1-5)

FFT (Cooley-Tukey) Algorithm

- **Goal:** compute ($\text{DFT}_n(a)$) in ($(n \log n)$) time
- **Input:** coefficient vector ($a = (a_0, \dots, a_{n-1})$), (n) is exact power of 2
- **Output:** DFT vector ($y = (y_0, \dots, y_{n-1})$)
- **Steps:**
 1. Base case: if ($n = 1$), return (a)
 2. Split: ($a_{\text{even}} = (a_0, a_2, \dots, a_{n-2})$), ($a_{\text{odd}} = (a_1, a_3, \dots, a_{n-1})$)
 3. Recurse: ($y_{\text{even}} = \text{FFT}(a_{\text{even}}, n/2)$), ($y_{\text{odd}} = \text{FFT}(a_{\text{odd}}, n/2)$)
 4. Combine: for ($k = 0$) to ($n/2 - 1$): ($y_k = y_{\text{even}}[k] + n^k y_{\text{odd}}[k]$) ($y_{k+n/2} = y_{\text{even}}[k] - n^k y_{\text{odd}}[k]$)
 5. Return (y)
- **Complexity:** ($T(n) = 2T(n/2) + (n) = (n \log n)$)
- **Example:** For input ($a = (0, 1, 2, 3)$) with ($n=4$): ($a_{\text{even}} = (0, 2)$), ($a_{\text{odd}} = (1, 3)$); recursively compute DFTs of size 2; combine with twiddle factors to get ($y = (6, -2+2i, -2, -2-2i)$)

Inverse DFT

- **Goal:** interpolate from point-value form to coefficient form
- **Method:** Run FFT with ($\omega_n^{-1} = e^{-2\pi i / n}$) instead of (ω_n), and divide each result by (n)
- **Steps:** (1) Replace (ω_n) by (ω_n^{-1}) (2) After FFT, multiply each element by ($1/n$)
- **Complexity:** ($(n \log n)$)

Bit-Reversal Permutation

- **Goal:** rearrange input for FFT circuit
- **Steps:** element (a_k) moves to position ((k)) where ((k)) is the (n)-bit reversal of (k)
- **Example:** for ($n=8$), order becomes ($0, 4, 2, 6, 1, 5, 3, 7$) (binary: 000, 100, 010, 110, 001, 101, 011, 111)

FFT Circuit (Parallel)

- **Structure:** (n) inputs, (n) stages, each stage has ($n/2$) butterflies operating in parallel
- **Depth:** ((n))
- **Total operations:** ($(n \log n)$)
- **Steps:** (1) Bit-reversal permutation of inputs (2) For ($s = 1$) to ($\log n$): stage (s) has ($n/2^s$) groups with (2^{s-1}) butterflies per group; twiddle factors are (ω_m^k) where ($m = 2^s$)
- **Base case:** (FFT_1) does nothing; (FFT_2) is a single butterfly with twiddle factor ($\omega_2^0 = 1$)

Formulas & Equations

Polynomial Addition

$$[c_j = a_j + b_j \quad j = 0, 1, \dots, n-1]$$

Polynomial Multiplication (Convolution)

$$[c_j = \sum_{k=0}^j a_k b_{j-k} \quad j = 0, 1, \dots, 2n-2]$$

Horner's Rule

$$[A(x) = a_0 + x(a_1 + x(a_2 + \dots + x(a_{n-1})))]$$

Vandermonde Matrix

$$[V(x_0, \dots, x_{n-1}) = \\ [V = \omega_{j,k} = (x_k - x_j)]]$$

Lagrange Interpolation

$$[A(x) = \sum_{k=0}^{n-1} y_k L_k(x)]$$

Complex Roots of Unity

$$[\omega_n^k = e^{2\pi i k / n} = \cos(2\pi k/n) + i \sin(2\pi k/n)] [\omega_n^n = 1, \omega_n^k \neq 1 \quad 0 < k < n]$$

Cancellation Lemma (Lemma 30.3)

$$[\omega_n^{dn} = \omega_n^k]$$

Corollary 30.4

$$[\omega_n^{n/2} = -1]$$

Halving Lemma (Lemma 30.5)

If $(n > 0)$ is even, the squares of the (n) complex (n) th roots of unity are the $(n/2)$ complex $(n/2)$ th roots of unity.

Summation Lemma (Lemma 30.6)

$$\left[\sum_{j=0}^{n-1} (\omega_n^k)^j = 0 \mid k \neq 0 \right]$$

DFT Definition

$$\left[y_k = \sum_{j=0}^{n-1} a_j \omega_n^{kj} \mid k = 0, 1, \dots, n-1 \right]$$

Matrix Form of DFT

$$\left[y = V_n a, \quad V_n[k, j] = \omega_n^{kj} \right]$$

Inverse DFT

$$\left[a_j = \sum_{k=0}^{n-1} y_k \omega_n^{-kj} \mid j = 0, 1, \dots, n-1 \right]$$

Convolution Theorem (Theorem 30.8)

$$\left[a * b = \mathcal{F}^{-1}(\mathcal{F}(a) \mathcal{F}(b)) \right]$$

FFT Recurrence

$$\left[T(n) = 2T(n/2) + (n) = (n \log n) \right]$$

Chirp Transform

$\left[y_k = \sum_{j=0}^{n-1} a_j z^{kj} \right]$ Can be evaluated in $(O(n \log n))$ time using convolution: $\left(y_k = z^{k^2/2} \sum_{j=0}^{n-1} (a_j z^{j^2/2}) z^{-(k-j)^2/2} \right)$

Rules, Laws & Theorems

Theorem 30.1 (Uniqueness of Interpolating Polynomial)

- **Statement:** For any set of (n) point-value pairs with distinct (x_k) , there is a unique polynomial of degree-bound (n) such that $(y_k = A(x_k))$

Theorem 30.2

- **Statement:** Two polynomials of degree-bound (n) can be multiplied in $(n \log n)$ time using FFT

Lemma 30.3 (Cancellation Lemma)

- **Statement:** $(\sum_{k=0}^{d-1} \omega_n^{dk} = 0)$ for any integers $(n > 0, d > 0)$

Lemma 30.5 (Halving Lemma)

- **Statement:** If $(n > 0)$ is even, the squares of the complex (n) th roots of unity are the $(n/2)$ complex $(n/2)$ th roots of unity

Lemma 30.6 (Summation Lemma)

- **Statement:** $(\sum_{j=0}^{n-1} \omega_n^{kj} = 0)$ for any integer (n) and nonzero (k) not divisible by (n)

Theorem 30.7 (Inverse DFT Matrix)

- **Statement:** The $((j,k))$ entry of (V_n^{-1}) is (ω_n^{-jk}/n)

Theorem 30.8 (Convolution Theorem)

- **Statement:** $(a * b = \text{DFT}_{2n}^{-1}(\text{DFT}_{2n}(a) \cdot \text{DFT}_{2n}(b)))$ where (a, b) are padded with 0s to length $(2n)$

Comparisons & Trade-offs

Dimension	Coefficient Form	Point-Value Form
Addition	(n)	(n)
Evaluation at one point	(n) (Horner)	Must convert to coefficient form first
Multiplication	(n^2)	(n) (pointwise multiply)
Conversion between forms	—	$(n \log n)$ via FFT
Interpolation	—	(n^2) (Lagrange) or $(n \log n)$ (inverse FFT)

Edge Cases & Pitfalls

- FFT requires (n) to be an exact power of 2; otherwise pad with zeros
- Numerical round-off errors can accumulate when using FFT with floating point; modular arithmetic variants can give exact results
- Lagrange interpolation can be numerically unstable
- The product of degree-bound (n) polynomials has degree-bound $(2n)$, so point-value representations must use $(2n)$ points for correct interpolation

- The twiddle factors (ω_n^k) computed iteratively can accumulate round-off errors for large inputs
- For polynomial division in point-value form, dividing (y)-values only works if the division comes out exactly (remainder is zero polynomial)
- FFTW (Fastest Fourier Transform in the West) handles any problem size (n), not just powers of 2

End-of-Chapter Material

Exercises 30.1-1 through 30.1-7 (polynomial multiplication, evaluation via division, reversed polynomials, necessity of (n) point-value pairs, Lagrange interpolation in (n^2)), division in point-value form, Cartesian sum)

Exercises 30.2-1 through 30.2-8 (prove Corollary 30.4, compute DFT, multiply via FFT, inverse FFT pseudocode, FFT for powers of 3, FFT modulo (m), polynomial with given zeros, chirp transform)

Exercises 30.3-1 through 30.3-5 (FFT circuit wire values, butterfly wiring, bit-reversal matrix, bit-reversal permutation pseudocode, faulty adder detection)

Problems: - **30-1 Divide-and-conquer multiplication:** multiply linear polynomials with 3 multiplications; give ($n^{\{\}}$) algorithms; multiply (n)-bit integers in ($O(n^{\{\}})$) - **30-2 Multidimensional FFT:** compute (d)-dimensional DFT by 1D DFTs on each dimension; order doesn't matter; total time ($O(n^d)$) independent of (d) - **30-3 Evaluating all derivatives:** compute all derivatives at a point in ($O(n^d)$) time - **30-4 Polynomial evaluation at multiple points:** evaluate at (n) arbitrary points in ($O(n^2 d)$) time using remainder trees - **30-5 FFT using modular arithmetic:** find ($p = kn+1$) prime; use generator of (ω_p^k); FFT modulo (p) in ($O(n^2)$); compute DFT modulo 17

Ch. 31 – Number-Theoretic Algorithms

Named Entities (Terms & Definitions)

- **Divisor:** ($d \mid a$) means ($a = kd$) for some integer (k); if ($d \mid a$) and ($d \mid b$), (d) is a divisor of (a)
- **Trivial divisors:** 1 and (a) itself for any positive integer (a)
- **Factors:** nontrivial divisors of (a)
- **Prime:** integer ($a > 1$) whose only divisors are 1 and itself
- **Composite:** integer ($a > 1$) that is not prime
- **Unit:** the integer 1 (neither prime nor composite)
- **Quotient:** ($q = a/n$) in division theorem
- **Remainder (residue):** ($r = a \bmod n$), satisfying ($0 \leq r < n$)
- **Equivalence class modulo (n):** ($[a]_n = \{ a + kn : k \in \mathbb{Z} \}$)

- **$*(\mathbb{Z}_n)^*$** : set ($\{0,1,\dots,n-1\}$) of equivalence classes modulo (n)
- **Common divisor**: (d) that divides both (a) and (b)
- **Greatest common divisor (gcd)**: ((a,b)), the largest common divisor of (a) and (b); ($(0,0) = 0$)
- **Relatively prime (coprime)**: ($(a,b) = 1$)
- **Pairwise relatively prime**: ($(n_i, n_j) = 1$) for all ($i < j$)
- **Unique prime factorization**: any composite integer can be written uniquely as ($a = p_1^{e_1} p_2^{e_2} \dots p_r^{e_r}$) with primes ($p_1 < p_2 < \dots < p_r$) and positive exponents (e_i)
- **Group**: set (S) with binary operation ($\cdot$) satisfying closure, identity, associativity, inverses
- **Abelian group**: group also satisfying commutativity ($ab = ba$)
- **Finite group**: group with finite size ($|S|$)
- **Additive group modulo (n)**: ($(\mathbb{Z}_n, +_n)$)
- **Multiplicative group modulo (n)**: ($(\mathbb{Z}_n^*, \cdot_n)$) where ($\mathbb{Z}_n^* = \{a \in \mathbb{Z}_n : (a,n) = 1\}$)
- **Euler's phi function**: ($\phi(n) = |\mathbb{Z}_n^*| = n \prod_{p|n} (1 - 1/p)$)
- **Subgroup**: subset (S') of group (S) that is itself a group under same operation
- **Subgroup generated by (a)**: ($\langle a \rangle = \{a^k : k \in \mathbb{Z}\}$)
- **Generator**: element (a) that generates the whole group
- **Order of (a)**: smallest positive (t) such that ($a^t = e$); ($\text{ord}(a) = |a|$)
- **Primitive root (generator of ($\mathbb{Z}_n^*$))**: (g) such that every element of ($\mathbb{Z}_n^*$) is a power of (g) modulo (n); ($\mathbb{Z}_n^*$) is cyclic if it has a primitive root
- **Discrete logarithm (index)**: ($\log_{g,n}(a)$) is the (z) such that ($g^z = a$)
- **Euler's totient**: ($\phi(n)$)
- **Multiplicative inverse**: ($a^{-1} \pmod n$) exists iff ($(a,n) = 1$)
- **Chinese remainder theorem (CRT)**: bijection between ($\mathbb{Z}_n$) and ($\{\mathbb{Z}_{n_1} \times \dots \times \mathbb{Z}_{n_k}\}$) for pairwise coprime (n_i) with product (n)
- **Modular exponentiation**: computing ($a^b \pmod n$) efficiently
- **Repeated squaring**: method for modular exponentiation using ($a^b = (a^{b/2})^2$) for even (b) and ($a a^{b-1}$) for odd (b)
- **RSA public-key cryptosystem**: asymmetric crypto using large primes; public key ((e,n)), secret key ((d,n))
- **Public key**: ($P = (e,n)$) published openly
- **Secret key**: ($S = (d,n)$) kept secret
- **Encryption**: ($C = P(M) = M^e \pmod n$)
- **Decryption**: ($M = S(C) = C^d \pmod n$)
- **Digital signature**: ($S = S(M')$); verified by ($M' = P(S)$)
- **Certificate**: signed message from trusted authority linking identity to public key
- **Carmichael numbers**: composite (n) such that ($a^{n-1} \equiv 1 \pmod n$) for all ($a \in \mathbb{Z}_n^*$); first three: 561, 1105, 1729
- **Base-(a) pseudoprime**: composite (n) with ($a^{n-1} \equiv 1 \pmod n$)

- **Witness:** value (a) such that $\text{WITNESS}(a,n)$ returns TRUE (proves (n) is composite)
- **Nonwitness:** value (a) that fails to prove compositeness
- **Miller-Rabin primality test:** randomized test using multiple bases and checking for nontrivial square roots of 1
- **Trial division:** testing primality by dividing by all integers up to ()
- **Prime distribution function:** ($\pi(n)$) number of primes (n)
- **Nontrivial square root of 1:** (x) such that ($x^2 \equiv 1 \pmod{n}$) but ($x \not\equiv \pm 1 \pmod{n}$)
- **Legendre symbol:** ($\left(\frac{a}{p}\right)$) if (a) is quadratic residue mod (p), (-1) otherwise
- **Quadratic residue:** (a) such that ($x^2 \equiv a \pmod{p}$) has a solution
- **Least common multiple:** ($\text{lcm}(a_1, \dots, a_n)$), smallest nonnegative integer multiple of each (a_i)
- **Binary gcd algorithm:** gcd algorithm using subtraction, parity tests, and halving instead of remainder

Processes / Algorithms / Pathways

Euclid's Algorithm (GCD)

- **Goal:** compute ($\text{gcd}(a,b)$) for nonnegative integers (a,b)
- **Input:** nonnegative integers (a,b)
- **Output:** ($\text{gcd}(a,b)$)
- **Steps:** (1) If ($b = 0$), return (a) (2) Else return ($\text{gcd}(b, a \bmod b)$)
- **Complexity:** ($O(b)$) recursive calls; ($O(1)$) arithmetic operations and ($O(\log b)$) bit operations for ($\log b$)-bit numbers
- **Example:** ($\text{gcd}(30,21) = \text{gcd}(21,9) = \text{gcd}(9,3) = \text{gcd}(3,0) = 3$)
- **Worst case:** consecutive Fibonacci numbers (F_{k+1}, F_k) make the most recursive calls (Lamé's theorem)

Extended Euclid's Algorithm

- **Goal:** compute ($d = \text{gcd}(a,b)$) and coefficients (x,y) such that ($d = ax + by$)
- **Input:** nonnegative integers (a,b)
- **Output:** ((d,x,y)) where ($d = \text{gcd}(a,b) = ax + by$)
- **Steps:** (1) If ($b = 0$), return ($(a,1,0)$) (2) Else recursively call ($(d',x',y') = \text{ExtendedEuclid}(b, a \bmod b)$) (3) Return ($(d', x' - a/b y')$)
- **Complexity:** same as EUCLID – ($O(b)$) recursive calls
- **Example:** ($\text{ExtendedEuclid}(99,78) = (3,-11,14)$); check: ($3 = 99(-11) + 78(14)$)

Solving Modular Linear Equations

- **Goal:** find all (x) modulo (n) satisfying ($ax \equiv b \pmod{n}$)
- **Input:** positive integers (a,n), integer (b)
- **Output:** all solutions (x) modulo (n), or "no solutions"

- **Algorithm** (MODULAR-LINEAR-EQUATION-SOLVER):
 1. ($(d, x', y') = (a, n)$)
 2. If ($d \mid b$), print "no solutions"
 3. Else ($x_0 = x'(b/d) \pmod n$)
 4. For ($i = 0$) to ($d-1$): print ($(x_0 + i \cdot n/d) \pmod n$)
- **Complexity:** ($O(n + (a, n))$) arithmetic operations
- **Example:** ($14x \equiv 1 \pmod{15}$): ($d=2, x'=-7, y'=1, x_0 = (-7)(15) = 95$); solutions: 95, 45

Modular Exponentiation (Repeated Squaring)

- **Goal:** compute ($a^b \pmod n$) efficiently
- **Input:** nonnegative integers (a, b), positive integer (n)
- **Output:** ($a^b \pmod n$)
- **Steps:** (1) If ($b = 0$), return 1 (2) If (b) is even: ($d = (a, b/2, n)$); return ($(d \cdot d) \pmod n$) (3) If (b) is odd: ($d = (a, b-1, n)$); return ($(a \cdot d) \pmod n$)
- **Complexity:** ($O(b)$) arithmetic operations, ($O(b^3)$) bit operations for (b)-bit numbers
- **Example:** ($(7, 560, 561) = 1$)

RSA Key Generation

- **Steps:** (1) Select two large random distinct primes (p, q) (e.g., 1024 bits each) (2) Compute ($n = pq$) (3) Select small odd (e) with ($(e, (n)) = 1$) (4) Compute ($d = e^{-1} \pmod n$) via extended Euclid (5) Public key: ($P = (e, n)$) (6) Secret key: ($S = (d, n)$)

RSA Encryption/Decryption

- **Encryption:** ($C = P(M) = M^e \pmod n$)
- **Decryption:** ($M = S(C) = C^d \pmod n$)

RSA Digital Signature

- **Signing:** ($S = S(M') = (M')^d \pmod n$)
- **Verification:** check ($M' = P(S) = S^e \pmod n$)

Pseudoprime Test (PSEUDOPRIME)

- **Goal:** test primality with base 2
- **Input:** odd integer ($n > 2$)
- **Output:** COMPOSITE (definite) or PRIME (probable)
- **Steps:** (1) If ($2^{n-1} \equiv 1 \pmod n$), return COMPOSITE (2) Else return PRIME
- **Limitation:** fools base-2 pseudoprimes and Carmichael numbers

Miller-Rabin Primality Test

- **Goal:** randomized primality test with controllable error rate

- **Input:** odd integer ($n > 2$), integer (s) (number of trials)
- **Output:** COMPOSITE (definite) or PRIME (almost surely)
- **Steps:** (1) For ($j = 1$) to (s): (a) Choose random ($a \in \{2, 3, \dots, n-2\}$) (b) If WITNESS((a, n)) returns TRUE, return COMPOSITE (2) Return PRIME
- **Complexity:** ($O(s)$) arithmetic operations, ($O(s^3)$) bit operations
- **Error rate:** at most (2^{-s})

WITNESS Subroutine

- **Steps:** (1) Write ($n-1 = 2^t u$) where (u) is odd (2) ($x_0 = a^u \pmod n$) (3) For ($i = 1$) to (t): ($x_i = x_{i-1}^2 \pmod n$); if ($x_i = 1$) and ($x_{i-1} \neq 1$) and ($x_{i-1} \neq n-1$), return TRUE (nontrivial square root found) (4) If ($x_t = n-1$), return TRUE (Fermat witness) (5) Return FALSE (nonwitness)
- **Example:** For ($n = 561$), ($n-1 = 560 = 2^4 \cdot 35$), ($a=7$): ($x_0 = 7^{35} \pmod{561} = 241$), sequence ($241, 298, 166, 67, 1$); (67) is a nontrivial square root of 1 $\rightarrow$ returns TRUE

Formulas & Equations

Division Theorem

$$[a = qn + r, r < n, q = a/n, r = a \bmod n]$$

GCD Properties

$$[(a,b) = (b,a) = (\gcd(a), \gcd(b)) = (a, b - a) = (a, b \bmod a)] [(a,0) = |a|] [(an, bn) = n(a,b)] [d \mid a, d \mid b \Rightarrow d \mid (a,b)] [(a,b) \mid a, (a,b) \mid b]$$

GCD as Linear Combination (Theorem 31.2)

$$[(a,b) = \{ax + by : x, y \in \mathbb{Z}\}]$$

GCD from Prime Factorization

$$[(a,b) = \prod_{i=1}^r p_i^{\min\{e_i, f_i\}}]$$

Euler's Phi Function

$$[\phi(n) = n \prod_{p \mid n} (1 - \frac{1}{p})] [\phi(p) = p - 1] [\phi(p^e) = p^{e-1}(p-1)]$$

Euler's Theorem (Theorem 31.30)

$$[a^{\phi(n)} \equiv 1 \pmod n]$$

Fermat's Theorem (Theorem 31.31)

$$[a^{p-1} \equiv 1 \pmod{p}, a \not\equiv 0 \pmod{p}]$$

Euler's Theorem Lower Bound

$$[\phi(n) > n/2] \quad [n > 5] \quad [e^{-1/n} < 1 - 1/n]$$

Modular Linear Equation Solutions

$$[ax \equiv b \pmod{d} \iff (a,n) \mid b] \quad [dx_i \equiv x_0 + i(n/d) \pmod{d} \quad i = 0, \dots, d-1]$$

Modular Inverse

$$[a^{-1} \pmod{n} \text{ exists} \iff (a,n) = 1]$$

Chinese Remainder Theorem

For pairwise coprime (n_i) with $(n = n_1 n_2 \dots n_k)$: $[a \equiv (a_1, a_2, \dots, a_k) \pmod{n} \iff a \equiv a_i \pmod{n_i} \text{ for all } i]$
 $[a \equiv \sum_{i=1}^k a_i c_i \pmod{n}, c_i = m_i (m_i^{-1} \pmod{n_i}), m_i = n/n_i]$

Discrete Logarithm Theorem (Theorem 31.33)

$$[g^x \equiv g^y \pmod{p} \iff x \equiv y \pmod{p-1}]$$

RSA Correctness

$$[P(S(M)) = S(P(M)) = M^{ed} \pmod{M}] \text{ where } (ed \equiv 1 \pmod{\phi(M)})$$

Repeated Squaring Formula

$$[a^b = a^{b/2} \cdot a^{b/2} \text{ if } b \text{ is even}]$$

Prime Number Theorem (Theorem 31.37)

$$[\pi(n) \sim n / \ln n] \text{ Probability a random } (n)\text{-bit number is prime: } (1/n)$$

Miller-Rabin Error Probability

For odd composite (n) , number of witnesses to compositeness $(\leq (n-1)/2)$ Probability error $(\leq 2^{-s})$

Bayes for Miller-Rabin

$[A \mid B]$ where $(A = n)$ is prime, $(B = \text{Miller-Rabin returns PRIME})$

Carmichael Number Condition

$(n)^{n-1}$), where $(n) = \{(p_i^{e_i})\}$; $(p^e) = (p^e)$ for odd prime power, $(2) = 1$, $(4) = 2$, $(2^e) = 2^{e-2}$ for (e)

Lamé's Theorem (Theorem 31.11)

If $(a > b)$ and $(b < F_{k+1})$, then $\text{EUCLID}((a, b))$ makes fewer than (k) recursive calls. Number of recursive calls in EUCLID is $(O(b))$.

Bound on Recursive Calls

$[(a, b)] \leq 1 + b$ Improved: $(1 + (b/(a, b)))$

Rules, Laws & Theorems

Theorem 31.1 (Division Theorem)

- **Statement:** For any integer (a) and positive integer (n) , unique (q, r) exist with $(0 \leq r < n)$ and $(a = qn + r)$

Theorem 31.2 (GCD as Linear Combination)

- **Statement:** (a, b) is the smallest positive element of $(\{ax + by : x, y\})$

Corollary 31.3

- **Statement:** If $(d|a)$ and $(d|b)$, then $(d|(a, b))$

Corollary 31.4

- **Statement:** $(an, bn) = n(a, b)$

Corollary 31.5

- **Statement:** If $(n|ab)$ and $(a, n) = 1$, then $(n|b)$

Theorem 31.6

- **Statement:** $(ab, p) = 1$ iff $(a, p) = 1$ and $(b, p) = 1$

Theorem 31.7

- **Statement:** For prime (p) , if $(p|ab)$ then $(p|a)$ or $(p|b)$ (or both)

Theorem 31.8 (Unique Prime Factorization)

- **Statement:** Every composite integer has a unique factorization into primes

Theorem 31.9 (GCD Recursion Theorem)

- **Statement:** $(a, b) = (b, a \bmod b)$ for nonnegative a and positive b

Lemma 31.10 (Lower bound for EUCLID)

- **Statement:** If $\text{EUCLID}(a, b)$ performs k recursive calls and $a > b$, then $a \geq F_{k+2}$ and $b \geq F_{k+1}$

Theorem 31.11 (Lamé's Theorem)

- **Statement:** If $a > b$ and $b < F_{k+1}$, then $\text{EUCLID}(a, b)$ makes fewer than k recursive calls

Theorem 31.12

- **Statement:** $(\mathbb{Z}_n, +_n)$ is a finite abelian group

Theorem 31.13

- **Statement:** $(\mathbb{Z}_n^*, \cdot_n)$ is a finite abelian group

Theorem 31.14 (Nonempty Closed Subset of Finite Group is Subgroup)

- **Statement:** If $(S, \cdot)$ is a finite group and $S' \subseteq S$ is nonempty and closed under $\cdot$, then $(S', \cdot)$ is a subgroup

Theorem 31.15 (Lagrange's Theorem)

- **Statement:** If $(S', \cdot)$ is a subgroup of finite group $(S, \cdot)$, then $|S'|$ divides $|S|$

Corollary 31.16

- **Statement:** If $(S', \cdot)$ is a proper subgroup of finite group $(S, \cdot)$, then $|S'| \leq |S|/2$

Theorem 31.17

- **Statement:** For any finite group, $\sum_{a \in G} (a) = |G|$

Corollary 31.18

- **Statement:** The sequence $(a^{\{1\}}, a^{\{2\}}, \dots)$ is periodic with period $(t = (a))$

Corollary 31.19

- **Statement:** For finite group $(S, \cdot)$ with identity (e) , $(a^{\{|S|\}} = e)$ for all $(a \in S)$

Theorem 31.20

- **Statement:** $(a = d = \{0, d, 2d, \dots, ((n/d)-1)d\})$ in $(\mathbb{Z}_n)$, where $(d = (a, n))$; $(|a| = n/d)$

Corollary 31.21

- **Statement:** $(ax \equiv b)$ is solvable iff $(d \mid b)$ where $(d = (a, n))$

Corollary 31.22

- **Statement:** $(ax \equiv b)$ has either $(d = (a, n))$ distinct solutions or no solutions

Theorem 31.23

- **Statement:** If $(d = (a, n) \mid b)$, then $(x_0 = x'(b/d) \cdot n)$ is a solution to $(ax \equiv b)$

Theorem 31.24

- **Statement:** If solvable, the $(d = (a, n))$ solutions are $(x_i = x_0 + i(n/d))$ for $(i = 0, \dots, d-1)$

Corollary 31.25

- **Statement:** If $((a, n) = 1)$, then $(ax \equiv b)$ has a unique solution modulo (n)

Corollary 31.26

- **Statement:** If $((a, n) = 1)$, then (a) has a unique multiplicative inverse modulo (n) ; otherwise no inverse exists

Theorem 31.27 (Chinese Remainder Theorem)

- **Statement:** For pairwise coprime (n_i) with product (n) , the mapping $(a \mapsto (a \bmod n_1, \dots, a \bmod n_k))$ is a bijection between $(\mathbb{Z}_n)$ and $(\mathbb{Z}_{n_1} \times \dots \times \mathbb{Z}_{n_k})$

Corollary 31.28

- **Statement:** The system $(x \equiv a_i \pmod{n_i})$ with pairwise coprime (n_i) has a unique solution modulo $(n = n_i)$

Corollary 31.29

- **Statement:** For pairwise coprime (n_i) with product (n) , $(x \equiv a \pmod{n})$ for all (i) iff $(x \equiv a \pmod{n_i})$

Theorem 31.30 (Euler's Theorem)

- **Statement:** $a^{\phi(n)} \equiv 1 \pmod{n}$ for all (a) with $(a, n) = 1$

Theorem 31.31 (Fermat's Theorem)

- **Statement:** $a^{p-1} \equiv 1 \pmod{p}$ for prime (p) and (a)

Theorem 31.32 (Values for which $(\mathbb{Z}_n^*)$ is cyclic)

- **Statement:** $(\mathbb{Z}_n^*)$ is cyclic for $(n = 2, 4, p^e, 2p^e)$ where $(p > 2)$ is prime and (e)

Theorem 31.33 (Discrete Logarithm Theorem)

- **Statement:** If (g) is primitive root of $(\mathbb{Z}_n^*)$, then $(g^x \equiv g^y \pmod{n})$ iff $(x \equiv y \pmod{\phi(n)})$

Theorem 31.34

- **Statement:** If (p) is odd prime and (e) , equation $(x^2 \equiv a \pmod{p^e})$ has only solutions (x)

Corollary 31.35

- **Statement:** If a nontrivial square root of 1 modulo (n) exists, then (n) is composite

Theorem 31.36 (Correctness of RSA)

- **Statement:** RSA equations define inverse transformations: $(P(S(M)) = S(P(M)) = M^{\{ed\}} \pmod{n})$ for all $(M \in \mathbb{Z}_n^*)$

Theorem 31.37 (Prime Number Theorem)

- **Statement:** $(\pi(n) \sim n / \ln n)$

Lemma 31.38 (Correctness of WITNESS)

- **Statement:** If $\text{WITNESS}((a, n))$ returns TRUE, then a proof that (n) is composite can be constructed using (a) as a witness

Theorem 31.39

- **Statement:** If (n) is odd composite, the number of witnesses to compositeness is at least $((n-1)/2)$

Theorem 31.40 (Miller-Rabin Error Bound)

- **Statement:** Probability that $\text{MILLER-RABIN}((n, s))$ errs is at most (2^{-s})

Comparisons & Trade-offs

Dimension	Pseudoprime Test	Miller-Rabin
Bases used	Only $(a=2)$	Random (a) chosen (s) times
Error on Carmichael numbers	Always wrong	Detects compositeness
Error probability	Can be high for bad inputs	(2^{-s}) (no bad inputs)
Runtime	$(O(n^3))$	$(O(s^3))$
Determinism	Deterministic but wrong on pseudoprimes	Randomized

Dimension	Trial Division	Miller-Rabin
Time	$(O(n))$ – exponential in (n)	$(O(s^3))$ – polynomial in (s)
Factorization	Produces a factor if composite	Only proves compositeness
Suitable for	Small numbers only	Large numbers

Dimension	Symmetric-Key Crypto	Public-Key Crypto (RSA)
Keys	Same key for encryption/decryption	Different keys (public/secret)
Speed	Fast	Slow
Key distribution	Must be kept secret	Public key can be published
Typical use	Bulk encryption	Key exchange, signatures

Edge Cases & Pitfalls

- $(0, 0) = 0$ by convention
- $(a, b) = (|a|, |b|)$; Euclid's algorithm works with nonnegative inputs

- The equation $(ax \equiv b \pmod{n})$ may have 0, 1, or multiple solutions depending on whether $(d = (a,n))$ divides (b)
- Modular inverse exists iff $((a,n) = 1)$
- Carmichael numbers (e.g., 561) pass Fermat's test for all $(a \in \mathbb{Z}_n^*)$; Miller-Rabin detects them via nontrivial square roots
- RSA requires $(p \neq q)$ to avoid easy factoring
- The security of RSA depends on difficulty of factoring; unproven that breaking RSA requires factoring
- In practice, RSA moduli should be at least 2048-4096 bits (as of 2021)
- $((a,n) = 1)$ is necessary for (a) to be a nonwitness in Miller-Rabin; nonwitnesses are always in $(\mathbb{Z}_n^*)$
- Fermat's theorem gives $(a^{p-1} \equiv 1 \pmod{p})$, but also $(a^p \equiv a \pmod{p})$ for all (a) (including multiples of (p))
- Binary gcd algorithm avoids remainder computations using subtraction, parity tests, and halving; runs in $(O(\log a))$ time
- The pseudoprime test with base 2 errs on base-2 pseudoprimes (first: 341, 561, 645, 1105)
- A Carmichael number must be square-free and the product of at least three primes

End-of-Chapter Material

Exercises 31.1-1 through 31.1-14 (mod properties, infinite primes, transitivity of divisibility, prime/ k gcd, Corollary 31.5, binomial/Fermat, nested mod, k th powers, gcd equations, gcd associativity, unique factorization, division algorithm, binary-to-decimal, lightbulb puzzle)

Exercises 31.2-1 through 31.2-9 (gcd from prime factors, EXTENDED-EUCLID(899,493), gcd and modular inverse, iterative EUCLID, tighter bound on recursive calls, EXTENDED-EUCLID on Fibonacci, gcd for multiple arguments, lcm computation, pairwise relative primality)

Exercises 31.3-1 through 31.3-5 (group operation tables, subgroups of $(\mathbb{Z}_9)$ and $(\mathbb{Z}_{13}^*)$, prove Theorem 31.14, ϕ of prime power, permutation property)

Exercises 31.4-1 through 31.4-4 ($35x \equiv 10 \pmod{50}$, cancellation with gcd=1, modified solver line, zeros of polynomials modulo prime)

Exercises 31.5-1 through 31.5-4 (CRT examples, CRT and gcd, polynomial roots)

Exercises 31.6-1 through 31.6-5 (order table for $(\mathbb{Z}_{11}^*)$, square roots mod (p^e) , rewrite modular exponentiation, iterative version, inverse via Euler)

Exercises 31.7-1 through 31.7-3 (RSA computation with $p=11, q=29$, factoring when $e=3$ and d known, RSA multiplicativity)

Exercises 31.8-1 through 31.8-3 (nontrivial square root for non-prime-power composite, Carmichael number properties, nontrivial square root gives factor)

Problems: - **31-1 Binary gcd algorithm:** (a) both even: $\text{gcd} \times 2$ (b) odd/even: halve (c) both odd: subtract and halve (d) $O(a)$ time - **31-2 Bit operations in Euclid:** (a) long division uses $O((1+q)b)$ bit ops (b) $(a, b) = (1+a)(1+b)$ decreases (c) $O(n^2)$ bit ops - **31-3 Fibonacci algorithms:** recursive (exponential), memoization ($O(n)$), matrix exponentiation ($O(\lg n)$); cost under bit-operation model - **31-4 Quadratic residues:** (a) exactly $(p-1)/2$ residues (b) Legendre symbol and Euler's criterion (c) square root for $p=4k+3$ (d) randomized nonresidue finder

Extractions: Chapters 32–33

Ch. 32 – String Matching

Named Entities (Terms & Definitions)

- **String-matching problem:** Given text array $T[1:n]$ of length n and pattern array $P[1:m]$ of length $m \leq n$, find all valid shifts s ($0 \leq s \leq n-m$) such that $T[s+1:s+m] = P[1:m]$.
- **Alphabet Σ :** Finite set of characters from which P and T are drawn.
- **Shift s :** A position in T where P is aligned; valid if $T[s+1:s+m] = P[1:m]$, invalid otherwise.
- **Valid shift:** Shift s where pattern P occurs in text T .
- **Empty string ϵ :** The 0-length string belonging to Σ^* (the set of all finite-length strings over Σ).
- **Concatenation xy :** String consisting of characters from x followed by characters from y ; $|xy| = |x| + |y|$.
- **Prefix $w \sqsubset x$:** $x = wy$ for some string $y \in \Sigma^*$; $|w| \leq |x|$.
- **Suffix $w \sqsupset x$:** $x = yw$ for some $y \in \Sigma^*$; $|w| \leq |x|$.
- **Proper prefix/suffix:** $w \sqsubset x$ and $|w| < |x|$ (or $w \sqsupset x$ and $|w| < |x|$).
- **$P[:k]$:** k -character prefix $P[1:k]$ of pattern P .
- **$T[:k]$:** k -character prefix of text T .
- **Finite automaton M :** 5-tuple $(Q, q_0, A, \Sigma, \delta)$ where Q is finite set of states, $q_0 \in Q$ is start state, $A \subseteq Q$ is set of accepting states, Σ is finite input alphabet, $\delta : Q \times \Sigma \rightarrow Q$ is the transition function.
- **Final-state function ϕ :** Maps Σ^* to Q ; $\phi(w)$ is the state M ends up in after reading string w . Defined: $\phi(\epsilon) = q_0$, $\phi(wa) = \delta(\phi(w), a)$.
- **Suffix function $\sigma(x)$:** Length of the longest prefix of P that is also a suffix of x . Maps Σ^* to $\{0, 1, \dots, m\}$.

- **Prefix function $\pi[q]$:** For pattern $P[1:m]$, $\pi[q] = \max\{k : k < q \text{ and } P[:k] \sqsubseteq P[:q]\}$. Length of longest proper prefix of $P[:q]$ that is also a suffix of $P[:q]$.
- **$\pi^*[q]$:** Set $\{\pi[q], \pi(2)[q], \pi(3)[q], \dots, \pi(t)[q]\}$ where $\pi(i)[q] = \pi(\pi(i-1)[q])$; iteration stops at 0.
- **Suffix array $SA[1:n]$:** If $SA[i] = j$, then $T[j:]$ is the i th suffix of T in lexicographic order.
- **Longest common prefix array $LCP[1:n]$:** $LCP[i]$ = length of longest common prefix between i th and $(i-1)$ st suffixes in sorted order; $LCP[1] = 0$.
- **Rank array:** Inverse of SA ; $rank[SA[i]] = i$ for $i = 1, \dots, n$.
- **Spurious hit:** When $ts \equiv p \pmod{q}$ but $ts \neq p$ (i.e., the hash values match but the strings don't).
- **Hit:** When $ts \equiv p \pmod{q}$ in Rabin-Karp.
- **Radix-d notation:** Each character treated as a digit in base d , where $d = |\Sigma|$.
- **Gap character ϵ :** Special character that matches an arbitrary string (including length 0).
- **Nonoverlappable pattern:** $P[:k] \sqsubseteq P[:q]$ implies $k = 0$ or $k = q$.
- **Lexicographic order:** "Alphabetical order" in the underlying character set.
- **Burrows-Wheeler transform (BWT):** Append $\$$ (smallest character), list all cyclic rotations, sort lexicographically, take rightmost column.

Processes / Algorithms / Pathways

The Naive String-Matching Algorithm (NAIVE-STRING-MATCHER)

- **Goal:** Find all valid shifts of pattern P in text T .
- **Input/Output:** Input: text $T[1:n]$, pattern $P[1:m]$. Output: prints all valid shifts s .
- **Steps:** (1) For $s = 0$ to $n-m$: (2) If $P[1:m] == T[s+1:s+m]$, then (3) print shift s .
- **Complexity:** Preprocessing $O(\theta)$. Matching $O((n-m+1)m)$. Worst case $\theta((n-m+1)m) = \theta(n^2)$ when $m = \lfloor n/2 \rfloor$.
- **Example:** Text $T = \text{"acaabc"}$, Pattern $P = \text{"aab"}$. Shift $s=2$ is valid. For each $s=0..3$, compare all m characters. At $s=2$, $T[3:5] = \text{"aab"} = P$.
- **Pitfall:** Ignores information gained from previous shifts; can be very slow for repeated characters (e.g., $P = a^m$, $T = a^n$ yields $\theta((n-m+1)m)$).

Rabin-Karp Algorithm (RABIN-KARP-MATCHER)

- **Goal:** Find all occurrences of pattern P in text T using rolling hash.
- **Input/Output:** Input: T , P , n , m , radix d , prime q . Output: prints valid shifts.
- **Steps:** (1) Precompute $h = d^{m-1} \bmod q$. (2) Compute $p = \text{value of } P \bmod q$ and $t_i = \text{value of } T[i:m] \bmod q$ in $\theta(m)$ time

- using Horner's rule. (3) For each shift $s = 0$ to $n-m$: (a) If $p == t_s$ (a "hit"), verify explicitly with $P[1:m] == T[s+1:s+m]$. (b) If $s < n-m$, compute $t_{s+1} = (d(t_s - T[s+1] \cdot h) + T[s+m+1]) \bmod q$.
- **Recurrence:** $t_{s+1} = (d \cdot (t_s - T[s+1] \cdot h) + T[s+m+1]) \bmod q$, where $h = d^{m-1} \bmod q$.
 - **Complexity:** Preprocessing $\Theta(m)$. Worst-case matching $\Theta((n-m+1)m)$. Expected matching $\Theta(n+m)$ if $v = O(1)$ valid shifts and $q \geq m$. Expected time: $\Theta(n) + \Theta(m(v + n/q))$.
 - **Example:** Figure 32.4. Text = digits, $q = 13$, $P = 31415$. $p \bmod 13 = 7$. Find windows with value 7 mod 13. Window at position 7 is a real match; window at position 13 is a spurious hit.
 - **Edge case:** Spurious hits require extra verification. Choose $q \geq m$ and prime so $d \cdot q$ fits in a word.

String Matching with Finite Automata (FINITE-AUTOMATON-MATCHER)

- **Goal:** Build automaton for pattern P that scans text once, constant time per character.
- **Input/Output:** Input: transition function δ (precomputed), text $T[1:n]$. Output: prints shifts where pattern occurs.
- **Steps:** (1) $q = 0$. (2) For $i = 1$ to n : $q = \delta(q, T[i])$. If $q == m$, print shift $i-m$.
- **Transitions:** $\delta(q, a) = \sigma(P[:q]a)$ where $\sigma(x)$ = length of longest prefix of P that is a suffix of x .
- **State set:** $Q = \{0, 1, \dots, m\}$; start state 0; accepting state m .
- **Complexity:** Preprocessing $O(m|\Sigma|)$ for straightforward construction (naïve $O(m^3|\Sigma|)$). Matching $\Theta(n)$.
- **Theorem 32.4:** $\phi(T[:i]) = \sigma(T[:i])$ for $i = 0, 1, \dots, n$. The automaton maintains the invariant that state = length of longest prefix of P matching a suffix of the text read so far.
- **Lemma 32.2 (Suffix-function inequality):** $\sigma(xa) \leq \sigma(x) + 1$.
- **Lemma 32.3 (Suffix-function recursion):** If $q = \sigma(x)$, then $\sigma(xa) = \sigma(P[:q]a)$.

Compute-Transition-Function

- **Goal:** Compute δ for string-matching automaton.
- **Steps:** (1) For $q = 0$ to m : (2) For each $a \in \Sigma$: (3) $k = \min(m, q+1)$; (4) While $P[:k]$ is not a suffix of $P[:q]a$: (5) $k = k-1$; (6) $\delta(q, a) = k$.
- **Complexity:** $O(m^3|\Sigma|)$ naïve; can be improved to $O(m|\Sigma|)$.
- **Use:** Converts pattern into automaton for $\Theta(n)$ matching.

Knuth-Morris-Pratt Algorithm (KMP-MATCHER)

- **Goal:** Linear-time string matching without precomputing full δ ; uses prefix function π .
- **Input/Output:** Input: text $T[1:n]$, pattern $P[1:m]$. Output: prints valid shifts.

- **Steps:** (1) Compute prefix function $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P, m)$. (2) $q = 0$. (3) For $i = 1$ to n : (4) While $q > 0$ and $P[q+1] \neq T[i]$: $q = \pi[q]$. (6) If $P[q+1] == T[i]$: $q = q+1$. (8) If $q == m$: print shift $i-m$; $q = \pi[q]$.
- **Complexity:** Preprocessing $\Theta(m)$. Matching $\Theta(n)$. Total $\Theta(n+m)$.
- **Example:** $P = \text{"ababaca"}$, $\pi[5] = 3$. When matching fails at $q=5$, next state is $\pi[5]=3$ (if $P[4]=b$ matches) else $\pi[3]=1$ else $\pi[1]=0$.

Compute-Prefix-Function (COMPUTE-PREFIX-FUNCTION)

- **Goal:** Compute π array for pattern P .
- **Steps:** (1) $\pi[1] = 0$, $k = 0$. (2) For $q = 2$ to m : (3) While $k > 0$ and $P[k+1] \neq P[q]$: $k = \pi[k]$. (5) If $P[k+1] == P[q]$: $k = k+1$. (6) $\pi[q] = k$.
- **Complexity:** $\Theta(m)$ amortized (aggregate analysis: while loop runs at most $m-1$ times total).
- **Lemma 32.5 (Prefix-function iteration lemma):** $\pi^*[q] = \{k : k < q \text{ and } P[:k] \sqsubseteq P[:q]\}$.
- **Lemma 32.6:** If $\pi[q] > 0$, then $\pi[q] - 1 \in \pi^*[q-1]$.
- **Corollary 32.7:** $\pi[q] = 0$ if $E_{q-1} = \emptyset$; otherwise $\pi[q] = 1 + \max E_{q-1}$.

Suffix Array Construction (COMPUTE-SUFFIX-ARRAY)

- **Goal:** Compute suffix array SA for text $T[1:n]$ in $O(n \lg n)$ time.
- **Steps:** (1) Initialize substr-rank with ord values for length-2 substrings (left-rank = $\text{ord}(T[i])$, right-rank = $\text{ord}(T[i+1])$ or 0). Sort. (2) For $l = 2, 4, 8, \dots$ while $l < n$: (a) MAKE-RANKS: assign ranks from $1..n$ to sorted substrings. (b) Rebuild substr-rank with left-rank = $\text{rank}[i]$, right-rank = $\text{rank}[i+l]$ (or 0). (c) Sort by left-rank, break ties by right-rank. (3) After loop, $SA[i] = \text{substr-rank}[i].\text{index}$.
- **Complexity:** $O(n \lg n)$ using radix sort ($\Theta(n)$ per iteration, $\lg n$ iterations). Naïve version $O(n \lg^2 n)$.
- **Key idea:** Represent substrings by integer ranks; doubling substring length each pass.

MAKE-RANKS

- **Goal:** Assign ranks to sorted substrings.
- **Steps:** (1) $r = 1$. (2) For $i = 1$ to n : if $\text{substr-rank}[i]$ differs from $\text{substr-rank}[i-1]$, $r = r+1$. (3) Set $\text{rank}[\text{substr-rank}[i].\text{index}] = r$.
- **Complexity:** $\Theta(n)$.

Longest Common Prefix Array (COMPUTE-LCP)

- **Goal:** Compute LCP array from suffix array and text.
- **Steps:** (1) Build rank array: $\text{rank}[SA[i]] = i$. (2) $l = 0$. (3) For $i = 1$ to n : if $\text{rank}[i] > 1$: $j = SA[\text{rank}[i]-1]$; while $m+l$

- $\leq n$ and $T[i+1] == T[j+1]$: $l++$. $LCP[rank[i]] = l$; if $l > 0$: $l--$.
- **Lemma 32.8**: If $LCP[rank[i-1]] = l > 1$, then $LCP[rank[i]] \geq l-1$.
 - **Complexity**: $\Theta(n)$ (aggregate analysis: l incremented $< 2n$ times).
 - **Example**: $T = \text{"ratatat"}$, $LCP[3] = 4$ (max). Longest repeated substring: $T[2:5] = \text{"atat"}$.

Burrows-Wheeler Transform (BWT)

- **Goal**: Transform text for compression (tends to group identical characters).
- **Procedure**: (1) Append \$ (smaller than all chars). (2) List all n cyclic rotations. (3) Sort lexicographically. (4) BWT = rightmost column, top to bottom.
- **Inverse BWT**: Use BWT and rank array to reconstruct from back to front, starting with \$.
- **Example**: $T = \text{"rutabaga"} \rightarrow T' = \text{"rutabaga\$"}$. BWT = $\text{"agtbaa\$ur"}$.

Repetition-Matcher (Problem 32-1)

- **Goal**: String matching using repetition factor $\rho^*(P)$.
- **Steps**: (1) $k = 1 + \rho^*(P)$. (2) $q = 0$, $s = 0$. (3) While $s \leq n-m$: if $T[s+q+1] == P[q+1]$: $q++$. If $q == m$: print shift. If $q == m$ or mismatch: $s += \max(1, \lceil q/k \rceil)$; $q = 0$.
- **Complexity**: $O(\rho^*(P) \cdot n + m)$.
- $\rho(x)$: Largest r such that $x = y^r$ for some y .
- $\rho^*(P)$: $\max \rho(P[:i])$ over i .

Comparisons & Trade-offs

Dimension	Naive	Rabin-Karp	Finite Automaton	KMP	Suffix Array
Preprocessing	Θ	$\Theta(m)$	$O(m \Sigma)$	$\Theta(m)$	$O(n \lg n)$
Matching	$O((n-m+1)m)$	$O((n-m+1)m)$ worst, $O(n)$ expected	$\Theta(n)$	$\Theta(n)$	$O(m \lg n + km)$
Space	$O(1)$	$O(1)$	$O(m \Sigma)$	$\Theta(m)$	$O(n)$
Extra capabilities	None	Generalizes to 2D, multiple patterns	None	None	Longest repeated substring, LCS, BWT, palindromes
Best for	Small texts	Average-case fast, multiple patterns	When Σ small	General purpose, worst-case linear	Multiple queries, additional problems

Formulas & Equations

Rolling Hash Recurrence (Rabin-Karp)

$$t_{\{s+1\}} = (d \cdot (t_s - T[s+1] \cdot h) + T[s+m+1]) \bmod q$$

$$\text{where } h = d^{\{m-1\}} \bmod q$$

Expected Running Time of Rabin-Karp

$$O(n) + O(m(v + n/q))$$

where v = number of valid shifts, q = modulus.

Suffix Function Definition

$$\sigma(x) = \max\{k : P[:k] \sqsubseteq x, 0 \leq k \leq m\}$$

Automaton Transition Function

$$\delta(q, a) = \sigma(P[:q]a)$$

Prefix Function Definition

$$\pi[q] = \max\{k : k < q \text{ and } P[:k] \sqsubseteq P[:q]\}$$

KMP Next Shift

$$s' = s + (q - \pi[q])$$

Objective Function for k-means

$$f(S, C) = \sum_{\ell=1}^k \sum_{x \in S^{\{(\ell)\}}} \Delta(x, c^{\{(\ell)\}})$$

$$\text{where } \Delta(x, y) = \sum_{a=1}^d (x_a - y_a)^2$$

Cluster Centroid

$$c_a^{\{(\ell)\}} = (1/|S^{\{(\ell)\}}|) \sum_{x \in S^{\{(\ell)\}}} x_a \text{ for each attribute } a = 1, \dots, d$$

Weighted-Majority Bound

$$m(T') \leq 2(1+\gamma)m_i(T') + (2 \ln n)/\gamma \text{ for every expert } i \text{ and every } T' \leq T.$$

Gradient Descent Update

$$x^{\{(t+1)\}} = x^{\{(t)\}} - \gamma \cdot (\nabla f)(x^{\{(t)\}})$$

Gradient Descent Error Bound

$f(x_{\text{avg}}) - f(x^*) \leq \epsilon$ where $\epsilon = RL\sqrt{(2/T)}$ with $R = \|x^{\{(0)\}} - x^*\|$ and $L = \max_t \|\nabla f(x^{\{(t)\}})\|$.

Number of iterations needed: $T = R^2 L^2 / \epsilon^2$.

Potential Function for Gradient Descent

$$\Phi(t) = (1/(2\gamma)) \cdot \|x^{\{(t)\}} - x^*\|^2$$

Amortized Progress

$$p(t) = f(x^{\{(t)\}}) - f(x^*) + \Phi(t+1) - \Phi(t)$$

Rules, Laws & Theorems

Lemma 32.1 (Overlapping-suffix lemma)

- **Statement:** Suppose $x \sqsubseteq z$ and $y \sqsubseteq z$. If $|x| \leq |y|$ then $x \sqsubseteq y$. If $|x| \geq |y|$ then $y \sqsubseteq x$. If $|x| = |y|$ then $x = y$.

Theorem 32.4 (Automaton correctness)

- **Statement:** If ϕ is the final-state function of a string-matching automaton for pattern P and T is an input text, then $\phi(T[:i]) = \sigma(T[:i])$ for $i = 0, 1, \dots, n$.

Lemma 32.2 (Suffix-function inequality)

- **Statement:** For any string x and character a , $\sigma(xa) \leq \sigma(x) + 1$.

Lemma 32.3 (Suffix-function recursion lemma)

- **Statement:** For any string x and character a , if $q = \sigma(x)$ then $\sigma(xa) = \sigma(P[:q]a)$.

Lemma 32.5 (Prefix-function iteration lemma)

- **Statement:** For pattern P with prefix function π , $\pi^*[q] = \{k : k < q \text{ and } P[:k] \sqsubseteq P[:q]\}$ for $q = 1, \dots, m$.

Lemma 32.6

- **Statement:** If $\pi[q] > 0$, then $\pi[q] - 1 \in \pi^*[q-1]$.

Corollary 32.7

- **Statement:** $\pi[q] = 0$ if $E_{q-1} = \emptyset$, otherwise $\pi[q] = 1 + \max E_{q-1}$, where $E_{q-1} = \{k \in \pi^*[q-1] : P[k+1] = P[q]\}$.

Theorem 33.1 (Centroid optimality)

- **Statement:** For a nonempty cluster $S^{\{\ell\}}$, the centroid (mean) is the unique cluster center minimizing $\sum_{x \in S^{\{\ell\}}} \Delta(x, c^{\{\ell\}})$.

Theorem 33.2 (Nearest-center rule optimality)

- **Statement:** Given a set S of points and centers $\{c^{\{1\}}, \dots, c^{\{k\}}\}$, a clustering minimizes $\sum_{\ell} \sum_{x \in S^{\{\ell\}}} \Delta(x, c^{\{\ell\}})$ iff it assigns each point x to a cluster with a nearest center.

Lemma 33.3 (Perfect expert bound)

- **Statement:** If one expert always predicts correctly for all T events, then there is an algorithm making at most $\lg n$ mistakes.

Theorem 33.4 (Weighted-Majority bound)

- **Statement:** For every expert E_i and every $T' \leq T$, $m(T') \leq 2(1+\gamma)m_i(T') + (2 \ln n)/\gamma$.

Corollary 33.5 (Best expert comparison)

- **Statement:** At end of WEIGHTED-MAJORITY, $m \leq 2(1+\gamma)m^* + (2 \ln n)/\gamma$.

Lemma 33.6 (Convex function lies above tangent)

- **Statement:** For any convex differentiable $f: \mathbb{R}^n \rightarrow \mathbb{R}$ and all $x, y \in \mathbb{R}^n$, $f(y) \geq f(x) + \nabla f(x) \cdot (y - x)$.

Lemma 33.7 (Convexity averaging)

- **Statement:** For any convex $f: \mathbb{R}^n \rightarrow \mathbb{R}$, any integer $T \geq 1$, and all $x^{\{0\}}, \dots, x^{\{T-1\}} \in \mathbb{R}^n$, $f((1/T) \sum x^{\{t\}}) \leq (1/T) \sum f(x^{\{t\}})$.

Theorem 33.8 (Gradient descent error bound)

- **Statement:** Let x^* be minimizer of convex f , and x -avg returned by GRADIENT-DESCENT($f, x^{\{0\}}, \gamma, T$) where $\gamma = R/(L\sqrt{2T})$. Then $f(x\text{-avg}) - f(x^*) \leq RL\sqrt{2/T} = \epsilon$.

Lemma 33.9 (Per-iteration progress)

- **Statement:** For each point $x^{\{t\}}$ computed by GRADIENT-DESCENT, $f(x^{\{t\}}) - f(x^*) + \Phi(t+1) - \Phi(t) \leq \gamma L^2/2$.

Lemma 33.10 (Projection does not increase distance)

- **Statement:** For convex body $K \subseteq \mathbb{R}^n$, $a \in K$, $b' \in \mathbb{R}^n$, and $b = \Pi_K(b')$, $\|b-a\|^2 \leq \|b'-a\|^2$.

Theorem 33.11 (Constrained gradient descent bound)

- **Statement:** Same error bound as Theorem 33.8 holds for GRADIENT-DESCENT-CONSTRAINED.

Edge Cases & Pitfalls

- **Naive algorithm:** Worst case when $P = a^m$ and $T = a^n$ yields $O((n-m+1)m)$ comparisons.
- **Rabin-Karp spurious hits:** When $t_s \equiv p \pmod{q}$ but strings don't match; need expensive explicit verification. Choose $q \geq m$ and prime to reduce frequency.
- **Rabin-Karp large numbers:** p and t_s values may overflow; must work modulo q .
- **Finite automaton preprocessing:** Naïve construction is $O(m^3 |\Sigma|)$; large Σ makes preprocessing expensive.
- **KMP $\pi[q] < q$:** Always strict; $\pi[1] = 0$. The while loop in COMPUTE-PREFIX-FUNCTION only decreases k .
- **KMP line 10:** After finding a match, must set $q = \pi[q]$ to avoid out-of-bounds access $P[m+1]$.
- **Suffix array ties:** When substrings have equal left-rank and right-rank, ties are broken arbitrarily but consistently.
- **BWT:** The \$ character must compare lexicographically smaller than all other characters.
- **Clustering ties:** Must break ties so that a point is not reassigned unless the new center is strictly closer.
- **Lloyd's procedure:** May find only a local minimum; run multiple times with different initial centers.
- **Lloyd's procedure empty clusters:** If $k > \# \text{distinct points}$, some clusters may be empty; set center to zero vector.
- **Lloyd's procedure termination:** Guaranteed only because $\# \text{clusterings}$ is finite (k^n); in practice use threshold on %decrease.
- **k-means is NP-hard:** No polynomial-time algorithm for global optimum known.
- **Gradient descent step size:** Too large can overshoot ($f(x'') > f(x^{(0)})$). Too small converges slowly.
- **Gradient descent local minima:** For non-convex functions, may converge to a local (not global) minimum.
- **Gradient descent R and L:** Usually unknown; need to derive from problem-specific bounds or use line search.
- **Weighted-Majority γ :** Must be $\leq 1/2$. Choice affects bound: larger γ gives smaller multiplier but larger additive term.
- **Weighted-Majority tie-breaking:** If upweight == downweight, predict 1 (arbitrary but must be deterministic).

- **Data normalization:** Attributes with different scales can dominate dissimilarity; normalize to $[0,1]$ or zero mean/unit variance.
- **Missing attribute values:** Either ignore the example or fill with median value.
- **Regularization:** Needed to prevent overfitting; can add constraint on norm of weights.

End-of-Chapter Material

Chapter 32 Exercises

32.1-1: Show comparisons for $P = 0001$ in $T = 000010001010001$. [Manual tracing]

32.1-2: If all P characters are distinct, accelerate Naive to $O(n)$. When mismatch at $T[s+q+1] \neq P[q+1]$, skip to $s+q+1$ because $P[1] \neq T[s+q+1]$ and $P[1] \neq$ any earlier character in the window.

32.1-3: Expected comparisons for random strings from d -ary alphabet: $(n-m+1)(1 - 1/d^m)/(1 - 1/d) \approx (n-m+1) \cdot d/(d-1)$.

32.1-4: Pattern with gap character $\square$. Give $O(nm)$ algorithm (or better) treating $\square$ as matching arbitrary substring.

32.2-1: $T = 3141592653589793$, $P = 26$, $q = 11$. $p = 26 \bmod 11 = 4$. Find all windows $\equiv 4 \bmod 11$, then verify. Count spurious hits.

32.2-2: Extend Rabin-Karp to k patterns of same length: hash each pattern, check set membership. Different lengths: use multiple hash tables or hashing per length.

32.2-3: 2D Rabin-Karp: Hash rows, then hash columns of hashes (or 2D rolling hash).

32.2-4: Alice and Bob file comparison. $A(x) = \sum a_i x^i$. If $A \neq B$, probability $A(x) \equiv B(x) \bmod q \leq n/q < 1/1000$ by Schwartz-Zippel.

32.3-1: Automaton for $P = aabab$ over $\Sigma = \{a,b\}$. Diagram with states 0-5.

32.3-2: Automaton for long pattern $P = ababbabbababbababbabb$.

32.3-3: Nonoverlappable pattern: $\delta(q,a) = 0$ for all $a \neq P[q+1]$; $\delta(q,P[q+1]) = q+1$ for $q < m$; $\delta(m,a) = 0$ for all a .

32.3-4: If $x \sqsubseteq y$ (both prefixes of P), then $\sigma(x) \leq \sigma(y)$.

32.3-5: Build automaton for two patterns P and P' by constructing combined automaton (union of states, careful with accepting states).

32.3-6: Pattern with gap characters: build NFA, convert to DFA.

32.4-1: Compute π for pattern ababbabbabbababbabb.

32.4-2: $|\pi^*[q]| \leq \lceil \lg q \rceil + 1$. Tight for $P = ababa\dots$ (alternating). Example: $P = "ababac\dots"$ gives chain.

32.4-3: Occurrences by examining π on string PT . When π value = m at a position $> m$, pattern ends there.

32.4-4: Aggregate analysis for KMP matching time $\Theta(n)$: q increases $\Theta(n)$, while loop decreases q bound by total increases.

32.4-5: Potential function $\Phi = (\text{some function of } q)$ to show linear time.

32.4-6: Improve KMP by replacing π with π' in line 5: $\pi'[q] = \pi[q]$ if $P[\pi[q]+1] \neq P[q+1]$, else $\pi'[q] = \pi'[\pi[q]]$. Skips redundant comparisons.

32.4-7: Cyclic rotation: check if T is substring of $T'T'$ using KMP. Or use $T'T'$ and π .

32.4-8: Compute δ in $O(m|\Sigma|)$: $\delta(q, a) = \delta(\pi[q], a)$ if $q=m$ or $P[q+1] \neq a$; else $\delta(q, a) = q+1$.

32.5-1: Run COMPUTE-SUFFIX-ARRAY on "hippityhoppity". Show substr-rank, rank, SA, sorted suffixes, LCP.

32.5-2: Stop while loop early if all ranks are distinct (all suffixes separated). $O(1)$ iterations when text has no repeated substrings. Maximum iterations when all suffixes are identical (single character repeated).

32.5-3: Longest common substrings of T_1, T_2 : concatenate $T_1T_2\#$, build SA and LCP. LCP entries straddling the boundary give common substrings.

32.5-4: Professor Markram's palindrome method fails when a palindrome and its reverse are not consecutive in SA. Example needed.

Chapter 32 Problems

32-1 String matching based on repetition factors - a. Compute $\rho(P[:i])$ for $i=1..m$ efficiently. - b. Expected $\rho(P) = O(1)$ for random binary strings. - c. REPETITION-MATCHER correctness and $O(\rho(P)n + m)$ time.

32-2 A linear-time suffix-array algorithm (DC3) - a. Order of suffixes of P is same as order of sample suffixes of T . - b. Substep C (sorting metacharacters) in $\Theta(n)$ time. - c. Tuples in substep H are unique; can be sorted in $\Theta(n)$. - d. Compare sample vs nonsample in $\Theta(1)$ time. - e. Recurrence $T(n) \leq T(\lceil 2n/3 \rceil + 2) + \Theta(n)$ $\implies T(n) = \Theta(n)$.

32-3 Burrows-Wheeler transform - a. Compute BWT from suffix array of T' in $\Theta(n)$. - b. Compute rank array from BWT in $\Theta(n)$ time (constant alphabet). - c. Reconstruct T' from BWT and rank in $\Theta(n)$.

Ch. 33 – Machine-Learning Algorithms

Named Entities (Terms & Definitions)

- **Machine learning:** Subfield of AI; produces hypotheses from data to make predictions.
- **Supervised learning:** Training data has input-output pairs (labeled); goal is to predict labels for new data.
- **Unsupervised learning:** Training data is unlabeled; goal is to find structure (e.g., clusters).
- **Reinforcement learning:** Learner takes actions, receives feedback, updates model of environment.
- **Training phase:** Takes training data (input+label), produces hypothesis(es).
- **Prediction phase:** Uses hypothesis to predict labels on new data.
- **Online learning:** Training and prediction phases intermingled (used in multiplicative weights).
- **Hypothesis h_θ :** Formula/algorithm parameterized by θ that describes regularities or makes predictions.
- **Causal inference:** Finding explanatory model of how features affect labels.
- **Clustering:** Dividing n examples into k disjoint groups where similar points are grouped together.
- **k-clustering:** Decomposition of S into k disjoint subsets $S^{\{(1)\}}, \dots, S^{\{(k)\}}$.
- **k-means problem:** Find k centers C minimizing $f(S, C) = \sum_{\ell} \sum_{x \in S^{\{(\ell)\}}} \Delta(x, c^{\{(\ell)\}})$. NP-hard.
- **Feature vector:** d -dimensional vector $x = (x_1, \dots, x_d)$ representing an example's attributes.
- **Dissimilarity $\Delta(x, y)$:** Squared Euclidean distance $= \sum_{a=1}^d (x_a - y_a)^2$.
- **Centroid/mean:** Center of cluster; each coordinate = mean of that attribute over all points in cluster.
- **Nearest-center rule:** Point x belongs to cluster $S^{\{(\ell)\}}$ if $\Delta(x, c^{\{(\ell)\}}) = \min_j \Delta(x, c^{\{(j)\}})$.
- **Lloyd's procedure (k-means algorithm):** Iteratively assign points to nearest center, then recompute centers as centroids.
- **Vector quantization:** Compressing images by reducing distinct colors via clustering.
- **Regularization:** Penalizing complex hypotheses to avoid overfitting (e.g., bounding norm of weights).
- **Multiplicative-weights algorithms:** Class of online algorithms that maintain weights for experts/actions, update multiplicatively based on feedback.

- **Weighted-Majority algorithm:** Maintains weights for experts, predicts by weighted majority vote, decreases weights of mistaken experts by factor $(1-\gamma)$.
- **Regret:** $m - m^*$ where m = algorithm's mistakes, m^* = mistakes of the best expert (in hindsight).
- **Hedge algorithm:** Randomized variant; chooses expert probabilistically according to weights; update rule $w_i \leftarrow w_i \cdot (1-\eta)$ if mistaken.
- **Gradient:** $\nabla f : \mathbb{R}^n \rightarrow \mathbb{R}^n$, vector of n partial derivatives $(\partial f / \partial x_1, \dots, \partial f / \partial x_n)$. Direction of steepest increase.
- **Gradient descent:** Iterative algorithm to find local minimum by moving opposite to gradient.
- **Convex function $f : \mathbb{R}^n \rightarrow \mathbb{R}$:** $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$ for all x, y , $0 \leq \lambda \leq 1$. Any local minimum is global.
- **Closed convex body K :** Convex set containing its limit points.
- **Projection $\Pi_K(x)$:** Closest point in convex body K to x (minimum Euclidean distance).
- **Line search:** Binary-search-like routine to find good step size.
- **Stochastic gradient descent:** Online variant; gradient estimated from a single randomly chosen point per iteration.
- **Loss function:** Objective function measuring error between predictions and labels (e.g., least-squares error).
- **Least-squares error:** $\sum (f(x_i) - y_i)^2$.
- **Linear regression:** Finding linear function f to minimize least-squares error.
- **Hessian matrix $(\nabla^2 f)(x)$:** Matrix where entry $(i, j) = \partial^2 f / (\partial x_i \partial x_j)$. Convex when positive-semidefinite.
- **α -strongly convex:** $f(y) \geq f(x) + \nabla f(x) \cdot (y-x) + \frac{\alpha}{2} \|y-x\|^2$. Gradient descent converges faster (linear in $1/\alpha$).
- **β -smooth:** Converse inequality to strong convexity; also enables better bounds.
- **Newton's method:** Iterative algorithm to find root of function; uses tangent line. Can have quadratic convergence.

Processes / Algorithms / Pathways

Lloyd's Procedure (k-means clustering)

- **Goal:** Find a locally optimal k -clustering minimizing sum of squared distances.
- **Input/Output:** Input: set S of points in $\mathbb{R}^d$, integer k .
Output: k -clustering $\{S^{(1)}, \dots, S^{(k)}\}$ with centers $\{c^{(1)}, \dots, c^{(k)}\}$.
- **Steps:** (1) Initialize centers: pick k points randomly from S . Assign all points to $S^{(1)}$. (2) Assign points to clusters using nearest-center rule (break ties arbitrarily, don't change unless strictly closer). (3) If no change, stop and return. (4) Recompute each center as centroid of its cluster (zero vector if empty). Go to step 2.
- **Complexity:** $O(Tdkn)$ where T = iterations. Each iteration: $O(dkn)$ for assignment + $O(dn)$ for recomputing centers.

- **Termination:** Guaranteed because f strictly decreases each iteration (except last) and only k^n possible clusterings. In practice, stop when $\%decrease < threshold$.
- **Example:** Figure 33.1 – 49 US capitals, $k=4$. Initial centers: AR, KS, LA, TN. After 11 iterations, converges.
- **Pitfalls:** May find only local minimum. Run multiple times with different random initializations.
- **Vector quantization example:** Figure 33.2 – photo compressed from 24-bit color to $k=4, 16, 64, 256$ colors.

Weighted-Majority Algorithm (WEIGHTED-MAJORITY)

- **Goal:** Make online predictions nearly as good as the best expert.
- **Input/Output:** Input: experts $E = \{E_1, \dots, E_n\}$, T events, n , parameter $0 < \gamma \leq \frac{1}{2}$. Output: predictions $p^\wedge(t)$.
- **Steps:** (1) Initialize all $w_i^\wedge(1) = 1$. (2) For each event $t = 1..T$: (3) Each expert predicts $p_i^\wedge(t) \in \{0, 1\}$. (4) Compute upweight $= \sum w_i^\wedge(t)$ for experts predicting 1, downweight $= \sum w_i^\wedge(t)$ for experts predicting 0. (5) If upweight $\geq$ downweight, predict 1; else predict 0. (6) Outcome $o^\wedge(t)$ revealed. (7) For each expert i : if $p_i^\wedge(t) \neq o^\wedge(t)$, multiply $w_i^\wedge(t+1) = w_i^\wedge(t) \cdot (1-\gamma)$; else $w_i^\wedge(t+1) = w_i^\wedge(t)$.
- **Complexity:** $O(nT)$ ($O(n)$ per event).
- **Bound:** $m(T') \leq 2(1+\gamma)m_i(T') + (2 \ln n)/\gamma$ for every expert i and every $T' \leq T$.
- **Example:** $\gamma = \frac{1}{4}$, $n=20$, best expert makes 50 mistakes. Bound: at most 149 mistakes (85% success).
- **Key idea:** Potential function $W(t) = \sum w_i^\wedge(t)$. Initially $W = n$. Each mistake halves at least $1-\gamma$ of the total weight.

Gradient Descent (GRADIENT-DESCENT)

- **Goal:** Minimize convex function $f: \mathbb{R}^n \rightarrow \mathbb{R}$.
- **Input/Output:** Input: function f , initial point $x^\wedge(0)$, step-size $\gamma > 0$, number of steps T . Output: $x\text{-avg} = (1/T) \sum_{t=0}^{T-1} x^\wedge(t)$.
- **Steps:** (1) $sum = 0$. (2) For $t = 0$ to $T-1$: (3) $sum += x^\wedge(t)$. (4) $x^\wedge(t+1) = x^\wedge(t) - \gamma \cdot (\nabla f)(x^\wedge(t))$. (5) $x\text{-avg} = sum/T$. (6) Return $x\text{-avg}$.
- **Update rule:** $x^\wedge(t+1) = x^\wedge(t) - \gamma \cdot \nabla f(x^\wedge(t))$
- **Complexity:** Per iteration: $O(nd)$ where d = cost of computing gradient. Total: $O(Td)$.
- **Error bound:** $f(x\text{-avg}) - f(x) \leq RL\sqrt{(2/T)}$ where $R = \|x^\wedge(0) - x\|$, $L = \max \|\nabla f(x^\wedge(t))\|$.
- **Step size choice:** $\gamma = R/(L\sqrt{(2T)})$ balances two error terms.
- **Convergence:** $T = R^2 L^2 / \epsilon^2$ iterations needed for error $\leq \epsilon$ (quadratic in $1/\epsilon$).

Constrained Gradient Descent (GRADIENT-DESCENT-CONSTRAINED)

- **Goal:** Minimize convex f over closed convex body K .

- **Steps:** Same as GRADIENT-DESCENT but after update $x' = x(t) - \eta \cdot \nabla f(x(t))$, project: $x^{(t+1)} = \Pi_K(x')$.
- **Same bound:** Theorem 33.11 gives same error bound as unconstrained.
- **Example projection:** Ball constraint $\|w\| \leq B$: scale w' to have norm B if $\|w'\| > B$.

Stochastic Gradient Descent (Problem 33-4)

- **Goal:** Online/randomized gradient descent for linear regression.
- **Procedure:** For each randomly chosen point (x_i, y_i) , compute gradient of just that point's loss term; update a, b accordingly.
- **Expected error:** Similar bounds to batch gradient descent.

Newton's Method (Problem 33-1)

- **Goal:** Find root of $f: \mathbb{R} \rightarrow \mathbb{R}$.
- **Update rule:** $x^{(t+1)} = x^{(t)} - f(x^{(t)})/f'(x^{(t)})$
- **Convergence:** Quadratic (error squared each iteration) if close to root.
- **Application:** Finding minimizer when $f'(x) = 0$.

Hedge Algorithm (Problem 33-2)

- **Goal:** Randomized multiplicative weights.
- **Steps:** (1) At each iteration t , probability of choosing expert $i = w_i^{(t)} / \sum w_j^{(t)}$. (2) Predict according to chosen expert. (3) Update: $w_i^{(t+1)} = w_i^{(t)} \cdot (1 - \eta \cdot \text{mistake}_i^{(t)})$.
- **Bound:** Expected mistakes $\leq m^* + (\ln n)/\eta + \eta T$.

Learning from Experts (Lemma 33.3, perfect expert case)

- **Goal:** At most $\lceil \lg n \rceil$ mistakes when one expert is always correct.
- **Algorithm:** Maintain set S of experts who haven't made mistakes. Predict by majority of S . Remove mistaken experts.
- **Proof:** Each mistake eliminates at least half of S ; size halves at most $\lceil \lg n \rceil$ times; when $|S|=1$, never makes mistake again.

Generalized Expert Algorithm (Exercise 33.2-1)

- **Goal:** Handle case with no perfect expert.
- **Algorithm:** Same but reset S to all experts when S becomes empty.
- **Bound:** At most $m^* \cdot \lceil \lg n \rceil$ mistakes.

Comparisons & Trade-offs

Dimension	Batch Gradient Descent	Stochastic Gradient Descent	Newton's Method
Per iteration cost	$O(nm)$ (full gradient)	$O(m)$ (one point)	$O(f'(x)/f''(x))$
Convergence rate	$O(1/\epsilon^2)$	$O(1/\epsilon^2)$ (expected)	Quadratic (near root)
Guarantee	Deterministic	Expected	Local only
Convex required	Yes	Yes	No (but $f' \neq 0$)
Use case	Small/medium data	Large data, online	Root finding near solution

Dimension	Weighted-Majority	Hedge
Prediction	Deterministic (weighted vote)	Random (probabilistic)
Update	$w_i \leftarrow w_i \cdot (1-\gamma)$ if mistaken	$w_i \leftarrow w_i \cdot (1-\epsilon)$ if mistaken
Mistake bound	$2(1+\gamma)m^* + 2 \ln n / \gamma$	$m^* + (\ln n)/\epsilon + \epsilon T$ (expected)
Best multiplier	2 (with $\gamma \rightarrow 0$, additive term large)	$1 + \epsilon$ (approaches 1)

Formulas & Equations

Squared Euclidean Dissimilarity

$$\Delta(x, y) = \sum_{a=1}^d (x_a - y_a)^2$$

k-means Objective

$$f(S, C) = \sum_{\ell=1}^k \sum_{x \in S^{\{(\ell)\}}} \|x - c^{\{(\ell)\}}\|^2$$

Centroid Formula

$$c^{\{(\ell)\}} = (1/|S^{\{(\ell)\}}|) \sum_{x \in S^{\{(\ell)\}}} x$$

Alternative k-means Objective (Exercise 33.1-1)

$$f(S, C) = (1/2) \sum_{\ell=1}^k (1/|S^{\{(\ell)\}}|) \sum_{x, y \in S^{\{(\ell)\}}} \|x - y\|^2$$

Weighted-Majority Weight Evolution

$$w_i^{\{t\}} = (1-\gamma)^{m_i^{\{t\}}}$$

Weighted-Majority Potential Function

$$W(t) = \sum_{i=1}^n w_i^{(t)}$$

Weighted-Majority Key Inequality per Mistake

$$W(t+1) \leq (1 - \gamma/2) W(t) \text{ (when algorithm makes mistake)}$$

Weighted-Majority Final Bound (derivation)

$$\ln((1-\gamma)^{m_i}) \leq \ln(n) + m \cdot \ln(1-\gamma/2)$$

Using $\ln(1-\gamma) \geq -\gamma-\gamma^2$ (lower bound) and $\ln(1-\gamma/2) \leq -\gamma/2$ (upper bound):

$$-\gamma m_i - \gamma^2 m_i \leq \ln n - (\gamma/2) m$$

$$\rightarrow m \leq 2(1+\gamma)m_i + (2 \ln n)/\gamma$$

Hessian Matrix

$$(\nabla^2 f)(x)_{\{i,j\}} = \partial^2 f / (\partial x_i \partial x_j)$$

Linear Regression Hypothesis

$$f(x) = w_0 + w_1 x_1 + \dots + w_n x_n = w_0 + w \cdot x$$

Least-Squares Loss

$$L(w) = \sum_{i=1}^m (w_0 + w \cdot x^{(i)} - y^{(i)})^2$$

Gradient of Least-Squares

$$\partial L / \partial w_0 = 2 \sum_{i=1}^m (w_0 + w \cdot x^{(i)} - y^{(i)}) \cdot x_0^{(i)}$$

Potential Function for Gradient Descent

$$\Phi(t) = (1/(2\gamma)) \cdot \|x^{(t)} - x^*\|^2$$

Amortized Progress

$$p(t) = f(x^{(t)}) - f(x^*) + \Phi(t+1) - \Phi(t) \leq \gamma L^2/2$$

Total Error Bound

$$f(x\text{-avg}) - f(x^*) \leq R^2/(2\gamma T) + \gamma L^2/2 = RL\sqrt{(2/T)} \text{ (with optimal } \gamma = R/(L\sqrt{(2T)}) \text{)}$$

Required Iterations

$$T = R^2 L^2 / \epsilon^2$$

Projection onto Ball Constraint

$$w = w' \cdot B/\|w'\| \quad (\text{if } \|w'\| > B)$$

Newton's Method Update

$$x^{(t+1)} = x^{(t)} - f(x^{(t)})/f'(x^{(t)})$$

Quadratic Convergence

$$\|x^{(t+1)}\| \leq c \cdot (\|x^{(t)}\|)^2 \text{ for some constant } c.$$

α -Strongly Convex Convergence

$$f(x\text{-avg}) - f(x^*) \leq L^2/(\alpha(T+1)) \quad (\text{linear in } 1/T, \text{ better than standard gradient descent}).$$

Rules, Laws & Theorems

Theorem 33.1 (Centroid optimality)

- **Statement:** For a nonempty cluster $S^{(\ell)}$, the centroid (mean) is the unique minimizer of $\sum_{x \in S^{(\ell)}} \|x - c\|^2$.

Theorem 33.2 (Nearest-center rule optimality)

- **Statement:** Given a set S and centers $\{c^{(1)}, \dots, c^{(k)}\}$, a clustering minimizes $\sum_{\ell} \sum_{x \in S^{(\ell)}} \|x - c^{(\ell)}\|^2$ iff each point is assigned to a cluster with the nearest center.

Lemma 33.3 (Perfect expert bound)

- **Statement:** If one expert always correct, the majority-vote-with-removal algorithm makes at most $\lg n$ mistakes.

Theorem 33.4 (Weighted-Majority bound)

- **Statement:** For every expert E_i and every $T' \leq T$, $m(T') \leq 2(1+\gamma)m_i(T') + (2 \ln n)/\gamma$.

Corollary 33.5 (Best expert bound)

- **Statement:** At end, $m \leq 2(1+\gamma)m^* + (2 \ln n)/\gamma$.

Lemma 33.6 (Convex function above tangent)

- **Statement:** For convex differentiable $f: \mathbb{R}^n \rightarrow \mathbb{R}$ and all $x, y \in \mathbb{R}^n$, $f(y) \geq f(x) + \langle \nabla f(x), y-x \rangle$.

Lemma 33.7 (Convex averaging)

- **Statement:** For convex f , $f((1/T)\sum x^{\{(t)\}}) \leq (1/T)\sum f(x^{\{(t)\}})$.

Theorem 33.8 (Gradient descent error)

- **Statement:** With $\gamma = R/(L\sqrt{2T})$, $f(x\text{-avg}) - f(x^*) \leq RL\sqrt{2/T}$.

Lemma 33.9 (Per-iteration progress bound)

- **Statement:** $f(x^{\{(t)\}}) - f(x^*) + \Phi(t+1) - \Phi(t) \leq \gamma L^2/2$.

Lemma 33.10 (Projection lemma)

- **Statement:** For convex body K , $a \in K$, $b' \in \mathbb{R}^n$, $b = \Pi_K(b')$, $\|b-a\|^2 \leq \|b'-a\|^2$.

Theorem 33.11 (Constrained gradient descent)

- **Statement:** Same bound as Theorem 33.8 for GRADIENT-DESCENT-CONSTRAINED.

Convexity property

- **Statement:** For convex f , any local minimum is a global minimum.

Edge Cases & Pitfalls

- **Lloyd's local optimum:** Properties (centroid centers + nearest-center assignment) are necessary but not sufficient for optimality. Exercise 33.1-2: 4 points, $k=2$ where Lloyd's doesn't improve but not optimal.
- **Lloyd's empty clusters:** If many points are identical, some clusters may be empty; center = zero vector.
- **Lloyd's initialization:** Random picks may give duplicate centers if many repeated points (Exercise 33.1-3: maximize distinct picks).
- **Attribute scaling:** Attributes with different ranges dominate Euclidean distance; must normalize.
- **γ in Weighted-Majority:** Must be $0 < \gamma \leq 1/2$. Bound analysis uses $\ln(1-\gamma) \geq -\gamma-\gamma^2$ which holds for $\gamma \leq 1/2$.
- **Adversarial experts:** Weighted-Majority works even if experts collude to deceive.
- **Gradient descent step size:** Fixed γ may overshoot; use line search or decaying step size for strongly convex functions.
- **Constrained projection:** The projection step must be computationally feasible (e.g., ball constraint is easy, general convex bodies may not be).

- **Stochastic gradient descent:** Noisy gradient estimates; only bounded expected error.
- **Overfitting:** Minimizing training loss may not generalize; use regularization (e.g., weight norm constraint).
- **Missing attributes:** Either drop example or fill with median.
- **Gradient descent initial point:** For non-convex functions, may converge to poor local minimum.

End-of-Chapter Material

Chapter 33 Exercises

33.1-1: Show $f(S, C) = (1/2) \sum_{\ell} (1/|S^{\ell}|) \sum_{x, y \in S^{\ell}} \|x - y\|^2$.

33.1-2: Example in plane with $n=4$, $k=2$ where Lloyd's doesn't improve but not optimal. E.g., points at $(0,0)$, $(1,0)$, $(0,1)$, $(2,2)$ with centers at $(0,0)$ and $(2,2)$. No reassignment occurs but $(1,0)$ might belong to wrong cluster.

33.1-3: Maximize distinct random centers: sample without replacement (use reservoir sampling, Exercise 5.3-5).

33.1-4: Optimal 1D k -clustering in polynomial time: sort points, DP for optimal k segments minimizing sum of squared distances to centroid.

33.2-1: Generalized expert algorithm (reset S when empty) makes $\leq m^* \cdot \lg n$ mistakes.

33.2-2: Prove $\ln(1-x) \geq -x-x^2$ for $0 < x \leq 1/2$ using Taylor series and comparison with geometric series.

33.2-3: Randomized expert algorithm: choose uniformly from S , expected mistakes $\leq \lg n$.

33.2-4: Randomized Weighted-Majority: choose expert proportionally to weights. Expected mistakes $\leq (1+\epsilon)m^* + (\ln n)/\epsilon$.

33.3-1: Prove Lemma 33.6 (convex function above tangent hyperplane).

33.3-2: Prove Lemma 33.7 (convex averaging inequality).

33.3-3: Prove equation (33.29): $\|a+b\|^2 = \|a\|^2 + \|b\|^2 + 2\langle a, b \rangle$.

33.3-4: Show f from (33.32) is convex in w (sum of squares of linear functions).

33.3-5: Compute gradient of least-squares: $O(nm)$ time.

33.3-6: For $\|w\| \leq B$, $L = O(B)$.

33.3-7: Use gradient descent to solve k-means: treat centers as variables, objective is differentiable, compute gradient w.r.t. centers.

Chapter 33 Problems

33-1 Newton's method - a. Derive update rule $x^{(t+1)} = x^{(t)} - f(x^{(t)})/f'(x^{(t)})$. - b. Error bound: $|x^{(t+1)} - x^*| \leq (|x^{(t)} - x^*|)^2 \cdot |f''(x^*)| / (2|f'(x^*)|)$. - c. If $|x^{(t+1)} - x^*| \leq c \cdot (|x^{(t)} - x^*|)^2$ with $|x^{(0)} - x^*| < 1$, need $O(\lg \lg(1/\delta))$ iterations for accuracy δ . - d. $f(x) = (x-3)^2$, $x^* = 3.5$. Compare gradient descent vs Newton's method: Newton finds root in 1 iteration (f'/f''), gradient descent takes many small steps.

33-2 Hedge - Expected mistakes $\leq m^* + (\ln n)/\epsilon + \epsilon T$. - Analysis: potential function $= \sum w_i$. Expected weight update factor $\leq (1-\epsilon(1-\epsilon))$ per mistake. Use $\ln$ bounds.

33-3 Nonoptimality of Lloyd's in 1D - Example needed where Lloyd's converges to local but not global minimum on line.

33-4 Stochastic gradient descent - Pseudocode for SGD for linear regression. - Expected error: similar to batch GD but with T instead of n -sized batches.

Chapter Notes

- Lloyd's procedure also called "Lloyd-Forgy algorithm". k-means NP-hard in plane (Mahajan et al.). Approximation ratio $9+\epsilon$ exists (Kanungo et al.).
- Multiplicative weights are surveyed by Arora, Hazan, Kale. First use in machine learning: Littlestone's Winnow algorithm. Weighted-Majority by Littlestone and Warmuth. Related to boosting (Freund and Shapire) and perceptron.
- Gradient descent treatment draws from Bansal and Gupta (potential function + amortized analysis). Other works by Bubeck, Boyd and Vandenberghe, Nesterov.
- α -strongly convex functions allow variable step size $\eta_t = 1/(\alpha(t+1))$, giving bound $L^2/(\alpha(T+1))$ – linear in $1/T$.
- β -smooth functions give better bounds for gradient descent.

Ch. 34 – NP-Completeness

Named Entities (Terms & Definitions)

- **Tractable Problem:** a problem solvable by a polynomial-time algorithm ($O(n^k)$ for some constant k). Generally regarded as "easy."
- **Intractable Problem:** a problem requiring superpolynomial time to solve. Considered "hard."
- **NP-complete (NPC):** the class of problems that belong to NP and are as "hard" as any problem in NP. If any NP-complete

problem can be solved in polynomial time, then every problem in NP has a polynomial-time algorithm ($P = NP$).

- **Decision Problem:** a problem whose answer is simply “yes” or “no” (1 or 0). NP-completeness applies directly to decision problems.
- **Optimization Problem:** a problem in which each feasible solution has an associated value, and the goal is to find a feasible solution with the best value.
- **Instance:** the input to a particular problem.
- **Polynomial-time Reduction Algorithm:** a procedure that transforms any instance α of problem A into some instance β of problem B such that: (1) the transformation takes polynomial time, (2) the answers are the same (α is “yes” iff β is “yes”). Written as $L1 \leq_P L2$.
- **Certificate:** a proof that a solution exists, used by a verification algorithm. Must be of polynomial length ($O(|x|^c)$).
- **Verification Algorithm:** a two-argument algorithm A where one argument is an ordinary input string x and the other is a binary string y called a certificate. A verifies x if there exists a certificate y such that $A(x,y) = 1$.
- **Encoding:** a mapping e from a set S of abstract objects to the set of binary strings $\{0,1\}^*$.
- **Polynomially Related Encodings:** two encodings e1 and e2 such that there exist polynomial-time computable functions f12 and f21 converting between them.
- **Abstract Problem:** a binary relation on a set I of problem instances and a set S of problem solutions.
- **Concrete Problem:** a problem whose instance set is the set of binary strings.
- **Alphabet (Σ):** a finite set of symbols.
- **Language (L):** any set of strings made up of symbols from Σ .
- **Accept:** algorithm A accepts string x if $A(x) = 1$.
- **Reject:** algorithm A rejects string x if $A(x) = 0$.
- **Language Accepted by A:** $L = \{x \in \{0,1\}^* : A(x) = 1\}$.
- **Language Decided by A:** every binary string in L is accepted by A and every binary string not in L is rejected by A.
- **Complexity Class:** a set of languages, membership in which is determined by a complexity measure such as running time.
- **NP-hard:** a language L satisfying property 2 of NP-completeness ($L' \leq_P L$ for every $L' \in NP$) but not necessarily property 1 ($L \in NP$).
- **TAUTOLOGY:** the language of boolean formulas that are tautologies (evaluate to 1 for every assignment). TAUTOLOGY $\in$ co-NP.
- **co-NP:** the set of languages L such that $\bar{L} \in NP$ (the complement of each language in NP).
- **Gadget:** a component in a reduction graph that enforces certain properties (e.g., the 12-vertex gadget in VERTEX-COVER $\rightarrow$ HAM-CYCLE reduction).
- **Selector Vertices:** vertices in a reduction that select the k vertices of the cover (e.g., $s_1, s_2, \dots, s_k$ in the HAM-CYCLE reduction).

- **Halting Problem:** Turing's famous problem that cannot be solved by any computer, no matter how long you wait.
- **Noninstance:** a string $x \in \{0,1\}^*$ such that there is no instance i for which $e(i) = x$.

Key Pairs: Polynomial vs. NP-complete

- **Shortest vs. Longest Simple Paths:** Shortest paths from single source in $O(VE)$ even with negative weights. Longest simple path (deciding if a graph contains a simple path with at least a given number of edges) is NP-complete.
- **Euler Tour vs. Hamiltonian Cycle:** Euler tour (traverses each edge exactly once) can be found in $O(E)$ time for strongly connected directed graphs. Hamiltonian cycle (simple cycle containing each vertex exactly once) is NP-complete.
- **2-CNF vs. 3-CNF Satisfiability:** 2-CNF satisfiability has a polynomial-time algorithm. 3-CNF satisfiability is NP-complete.

Graphs & Definitions in Reductions

- **Clique:** a subset $V' \subseteq V$ of vertices, each pair connected by an edge (complete subgraph). Size = number of vertices.
- **Vertex Cover:** a subset $V' \subseteq V$ such that if $(u,v) \in E$, then $u \in V'$ or $v \in V'$ (or both). Size = number of vertices.
- **Hamiltonian Cycle:** a simple cycle that contains each vertex in V exactly once (in undirected graph).
- **Hamiltonian Path:** a simple path that visits every vertex exactly once.
- **Complement Graph ($\bar{G}$)** $\bar{G} = (V, \bar{E})$ where $\bar{E} = \{(u,v) : u,v \in V, u \neq v, \text{ and } (u,v) \notin E\}$.
- **Maximal Matching:** a matching to which no edges can be added and still have a matching.

Processes / Algorithms / Pathways

Algorithm for verifying PATH

- **Goal:** Verify that a path exists from u to v with at most k edges.
- **Input/Output:** Input: graph G , vertices u,v , integer k . Output: yes/no.
- **Steps:** (1) Verify G encodes an undirected graph. (2) Verify u and v are vertices in G . (3) Use BFS to compute shortest path from u to v . (4) Compare number of edges with k . If path length $\leq k$, output 1 and halt; otherwise output 0 and halt (for decision version).
- **Complexity:** $O(V+E)$ time.
- **Note:** PATH $\in$ P.

Polynomial-time Reduction Methodology

- **Goal:** Show problem B is NP-complete by reducing from known NP-complete A.
- **Input/Output:** Input: instance of A. Output: instance of B.
- **Steps:** (1) Prove $L \in \text{NP}$ (show certificate can be verified in polynomial time). (2) Prove L is NP-hard: (a) Select a known NP-complete language L' . (b) Describe algorithm computing function f mapping every instance x of L' to instance $f(x)$ of L. (c) Prove $x \in L' \iff f(x) \in L$ for all x . (d) Prove the algorithm computing f runs in polynomial time.
- **Key Lemma (34.8):** If $L' \leq_P L$ for some $L' \in \text{NPC}$, then L is NP-hard. If also $L \in \text{NP}$, then $L \in \text{NPC}$.

Proof that $P = \text{NP}$ (Theorem 34.2)

- **Steps:** (1) Let L be accepted by polynomial-time algorithm A in $O(n^k)$ time. (2) There exists constant c such that A accepts L in at most cn^k steps. (3) Construct A' that simulates A for cn^k steps: if A accepted, output 1; otherwise output 0. (4) A' runs in polynomial time and decides L. (5) Therefore $P = \{L : L \text{ is accepted by a polynomial-time algorithm}\}$.

Proof that CIRCUIT-SAT is NP-complete

- **NP membership (Lemma 34.5):** Certificate = assignment of boolean values to each wire in circuit. Verification checks each gate's output is correctly computed from inputs. If circuit output is 1, accept. Runs in polynomial time (linear with good implementation).
- **NP-hardness (Lemma 34.6):** For any $L \in \text{NP}$ with verification algorithm A running in $O(n^k)$ time: (1) Represent computation of A as sequence of configurations. (2) Build circuit M implementing computer hardware mapping configuration c_i to c_{i+1} . (3) Paste $T(n)$ copies of M together. (4) Wire inputs for program, program counter, input x to known values. (5) Only remaining inputs correspond to certificate y . (6) Circuit C is satisfiable iff there exists certificate y such that $A(x, y) = 1$. (7) C has size polynomial in n . (8) F constructs C in polynomial time.

SAT $\rightarrow$ 3-CNF-SAT Reduction (Theorem 34.10)

- **Goal:** Transform any boolean formula ϕ into a 3-CNF formula ϕ'' that is satisfiable iff ϕ is.
- **Steps:** (1) Construct binary parse tree of ϕ . Introduce variable y_i for output of each internal node. Rewrite ϕ as AND of root variable and clauses describing each node's operation (each clause has ≤ 3 literals). (2) Convert each clause to CNF using truth table: build DNF formula equivalent to $\neg$ clause from truth table rows evaluating to 0, negate and

apply DeMorgan's laws. Each clause yields ≤ 8 CNF clauses of ≤ 3 literals. (3) Convert to exactly 3 literals per clause: for 2-literal clauses, add $(l_1 \vee l_2 \vee p) \vee (l_1 \vee l_2 \vee \neg p)$; for 1-literal clauses, add $(l \vee p \vee q) \vee (l \vee p \vee \neg q) \vee (l \vee \neg p \vee q) \vee (l \vee \neg p \vee \neg q)$.

CIRCUIT-SAT $\rightarrow$ SAT Reduction (Theorem 34.9)

- **Steps:** (1) For each wire x_i in circuit C , create variable x_i in formula ϕ . (2) For each gate, construct a clause of the form (output $\leftrightarrow$ function of inputs). (3) $\phi =$ (circuit output variable) AND (conjunction of all gate clauses). (4) Total formula size polynomial in circuit size. (5) C is satisfiable iff ϕ is satisfiable.

3-CNF-SAT $\rightarrow$ CLIQUE Reduction (Theorem 34.11)

- **Steps:** (1) For 3-CNF formula ϕ with k clauses, each clause $C_r = (l_1^r \vee l_2^r \vee l_3^r)$. (2) Create graph G with $3k$ vertices, one triple per clause. (3) Add edge between vertices v_r^p and v_s^q if: $r \neq s$ (different triples) AND the corresponding literals are consistent (l_p^r is not the negation of l_q^s). (4) ϕ is satisfiable iff G has a clique of size k .

CLIQUE $\rightarrow$ VERTEX-COVER Reduction (Theorem 34.12)

- **Steps:** (1) Given instance $\langle G, k \rangle$ of CLIQUE, compute complement graph $\bar{G}$ in polynomial time. (2) Output instance $\langle \bar{G}, |V| - k \rangle$ of VERTEX-COVER. (3) G has clique of size k iff $\bar{G}$ has vertex cover of size $|V| - k$.

VERTEX-COVER $\rightarrow$ HAM-CYCLE Reduction (Theorem 34.13)

- **Steps:** (1) For each edge $(u, v) \in E$, create gadget Γ_{uv} with 12 vertices and 14 edges. (2) Add k selector vertices $s_1, \dots, s_k$. (3) For each vertex $u \in V$, order its adjacent vertices $u(1), \dots, u(\text{degree}(u))$. Add edges connecting gadgets: $([u, u(i), 6], [u, u(i+1), 1])$ for $i=1, \dots, \text{degree}(u)-1$. (4) Connect each selector vertex to first and last vertices of each gadget path: edges $(s_j, [u, u(1), 1])$ and $(s_j, [u, u(\text{degree}(u)), 6])$. (5) G has vertex cover of size k iff G' has a hamiltonian cycle.

HAM-CYCLE $\rightarrow$ TSP Reduction (Theorem 34.14)

- **Steps:** (1) Given $G = (V, E)$ instance of HAM-CYCLE. (2) Form complete graph $G' = (V, E')$ where $E' = \{(i, j) : i, j \in V, i \neq j\}$. (3) Cost function: $c(i, j) = 0$ if $(i, j) \in E$, $c(i, j) = 1$ if $(i, j) \notin E$. (4) G has hamiltonian cycle iff G' has TSP tour of cost ≤ 0 .

3-CNF-SAT $\rightarrow$ SUBSET-SUM Reduction (Theorem 34.15)

- **Steps:** (1) Given 3-CNF formula ϕ with n variables $x_1, \dots, x_n$ and k clauses $C_1, \dots, C_k$. (2) Create numbers in base 10 with $n+k$ digits (most significant n = variables, least significant k = clauses). (3) Target t : 1 in each variable digit, 4 in each clause digit. (4) For each variable x_i : create v_i (1 in x_i digit, 1 in C_j digit if $x_i \in C_j$) and v'_i (1 in x_i digit, 1 in C_j digit if $\neg x_i \in C_j$). (5) For each clause C_j : create slack variables s_j (1 in C_j digit) and s'_j (2 in C_j digit). (6) ϕ satisfiable iff there exists subset $S' \subseteq S$ summing to t .

Classifications & Hierarchies

Complexity Classes

- **P**: the set of concrete decision problems that are polynomial-time solvable (solvable in $O(n^k)$ time for some constant k). Closed under union, intersection, concatenation, complement, and Kleene star.
- **NP**: the class of languages that can be verified by a polynomial-time algorithm. $L \in \text{NP}$ iff there exist a two-input polynomial-time algorithm A and a constant c such that $L = \{x \in \{0,1\}^* : \text{there exists certificate } y \text{ with } |y| = O(|x|^c) \text{ and } A(x,y) = 1\}$. Closed under union, intersection, concatenation, and Kleene star (closure under complement is unknown).
- **NPC** (NP-complete): languages L such that (1) $L \in \text{NP}$, and (2) $L' \leq_P L$ for every $L' \in \text{NP}$. The "hardest" problems in NP.
- **co-NP**: the set of languages L such that $\bar{L} \in \text{NP}$. Question of whether $\text{NP} = \text{co-NP}$ is open.
- **$P \subseteq \text{NP} \cap \text{co-NP}$** : P is a subset of both NP and co-NP. Whether $P = \text{NP} \cap \text{co-NP}$ is unknown.
- **$\text{NP} \neq \text{co-NP} \rightarrow P \neq \text{NP}$** (Exercise 34.2-10): If NP is not closed under complement, then NP and P cannot be equal.

Most Likely Relationship (researchers' belief)

- $P \subseteq \text{NP}$, $\text{NP} \subseteq \text{NPC}$, $P \cap \text{NPC} = \emptyset$, $\text{NP} \cap \text{co-NP}$ may extend beyond P

Reductions

How to Prove NP-completeness

1. Prove $L \in \text{NP}$ (certificate + polynomial-time verification).
2. Prove L is NP-hard:
 1. Select known NP-complete language L' .
 2. Describe polynomial-time algorithm computing reduction function f mapping instances of L' to instances of L .
 3. Prove $x \in L'$ iff $f(x) \in L$.
 4. Prove algorithm runs in polynomial time.

Common Reduction Patterns

- **Go from general to specific:** Always start with arbitrary input to problem X. Restrict input to problem Y as much as desired.
- **Take advantage of structure:** Reduce from 3-CNF-SAT rather than SAT; from HAM-CYCLE rather than TSP.
- **Look for special cases:** If X is NP-hard and a special case of Y, then Y is NP-hard.
- **Select appropriate problem domain:** Graph $\rightarrow$ Graph (CLIQUE $\rightarrow$ VERTEX-COVER); cross-domain (3-CNF-SAT $\rightarrow$ CLIQUE, 3-CNF-SAT $\rightarrow$ SUBSET-SUM).
- **Make big rewards and big penalties:** Give low weight for "good" edges, high weight for "bad" edges.
- **Design gadgets:** Subgraph components that enforce properties (e.g., 12-vertex gadget in HAM-CYCLE reduction; slack variables in SUBSET-SUM).

Transitivity of $\leq_P$

- If $L1 \leq_P L2$ and $L2 \leq_P L3$, then $L1 \leq_P L3$ (Exercise 34.3-2).

Pitfalls

- **Wrong direction:** Reduction must be from known NP-complete X to problem Y, not Y to X.
- **NP-hard $\neq$ NP-complete:** Must also prove $L \in NP$.
- **$L \leq_P L$ iff $L \leq_P L$** (Exercise 34.3-3).

Table: NP-Completeness Proof Structure

Problem	Reduced From	Key Technique
CIRCUIT-SAT (first)		Configuration simulation
SAT	CIRCUIT-SAT	Gate $\rightarrow$ clause encoding
3-CNF-SAT	SAT	Parse tree $\rightarrow$ truth table $\rightarrow$ 3-CNF
CLIQUE	3-CNF-SAT	Triple per clause, edges for consistent literals
VERTEX-COVER	CLIQUE	Complement graph
HAM-CYCLE	VERTEX-COVER	12-vertex gadgets, selector vertices
TSP	HAM-CYCLE	0/1 edge costs
SUBSET-SUM	3-CNF-SAT	Base-10 digit encoding, slack variables

Comparisons & Trade-offs

Dimension	P	NP
Solving time	Polynomial	

Dimension	P	NP
		Unknown (likely exponential)
Verification time	Polynomial	Polynomial
Membership proof	Algorithm exists	Certificate exists
Complement closure	Closed	Unknown
Examples	PATH, 2-CNF-SAT, Euler tour	HAM-CYCLE, 3-CNF-SAT, CLIQUE
Dimension	Optimization Problem	Decision Problem
Answer	Value (min/max)	Yes/No
NP-completeness applies to	Indirectly (via decision version)	Directly
Trade-off	Hard $\rightarrow$ decision is hard	Easy $\rightarrow$ optimization is easy

Formulas & Equations

Approximation ratio lower bound for HAM-CYCLE reduction to TSP

$c(i,j) = 0$ if $(i,j) \notin E$, $c(i,j) = 1$ if $(i,j) \in E$ Tour cost = 0 iff hamiltonian cycle exists.

SUBSET-SUM target

$t_i = 1$ for $i \in$ variables, $t_i = 4$ for $i \in$ clauses No carries because max sum per digit = $6 < 10$.

Rules, Laws & Theorems

Lemma 34.1 (Encoding Independence)

- **Statement:** Let Q be an abstract decision problem on instance set I , and let e_1 and e_2 be polynomially related encodings on I . Then $e_1(Q) \in P$ iff $e_2(Q) \in P$.
- **Proof:** Forward direction: If $e_1(Q)$ solvable in $O(n^k)$ and $e_1(i)$ computable from $e_2(i)$ in $O(n^c)$, then $|e_1(i)| = O(n^c)$ and total time $O(n^{\{ck\}})$ which is polynomial.

Lemma 34.3 (Reduction $\in$ P-membership)

- **Statement:** If $L_1, L_2 \subseteq \{0,1\}^*$ are languages such that $L_1 \leq_P L_2$, then $L_2 \in P$ implies $L_1 \in P$.
- **Proof:** Let A_2 decide L_2 in polynomial time, F compute reduction f in polynomial time. Algorithm A_1 : on input x , compute $f(x)$ using F , run A_2 on $f(x)$, output A_2 's answer.

Theorem 34.4 (NPC and P vs NP)

- **Statement:** If any NP-complete problem is polynomial-time solvable, then $P = NP$. Equivalently, if any problem in NP is not polynomial-time solvable, then no NP-complete problem is polynomial-time solvable.
- **Proof:** If $L \in P$ and $L \in NPC$, then for any $L' \in NP$, $L' \leq_P L$, so by Lemma 34.3, $L' \in P$. Thus $P = NP$.

Theorem 34.7 (CIRCUIT-SAT is NP-complete)

- **Statement:** The circuit-satisfiability problem is NP-complete.
- **Proof:** From Lemmas 34.5 ($\in NP$) and 34.6 (NP-hard).

Theorem 34.9 (SAT is NP-complete)

- **Statement:** Satisfiability of boolean formulas is NP-complete.
- **Proof:** $SAT \in NP$ (certificate = satisfying assignment).
 $CIRCUIT-SAT \leq_P SAT$: express circuit as boolean formula with variable per wire and clause per gate.

Theorem 34.10 (3-CNF-SAT is NP-complete)

- **Statement:** Satisfiability of boolean formulas in 3-conjunctive normal form is NP-complete.
- **Proof:** $3\text{-CNF-SAT} \in NP$. $SAT \leq_P 3\text{-CNF-SAT}$ via parse tree $\rightarrow$ truth table $\rightarrow$ 3-CNF conversion.

Theorem 34.11 (CLIQUE is NP-complete)

- **Statement:** The clique problem is NP-complete.
- **Proof:** $CLIQUE \in NP$ (certificate = set of vertices). $3\text{-CNF-SAT} \leq_P CLIQUE$ via triple-per-clause graph construction.

Theorem 34.12 (VERTEX-COVER is NP-complete)

- **Statement:** The vertex-cover problem is NP-complete.
- **Proof:** $VERTEX\text{-COVER} \in NP$ (certificate = vertex set). $CLIQUE \leq_P VERTEX\text{-COVER}$ via complement graph: G has clique of size k iff G has vertex cover of size $|V| - k$.

Theorem 34.13 (HAM-CYCLE is NP-complete)

- **Statement:** The hamiltonian cycle problem is NP-complete.
- **Proof:** $HAM\text{-CYCLE} \in NP$ (certificate = vertex sequence).
 $VERTEX\text{-COVER} \leq_P HAM\text{-CYCLE}$ using 12-vertex gadgets per edge and k selector vertices.

Theorem 34.14 (TSP is NP-complete)

- **Statement:** The traveling-salesperson problem is NP-complete.

- **Proof:** TSP $\in$ NP (certificate = vertex sequence). HAM-CYCLE $\leq$ P TSP via 0/1 cost function.

Theorem 34.15 (SUBSET-SUM is NP-complete)

- **Statement:** The subset-sum problem is NP-complete.
- **Proof:** SUBSET-SUM $\in$ NP (certificate = subset). 3-CNF-SAT $\leq$ P SUBSET-SUM via base-10 encoding with $n+k$ digits.

Lemma 34.5 (CIRCUIT-SAT $\in$ NP)

- **Statement:** The circuit-satisfiability problem belongs to NP.
- **Proof:** Certificate = assignment of boolean values to each wire. Verify each gate's output vs inputs; if circuit output = 1, accept.

Lemma 34.6 (CIRCUIT-SAT is NP-hard)

- **Statement:** The circuit-satisfiability problem is NP-hard.
- **Proof:** For any $L \in$ NP with verification algorithm A , construct circuit C simulating $T(n)$ steps of A ; C is satisfiable iff there exists certificate y such that $A(x,y)=1$.

Complete List of NP-Complete Problems (from text)

1. **CIRCUIT-SAT:** Given a boolean combinational circuit of AND, OR, NOT gates, is it satisfiable? (First NP-complete problem)
2. **SAT:** Given a boolean formula, is it satisfiable?
3. **3-CNF-SAT:** Given a boolean formula in 3-conjunctive normal form, is it satisfiable?
4. **CLIQUE:** Does an undirected graph contain a clique of size k ?
5. **VERTEX-COVER:** Does an undirected graph have a vertex cover of size k ?
6. **HAM-CYCLE:** Does an undirected graph have a hamiltonian cycle?
7. **TSP:** Does a complete graph with edge costs have a traveling-salesperson tour of cost $\leq k$?
8. **SUBSET-SUM:** Given a set S of positive integers and target t , does there exist a subset summing to t ?
9. **LONGEST-PATH:** Does an undirected graph contain a simple path with at least k edges?
10. **SET-COVER** (decision version, Exercise 35.3-2): Does there exist a cover of size $\leq k$?
11. **GRAPH-ISOMORPHISM:** Are two graphs isomorphic? ($\in$ NP, Exercise 34.2-1)
12. **HAM-PATH:** Does there exist a hamiltonian path from u to v ? (Exercise 34.2-6, 34.5-6)
13. **SUBGRAPH-ISOMORPHISM:** Is G_1 isomorphic to a subgraph of G_2 ? (Exercise 34.5-1)
14. **0-1 INTEGER PROGRAMMING:** Does there exist $x \in \{0,1\}^n$ such that $Ax \leq b$? (Exercise 34.5-2)

15. **INTEGER LINEAR PROGRAMMING:** Same as 0-1 IP but x may be any integers. (Exercise 34.5-3)
16. **SET-PARTITION:** Can S be partitioned into two sets with equal sum? (Exercise 34.5-5)
17. **LONGEST-SIMPLE-CYCLE:** Determine a simple cycle of maximum length. (Exercise 34.5-7)
18. **HALF 3-CNF SAT:** Given 3-CNF ϕ with m clauses (m even), is there an assignment where exactly half evaluate to 1? (Exercise 34.5-8)
19. **INDEPENDENT-SET:** Find a maximum-size set of vertices with no edges between them. (Problem 34-1)
20. **3-COLOR:** Can a graph be colored with 3 colors such that adjacent vertices differ? (Problem 34-3)
21. **GRAPH-COLORING:** Determine minimum number of colors needed. (Problem 34-3)
22. **SCHEDULING WITH PROFITS AND DEADLINES:** Schedule tasks to maximize profit given processing times and deadlines. (Problem 34-4)
23. **TAUTOLOGY:** Is a boolean formula a tautology? (co-NP-complete, Exercise 34.4-4)

Edge Cases & Pitfalls

- **Encodings matter:** Unary encoding can make exponential problems polynomial (and vice versa). Standard encoding assumes binary or reasonable concise encoding.
- **Nonconstructive proof:** Theorem 34.2 proof is nonconstructive—exists bound but may not know it.
- **Isolated vertices** break the VERTEX-COVER $\rightarrow$ HAM-CYCLE reduction (Exercise 34.5-9). The reduction assumes no isolated vertices. If a graph has isolated vertices, they belong to any vertex cover but don't help construct the hamiltonian cycle.
- **Wrong reduction direction:** To show Y is NP-complete, reduce from known NP-complete X to Y , not Y to X .
- **Must prove both NP and NP-hard:** Showing NP-hard alone is insufficient for NP-completeness.
- **Naive circuit-to-formula reduction** can cause exponential formula size when gates have fan-out ≥ 2 . The correct method introduces a new variable per wire.
- **Incorrect use of truth-table only:** Professor Jagger's proposal to convert SAT to 3-CNF-SAT using only truth tables fails because the truth table has 2^n rows (exponential).
- **Restricted structural conclusions:** Showing CLIQUE is NP-hard only in graphs with vertices in triples suffices because polynomial-time algorithm for general CLIQUE would also solve restricted case.
- **Carries in SUBSET-SUM reduction:** Base 10 prevents carries (max sum per digit = 6). Any base $b \geq 7$ works.

End-of-Chapter Material

Key Terms (Ch. 34)

- P, NP, NPC (NP-complete), NP-hard, polynomial-time reduction, certificate, verification algorithm, decision problem, optimization problem, language, encoding, complement graph, clique, vertex cover, hamiltonian cycle, traveling-salesperson problem, subset-sum problem, gadget, reduction function, satisfiability, tautology, co-NP, 3-CNF, literal, clause, conjunctive normal form, disjunctive normal form, truth assignment, satisfying assignment

Review Questions (Exercises Key Ideas)

34.1-1: LONGEST-PATH-LENGTH optimization can be solved in polynomial time iff LONGEST-PATH $\in$ P. 34.1-2: Longest simple cycle decision problem: $\langle G, k \rangle$ – does G contain a simple cycle of length $\geq k$? 34.1-3: Adjacency matrix and adjacency list encodings are polynomially related. 34.1-4: The DP algorithm for 0-1 knapsack (Exercise 15.2-2) runs in $O(nW)$ time, which is pseudo-polynomial (not polynomial if W is large). 34.1-5: Constant number of calls to polynomial-time subroutines yields polynomial time. Polynomial number of calls may yield exponential time. 34.1-6: P is closed under union, intersection, concatenation, complement, and Kleene star. 34.2-1: GRAPH-ISOMORPHISM $\in$ NP: certificate = vertex mapping; verify edges are preserved. 34.2-2: Bipartite graph with odd number of vertices is nonhamiltonian because hamiltonian cycles must alternate between partitions, requiring equal size partitions. 34.2-3: If HAM-CYCLE $\in$ P, can list vertices by querying decision problem (removing vertices, testing). 34.2-4: NP closed under union, intersection, concatenation, Kleene star. Closure under complement is unknown. 34.2-5: Any language in NP can be decided in $O(2^{\{n\}})$ time (brute force over all certificates). 34.2-6: HAM-PATH $\in$ NP (certificate = vertex sequence). 34.2-7: Hamiltonian path in DAG can be solved in polynomial time via topological sort. 34.2-8: TAUTOLOGY $\in$ co-NP: certificate = falsifying assignment. 34.2-9: P $\in$ co-NP because P closed under complement and P $\in$ NP. 34.2-10: NP $\neq$ co-NP $\rightarrow$ P $\neq$ NP (by contrapositive of P = NP $\iff$ NP = co-NP). 34.2-11: For connected undirected G with ≥ 3 vertices, $G^{\wedge 3}$ (connecting vertices at distance ≤ 3) is hamiltonian. 34.3-1: Circuit in Figure 34.8(b) is unsatisfiable. 34.3-2: $\leq P$ is transitive. 34.3-3: $L \leq P L$ iff $L \leq P L$. 34.3-4: Satisfying assignment as certificate is easier than full wire assignment. 34.3-5: Contiguous memory assumption can be relaxed; scattered memory is reducible via bookkeeping. 34.3-6: Only $\emptyset$ and $\{0,1\}^*$ are not complete for P. 34.3-7: L is NP-complete iff L is co-NP-complete. 34.3-8: The existence of A, k is known (by definition of NP), even if exact constants are unknown. F uses existential guarantee. 34.4-1: A circuit of AND gates with all inputs wired to same variable, when naively expanded, causes exponential formula size. 34.4-2: Show the 3-CNF

formula from applying Theorem 34.10 on $\phi = ((x_1 \rightarrow x_2) \wedge \neg((\neg x_1 \rightarrow x_3) \wedge x_4)) \wedge \neg x_2$. 34.4-3: Truth table has 2^n rows – exponential time. 34.4-4: TAUTOLOGY is co-NP-complete. 34.4-5: DNF satisfiability is polynomial-time solvable (check if any AND clause has no complementary literals). 34.4-6: Use decision algorithm + binary search on variable assignments to find satisfying assignment in polynomial time. 34.4-7: 2-CNF-SAT $\in$ P by reducing to strongly connected components in implication graph: formula unsatisfiable iff some variable and its negation in same SCC. 34.5-1: SUBGRAPH-ISOMORPHISM is NP-complete (reduce from CLIQUE). 34.5-2: 0-1 INTEGER PROGRAMMING is NP-complete (reduce from 3-CNF-SAT). 34.5-3: INTEGER LINEAR PROGRAMMING is NP-complete. 34.5-4: SUBSET-SUM is in P if t is in unary (DP in $O(nt)$ time). 34.5-5: SET-PARTITION is NP-complete (reduce from SUBSET-SUM). 34.5-6: HAMILTONIAN-PATH is NP-complete. 34.5-7: LONGEST-SIMPLE-CYCLE decision problem is NP-complete. 34.5-8: HALF 3-CNF SAT is NP-complete. 34.5-9: Isolated vertices cause the reduction to fail because no edges connect gadget paths for that vertex, making the construction undefined.

Problems (Ch. 34)

34-1 Independent Set a. Decision: $\langle G, k \rangle$ – does G have independent set of size k ? NP-complete via CLIQUE reduction (independent set in G = clique in $\bar{G}$). b. Polynomial-time algorithm using black box: for each vertex, test if removing it still allows size k ; build independent set greedily. c. Degree-2 graphs: collection of paths and cycles; can solve in $O(V+E)$ time. d. Bipartite graphs: $|\text{maximum independent set}| = |V| - |\text{maximum matching}|$ (König's theorem). Use Hopcroft-Karp for matching.

34-2 Bonnie and Clyde a. Two denominations: polynomial-time (DP or reduce to subset-sum with 2 values). b. Powers of 2: polynomial-time (greedy binary representation). c. Arbitrary checks: NP-complete (SUBSET-SUM reduction). d. Difference $\leq \$100$: likely NP-complete (still general SUBSET-SUM).

34-3 Graph Coloring a. 2-coloring: BFS with alternating colors; $O(V+E)$. b. Decision: $\langle G, k \rangle$ – can G be colored with k colors? Polynomial iff optimization is polynomial. c. If 3-COLOR is NP-complete, then k -COLOR for $k \geq 3$ is NP-complete. d. In any 3-coloring with literal edges (TRUE, FALSE, RED triangle + x_i , $\neg x_i$, RED triangles): exactly one of $\{x_i, \neg x_i\} = c(\text{TRUE})$, other = $c(\text{FALSE})$. e. Clause gadget (5 vertices): 3-colorable iff at least one of $\{x, y, z\} = c(\text{TRUE})$. f. 3-COLOR is NP-complete (3-CNF-SAT $\leq_P$ 3-COLOR).

34-4 Scheduling with Profits and Deadlines a. Decision: given tasks and profit P , does there exist schedule with profit $\geq P$? b. NP-complete (reduce from SUBSET-SUM). c. Polynomial if processing times $\in \{1, \dots, n\}$: DP over time and tasks. d. Optimization version: DP for min-deadline schedule maximizing profit.

Ch. 35 – Approximation Algorithms

Named Entities (Terms & Definitions)

- **Approximation Algorithm:** an algorithm that returns near-optimal solutions in polynomial time for NP-complete optimization problems.
- **Approximation Ratio $\rho(n)$:** for any input of size n , cost C of solution produced by algorithm is within a factor $\rho(n)$ of optimal cost C^* . For minimization: $C/C^* \leq \rho(n)$. For maximization: $C^*/C \leq \rho(n)$. Always ≥ 1 . A 1-approximation algorithm produces an optimal solution.
- **$\rho(n)$ -approximation Algorithm:** an algorithm achieving approximation ratio $\rho(n)$.
- **Approximation Scheme:** an approximation algorithm that takes input instance + value $\epsilon > 0$ such that for any fixed ϵ , the scheme is a $(1+\epsilon)$ -approximation algorithm.
- **Polynomial-Time Approximation Scheme (PTAS):** for any fixed $\epsilon > 0$, runs in time polynomial in n (the input size). Running time may increase very rapidly as ϵ decreases (e.g., $O(n^{2/\epsilon})$).
- **Fully Polynomial-Time Approximation Scheme (FPTAS):** an approximation scheme whose running time is polynomial in both $1/\epsilon$ and n (e.g., $O((1/\epsilon)^2 n^3)$).
- **Minimum Spanning Tree (MST):** used as lower bound in TSP approximation.
- **Full Walk:** a walk of a tree that lists vertices when first visited and whenever returned to after visiting a subtree; traverses every edge exactly twice.
- **Preorder Tree Walk:** recursively visits every vertex in the tree, listing a vertex when first encountered, before visiting any of its children.
- **Triangle Inequality:** for all vertices u, v, w : $c(u, w) \leq c(u, v) + c(v, w)$. Holds for Euclidean distance and many practical cost functions.
- **Maximal Matching:** a matching to which no edges can be added. Used as lower bound for optimal vertex cover.
- **Vertex Cover:** subset $V' \subseteq V$ covering all edges (each edge has at least one endpoint in V').
- **Minimum-Weight Vertex Cover:** vertex cover minimizing sum of vertex weights.
- **Linear-Programming Relaxation:** replacing integrality constraints ($x(v) \in \{0, 1\}$) with fractional constraints ($0 \leq x(v) \leq 1$). Provides lower bound for optimization.
- **Makespan ($C_{\max}$):** the maximum completion time of any job in a schedule.
- **First-Fit Heuristic:** places each object into first bin that can accommodate it (for bin packing).
- **Merged List (subset-sum):** L_i = sorted list of all subset sums not exceeding t .
- **Trimming:** removing elements from a sorted list L such that for each removed y , there exists remaining z with $z \leq y \leq z \cdot (1+\delta)$.

- **Slack Variables:** s_j (value 1) and s'_j (value 2) in SUBSET-SUM reduction.
- **Randomized Approximation Algorithm:** approximation ratio is for expected cost.

Processes / Algorithms / Pathways

APPROX-VERTEX-COVER (2-approximation)

- **Goal:** Find vertex cover of size at most twice optimal.
- **Input/Output:** Input: undirected graph $G = (V, E)$. Output: vertex cover C .
- **Steps:** (1) $C = \emptyset$. (2) $E' = E$. (3) While $E' \neq \emptyset$: (a) Pick arbitrary edge (u, v) from E' . (b) $C = C \cup \{u, v\}$. (c) Remove from E' edge (u, v) and all edges incident on u or v . (4) Return C .
- **Complexity:** $O(V+E)$ time using adjacency lists.
- **Example:** Graph with 7 vertices, 8 edges produces cover $\{b, c, d, e, f, g\}$ of size 6 vs optimal $\{b, d, e\}$ of size 3. Ratio = 2.
- **Proof:** Let A = set of edges picked. (1) $|C| \geq |A|$ (each edge in A needs distinct vertex from optimal cover since A is a maximal matching). (2) $|C| = 2|A|$. (3) Therefore $|C| = 2|A| \leq 2|C|$.

APPROX-TSP-TOUR (2-approximation with triangle inequality)

- **Goal:** Find TSP tour of cost at most twice optimal.
- **Input/Output:** Input: complete undirected graph G , cost function c satisfying triangle inequality. Output: hamiltonian cycle H .
- **Steps:** (1) Select root vertex $r \in V$. (2) Compute MST T using MST-PRIM(G, c, r). (3) Let H be list of vertices in preorder walk of T (first visit order). (4) Return H .
- **Complexity:** $O(V^2)$ with simple Prim implementation.
- **Example:** Grid points, MST from root a , preorder walk gives tour cost ~ 19.074 vs optimal ~ 14.715 ($\approx 23\%$ shorter).
- **Proof:** (1) $c(T) \leq c(H)$ (MST weight lower bounds optimal tour). (2) Full walk W traverses each edge of T twice: $c(W) = 2c(T) \leq 2c(H)$. (3) By triangle inequality, shortcutting duplicates does not increase cost: $c(H) \leq c(W)$. (4) $c(H) \leq 2c(H^*)$.

Greedy Set Cover ($O(\lg |X|)$ -approximation)

- **Goal:** Find subfamily $C \subseteq \mathcal{C}$ covering X of minimum size.
- **Input/Output:** Input: finite set X , family $\mathcal{C}$ of subsets covering X . Output: subfamily $C \subseteq \mathcal{C}$ covering X .
- **Steps:** (1) $U_0 = X$, $C = \emptyset$, $i = 0$. (2) While $U_i \neq \emptyset$: (a) Select $S \in \mathcal{C}$ maximizing $|S \cap U_i|$. (b) $U_{i+1} = U_i - S$. (c) $C = C \cup \{S\}$. (d) $i = i+1$. (3) Return C .

- **Complexity:** $O(|X| \cdot |C| \cdot (|X| + |C|))$, can be $O(\sum_{S \in C} |S|)$ with efficient implementation.
- **Example:** $X = 12$ elements, $C = \{S_1, \dots, S_6\}$ produces cover $\{S_1, S_4, S_5, S_3\}$ of size 4 vs optimal $\{S_3, S_4, S_5\}$ of size 3.
- **Proof:** (1) Let $k = |C|$. (2) At each step, some set in C covers at least $|U_i|/k$ elements (since k optimal sets cover U_i). (3) Greedy picks set covering $\geq |U_i|/k$ elements. (4) $|U_{i+1}| \leq |U_i|(1-1/k)$. (5) $|U_i| \leq |X|(1-1/k)^i$. (6) Using $1+x \leq e^x$: $|U_i| \leq |X|e^{-i/k}$. (7) Algorithm stops when $|U_i| < 1$, requiring $i \geq k \cdot \ln|X|$. (8) $|C| \leq |C| \cdot \ln|X|$.

APPROX-MIN-WEIGHT-VC (LP-rounding 2-approximation)

- **Goal:** Find minimum-weight vertex cover.
- **Input/Output:** Input: undirected graph $G = (V, E)$, vertex weights $w(v) > 0$. Output: vertex cover C .
- **Steps:** (1) $C = \emptyset$. (2) Solve LP relaxation: minimize $\sum w(v)x(v)$ s.t. $x(u)+x(v) \geq 1 \ \forall (u,v) \in E$, $0 \leq x(v) \leq 1$. Let x be optimal solution. (3) For each vertex v : if $x(v) \geq 1/2$, $C = C \cup \{v\}$. (4) Return C .
- **Complexity:** polynomial (LP solver + $O(V)$ rounding).
- **Proof:** (1) $z^* = \text{optimal LP value} \leq w(C)$ (LP relaxation lower bound). (2) For any edge (u,v) : $x(u)+x(v) \geq 1$ $\Rightarrow$ at least one of $x(u), x(v) \geq 1/2 \Rightarrow C$ is a vertex cover. (3) $w(C) = \sum_{v \in C} w(v) \leq \sum_{v \in V} 2 \cdot w(v) \cdot x(v) \leq 2 \cdot \sum_{v \in V} w(v) \cdot x(v) = 2z \leq 2w(C^*)$.

Randomized MAX-3-CNF SAT (8/7-approximation)

- **Goal:** Satisfy as many clauses as possible.
- **Steps:** (1) Independently set each variable to 1 with probability $1/2$, 0 with probability $1/2$. (2) Return assignment.
- **Expected Ratio:** (1) For each clause i with 3 distinct literals, $\Pr[\text{satisfied}] = 1 - (1/2)^3 = 7/8$. (2) $E[Y] = m \cdot 7/8$ where m = number of clauses. (3) Expected approximation ratio $\leq m/(7m/8) = 8/7$.
- **Assumptions:** No clause contains both a variable and its negation. (Removable, Exercise 35.4-1.)

EXACT-SUBSET-SUM (Exponential exact algorithm)

- **Goal:** Find subset of S with largest sum $\leq t$.
- **Input/Output:** Input: set $S = \{x_1, \dots, x_n\}$, target t . Output: max value in L_n .
- **Steps:** (1) $L_0 = \{0\}$. (2) For $i = 1$ to n : $L_i = \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$. Remove elements $> t$ from L_i . (3) Return largest element in L_n .
- **Complexity:** $O(2^n)$ worst-case (list length can be 2^i).
- **Polynomial cases:** when t is polynomial in $|S|$ or all numbers bounded polynomially.

APPROX-SUBSET-SUM (FPTAS)

- **Goal:** Find subset sum within factor $(1+\epsilon)$ of optimal.
- **Input/Output:** Input: set $S = \{x_1, \dots, x_n\}$, target t , approximation parameter ϵ where $0 < \epsilon < 1$. Output: z^* such that $y/z \leq 1+\epsilon$ where $y^* = \text{optimal}$.
- **Steps:** (1) $L_0 = \emptyset$. (2) For $i = 1$ to n : (a) $L_i = \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$. (b) $L_i = \text{TRIM}(L_i, \epsilon/2n)$. (c) Remove elements $> t$ from L_i . (3) Return largest value in L_n .
- **Complexity:** $O(n \cdot \log_{1+\epsilon/2n} t) = O(n \cdot (\ln t)/(\epsilon/2n)) = O(n^2 \cdot (\ln t)/\epsilon)$. Polynomial in n , $1/\epsilon$, and input size.
- **Example:** $S = \{104, 102, 201, 101\}$, $t = 308$, $\epsilon = 0.40$. $\delta = 0.05$. Returns $z^* = 302$ vs optimal 307 ($< 2\%$ error well within 40%).

TRIM Procedure

- **Goal:** Remove near-duplicate values from sorted list.
- **Input/Output:** Input: sorted list L , parameter $\delta \in (0, 1)$. Output: trimmed list L' .
- **Steps:** (1) $L' = \{y_1\}$, $\text{last} = y_1$. (2) For $i = 2$ to m : if $y_i > \text{last} \cdot (1+\delta)$: append y_i to L' , $\text{last} = y_i$. (3) Return L' .
- **Complexity:** $O(m)$ time.
- **Property:** For every removed y , there exists remaining z such that $z \leq y \leq z \cdot (1+\delta)$.

First-Fit Heuristic (Bin Packing, Problem 35-1)

- **Steps:** (1) Maintain ordered list of bins. (2) For each object i in turn: place in lowest-numbered bin that can accommodate it. (3) If no bin can, open new bin with object i .
- **Approximation Ratio:** 2. At most one bin $\leq 1/2$ full.

Greedy Parallel Machine Scheduling (Problem 35-5)

- **Steps:** Whenever a machine is idle, schedule any unscheduled job.
- **Approximation Ratio:** 2. $C_{\max} \leq (\sum p_k)/m + \max p_k \leq 2 \cdot C^*_{\max}$.

0-1 Knapsack 2-Approximation (Problem 35-7)

- **Steps:** (1) Sort items by value $v_1 \geq v_2 \geq \dots \geq v_n$. (2) For $j = 1$ to n : form instance I_j requiring item j . Solve fractional knapsack for I_j greedily (by v_i/w_i). Delete fractional item to get R_j . (3) Return max-value solution from $\{R_1, \dots, R_n\}$.
- **Approximation Ratio:** 2. $v(R_j) \geq v(Q_j)/2 \geq v(P_j)/2$.

Classifications & Hierarchies

Approximation Classes

- **PTAS (Polynomial-Time Approximation Scheme):** runs in $O(n^{f(\epsilon)})$ time for any fixed $\epsilon > 0$.

- **FPTAS (Fully Polynomial-Time Approximation Scheme)**: runs in $O((1/\epsilon)^c \cdot n^d)$ time for constants c, d .
- **APX**: problems with constant-factor approximation algorithms.
- **No constant approximation possible (unless P=NP)**: general TSP (Theorem 35.3), clique? (open but believed).

Inapproximability Results

- **General TSP**: For any constant $p \geq 1$, no polynomial-time p -approximation exists unless $P = NP$ (Theorem 35.3).
- **General TSP with $|V|^c$ approximation**: No polynomial-time $|V|^c$ -approximation for any constant $c \geq 0$ unless $P = NP$ (Exercise 35.2-6).

Reductions

Proving Inapproximability (Gap Technique)

- Given NP-hard decision problem X , produce optimization problem Y such that:
 - "Yes" instances of $X \rightarrow$ instances of Y with optimal value $\leq k$
 - "No" instances of $X \rightarrow$ instances of Y with optimal value $> p \cdot k$
- A p -approximation algorithm could distinguish "yes" from "no", solving X in polynomial time.

Comparisons & Trade-offs

Algorithm	Problem	Ratio	Technique
APPROX-VERTEX-COVER	Vertex Cover	2	Maximal matching
APPROX-TSP-TOUR	TSP (triangle ineq.)	2	MST + preorder walk
Greedy Set Cover	Set Cover	$O(\lg X)$	Greedy by uncovered count
Random	MAX-3-CNF SAT	$8/7$ (expected)	Random assignment
LP Rounding	Min-Weight VC	2	LP + threshold rounding
APPROX-SUBSET-SUM	Subset Sum	$1+\epsilon$ (FPTAS)	Trimming + DP
First-Fit	Bin Packing	2	First-fit heuristic
Greedy	Parallel Machine Sched.	2	Idle machine scheduling
Greedy	0-1 Knapsack	2	Fractional + round down
Christofides	TSP (triangle ineq.)	$3/2$	MST + matching

Scheme	Running Time	Example
PTAS	$O(n^{f(\epsilon)})$	$O(n^{2/\epsilon})$
FPTAS	$O((1/\epsilon)^c \cdot n^d)$	$O((1/\epsilon)^2 n^3)$

Formulas & Equations

Approximation Ratio

$$\max(C/C^*, C^*/C) \leq \rho(n)$$

Triangle Inequality

$$c(u, w) \leq c(u, v) + c(v, w)$$

Full Walk Cost

$$c(W) = 2c(T) \text{ (each MST edge traversed twice)}$$

Greedy Set Cover Bound

$$|U_{i+1}| \leq |U_i| \cdot (1 - 1/k) \text{ where } k = |C^*|$$

$$|U_i| \leq |X| \cdot (1 - 1/k)^i \leq |X| \cdot e^{-i/k} \text{ Number of iterations } \leq k \cdot \lceil \ln |X| \rceil$$

LP Relaxation for Min-Weight VC

$$\begin{aligned} &\text{minimize } \sum w(v) \cdot x(v) \\ &\text{subject to } x(u) + x(v) \geq 1 \text{ for all } (u, v) \in E \\ &0 \leq x(v) \leq 1 \text{ for all } v \in V \end{aligned}$$

Subset-Sum Trimming Bound

$$z'/z > 1 + \delta \text{ (successive trimmed elements differ by factor } > 1+\delta)$$

$$\text{List length bound: } \log_{1+\epsilon/2n} t = (\ln t) / \ln(1+\epsilon/2n)$$

FPTAS Approximation Bound

$$(1+\epsilon/2n)^n \leq 1+\epsilon \text{ (proved via inequality: for } 0 < \epsilon < 1, (1+\epsilon/2n)^n \leq e^{\epsilon/2} \leq 1+\epsilon)$$

Optimal Makespan Lower Bounds (Problem 35-5)

$$C^*_{\max} \geq \max_j p_j \quad C^*_{\max} \geq (1/m) \cdot \sum p_j$$

Rules, Laws & Theorems

Theorem 35.1 (Vertex Cover 2-Approximation)

- **Statement:** APPROX-VERTEX-COVER is a polynomial-time 2-approximation algorithm.

- **Proof:** Edges A form a maximal matching, so $|C| \geq |A|$.
Algorithm returns $|C| = 2|A| \leq 2|C|$.

Theorem 35.2 (TSP 2-Approximation with Triangle Inequality)

- **Statement:** When triangle inequality holds, APPROX-TSP-TOUR is a polynomial-time 2-approximation algorithm.
- **Proof:** $c(T) \leq c(H)$, $c(W) = 2c(T) \leq 2c(H)$, shortcutting gives $c(H) \leq c(W) \leq 2c(H^*)$.

Theorem 35.3 (General TSP Inapproximability)

- **Statement:** If $P \neq NP$, then for any constant $p \geq 1$, no polynomial-time p -approximation exists for general TSP.
- **Proof:** (1) Assume p -approximation A exists. (2) Given HAM-CYCLE instance G , construct TSP instance G' with costs: $c(u,v)=1$ if $(u,v) \in E$, $c(u,v)=p|V|+1$ if $(u,v) \notin E$. (3) If G has hamiltonian cycle, optimal tour cost $= |V|$. (4) If G has no hamiltonian cycle, optimal tour cost $\geq p|V|+|V| > p \cdot |V|$. (5) A returns tour of cost $\leq p \cdot |V|$ iff G has hamiltonian cycle. This solves HAM-CYCLE in polynomial time, contradiction unless $P=NP$.

Theorem 35.4 (Greedy Set Cover $O(\lg|X|)$ -approximation)

- **Statement:** GREEDY-SET-COVER is a polynomial-time $O(\lg|X|)$ -approximation algorithm.
- **Proof:** $|C| \leq |C^*| \cdot \ln|X|$.

Theorem 35.5 (Randomized MAX-3-CNF SAT $8/7$ -approximation)

- **Statement:** Randomly setting each variable to 1 with probability $1/2$ yields a randomized $8/7$ -approximation algorithm.
- **Proof:** Each clause satisfied with probability $7/8$. Expected satisfied $= 7m/8$. Ratio $\leq m/(7m/8) = 8/7$.

Theorem 35.6 (LP-Rounding Weighted VC 2-approximation)

- **Statement:** APPROX-MIN-WEIGHT-VC is a polynomial-time 2-approximation algorithm.
- **Proof:** $z^* \leq w(C)$. Rounding: C is vertex cover since $x(u)+x(v) \geq 1$ means some $x \geq 1/2$. $w(C) \leq 2z \leq 2w(C^*)$.

Theorem 35.7 (SUBSET-SUM FPTAS)

- **Statement:** APPROX-SUBSET-SUM is a fully polynomial-time approximation scheme for the subset-sum problem.
- **Proof:** (1) For every $y \in [P_i \leq t, \sum z \in L_i]$ with $y/(1+\epsilon/2n)^n \leq z \leq y$. (2) Therefore $y/z \leq (1+\epsilon/2n)^n \leq 1+\epsilon$. (3) Runtime polynomial in n , $\lg t$, and $1/\epsilon$.

Edge Cases & Pitfalls

- **Triangle inequality must hold:** Without it, no constant-ratio approximation is possible for TSP (unless $P = NP$).
- **Weighted vertex cover:** The unweighted 2-approximation (maximal matching) does NOT work for weighted case (LP relaxation needed).
- **Greedy set cover ties:** Can produce exponential number of different solutions depending on tie-breaking (Exercise 35.3-5).
- **TSP cost non-negativity:** Triangle inequality implies $c(u,v) \geq 0$ for all u,v (Exercise 35.2-1).
- **Fractional vs 0-1 knapsack:** Rounding fractional solution down loses at most $1/2$ value in each restricted instance, leading to 2-approximation.
- **Subset-sum trimming parameter:** $\delta = \epsilon/2n$ (not ϵ) to control cumulative approximation error over n trimming steps.
- **Set cover can have large approximation ratio:** Greedy is $O(\lg |X|)$ which grows with input size (unlike constant-ratio approximations).
- **Approximation ratio definitions differ for max vs min problems:** For maximization, $\text{ratio} = C/C \geq 1$. For minimization, $\text{ratio} = C/C \geq 1$.

End-of-Chapter Material

Key Terms (Ch. 35)

- approximation ratio, $p(n)$ -approximation algorithm, approximation scheme, PTAS, FPTAS, triangle inequality, vertex cover, traveling-salesperson problem, set covering, MAX-3-CNF satisfiability, randomized approximation, linear-programming relaxation, rounding, trimming, subset-sum problem, fully polynomial-time approximation scheme, parallel machine scheduling, makespan, bin packing, first-fit heuristic, maximal matching, minimum-weight vertex cover

Review Exercises

35.1-1: Give graph where APPROX-VERTEX-COVER is suboptimal (e.g., star graph: $K_{1,n}$). 35.1-2: Edges picked by APPROX-VERTEX-COVER form a maximal matching (can't add another edge without sharing endpoint). 35.1-3: Degree-based heuristic (select highest-degree vertex) does NOT guarantee ratio 2. Counterexample: bipartite graph with uniform degree left, varying degree right. 35.1-4: Optimal vertex cover for tree in linear time: DP with states (covered by parent / not covered). 35.1-5: Relationship between VC and CLIQUE (complement) does NOT give constant approximation for CLIQUE. VC is min, CLIQUE is max; no known constant-ratio approximation for CLIQUE. 35.2-1: Triangle inequality $\Rightarrow c(u,v) \geq 0$ for all u,v in complete graph with ≥ 3 vertices. 35.2-2: Transform non-triangle-inequality TSP instance: add large

constant to all edge costs. Same optimal tours. Does NOT contradict Theorem 35.3 because this transformation changes approximation ratios. 35.2-3: Closest-point heuristic is 2-approximation for TSP with triangle inequality (similar to MST-based proof). 35.2-4: Bottleneck TSP 3-approximation using bottleneck spanning tree + careful shortcutting (skip ≤ 2 consecutive nodes). 35.2-5: Optimal TSP tour on euclidean points never crosses itself. 35.2-6: Adapt Theorem 35.3 for $|V|^c$ approximation ratio using cost = 1 for edges in G , cost = $|V|^{c+1}$ for edges not in G . 35.3-1: Words as letter sets: apply greedy breaking ties by dictionary order. 35.3-2: Vertex-cover $\leq$ P set-cover: elements = edges, sets = vertices covering incident edges. 35.3-3: Implement GREEDY-SET-COVER in $O(\sum |S_i|)$ time using precomputed counts and linked lists. 35.3-4: Trivial bound: $|C| \leq |C^*| \cdot \max\{|S_i| : S_i \in \mathcal{S}\}$. 35.3-5: BAD-SET-COVER-INSTANCE(n) exponential in n . 35.4-1: Even with variable and negation in same clause, random assignment gives $8/7$ ratio. 35.4-2: MAX-CNF SAT: random assignment gives 2-approximation (each clause of k literals has $\Pr[\text{satisfied}] = 1 - 2^{-k} \geq 1/2$). 35.4-3: MAX-CUT: random assignment gives 2-approximation (each edge crosses cut with probability $1/2$). 35.4-4: Removing $x(v) \leq 1$ from LP: any optimal solution automatically satisfies $x(v) \leq 1$ (by edge constraints and non-negativity). 35.5-1: Prove $P_i = P_{i-1} \cup \{x_i\}$ by induction. L_i contains all elements of $P_i \leq t$. 35.5-2: Inductive proof of $y/(1+\epsilon/2n)^i \leq z \leq y$ for trimmed lists. 35.5-3: Prove $(1+\epsilon/2n)^n \leq e^{\epsilon/2} \leq 1+\epsilon$ for $0 < \epsilon < 1$. 35.5-4: Modify to find smallest sum $\geq t$ (similar trimming, track complement). 35.5-5: Track subsets with each sum by storing predecessor pointers. 35.3-1 (additional): Greedy might give different covers depending on tie-breaking; dictionary-order tie-breaking example. 35.3-5: Instance where Greedy has exponential number of possible covers due to tie-breaking. 35.4-4: LP constraint $x(v) \leq 1$ is redundant.

Problems (Ch. 35)

35-1 Bin Packing a. Subset-sum reduces to bin packing (NP-hard). b. Optimal $\geq \lceil S \rceil$ (total size). c. First-fit leaves at most one bin $\leq 1/2$ full. d. First-fit uses $\leq \lceil 2S \rceil$ bins. e. Approximation ratio = 2. f. $O(n \log n)$ implementation using balanced BST.

35-2 Approximating Maximum Clique a. For $G^{(k)}$: maximum clique size = (max clique size in G) ^{k} . b. If constant-factor approximation exists for CLIQUE, then PTAS exists (use $G^{(k)}$ to amplify ratio).

35-3 Weighted Set Cover Natural greedy: pick set minimizing cost per uncovered element ($w_i/|S_i \cap U|$). Ratio = $H(d)$ where $d = \max |S_i|$ (d -th harmonic number).

35-4 Maximum Matching a. Maximal $\neq$ maximum matching (counterexample: path of 4 vertices). b. Greedy $O(E)$ -time maximal matching: iterate edges, add to M , remove incident vertices.

c. Size of any matching $\leq$ size of any vertex cover. d. Maximal matching M: graph induced by vertices not incident to M has no edges. e. $2|M|$ is vertex cover size ($2|M|$ vertices incident to M cover all edges). f. Greedy matching is 2-approximation for maximum matching.

35-5 Parallel Machine Scheduling a. $C_{\max} \geq \max p_j$. b. $C_{\max} \geq (1/m) \sum p_j$. c. Greedy pseudocode: keep array of machine finish times; for each job, assign to earliest available machine. d. $C_{\max} \leq (\sum p_j)/m + \max p_j \leq 2 \cdot C^*_{\max}$. This is a polynomial-time 2-approximation algorithm.

35-6 Approximating Maximum Spanning Tree a. SG = TG example: star graph. b. SG $\neq$ TG example: 4-cycle. c. SG $\supseteq$ TG for any graph G. d. $w(\text{SG}) \geq w(\text{TG})/2$. e. $O(V+E)$ 2-approximation: compute SG (max incident edge per vertex) and sum.

35-7 0-1 Knapsack 2-Approximation a. Optimal solution to I is one of $\{P_1, \dots, P_n\}$ (P_j requires item j). b. Fractional greedy (by v_i/w_i) for instance I_j . c. Optimal fractional solution has at most one fractional item. d. $v(R_j) \geq v(Q_j)/2 \geq v(P_j)/2$ (remove fractional item loses at most half). e. Algorithm: try each j , compute R_j , return max; polynomial-time 2-approximation.

Appendix A – Summations

Formulas & Equations

Definition of Summation: - Given a sequence $a_1, a_2, \dots, a_n$ of numbers where n is a nonnegative integer, the finite sum is expressed as $\sum_{k=1}^n a_k$. If $n = 0$, the value is defined to be 0. - Infinite sum: $\sum_{k=1}^{\infty} a_k = \lim_{n \rightarrow \infty} \sum_{k=1}^n a_k$. If the limit does not exist, the series diverges; otherwise it converges. - Absolutely convergent series: $\sum_{k=1}^{\infty} a_k$ such that $\sum_{k=1}^{\infty} |a_k|$ also converges.

Linearity: - $\sum_{k=1}^n (c a_k + b_k) = c \sum_{k=1}^n a_k + \sum_{k=1}^n b_k$ for any real number c and finite sequences $a_1, \dots, a_n, b_1, \dots, b_n$. - Also applies to infinite convergent series and asymptotic notation: $\sum_{k=1}^n \theta(f(k)) = \theta(\sum_{k=1}^n f(k))$.

Arithmetic Series: - $\sum_{k=1}^n k = n(n+1)/2 = \theta(n^2)$ - General arithmetic series: $\sum_{k=1}^n (a + bk) = \theta(n^2)$ for $a \geq 0, b > 0$

Sums of Squares and Cubes: - $\sum_{k=0}^n k^2 = n(n+1)(2n+1)/6$ - $\sum_{k=0}^n k^3 = n^2(n+1)^2/4$

Geometric Series: - $\sum_{k=0}^n x^k = (x^{n+1} - 1)/(x - 1)$ for real $x \neq 1$ - Infinite decreasing geometric series ($|x| < 1$): $\sum_{k=0}^{\infty} x^k = 1/(1 - x)$ - If $0^0 = 1$, formulas hold even when $x = 0$.

Harmonic Series: - $H_n = \sum_{k=1}^n 1/k = \ln n + O(1)$ - Stronger bounds: $\ln(n+1) \leq H_n \leq 1 + \ln n$ (from integral approximation)

Integrating and Differentiating Series: - Differentiating infinite geometric series and multiplying by x : $\sum_{k=0}^{\infty} k x^k = x/(1-x)^2$ for $|x| < 1$

Telescoping Series: - $\sum_{k=1}^n (a_k - a_{k-1}) = a_n - a_0$ - $\sum_{k=0}^{n-1} (a_k - a_{k+1}) = a_0 - a_n$ - Example: $\sum_{k=1}^n 1/(k(k+1)) = 1 - 1/(n+1)$ by rewriting $1/(k(k+1)) = 1/k - 1/(k+1)$

Reindexing: - $\sum_{k=0}^n a_{n-k} = \sum_{j=0}^n a_j$ by letting $j = n - k$ - Example: $\sum_{k=1}^n 1/(n-k+1) = H_n$ by setting $j = n-k+1$

Products: - $\prod_{k=1}^n a_k = a_1 a_2 \dots a_n$. If $n = 0$, value is 1. - Conversion: $\lg(\prod_{k=1}^n a_k) = \sum_{k=1}^n \lg a_k$

Processes

Bounding Summations:

1. Mathematical Induction:

- Prove arithmetic series $\sum_{k=1}^n k = n(n+1)/2$ by induction.
- Prove asymptotic upper bounds by induction (e.g., $\sum_{k=0}^n 3^k = O(3^{n+1})$).
- Warning: The "constant" hidden by big-O must not grow with n . Fallacious proof of $\sum_{k=1}^n k = O(n)$ demonstrates the bug.

2. Bounding Terms:

- Use largest term to bound others: $\sum_{k=1}^n a_k \leq n \cdot a_{\max}$ where $a_{\max} = \max\{a_k : 1 \leq k \leq n\}$
- If $a_{k+1}/a_k \leq r$ for all $k \geq 0$ with $0 < r < 1$, then $\sum_{k=0}^n a_k \leq \sum_{k=0}^{\infty} a_0 r^k = a_0/(1-r)$
- Common bug: ratio < 1 is insufficient; need a constant $r < 1$ that works for all consecutive pairs.

3. Splitting Summations:

- Partition index range into two or more series, bound each separately.
- Example: $\sum_{k=1}^n k \geq \sum_{k=n/2+1}^n k \geq (n/2) \cdot (n/2) = \Omega(n^2)$
- For $\sum_{k=1}^{\infty} k/2^k$, split at $k \geq 3$ where ratio $\leq 7/8$.
- Harmonic series bound: Split 1 to n into $\lceil \lg n \rceil + 1$ pieces, each bounded by 1, giving $H_n = O(\lg n)$.

4. Approximation by Integrals:

- For monotonically increasing $f(k)$: $\int_0^n f(x) dx \leq \sum_{k=1}^n f(k) \leq \int_1^{n+1} f(x) dx$
- For monotonically decreasing $f(k)$: $\int_1^{n+1} f(x) dx \leq \sum_{k=1}^n f(k) \leq \int_0^n f(x) dx$
- Used to prove bounds on harmonic numbers: $\ln(n+1) \leq H_n \leq 1 + \ln n$

Exercises

A.1-1: Prove that $\sum_{k=1}^n (2k-1) = n^2$ by using the linearity property of summations.

A.1-2: Find a simple formula for $\sum_{k=1}^n (2k-1)$.

A.1-3: Interpret the decimal number 111,111,111 in light of equation (A.6) (geometric series formula).

A.1-4: Evaluate the infinite series $\sum_{k=1}^{\infty} k/2^k$.

A.1-5: Let $c \geq 0$ be a constant. Show that $\sum_{k=1}^n k^c = \Theta(n^{c+1})$.

A.1-6: Show that $\sum_{k=1}^{\infty} k^2 x^k = x(1+x)/(1-x)^3$ for $|x| < 1$.

A.1-7: Prove that $\sum_{k=1}^n k^2 = n(n+1)(2n+1)/6$. (Hint: Show the asymptotic upper and lower bounds separately.)

□ **A.1-8:** Show that $\sum_{k=1}^n 1/k^2 = O(1)$ by manipulating the harmonic series.

□ **A.1-9:** Show that $\sum_{k=1}^n 1/k = \ln n + O(1)$.

□ **A.1-10:** Evaluate the sum $\sum_{k=1}^n k \cdot 2^k$.

□ **A.1-11:** Evaluate the product $\prod_{k=2}^n (1 - 1/k^2)$.

A.2-1: Show that $\sum_{k=1}^{\infty} 1/k^2$ is bounded above by a constant.

A.2-2: Find an asymptotic upper bound on the summation $\sum_{k=0}^{\infty} k^{2/2} k$.

A.2-3: Show that the n th harmonic number is $\Omega(\lg n)$ by splitting the summation.

A.2-4: Approximate $\sum_{k=1}^n \sqrt{k}$ with an integral.

A.2-5: Why can't you use the integral approximation (A.19) directly on $f(x) = 1/x$ to obtain an upper bound on the n th harmonic number?

Problems

A-1 Bounding summations Give asymptotically tight bounds on the following summations. Assume that $r \geq 0$ and $s \geq 0$ are constants.
a. $\sum_{k=1}^n k^r$ b. $\sum_{k=1}^n \lg^s k$ c. $\sum_{k=1}^n k^r \lg^s k$

Appendix B – Sets, Etc.

Named Entities

Set Definitions: - A set is a collection of distinguishable objects called members or elements. - Notation: $x \in S$ (x is a member of S), $x \notin S$ (x is not a member) - Sets are described with braces: $S = \{1, 2, 3\}$ - Sets cannot contain the same object more than once; elements are not ordered. - Two sets are equal ($A = B$) if they contain the same elements.

Special Sets: - $\emptyset$ = empty set (set containing no members) - $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ (integers) - $\mathbb{R}$ = real numbers - $\mathbb{N} = \{0, 1, 2, \dots\}$ (natural numbers)

Subset Relations: - $A \subseteq B$: A is a subset of B (all elements of A are in B) - $A \subset B$: A is a proper subset of B ($A \subseteq B$ but $A \neq B$) - Every set is a subset of itself: $A \subseteq A$ - $A = B$ iff $A \subseteq B$ and $B \subseteq A$ - Subset and proper-subset relations are transitive. - Empty set is a subset of all sets: $\emptyset \subseteq A$

Set Operations: - Intersection: $A \cap B = \{x : x \in A \text{ and } x \in B\}$ - Union: $A \cup B = \{x : x \in A \text{ or } x \in B\}$ - Difference: $A - B = \{x : x \in A \text{ and } x \notin B\}$ - Complement (given universe U): $\bar{A} = U - A = \{x : x \in U \text{ and } x \notin A\}$

Set Laws: - Empty set laws: $A \cap \emptyset = \emptyset$, $A \cup \emptyset = A$ - Idempotency laws: $A \cap A = A$, $A \cup A = A$ - Commutative laws: $A \cap B = B \cap A$, $A \cup B = B \cup A$ - Associative laws: $A \cap (B \cap C) = (A \cap B) \cap C$, $A \cup (B \cup C) = (A \cup B) \cup C$ - Distributive laws: $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$, $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ - Absorption laws: $A \cap (A \cup B) = A$, $A \cup (A \cap B) = A$ - DeMorgan's laws: $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$; $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ - Complement laws: $\bar{\bar{A}} = A$, $A \cap \bar{A} = \emptyset$, $A \cup \bar{A} = U$ - DeMorgan's laws (complement form): $\overline{(B \cap C)} = \bar{B} \cup \bar{C}$, $\overline{(B \cup C)} = \bar{B} \cap \bar{C}$

Disjoint Sets: - A and B are disjoint if $A \cap B = \emptyset$. - A collection $S = \{S_i\}$ forms a partition of set S if: (1) sets are pairwise disjoint ($S_i, S_j \subseteq S$, $i \neq j \Rightarrow S_i \cap S_j = \emptyset$), (2) their union is S ($\bigcup \{S_i \subseteq S\} S_i = S$).

Cardinality: - $|S|$ = number of elements in set S (cardinality or size) - $|\emptyset| = 0$ - Sets are finite if cardinality is a natural number, otherwise infinite. - Countably infinite: can be put in one-to-one correspondence with $\mathbb{N}$ (e.g., $\mathbb{Z}$ is countable; $\mathbb{R}$ is uncountable). - Identity: $|A \cup B| = |A| + |B| - |A \cap B|$ - $|A \cap B| \leq |A| + |B|$ - If A and B are disjoint: $|A \cup B| = |A| + |B|$ - If $A \subseteq B$: $|A| \leq |B|$

Special Sets: - n -set: a finite set of n elements - 1-set (singleton): a set with one element - k -subset: a subset of k elements - Power set: $2^S =$ set of all subsets of S , including $\emptyset$ and S ; $|2^S| = 2^{|S|}$

Ordered Structures: - Ordered pair: $(a, b) = \{a, \{a, b\}\}$ (note: $(a, b) \neq (b, a)$) - Cartesian product: $A \times B = \{(a, b) : a \in A \text{ and } b \in B\}$; $|A \times B| = |A| \cdot |B|$ - n-fold Cartesian product: $A_1 \times A_2 \times \dots \times A_n = \{(a_1, a_2, \dots, a_n) : a_i \in A_i \text{ for } i = 1, 2, \dots, n\}$; $|A_1 \times A_2 \times \dots \times A_n| = |A_1| \cdot |A_2| \cdot \dots \cdot |A_n|$ - $A^n = A \times A \times \dots \times A$ (n times); $|A^n| = |A|^n$

Intervals: - Closed interval $[a, b] = \{x \in \mathbb{R} : a \leq x \leq b\}$ - Open interval $(a, b) = \{x \in \mathbb{R} : a < x < b\}$ - Half-open intervals: $[a, b)$ and $(a, b]$ - Intervals can also be defined on integers by replacing $\mathbb{R}$ with $\mathbb{Z}$.

Relations: - Binary relation R on sets A and B : a subset of $A \times B$. If $(a, b) \in R$, write $a R b$. - n-ary relation: subset of $A_1 \times A_2 \times \dots \times A_n$. - Reflexive: $a R a$ for all $a \in A$. - Symmetric: $a R b \implies b R a$ for all $a, b \in A$. - Transitive: $a R b$ and $b R c \implies a R c$ for all $a, b, c \in A$. - Equivalence relation: reflexive, symmetric, and transitive. - Equivalence class of a : $[a] = \{b \in A : a R b\}$ - Theorem B.1: The equivalence classes of any equivalence relation R on a set A form a partition of A , and any partition of A determines an equivalence relation on A for which the sets in the partition are the equivalence classes. - Antisymmetric: $a R b$ and $b R a \implies a = b$. - Partial order: reflexive, antisymmetric, and transitive; the set is a partially ordered set. - Maximal element: a such that for no $b \neq a$ is $a R b$. - Total relation: for all $a, b \in A$, $a R b$ or $b R a$ (or both). - Total order (linear order): a partial order that is also a total relation. - Total preorder: a total relation that is transitive (not necessarily symmetric or antisymmetric).

Functions: - Function $f: A \rightarrow B$: a binary relation such that for all $a \in A$, there exists precisely one $b \in B$ with $(a, b) \in f$. - Domain = A , codomain = B . - Argument: a ; value: $b = f(a)$. - Finite sequence (length n): function f with domain $\{0, 1, \dots, n-1\}$, denoted $\langle f(0), f(1), \dots, f(n-1) \rangle$. - Infinite sequence: function with domain $\mathbb{N}$. - Image of $A' \subseteq A$ under f : $f(A') = \{b \in B : b = f(a) \text{ for some } a \in A'\}$. - Range: image of the domain, $f(A)$. - Surjection (onto): range = codomain. - Injection (one-to-one): $a \neq a' \implies f(a) \neq f(a')$. - Bijection (one-to-one correspondence): both injective and surjective. - Permutation: bijection from a set to itself. - Inverse: $f^{-1}(b) = a$ iff $f(a) = b$ (defined when f is bijective).

Graphs: - Directed graph (digraph) $G = (V, E)$: V = finite vertex set, E = binary relation on V (edges are ordered pairs). Self-loops allowed. - Undirected graph $G = (V, E)$: edges are unordered pairs $\{u, v\}$ with $u \neq v$ (no self-loops). - Edge (u, v) in directed: incident from/leaves u , incident to/enters v . - Edge (u, v) in undirected: incident on u and v . - Adjacent: v adjacent to u if $(u, v) \in E$. In undirected, adjacency symmetric; in directed, not necessarily. - Degree (undirected): number of edges incident on a vertex. Isolated: degree 0. - In-degree (directed): number of edges entering. Out-degree: number of edges leaving. Degree = in-degree + out-degree. - Path of length k : sequence

$\langle v_0, v_1, \dots, v_k \rangle$ where $(v_{i-1}, v_i) \in E$. Length = number of edges. - Simple path: all vertices distinct. - Subpath: contiguous subsequence of vertices. - Cycle (directed): path $\langle v_0, \dots, v_k \rangle$ with $v_0 = v_k$ and at least one edge. Simple: $v_1, \dots, v_k$ are distinct. - Self-loop: cycle of length 1. - Cycle (undirected): path with $k > 0$, $v_0 = v_k$, and all edges distinct. Simple: $v_1, \dots, v_k$ distinct. - Acyclic graph: no simple cycles. - Connected (undirected): every vertex reachable from all others. - Connected components: equivalence classes under "is reachable from." - Strongly connected (directed): every two vertices mutually reachable. - Strongly connected components: equivalence classes under "are mutually reachable." - Isomorphic graphs: exists bijection $f: V \rightarrow V'$ such that $(u, v) \in E$ iff $(f(u), f(v)) \in E'$. - Subgraph $G' = (V', E')$ of $G = (V, E)$: $V' \subseteq V$, $E' \subseteq E$. - Induced subgraph by V' : $G' = (V', E')$ where $E' = \{(u, v) \in E : u, v \in V'\}$. - Directed version of undirected G : each undirected edge (u, v) becomes two directed edges (u, v) and (v, u) . - Undirected version of directed G : contains edges with directions removed and self-loops eliminated. - Neighbor: in directed graph, vertex adjacent in undirected version; in undirected graph, same as adjacent.

Graph Types: - Complete graph: undirected graph where every pair of vertices is adjacent. - Bipartite graph: V can be partitioned into V_1, V_2 such that every edge goes between the sets. - Forest: acyclic, undirected graph. - Free tree: connected, acyclic, undirected graph. - Dag: directed acyclic graph. - Multigraph: undirected graph with possible multiple edges and self-loops. - Hypergraph: hyperedges connect arbitrary subsets of vertices.

Contraction: - Contraction of undirected graph G by edge $e = (u, v)$: $G' = (V', E')$ where $V' = V - \{u, v\} \cup \{x\}$ (new vertex), E' formed by deleting (u, v) and replacing edges incident to u or v with edges to x .

Trees:

Free Trees: - Free tree: connected, acyclic, undirected graph. - Forest: acyclic, undirected graph (possibly disconnected). - Theorem B.2 (Properties of free trees): Let $G = (V, E)$ be undirected. The following are equivalent: 1. G is a free tree. 2. Any two vertices in G are connected by a unique simple path. 3. G is connected, but removing any edge disconnects the graph. 4. G is connected and $|E| = |V| - 1$. 5. G is acyclic and $|E| = |V| - 1$. 6. G is acyclic, but adding any edge creates a cycle.

Rooted Trees: - Rooted tree: free tree with one distinguished vertex (the root). Vertices are called nodes. - Ancestor/descendant: y is ancestor of x if y is on unique simple path from root to x . Every node is ancestor and descendant of itself. - Proper ancestor/descendant: ancestor/descendant but not equal. - Subtree rooted at x : tree induced by descendants of x , rooted at x . - Parent/child: if last edge on path from root to x is (y, x) , y is parent, x is child. - Root is the only node with no parent. - Siblings: two nodes with the same parent. - Leaf (external

node): node with no children. Internal node: nonleaf node. - Degree of node in rooted tree: number of children. - Depth of node x : length of simple path from root to x . - Level: set of all nodes at the same depth. - Height of node: number of edges on longest simple downward path to a leaf. Height of tree = height of root = largest depth of any node.

Ordered Trees: - Ordered tree: rooted tree where children of each node are ordered (first child, second child, etc.).

Binary and Positional Trees: - Binary tree (recursive definition): either empty (NIL) or composed of a root node, a left subtree, and a right subtree. - Left child/right child: roots of left/right subtrees. - In a binary tree, position matters: a sole child is distinguished as left or right (unlike ordered trees). - Full binary tree: each node is either a leaf or has degree exactly 2 (no degree 1 nodes). - Positional tree: children of a node are labeled with distinct positive integers. The i th child is absent if no child is labeled i . - k -ary tree: positional tree where all children with labels $> k$ are missing. Binary tree = k -ary tree with $k = 2$. - Complete k -ary tree: all leaves have the same depth and all internal nodes have degree k . - Number of nodes at depth d : k^d - Number of leaves at depth h : k^h - Height of complete k -ary tree with n leaves: $\log_k n$ - Internal nodes in complete k -ary tree of height h : $(k^h - 1)/(k - 1)$ - Complete binary tree of height h : 2^h leaves, $2^h - 1$ internal nodes.

Kraft Inequality (B.5-7): - Weight $w(x) = 2^{-d}$ for leaf x at depth d in binary tree T . Let L be set of leaves. Then $\sum_{x \in L} w(x) \leq 1$.

Classifications

Graph Types: - Directed vs. undirected - Simple graph (no self-loops in directed) - Multigraph (multiple edges, self-loops) - Hypergraph (hyperedges connect arbitrary subsets) - Complete graph (every pair adjacent) - Bipartite graph (vertices partitionable into two sets, edges only between sets) - Forest (acyclic undirected) - Tree (connected acyclic undirected) - Dag (directed acyclic graph)

Tree Types: - Free tree (connected, acyclic, undirected) - Rooted tree (free tree with distinguished root) - Ordered tree (rooted tree with ordered children) - Binary tree (each node has left/right child, position matters) - Full binary tree (every node has degree 0 or 2) - k -ary tree (positional tree, max label k) - Complete k -ary tree (all leaves same depth, all internal nodes degree k)

Exercises

B.1-1: Draw Venn diagrams that illustrate the first of the distributive laws.

B.1-2: Prove the generalization of DeMorgan's laws to any finite collection of sets: $A_1 \cap A_2 \cap \dots \cap A_n = A_1 \cup A_2 \cup \dots \cup A_n$
 $A_1 \cup A_2 \cup \dots \cup A_n = A_1 \cap A_2 \cap \dots \cap A_n$

B.1-3: Prove the generalization of the inclusion-exclusion principle: $|A_1 \cup A_2 \cup \dots \cup A_n| = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots + (-1)^{n-1} |A_1 \cap A_2 \cap \dots \cap A_n|$

B.1-4: Show that the set of odd natural numbers is countable.

B.1-5: Show that for any finite set S , the power set 2^S has $2^{|S|}$ elements.

B.1-6: Give an inductive definition for an n -tuple by extending the set-theoretic definition for an ordered pair.

B.2-1: Prove that the subset relation " $\subseteq$ " on all subsets of Z is a partial order but not a total order.

B.2-2: Show that for any positive integer n , the relation "equivalent modulo n " is an equivalence relation on the integers. Into what equivalence classes does this relation partition the integers?

B.2-3: Give examples of relations that are: a. reflexive and symmetric but not transitive b. reflexive and transitive but not symmetric c. symmetric and transitive but not reflexive

B.2-4: Let S be a finite set, and let R be an equivalence relation on $S \times S$. Show that if in addition R is antisymmetric, then the equivalence classes of S with respect to R are singletons.

B.2-5: Professor Narcissus claims that if a relation R is symmetric and transitive, then it is also reflexive. He offers the following proof. By symmetry, $a R b$ implies $b R a$. Transitivity, therefore, implies $a R a$. Is the professor correct?

B.3-1: Let A and B be finite sets, and let $f: A \rightarrow B$ be a function. Show: a. If f is injective, then $|A| \leq |B|$. b. If f is surjective, then $|A| \geq |B|$.

B.3-2: Is the function $f(x) = x + 1$ bijective when the domain and codomain are N ? Is it bijective when the domain and codomain are Z ?

B.3-3: Give a natural definition for the inverse of a binary relation such that if a relation is in fact a bijective function, its relational inverse is its functional inverse.

□ **B.3-4:** Give a bijection from Z to $Z \times Z$.

B.4-1 (Handshaking Lemma): If $G = (V, E)$ is an undirected graph, then $\sum_{v \in V} \text{degree}(v) = 2|E|$.

B.4-2: Show that if a directed or undirected graph contains a path between vertices u and v , then it contains a simple path between u and v . Show that if a directed graph contains a cycle, then it contains a simple cycle.

B.4-3: Show that any connected, undirected graph $G = (V, E)$ satisfies $|E| \geq |V| - 1$.

B.4-4: Verify that in an undirected graph, the “is reachable from” relation is an equivalence relation on the vertices. Which properties hold for directed graphs?

B.4-5: What is the undirected version of the directed graph in Figure B.2(a)? What is the directed version of the undirected graph in Figure B.2(b)?

B.4-6: Show how a bipartite graph can represent a hypergraph by letting incidence in the hypergraph correspond to adjacency in the bipartite graph.

B.5-1: Draw all free trees composed of vertices x, y, z . Draw all rooted trees with nodes x, y, z and x as root. Draw all ordered trees with nodes x, y, z and x as root. Draw all binary trees with nodes x, y, z and x as root.

B.5-2: Let $G = (V, E)$ be a directed acyclic graph with vertex $v_0 \in V$ such that there exists a unique path from v_0 to every vertex $v \in V$. Prove that the undirected version of G forms a tree.

B.5-3: Show by induction that the number of degree-2 nodes in any nonempty binary tree is one less than the number of leaves. Conclude that the number of internal nodes in a full binary tree is one less than the number of leaves.

B.5-4: Prove that for any integer $k \geq 1$, there is a full binary tree with k leaves.

B.5-5: Use induction to show that a nonempty binary tree with n nodes has height at least $\lceil \lg n \rceil$.

□ **B.5-6:** The internal path length of a full binary tree is the sum over all internal nodes of the depth of each node. The external path length is the sum over all leaves of the depth of each leaf. Consider a full binary tree with n internal nodes, internal path length i , and external path length e . Prove that $e = i + 2n$.

□ **B.5-7 (Kraft inequality):** Associate weight $w(x) = 2^{-d}$ with each leaf x at depth d in binary tree T , and let L be the set of leaves. Prove $\sum_{x \in L} w(x) \leq 1$.

□ **B.5-8:** Show that if $L \geq 2$, then every binary tree with L leaves contains a subtree having between $L/3$ and $2L/3$ leaves, inclusive.

Problems

B-1 Graph coloring A k -coloring of undirected graph $G = (V, E)$ is a function $c: V \rightarrow \{1, \dots, k\}$ such that $c(u) \neq c(v)$ for every edge $(u, v) \in E$. a. Show that any tree is 2-colorable. b. Show that the following are equivalent: (1) G is bipartite, (2) G is 2-colorable, (3) G has no cycles of odd length. c. Let d be the maximum degree in G . Prove G can be colored with $d + 1$ colors. d. Show that if G has $O(|V|)$ edges, then G can be colored with $O(\sqrt{|V|})$ colors.

B-2 Friendly graphs Reword each statement as a theorem about undirected graphs and prove it. Friendship is symmetric but not reflexive. a. Any group of at least two people contains at least two with the same number of friends in the group. b. Every group of six people contains either ≥ 3 mutual friends or ≥ 3 mutual strangers. c. Any group can be partitioned into two subgroups such that at least half the friends of each person belong to the other subgroup. d. If everyone is the friend of at least half the people, then the group can be seated around a table so everyone is between two friends.

B-3 Bisecting trees a. Show that vertices of any n -vertex binary tree can be partitioned into two sets A and B , $|A| \leq 3n/4$ and $|B| \leq 3n/4$, by removing a single edge. b. Show constant $3/4$ is optimal by giving an example where the most balanced partition upon removal of single edge has $|A| = 3n/4$. c. Show that by removing at most $O(\lg n)$ edges, we can partition vertices into A and B with $|A| = \lfloor n/2 \rfloor$ and $|B| = \lfloor n/2 \rfloor$.

Appendix C – Counting and Probability

Formulas

Counting – Rules of Sum and Product: - Rule of sum (disjoint sets): $|A \cup B| = |A| + |B|$ - Rule of product: $|A \times B| = |A| \cdot |B|$

Strings: - A string over finite set S : sequence of elements of S .
- Number of k -strings over set S : $|S|^k$ - Number of binary k -strings: 2^k

Permutations: - Number of permutations of an n -set: $n!$ - Number of k -permutations of an n -set: $P(n, k) = n! / (n - k)! = n(n - 1) \cdots (n - k + 1)$

Combinations: - Number of k -combinations of an n -set (binomial coefficient): $C(n, k) = n! / (k!(n - k)!)$ - Symmetry: $C(n, k) = C(n, n - k)$ - For $k = 0$: $C(n, 0) = 1$

Binomial Theorem: - $(x + y)^n = \sum_{k=0}^n x^k y^{n-k}$ for $n \in \mathbb{N}$, $x, y \in \mathbb{R}$ - Special case ($x=y=1$): $\sum_{k=0}^n 1 = 2^n$

Binomial Bounds ($1 \leq k \leq n$): - Lower bound: $\geq (n/k)^k$ - Upper bound (using $k! \geq (k/e)^k$ from Stirling's approximation): $\leq (en/k)^k$ - For all $0 \leq k \leq n$: $\leq n^{k/2} \binom{n-1}{k-1}$ (from induction) - For $k = \lambda n$ ($0 \leq \lambda \leq 1$): $\leq 2^{n H(\lambda)}$ where $H(\lambda) = -\lambda \lg \lambda - (1-\lambda) \lg(1-\lambda)$ is the binary entropy function, with $0 \lg 0 = 0$ so $H(0) = H(1) = 0$.

Probability Axioms: - Sample space S : set of outcomes/elementary events. - Event: subset of S . - Probability distribution $\Pr\{\cdot\}$ on S is a mapping from events to real numbers satisfying: 1. $\Pr\{A\} \geq 0$ for any event A 2. $\Pr\{S\} = 1$ 3. $\Pr\{A \cup B\} = \Pr\{A\} + \Pr\{B\}$ for any two mutually exclusive events A, B . More generally, for finite or countably infinite pairwise mutually exclusive events: $\Pr\{\cup_i A_i\} = \sum \Pr\{A_i\}$

Probability Consequences: - $\Pr\{\emptyset\} = 0$ - If $A \subset B$ then $\Pr\{A\} \leq \Pr\{B\}$ - $\Pr\{\bar{A}\} = 1 - \Pr\{A\}$ - $\Pr\{A \cap B\} = \Pr\{A\} + \Pr\{B\} - \Pr\{A \cup B\}$

Discrete Probability Distributions: - For finite or countably infinite S : $\Pr\{A\} = \sum_{s \in A} \Pr\{s\}$ - Uniform distribution on finite S : $\Pr\{s\} = 1/|S|$ for all $s \in S$ - Flipping fair coin n times: sample space $S = \{H, T\}^n$, $|S| = 2^n$, each string probability $1/2^n$ - Probability of exactly k heads in n flips: $\Pr\{\text{exactly } k \text{ heads}\} = \binom{n}{k} / 2^n$

Continuous Uniform Distribution: - Defined over $[a, b]$ ($a < b$) on $\mathbb{R}$. - For $[c, d]$ with $a \leq c \leq d \leq b$: $\Pr\{[c, d]\} = (d - c)/(b - a)$ - Probability of a single point: 0 - Open interval (c, d) has same probability as $[c, d]$.

Conditional Probability: - $\Pr\{A | B\} = \Pr\{A \cap B\} / \Pr\{B\}$ (when $\Pr\{B\} \neq 0$) - Events A and B are independent if $\Pr\{A \cap B\} = \Pr\{A\} \Pr\{B\}$ - Equivalent: $\Pr\{A | B\} = \Pr\{A\}$ (when $\Pr\{B\} \neq 0$) - Pairwise independent: $\Pr\{A_i \cap A_j\} = \Pr\{A_i\} \Pr\{A_j\}$ for all $i < j$ - Mutually independent: $\Pr\{\cap_{i \in I} A_i\} = \prod_{i \in I} \Pr\{A_i\}$ for every k -subset I ($2 \leq k \leq n$)

Bayes' Theorem: - From definition: $\Pr\{A | B\} \Pr\{B\} = \Pr\{B | A\} \Pr\{A\}$ - Bayes' theorem: $\Pr\{A | B\} = \Pr\{B | A\} \Pr\{A\} / \Pr\{B\}$ - With denominator expanded: $\Pr\{B\} = \Pr\{B \cap A\} + \Pr\{B \cap \bar{A}\} = \Pr\{A\} \Pr\{B | A\} + \Pr\{\bar{A}\} \Pr\{B | \bar{A}\}$ - Equivalent form: $\Pr\{A | B\} = \Pr\{A\} \Pr\{B | A\} / (\Pr\{A\} \Pr\{B | A\} + \Pr\{\bar{A}\} \Pr\{B | \bar{A}\})$

Random Variables: - Discrete random variable X : function from finite or countably infinite sample space S to $\mathbb{R}$. - Event $X = x$: $\{s \in S : X(s) = x\}$ - $\Pr\{X = x\} = \sum_{s: X(s)=x} \Pr\{s\}$ - Probability density function: $f(x) = \Pr\{X = x\}$, with $f(x) \geq 0$ and $\sum_x \Pr\{X = x\} = 1$ - Joint probability density: $f(x, y) = \Pr\{X = x \text{ and } Y = y\}$ - Conditional: $\Pr\{X = x | Y = y\} = \Pr\{X = x \text{ and } Y = y\} / \Pr\{Y = y\}$ - Independence: $\Pr\{X = x \text{ and } Y = y\} = \Pr\{X = x\} \Pr\{Y = y\}$ for all x, y .

Expectation: - $E[X] = \sum_x x \cdot \Pr\{X = x\}$ (defined if sum finite or converges absolutely); also denoted μ_X or μ . - Linearity of expectation: $E[X + Y] = E[X] + E[Y]$ (even when X, Y not independent) - For any function g : $E[g(X)] = \sum_x g(x) \Pr\{X = x\}$ - For constant a : $E[aX] = a E[X]$ - Expectations are linear: $E[aX + Y] = a E[X] + E[Y]$ - Independent X, Y : $E[XY] = E[X] E[Y]$ - For n mutually independent X_i : $E[\prod X_i] = \prod E[X_i]$ - For X taking values in $N = \{0, 1, 2, \dots\}$: $E[X] = \sum_{i=1}^{\infty} \Pr\{X \geq i\}$

Jensen's Inequality: - Convex function f : $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$ for all x, y , $0 \leq \lambda \leq 1$ - Jensen: $f(E[X]) \leq E[f(X)]$ for convex f

Variance and Standard Deviation: - $\text{Var}[X] = E[(X - E[X])^2] = E[X^2] - E^2[X]$ - $\text{Var}[aX] = a^2 \text{Var}[X]$ - Independent X, Y : $\text{Var}[X + Y] = \text{Var}[X] + \text{Var}[Y]$ - For n pairwise independent X_i : $\text{Var}[\sum X_i] = \sum \text{Var}[X_i]$ - Standard deviation: $\sigma_X = \sqrt{\text{Var}[X]}$; variance denoted σ^2

Geometric Distribution: - Bernoulli trial: success with probability p , failure with probability $q = 1 - p$. - X = number of trials needed to obtain first success. - $\Pr\{X = k\} = q^{k-1} p$ for $k \geq 1$ - $E[X] = 1/p$ - $\text{Var}[X] = q/p^2$

Binomial Distribution: - X = number of successes in n Bernoulli trials (each success probability p). - $b(k; n, p) = \Pr\{X = k\} = p^k q^{n-k}$ for $k = 0, 1, \dots, n$ - $\sum_{k=0}^n b(k; n, p) = (p + q)^n = 1$ - $E[X] = np$ - $\text{Var}[X] = npq$ - Ratio of successive terms: $b(k; n, p)/b(k-1; n, p) = 1 + ((n+1)p - k)/(kq)$ - Distribution increases for $k < (n+1)p$, decreases for $k > (n+1)p$. - Maximum at $k \approx np$ (if $(n+1)p$ integer, two maxima at $(n+1)p$ and $(n+1)p-1 = np-q$). - Lemma C.1: $b(k; n, p) \leq (np/k)^k (nq/(n-k))^{n-k}$

Tail Bounds (Binomial):

- Theorem C.2 (Right tail, union bound): For $0 \leq k \leq n$, $\Pr\{X \geq k\} \leq p^k$
- Corollary C.3 (Left tail): For $0 \leq k \leq n$, $\Pr\{X \leq k\} \leq q^{n-k}$
- Theorem C.4 (Left tail bound using geometric series): For $0 < k < np$, $\Pr\{X < k\} < (kq/(np - k)) \cdot b(k; n, p) = (kq/(np - k)) \cdot p^k q^{n-k}$
- Corollary C.5: For $0 < k \leq np/2$, $\Pr\{X < k\} < (1/2) \Pr\{X < k+1\}$
- Corollary C.6 (Right tail): For $np < k < n$, $\Pr\{X > k\} < ((n-k)p/(k - np)) b(k; n, p)$
- Corollary C.7: For $(np + n)/2 < k < n$, $\Pr\{X > k\} < (1/2) \Pr\{X > k-1\}$
- Theorem C.8 (Chernoff bound, general p_i): For Bernoulli trials with probabilities p_i , let X = total successes, $\mu =$

$E[X]$. For $r > \mu$: $\Pr\{X \geq r\} \leq (e^{\mu-r} \mu^r)^r = (e\mu/r)^r e^{-\mu}$

- Corollary C.9: For same-p Bernoulli trials (each prob p), $\mu = np$. For $r > np$: $\Pr\{X \geq r\} \leq (e\mu/r)^r e^{-\mu} = (enp/r)^r e^{-np}$

Markov's Inequality: - For nonnegative random variable X and $t > 0$: $\Pr\{X \geq t\} \leq E[X]/t$

Boole's Inequality (Union Bound): - For finite or countably infinite sequence of events $A_1, A_2, \dots$: $\Pr\{\bigcup A_i\} \leq \sum \Pr\{A_i\}$

Processes

Counting Techniques: 1. Rule of sum: count union of disjoint sets by adding cardinalities. 2. Rule of product: count Cartesian product by multiplying cardinalities. 3. Strings: count k -strings over n -set as n^k . 4. Permutations: count ordered selections without replacement as $n!/(n-k)!$. 5. Combinations: count unordered selections without replacement as $n!/(k!(n-k)!)$. 6. Binomial theorem: use to sum binomial coefficients; expand $(x+y)^n$. 7. Pascal's triangle (Exercise C.1-8): construct table of binomial coefficients with recursive relation $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$. 8. Inclusion-exclusion: for union of overlapping sets.

Indicator Random Variables: - Used to simplify expectation calculations in probabilistic analysis. - Define $X_i = I\{\text{event}\} = 1$ if event occurs, 0 otherwise. - $E[X_i] = \Pr\{\text{event occurs}\}$. - By linearity of expectation: $E[\sum X_i] = \sum E[X_i]$. - Used in binomial distribution analysis: $E[X] = E[\sum X_i] = \sum p = np$.

Tail Bounds Techniques: - Theorem C.2 and Corollary C.3: simple union-bound-style bounds. - Theorem C.4: bound left tail by geometric series technique (from Appendix A.2). - Corollaries C.5 and C.7: show exponential decay far from mean. - Theorem C.8 (Chernoff-Hoeffding): general exponential bound using moment-generating function $e^{\alpha X}$ and Markov's inequality. Choose $\alpha = \ln(r/\mu)$ to minimize bound.

Exercises

C.1-1: How many k -substrings does an n -string have? How many substrings total?

C.1-2: An n -input, m -output boolean function is a function from $\{0,1\}^n$ to $\{0,1\}^m$. How many n -input, 1-output boolean functions are there? How many n -input, m -output?

C.1-3: In how many ways can n professors sit around a circular conference table? (Two seatings same if rotation equivalent.)

C.1-4: In how many ways to choose three distinct numbers from $\{1, \dots, 99\}$ so their sum is even?

C.1-5: Prove identity $\sum_{k=0}^n \binom{n}{k} = 2^n$ for $0 \leq k \leq n$ (Vandermonde's convolution).

C.1-6: Prove identity $\sum_{k=0}^n \binom{n}{k} = 2^n$ for $0 \leq k < n$.

C.1-7: Use distinguished-object argument to prove $\sum_{k=0}^n \binom{n}{k} = 2^n$ (Pascal's identity).

C.1-8: Using C.1-7, make Pascal's triangle table for $n = 0, \dots, 6$.

C.1-9: Prove $\sum_{k=0}^n \binom{n}{k} = 2^n$.

C.1-10: Show that $\binom{n}{k}$ achieves its maximum at $k = \lfloor n/2 \rfloor$ or $\lceil n/2 \rceil$.

C.1-11: Argue that $\binom{n}{k} = \binom{n}{n-k}$ for $n \geq 0, j \geq 0, k \geq 0, j+k \leq n$. Provide algebraic and combinatorial proofs. Give example where equality does not hold.

C.1-12: Use induction on $k \leq n/2$ to prove $\binom{n}{k} \leq \binom{n}{k-1}$, and extend to all $0 \leq k \leq n$ using symmetry.

C.1-13: Use Stirling's approximation to prove that $\binom{2n}{n} = \Theta(2^{2n}/\sqrt{n})$.

C.1-14: By differentiating the entropy function $H(\lambda)$, show that it achieves its maximum value at $\lambda = 1/2$. What is $H(1/2)$?

C.1-15: Show that for any integer $n \geq 0$, $\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}$.

C.1-16: Inequality (C.5) provides a lower bound on $\binom{n}{k}$. For small values of k , a stronger bound holds. Prove that $\binom{n}{k} \geq (n/k)^k$ for $1 \leq k \leq n$.

C.2-1: Professor Rosencrantz flips a fair coin twice. Professor Guildenstern flips a fair coin once. What is the probability that Rosencrantz obtains strictly more heads than Guildenstern?

C.2-2: Prove Boole's inequality: For any finite or countably infinite sequence of events $A_1, A_2, \dots$, $\Pr\{\bigcup_i A_i\} \leq \sum \Pr\{A_i\}$.

C.2-3: You shuffle a deck of 10 cards numbered 1-10, then remove three cards one at a time. What is the probability the three cards are in sorted (increasing) order?

C.2-4: Prove that $\Pr\{A \mid B\} + \Pr\{\bar{A} \mid B\} = 1$.

C.2-5: Prove that for any collection of events $A_1, A_2, \dots, A_n$, $\Pr\{\bigcup_{i=1}^n A_i\} \leq \sum_{i=1}^n \Pr\{A_i\}$.

C.2-6: Show how to construct a set of n events that are pairwise independent but such that no subset of $k > 2$ of them is mutually independent.

□ **C.2-7:** Two events A and B are conditionally independent, given C, if $\Pr\{A \cap B \mid C\} = \Pr\{A \mid C\} \cdot \Pr\{B \mid C\}$. Give a simple but nontrivial example of two events that are not independent but are conditionally independent given a third event.

□ **C.2-8:** Professor Gore teaches a music class. Three students—Jeff, Tim, and Carmine—are in danger of failing. Professor Gore tells them one will pass, two will fail. Carmine asks privately which of Jeff and Tim will fail. Professor Gore tells Carmine that Jeff will fail. Carmine figures either he or Tim will pass, so his probability of passing is now $1/2$. Is Carmine correct, or is his chance still $1/3$? Explain.

C.3-1: You roll two ordinary 6-sided dice. What is the expectation of the sum? What is the expectation of the maximum?

C.3-2: An array $A[1:n]$ contains n distinct numbers randomly ordered (each permutation equally likely). What is the expectation of the index of the maximum? Of the minimum?

C.3-3: A carnival game consists of three dice. Player bets on number 1-6. If number appears on exactly k of the three dice ($k=1,2,3$), player keeps the dollar and wins k more dollars. If number does not appear, player loses the dollar. What is the expected gain?

C.3-4: Argue that if X and Y are nonnegative random variables, then $E[\max\{X, Y\}] \leq E[X] + E[Y]$.

□ **C.3-5:** Let X and Y be independent random variables. Prove that $f(X)$ and $g(Y)$ are independent for any choice of functions f and g .

□ **C.3-6:** Let X be a nonnegative random variable with well-defined $E[X]$. Prove Markov's inequality: $\Pr\{X \geq t\} \leq E[X]/t$ for all $t > 0$.

□ **C.3-7:** Let S be a sample space, and let X and X' be random variables such that $X(s) \geq X'(s)$ for all $s \in S$. Prove that for any real constant t , $\Pr\{X \geq t\} \geq \Pr\{X' \geq t\}$.

C.3-8: Which is larger: the expectation of the square of a random variable, or the square of its expectation?

C.3-9: Show that for any random variable X that takes on only values 0 and 1, $\text{Var}[X] = E[X] E[1-X]$.

C.3-10: Prove that $\text{Var}[aX] = a^2 \text{Var}[X]$ from the definition of variance.

C.4-1: Verify axiom 2 of the probability axioms for the geometric distribution.

C.4-2: How many times on average do you need to flip six fair coins before obtaining three heads and three tails?

C.4-3: Show that the variance of the geometric distribution is q/p^2 . (Hint: Use Exercise A.1-6.)

C.4-4: Show that $b(k; n, p) = b(n-k; n, q)$, where $q = 1 - p$.

C.4-5: Show that the maximum of the binomial distribution $b(k; n, p)$ is approximately $1/\sqrt{2\pi npq}$, where $q = 1 - p$.

□ **C.4-6:** Show that probability of no successes in n Bernoulli trials, each with $p = 1/n$, is approximately $1/e$. Show that probability of exactly one success is also approximately $1/e$.

□ **C.4-7:** Professor Rosencrantz flips a fair coin n times, so does Professor Guildenstern. Show the probability they get the same number of heads is $1/2^{2n}$. Use this to verify the identity $\sum_{k=0}^n \binom{n}{k}^2 = 2^{2n}$.

□ **C.4-8:** Show that for $0 \leq k \leq n$, $b(k; n, 1/2) \leq 2^{n H(k/n) - k}$, where $H(x)$ is the entropy function.

□ **C.4-9:** Consider n Bernoulli trials, where trial i has probability p_i of success. Let X be total successes. Let $p \geq p_i$ for all i . Prove that for $1 \leq k \leq n$, $\Pr\{X \geq k\} \leq p^k$.

□ **C.4-10:** Let X be total successes in set A of n Bernoulli trials (trial i has prob p_i). Let X' be total successes in set A' (trial i has prob $p'_i \geq p_i$). Prove that for $0 \leq k \leq n$, $\Pr\{X' \geq k\} \geq \Pr\{X \geq k\}$.

□ **C.5-1:** Which is more likely: getting exactly n heads in $2n$ flips of a fair coin, or n heads in n flips of a fair coin?

□ **C.5-2:** Prove Corollaries C.6 and C.7.

□ **C.5-3:** Show that $\sum_{k=0}^{\lfloor na/(a+1) \rfloor} \binom{n}{k} a^k \leq a^{\lfloor na/(a+1) \rfloor}$ for all $a > 0$ and all k with $0 < k < na/(a+1)$.

□ **C.5-4:** Prove that if $0 < k < np$, where $0 < p < 1$ and $q = 1 - p$, then $\Pr\{X < k\} < (npq/(k-np)^2) b(k; n, p)$.

□ **C.5-5:** Use Theorem C.8 to show that $\Pr\{X \geq r\} \leq (\mu/r)^r e^{r-\mu}$ for $r > \mu$. Similarly, use Corollary C.9 to show that $\Pr\{X \geq r\} \leq (np/r)^r e^{r-np}$ for $r > np$.

□ **C.5-6:** Consider Bernoulli trials with probabilities p_i . Let X be total successes, $\mu = E[X]$. Show that for $r \geq 0$, $\Pr\{X \geq r\} \leq \prod_{i=1}^n (p_i e/r)^{r/n}$... [Alternative Chernoff bound].

□ **C.5-7:** Show that choosing $\alpha = \ln(r/\mu)$ minimizes the right-hand side of inequality (C.51).

Problems

C-1 The Monty Hall problem a. First pick random (prob $1/3$ right). Monty always offers switch. Prove switching is better. Probability of winning? b. What are the six outcomes in the sample space? Which correspond to winning? Probabilities in terms of p_{right} , p_{wrong} , p_{switch} ? c. Prove probability of winning is $(1/3)(1 + p_{\text{switch}}(p_{\text{right}} - p_{\text{wrong}}) + p_{\text{switch}} p_{\text{wrong}})$. d. If $p_{\text{switch}} > 0$, what is Monty's best strategy (choice of p_{right} , p_{wrong})? e. If $p_{\text{switch}} = 0$, argue all of Monty's strategies are optimal. f. If you know p_{right} and p_{wrong} , what is your best p_{switch} ? g. If you don't know p_{right} and p_{wrong} , what p_{switch} maximizes the minimum winning probability? h. Argue conditional probability of winning given Monty offers switch is $(p_{\text{right}} + 2p_{\text{wrong}} + p_{\text{switch}}(p_{\text{right}} - p_{\text{wrong}}))/(p_{\text{right}} + 2p_{\text{wrong}})(3)$. i. What is expression value when $p_{\text{switch}} = 1/2$? Show $p_{\text{switch}} < 1/2$ or $> 1/2$ allows Monty to lower it. j. Explain why $p_{\text{switch}} = 1/2$ is a good strategy. Summarize what you learned.

C-2 Balls and bins a. n distinct balls, order within bin doesn't matter: b^n ways. b. Distinct balls, ordered within bins: $(b+n-1)!/(b-1)!$ ways. c. Identical balls, order doesn't matter: ways. d. Identical balls, at most one per bin, $n \leq b$: ways. e. Identical balls, no empty bin, $n \geq b$: ways.

Appendix D – Matrices

Named Entities

Matrix Types: - **Matrix:** rectangular array of numbers. $A = (a_{ij})$ where i = row index, j = column index. Uppercase letters denote matrices, lowercase subscripted letters denote elements. - **Transpose:** A^T obtained by exchanging rows and columns ($a_{Tji} = a_{ij}$). - **Vector:** one-dimensional array of numbers (n -vector). Lowercase letters. Column vector = $n \times 1$ matrix; row vector = transpose of column vector. - **Unit vector e_i :** vector whose i th element is 1, all others 0. - **Zero matrix:** all entries 0, denoted 0. - **Square matrix:** $n \times n$ matrix.

Special Square Matrices: 1. **Diagonal matrix:** $a_{ij} = 0$ whenever $i \neq j$. Diagonal entries listed: $\text{diag}(a_{11}, a_{22}, \dots, a_{nn})$. 2. **Identity matrix I_n :** diagonal matrix with 1s on diagonal. I without subscript: size from context. i th column = unit vector e_i . 3. **Tridiagonal matrix T :** $t_{ij} = 0$ if $|i-j| > 1$. Nonzero only on main diagonal, superdiagonal ($t_{i,i+1}$), and subdiagonal ($t_{i+1,i}$). 4. **Upper-triangular matrix U :** $u_{ij} = 0$ if $i > j$. All entries below diagonal are 0. - **Unit upper-triangular:** all 1s on diagonal. 5. **Lower-triangular matrix L :** $l_{ij} = 0$ if $i < j$. All entries above diagonal are 0. - **Unit lower-triangular:** all 1s on diagonal. 6. **Permutation matrix P :** exactly one 1 in each row and column, 0s elsewhere. Multiplying x by P permutes elements. 7. **Symmetric matrix A :** $A = A^T$ (e.g., matrix in equation (D.7)). 8. **Positive-**

definite matrix: $n \times n$ matrix A such that $x^T A x > 0$ for all n -vectors $x \neq 0$. - Example: identity matrix is positive-definite. - Theorem D.6: For any matrix A with full column rank, $A^T A$ is positive-definite. 9. **Positive-semidefinite matrix:** $x^T A x \geq 0$ for all n -vectors $x \neq 0$. 10. **Singular (noninvertible) matrix:** a matrix without an inverse. 11. **Nonsingular (invertible) matrix:** a matrix that has an inverse. 12. **Vandermonde matrix** (Problem D-1): $V = [x_i^j]$ for $i, j = 0, \dots, n-1$; $\det(V) = \prod_{0 \leq i < j \leq n-1} (x_j - x_i)$. 13. **0-1 matrix:** entries are 0 or 1, used over $GF(2)$.

Formulas & Equations

Matrix Operations:

- **Matrix addition:** $C = A + B$ where $c_{ij} = a_{ij} + b_{ij}$ (componentwise). $A + 0 = A = 0 + A$.
- **Scalar multiplication:** $\lambda A = (\lambda a_{ij})$. Negative: $-A = (-1) \cdot A$. Subtraction: $A - B = A + (-B)$.
- **Matrix multiplication:** If A ($p \times q$) and B ($q \times r$), then $C = AB$ is $p \times r$ where $c_{ij} = \sum_{k=1}^q a_{ik} b_{kj}$.
 - Runtime: $\Theta(pqr)$ for straightforward algorithm; $\Theta(n^3)$ for $n \times n$; $\Theta(n^{\lg 7})$ by Strassen's algorithm.
- **Identity:** $I_m A = A I_n = A$ for any $m \times n$ matrix A .
- **Zero product:** $A \cdot 0 = 0$.
- **Associativity:** $A(BC) = (AB)C$ for compatible matrices.
- **Distributivity:** $A(B+C) = AB + AC$, $(B+C)D = BD + CD$.
- **Non-commutativity:** For $n > 1$, matrix multiplication is not commutative ($AB \neq BA$ in general).
- **Inner product:** $x^T y = \sum_{i=1}^n x_i y_i = \langle x, y \rangle$ (scalar, commutative: $\langle x, y \rangle = \langle y, x \rangle$).
- **Outer product:** xy^T is $n \times n$ matrix Z with $z_{ij} = x_i y_j$.
- **Norm (Euclidean):** $\|x\| = (\sum_{i=1}^n x_i^2)^{1/2} = \sqrt{x^T x}$. Length in n -dimensional Euclidean space.
 - Fact: For any real a and n -vector x : $\|a x\| = |a| \cdot \|x\|$.
- **$(AB)^T = B^T A^T$; $A^T A$ is always symmetric.**
- **Transpose of inverse:** $(A^{-1})^T = (A^T)^{-1}$.
- **Inverse of product:** $(BA)^{-1} = A^{-1} B^{-1}$.

Determinants: - Minor $A[ij]$ of $n \times n$ matrix A ($n > 1$): $(n-1) \times (n-1)$ matrix obtained by deleting i th row and j th column. - Cofactor of a_{ij} : $(-1)^{i+j} \det(A[ij])$. - Determinant (recursive): $\det(A) = \sum_{j=1}^n a_{ij} \cdot (-1)^{i+j} \det(A[ij])$ (for any fixed i), or $= \sum_{i=1}^n a_{ij} \cdot (-1)^{i+j} \det(A[ij])$ (for any fixed j).

Theorem D.4 (Determinant Properties): - If any row or any column of A is zero, then $\det(A) = 0$. - If entries of any one row (or column) are all multiplied by λ , then $\det(A)$ is multiplied by λ . - Adding entries of one row (column) to another row (column) leaves $\det(A)$ unchanged. - $\det(A) = \det(A^T)$. - Exchanging any two rows (or columns) multiplies $\det(A)$ by -1 . - For any square matrices A and B : $\det(AB) = \det(A) \det(B)$.

Theorem D.5: An $n \times n$ matrix A is singular iff $\det(A) = 0$.

Determinant of Vandermonde matrix (Problem D-1): - $\det(V) = \prod_{0 \leq i < j \leq n-1} (x_j - x_i)$ where $V = [V_{ij}] = [x_i^{n-j}]$.

Matrix Inverses: - Inverse A^{-1} (if exists): $AA^{-1} = I_n = A^{-1}A$. - Inverses are unique (Exercise D.2-1). - Example: $[[1,1],[1,0]]^{-1} = [[0,1],[1,-1]]$. - Singular (noninvertible) example: $[[1,1],[1,1]]$. - $(BA)^{-1} = A^{-1}B^{-1}$. - $(A^{-1})^T = (A^T)^{-1}$.

Rank: - Row rank = Column rank = rank of A (fundamental property). - rank of $m \times n$ matrix is an integer between 0 and $\min(m,n)$. - Alternative definition: smallest r such that $A = BC$ with B ($m \times r$) and C ($r \times n$). - Full rank (square): rank = n (nonsingular iff full rank – Theorem D.1). - Full column rank: rank = n . - $\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$; equality holds if either A or B is nonsingular square.

Theorem D.1: A square matrix has full rank iff it is nonsingular.

Theorem D.2: A matrix has full column rank iff it does not have a null vector ($Ax = 0 \implies x = 0$).

Corollary D.3: A square matrix is singular iff it has a null vector.

Linear Dependence/Independence: - Vectors $x_1, \dots, x_n$ are linearly dependent if there exist coefficients $c_1, \dots, c_n$ not all zero such that $\sum c_i x_i = 0$. - Otherwise, they are linearly independent. - Null vector: nonzero vector x such that $Ax = 0$.

Positive-Definite Matrices: - Definition: $x^T A x > 0$ for all $x \neq 0$. - Identity matrix is positive-definite: $x^T I x = \|x\|^2 > 0$ for $x \neq 0$. - Theorem D.6: For any matrix A with full column rank, $A^T A$ is positive-definite. - Proof: $x^T (A^T A) x = (Ax)^T (Ax) = \|Ax\|^2 > 0$ for $x \neq 0$.

Exercises

D.1-1: Show that if A and B are symmetric $n \times n$ matrices, then so are $A + B$ and $A - B$.

D.1-2: Prove that $(AB)^T = B^T A^T$ and that $A^T A$ is always a symmetric matrix.

D.1-3: Prove that the product of two lower-triangular matrices is lower-triangular.

D.1-4: Prove that if P is an $n \times n$ permutation matrix and A is an $n \times n$ matrix, then PA is A with rows permuted, and AP is A with columns permuted. Prove that the product of two permutation matrices is a permutation matrix.

D.2-1: Prove that matrix inverses are unique: if B and C are inverses of A , then $B = C$.

D.2-2: Prove that the determinant of a lower-triangular or upper-triangular matrix equals the product of its diagonal elements. Prove that the inverse of a lower-triangular matrix, if it exists, is lower-triangular.

D.2-3: Prove that if P is a permutation matrix, then P is invertible, its inverse is P^T , and P^T is a permutation matrix.

D.2-4: Let A and B be $n \times n$ matrices such that $AB = I$. Prove that if A' is obtained from A by adding row j into row i ($i \neq j$), then subtracting column i from column j of B yields the inverse B' of A' .

D.2-5: Let A be a nonsingular $n \times n$ matrix with complex entries. Show that every entry of A^{-1} is real iff every entry of A is real.

D.2-6: Show that if A is nonsingular, symmetric, $n \times n$, then A^{-1} is symmetric. Show that if B is an arbitrary $m \times n$ matrix, then the $m \times m$ matrix BAB^T is symmetric.

D.2-7: Prove Theorem D.2: a matrix A has full column rank iff $Ax = 0$ implies $x = 0$. (Hint: Express linear dependence of one column on others as matrix-vector equation.)

D.2-8: Prove that $\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$, where equality holds if either A or B is a nonsingular square matrix. (Hint: Use the alternate definition of rank.)

Problems

D-1 Vandermonde matrix Given numbers $x_0, x_1, \dots, x_{n-1}$, prove that the determinant of the Vandermonde matrix $V = [x_i^j]$ is $\det(V) = \prod_{0 \leq i < j \leq n-1} (x_j - x_i)$. (Hint: Multiply column i by $-x_0$ and add to column $i+1$ for $i = n-1, n-2, \dots, 1$, then use induction.)

D-2 Permutations defined by matrix-vector multiplication over $\text{GF}(2)$ Let $S_n = \{0, 1, \dots, 2^n - 1\}$. For $x \in S_n$, view binary representation as n -bit vector. For $n \times n$ 0-1 matrix A (over $\text{GF}(2)$ where $1+1=0$), define permutation $\pi_A: x \rightarrow Ax$. a. If $r = \text{rank}(A)$, prove $|R(A)| = 2^r$, where $R(A) = \{y : y = Ax \text{ for some } x \in S_n\}$. Conclude A defines permutation on S_n only if full rank. b. For $y \in R(A)$, preimage $P(A, y) = \{x : Ax = y\}$. If $r = \text{rank}(A)$, prove $|P(A, y)| = 2^{n-r}$. c. Let $r = \text{rank}$ of lower left $(n-m) \times m$ submatrix of A . Let S be any size- 2^m block of S_n , $S' = \{y : y = Ax \text{ for some } x \in S\}$. Prove $|B(S', m)| = 2^r$ and exactly 2^{m-r} numbers in S map to each block. d. Use counting argument to show linear permutations ($x \rightarrow Ax + c$, A full rank, c n -bit vector) are far fewer than all permutations of S_n . e. Give example of n and permutation of S_n not achievable by any linear permutation.

(Hint: Consider multiplying matrix by unit vector relates to columns.)

Cross-Cutting Topics

Design Paradigms & Meta-Methods

Divide-and-Conquer (Ch 2, 4)

- **Structure:** (1) Divide problem into smaller subproblems, (2) Conquer subproblems recursively, (3) Combine solutions
- **Recurrence:** $T(n) = aT(n/b) + D(n) + C(n)$ where a = number of subproblems, n/b = size of each, $D(n)$ = divide cost, $C(n)$ = combine cost
- **Key examples:** Merge Sort ($a=2$, $b=2$, $D=O(1)$, $C=O(n) \rightarrow T(n)=2T(n/2)+O(n) \rightarrow \Theta(n \lg n)$); Strassen's matrix multiplication ($a=7$, $b=2$, $C=O(n^2) \rightarrow \Theta(n^{\lg 7}) \approx \Theta(n^{2.81})$)
- **Solving recurrences:** Substitution (guess+induction), Recursion-tree, Master Theorem, Akra-Bazzi

Dynamic Programming (Ch 14)

- **Optimal substructure:** An optimal solution contains within it optimal solutions to subproblems
- **Overlapping subproblems:** Recursive algorithm revisits the same subproblem repeatedly $\rightarrow$ store solutions in table
- **Steps:** (1) Characterize structure of optimal solution, (2) Define value recursively, (3) Compute value bottom-up, (4) Construct solution
- **Variants:** Top-down (memoization) vs bottom-up (tabulation)
- **Classic problems:** Rod cutting, matrix-chain multiplication (optimal parenthesization), longest common subsequence (LCS), optimal binary search tree
- **Subproblem graph:** $G = (V, E)$ where V = subproblems, E = dependencies; DP runs in $O(V+E)$

Greedy Algorithms (Ch 15)

- **Greedy-choice property:** A globally optimal solution can be arrived at by making a locally optimal (greedy) choice
- **Optimal substructure:** Same as DP – after greedy choice, remaining subproblem must be optimal
- **Steps:** (1) Cast as optimization with greedy choice, (2) Show greedy choice is safe (no future choice invalidates it), (3) Show optimal substructure
- **Key examples:** Activity selection (earliest finish time), Huffman codes (merge least frequent), offline caching (furthest-in-future)
- **DP vs Greedy:** DP considers all subproblems; greedy makes one irrevocable choice then solves single remaining subproblem

Amortized Analysis (Ch 16)

- **Goal:** Average time per operation over a sequence, even if individual operations are expensive
- **Aggregate analysis:** Sum total cost of n operations, divide by $n \rightarrow$ amortized cost per operation
- **Accounting method:** Assign different charges to operations; credit stored for future expensive ops
- **Potential method:** Define potential function $\Phi(D_i)$ for data structure D_i ; amortized cost $c_i' = c_i + \Phi(D_i) - \Phi(D_{i-1})$
- **Key examples:** Multipop stack ($O(n)$ total $\rightarrow O(1)$ amortized per op), binary counter ($O(1)$ amortized per increment), dynamic table ($O(1)$ amortized per insertion, $O(1)$ per operation with deletion)

Augmenting Data Structures (Ch 17)

- **Four-step methodology:** (1) Choose underlying data structure, (2) Determine additional information, (3) Verify maintenance, (4) Develop new operations
- **Examples:** Order-statistic trees (augment RB tree with `node.size` for OS-SELECT/OS-RANK), interval trees (augment RB tree with `node.max` for INTERVAL-SEARCH)

Online Algorithms & Competitive Analysis (Ch 27)

- **Competitive ratio:** An online algorithm A is c -competitive if $C_A(I) \leq c \cdot C_{OPT}(I)$ for all input sequences I
- **Key algorithm:** MOVE-TO-FRONT achieves competitive ratio 4 for list searching
- **Key algorithm:** LRU caching achieves competitive ratio $O(k)$ for k -page cache
- **Lower bound:** Any deterministic online caching algorithm has competitive ratio $\geq k$
- **Randomized marking:** Achieves $O(\lg k)$ competitive ratio

Fork-Join Parallelism (Ch 26)

- **Work (T_1):** Total time on one processor (sum of all operations)
- **Span (T_∞):** Longest strand of dependencies (critical path)
- **Parallelism:** $T_1/T_\infty =$ average amount of work per step along critical path
- **Greedy scheduler:** $T_P \leq T_1/P + T_\infty$ achieves within factor 2 of optimal
- **Key law:** Work law $T_P \geq T_1/P$; Span law $T_P \geq T_\infty$

Proof & Argument Patterns

Loop Invariants

- **Structure:** Initialization (true before first iteration), Maintenance (if true before iteration, stays true after), Termination (when loop ends, invariant gives useful property)
- **Used for:** Proving correctness of iterative algorithms (Insertion sort, BFS, etc.)

Induction

- **Simple induction:** Prove base case; assume $P(k)$ for $k \geq 1$, prove $P(k+1)$
- **Strong induction:** Assume $P(1) \dots P(k)$ to prove $P(k+1)$
- **Structural induction:** Prove property for base structures; assume for substructures, prove for composite

Contradiction / Contrapositive

- To prove "if P then Q ", assume P and not Q , derive contradiction
- Used in cut property, cycle property, correctness proofs

Exchange Argument (Greedy)

- Show that any optimal solution can be transformed into the greedy solution without decreasing optimality
- Used for activity selection, Huffman codes, MST

Greedy Stays Ahead

- Show that at each step, greedy choice does at least as well as any optimal solution on a measure
- Example: earliest-finish-time greedy for activity selection stays ahead of OPT on count

Potential Function (Amortized Analysis)

- Define Φ that maps state to real number
- Show: $\Phi \geq 0$, $\Phi(\text{initial})=0$; actual cost = amortized cost - $\Delta\Phi$
- Sum telescopes: $\sum \text{actual} = \sum \text{amortized} + \Phi_{\text{final}} - \Phi_{\text{initial}}$

Cut-and-Paste (Optimal Substructure)

- Assume optimal solution contains suboptimal sub-solution; "cut out" suboptimal, "paste in" optimal $\rightarrow$ better overall solution
- Used in DP correctness, shortest paths, MST

Min-Cut Max-Flow Theorem Proof

- **Key equivalence:** Three statements equivalent for flow f : (1) f is max flow, (2) residual network G_f has no augmenting path, (3) $|f| = \text{capacity of some cut } (S,T)$
- **Proof structure:** (1) $\Leftrightarrow$ (2) by contradiction; (2) $\Leftrightarrow$ (3) by constructing cut from reachable vertices; (3) $\Leftrightarrow$ (1) since $|f| \leq \text{cap}(S,T)$ for any cut

NP-Completeness Reductions

- **Reduction types:** Mapping (function f converts instance of A to instance of B); $A \leq_P B$ means A is no harder than B
- **Proof structure:** (1) Show $B \in \text{NP}$ (give certificate), (2) Find known NP-complete A , construct polynomial reduction f from A to B , (3) Show $x \in A \Leftrightarrow f(x) \in B$, (4) Show f runs in polynomial time

Probability & Statistics Foundation (from Appendix C)

Counting Basics

- **Permutations:** $P(n,k) = n!/(n-k)!$ – ordered arrangements
- **Combinations:** $C(n,k) = n!/(k!(n-k)!)$ – unordered subsets
- **Binomial theorem:** $(x+y)^n = \sum_{k=0}^n C(n,k) x^k y^{n-k}$

Probability Axioms

- $0 \leq \text{Pr}[A] \leq 1$; $\text{Pr}[S] = 1$; $\text{Pr}[\emptyset] = 0$
- **Union bound:** $\text{Pr}[A \cup B] \leq \text{Pr}[A] + \text{Pr}[B]$
- **Conditional probability:** $\text{Pr}[A|B] = \text{Pr}[A \cap B] / \text{Pr}[B]$ (if $\text{Pr}[B] > 0$)
- **Bayes' theorem:** $\text{Pr}[A|B] = \text{Pr}[B|A]\text{Pr}[A] / \text{Pr}[B]$
- **Independence:** $\text{Pr}[A \cap B] = \text{Pr}[A]\text{Pr}[B]$; $\text{Pr}[A|B] = \text{Pr}[A]$

Random Variables

- **Expectation:** $E[X] = \sum_x x \cdot \text{Pr}[X=x]$; linearity: $E[X+Y] = E[X] + E[Y]$ always
- **Variance:** $\text{Var}[X] = E[(X-E[X])^2] = E[X^2] - (E[X])^2$
- **Indicator variables:** $I\{A\} = 1$ if A occurs, 0 otherwise; $E[I\{A\}] = \text{Pr}[A]$
- **Bernoulli trial:** One trial, success prob p ; indicator
- **Binomial distribution:** $B(n,p)$: X = number of successes in n Bernoulli trials; $\text{Pr}[X=k] = C(n,k) p^k (1-p)^{n-k}$; $E[X] = np$; $\text{Var}[X] = np(1-p)$
- **Geometric distribution:** Number of trials until first success; $\text{Pr}[X=k] = (1-p)^{k-1}p$; $E[X] = 1/p$; $\text{Var}[X] = (1-p)/p^2$

Tail Bounds

- **Markov's inequality:** $\Pr[X \geq t] \leq E[X]/t$ for nonnegative random variable X
- **Chebyshev's inequality:** $\Pr[|X - E[X]| \geq t] \leq \text{Var}[X]/t^2$
- **Chernoff bounds:** For sum of independent Bernoulli r.v.s with $\mu = E[X]$: $\Pr[X \geq (1+\delta)\mu] \leq (e^{\delta/(1+\delta)})^{1+\delta} \mu$; $\Pr[X \leq (1-\delta)\mu] \leq e^{-\mu\delta^2/2}$

Asymptotic Notation Properties (Ch 3)

- $\Theta(g(n)) = \{f(n) : \exists c_1, c_2 > 0, n_0 \text{ s.t. } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ } \forall n \geq n_0\}$
- $O(g(n)) = \{f(n) : \exists c > 0, n_0 \text{ s.t. } 0 \leq f(n) \leq c g(n) \text{ } \forall n \geq n_0\}$
- $\Omega(g(n)) = \{f(n) : \exists c > 0, n_0 \text{ s.t. } 0 \leq c g(n) \leq f(n) \text{ } \forall n \geq n_0\}$
- **Theorem 3.1:** $f(n) = \Theta(g(n)) \iff f(n) = O(g(n)) \text{ and } f(n) = \Omega(g(n))$
- **Transitivity:** $O, \Omega, \Theta, o, \omega$ are transitive
- **Reflexivity:** $f(n) = \Theta(f(n)), O(f(n)), \Omega(f(n))$
- **Symmetry:** $f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$

People & Dates

Person	Contribution	Chapter(s)
Thomas H. Cormen	Co-author, CLRS	All
Charles E. Leiserson	Co-author, CLRS	All
Ronald L. Rivest	Co-author, CLRS; RSA (R)	31
Clifford Stein	Co-author, CLRS	All
Volker Strassen	Strassen's matrix multiplication (1969)	4
Edsger Dijkstra	Dijkstra's shortest path algorithm	22
Richard Bellman	Bellman-Ford algorithm, dynamic programming	14, 22
Lester Ford	Ford-Fulkerson method	24
Delbert Fulkerson	Ford-Fulkerson method	24
Robert W. Floyd	Floyd-Warshall algorithm	23
Stephen Warshall	Floyd-Warshall algorithm	23
Donald Knuth	KMP algorithm, Knuth-Morris-Pratt	32
James H. Morris	KMP algorithm	32
Vaughan Pratt	KMP algorithm	32
Richard M. Karp	Edmonds-Karp algorithm, Karp reduction	24, 34
Jack Edmonds	Edmonds-Karp algorithm, matching	24, 25
David Huffman	Huffman coding	15
Joseph Kruskal	Kruskal's MST algorithm	21
Robert Prim	Prim's MST algorithm	21

Person	Contribution	Chapter(s)
Michael O. Rabin	Rabin-Karp algorithm	32
Ronald L. Rivest	RSA	31
Adi Shamir	RSA	31
Leonard Adleman	RSA	31
John Hopcroft	Hopcroft-Karp matching	25
David Gale	Gale-Shapley stable marriage	25
Lloyd Shapley	Gale-Shapley stable marriage	25
Harold Kuhn	Hungarian algorithm	25
James Cooley	Cooley-Tukey FFT	30
John Tukey	Cooley-Tukey FFT	30
Stephen Cook	Cook-Levin theorem (NP-completeness)	34
Leonid Levin	Cook-Levin theorem (NP-completeness)	34
Eugene Lawler	Key contributions to approximation algorithms	35

Mnemonics & Memory Aids

Big-O Ordering (slowest to fastest growth)

Mnemonic: "0(1) Cute Little Logs Line Up, N-Log-N Squared, N-Factorial!" $0(1) < O(\lg n) < O(\lg^2 n) < O(n) < O(n \lg n) < O(n^2) < O(2^n) < O(n!)$

7 Strassen's Matrices

$P_1 = a(f-h)$, $P_2 = (a+b)h$, $P_3 = (c+d)e$, $P_4 = d(g-e)$, $P_5 = (a+d)(e+h)$, $P_6 = (b-d)(g+h)$, $P_7 = (a-c)(e+f)$

Master Theorem Cases

"Dominant wins, equal splits log, smaller is irrelevant" - Case 1 (leaf-heavy): $f(n) = O(n^{\log_b a - \epsilon}) \rightarrow T(n) = \Theta(n^{\log_b a})$ - Case 2 (balanced): $f(n) = \Theta(n^{\log_b a} \lg^k n) \rightarrow T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$ - Case 3 (root-heavy): $f(n) = \Omega(n^{\log_b a + \epsilon})$ AND regularity $\rightarrow T(n) = \Theta(f(n))$

Five properties of Red-Black Trees

"Red Roots Are Every Black Leaf" (1. Red property, 2. Root black, 3. Red's children black, 4. Equal black height {every path}, 5. Black leaf sentinels)

Fibonacci Sequence

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ... **Count:** $F_0=0$, $F_1=1$; $F_i = F_{i-1} + F_{i-2}$ **Growth:** $F_i = \phi^i / \sqrt{5}$ rounded, where $\phi = (1+\sqrt{5})/2 \approx 1.618$

Sorting Algorithm Complexities

Algorithm	Best	Average	Worst	Space	Stable?
Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Yes
Merge	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n)$	Yes
Heap	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$O(1)$	No
Quick (random)	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n^2)$	$O(\lg n)$	No
Counting	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(k)$	Yes
Radix	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$\Theta(d(n+k))$	$\Theta(n+k)$	Yes
Bucket	$\Theta(n)$	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n)$	Yes

Exam Questions by Type

MCQ

- Q:** Which of the following is the worst-case running time of quicksort?
1. $\Theta(n)$ B) $\Theta(n \lg n)$ C) $\Theta(n^2)$ D) $\Theta(\lg n)$ **A:** C. Quicksort has worst-case $\Theta(n^2)$ when the partition is always unbalanced. **Distractor B:** $\Theta(n \lg n)$ is the expected/average case.
- Q:** Which data structure supports INSERT, DELETE, and SEARCH in $O(1)$ expected time?
1. Binary search tree B) Heap C) Hash table D) Red-black tree **A:** C. Hash tables support dictionary operations in $O(1)$ average time. **Distractor A:** BST supports $O(\lg n)$ on average.
- Q:** The Floyd-Warshall algorithm computes:
1. Single-source shortest paths B) All-pairs shortest paths C) Minimum spanning tree D) Maximum flow **A:** B. Floyd-Warshall finds shortest paths between all pairs of vertices. **Distractor A:** Bellman-Ford and Dijkstra compute single-source shortest paths.
- Q:** In a red-black tree, the black-height property means:
1. The root is always black B) Every leaf is black C) All paths from root to leaves have the same number of black nodes D) Red nodes have only black children **A:** C. Property 4: every path from root to leaf has the same number of black nodes. **Distractor A:** True but not the black-height property – it's property 2.
- Q:** Strassen's algorithm multiplies two $n \times n$ matrices in:
1. $\Theta(n^3)$ B) $\Theta(n^{2.81})$ C) $\Theta(n^2)$ D) $\Theta(n \lg n)$ **A:** B. Strassen's runs in $\Theta(n^{\lceil \lg 7 \rceil}) \approx \Theta(n^{2.81})$. **Distractor A:** $\Theta(n^3)$ is the standard iterative algorithm.
- Q:** The maximum number of leaves in a binary tree of height h is:
1. h B) $2h$ C) 2^h D) 2^{h+1} **A:** C. A binary tree of height h has at most 2^h leaves (complete binary tree). **Distractor D:** $2^{h+1}-1$ is the maximum number of nodes.

7. **Q:** Which algorithm uses the greedy choice of earliest finish time?
1. Huffman coding B) Activity selection C) Prim's algorithm D) Kruskal's algorithm **A:** B. Activity selection greedily selects the activity with the earliest finish time.
Distractor A: Huffman codes greedily merge the two least frequent characters.
8. **Q:** The Bellman-Ford algorithm can detect:
1. Only positive-weight cycles B) Only negative-weight cycles C) Both positive and negative-weight cycles D) Neither **A:** B. Bellman-Ford detects negative-weight cycles reachable from the source. **Distractor C:** Positive-weight cycles don't affect shortest paths.

Trace / Apply

1. **Insertion sort trace:** Sort $A = [5, 2, 4, 6, 1, 3]$. **Expected:**
 - $j=2$: $\text{key}=2$, compare with 5 $\rightarrow [2, 5, 4, 6, 1, 3]$
 - $j=3$: $\text{key}=4$, compare 5 $\rightarrow$ shift, compare 2 $\rightarrow$ stop $\rightarrow [2, 4, 5, 6, 1, 3]$
 - $j=4$: $\text{key}=6$, no shifts needed $\rightarrow [2, 4, 5, 6, 1, 3]$
 - $j=5$: $\text{key}=1$, shift all $\rightarrow [1, 2, 4, 5, 6, 3]$
 - $j=6$: $\text{key}=3$, shift 4, 5, 6 $\rightarrow [1, 2, 3, 4, 5, 6]$
2. **Build max-heap:** Build a max-heap from $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$ **Expected:**
 - Calls MAX-HEAPIFY from $i = \lfloor n/2 \rfloor$ down to 1
 - $i=5$: $16 \leftrightarrow 7 \rightarrow [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$
 - $i=4$: $14 \leftrightarrow 2 \rightarrow [4, 1, 3, 14, 16, 9, 10, 2, 8, 7]$
 - $i=3$: $10 \leftrightarrow 3 \rightarrow [4, 1, 10, 14, 16, 9, 3, 2, 8, 7]$
 - $i=2$: $16 \leftrightarrow 1 \rightarrow [4, 16, 10, 14, 1, 9, 3, 2, 8, 7] \rightarrow$ then $1 \leftrightarrow 7 \rightarrow [4, 16, 10, 14, 7, 9, 3, 2, 8, 1]$
 - $i=1$: $16 \leftrightarrow 4 \rightarrow [16, 4, 10, 14, 7, 9, 3, 2, 8, 1] \rightarrow$ then $14 \leftrightarrow 4 \rightarrow [16, 14, 10, 4, 7, 9, 3, 2, 8, 1] \rightarrow$ then $8 \leftrightarrow 4 \rightarrow [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$
3. **Dijkstra's algorithm:** Find shortest paths from $s=0$ given graph: $(0 \rightarrow 1:4, 0 \rightarrow 2:1, 2 \rightarrow 1:2, 1 \rightarrow 3:1, 2 \rightarrow 3:5)$ **Expected:**
 - Extract 0 ($d=0$); relax $1 \rightarrow 4, 2 \rightarrow 1$
 - Extract 2 ($d=1$); relax $1 \rightarrow 3, 3 \rightarrow 6$
 - Extract 1 ($d=3$); relax $3 \rightarrow 4$
 - Extract 3 ($d=4$)
 - Result: $\text{dist}[0]=0, \text{dist}[1]=3, \text{dist}[2]=1, \text{dist}[3]=4$
4. **Rod cutting (DP):** Price table $p = [1, 5, 8, 9, 10]$ for lengths 1..5. Compute max revenue for $n=4$. **Expected:**
 - $r[0]=0; r[1]=1; r[2]=\max(2, 1+1)=5; r[3]=\max(3, 1+5, 5+1, 8)=8; r[4]=\max(4, 1+8, 5+5, 8+1, 9)=10$
 - Optimal cut: length $2+2 \rightarrow$ revenue 10
5. **LCS:** Find LCS of $X = \langle A, B, C, B, D, A, B \rangle$ and $Y = \langle B, D, C, A, B, A \rangle$ **Expected:**
 - Table computed: LCS length = 4
 - LCS = $\langle B, C, B, A \rangle$ or $\langle B, D, A, B \rangle$

Short Answer

1. **Define asymptotic notation and explain the differences between O , Ω , Θ , o , and ω .** *Rubric:* O =asymptotic upper bound, Ω =lower bound, Θ =tight bound (both O and Ω), o =strict upper bound (not tight), ω =strict lower bound. Must include formal definitions with existential quantifiers.
2. **Explain the min-cut max-flow theorem.** *Rubric:* The value of a maximum flow equals the capacity of a minimum cut. Prove via three equivalent conditions.
3. **What is the difference between dynamic programming and greedy algorithms?** *Rubric:* Both require optimal substructure. DP requires overlapping subproblems and considers all choices; greedy makes irrevocable local choice and solves single remaining subproblem. DP is correct for more problems but slower.
4. **Describe the four-step methodology for augmenting a data structure.** *Rubric:* (1) Choose base structure, (2) Determine additional info, (3) Verify maintenance under modifications, (4) Develop new operations.
5. **Explain why NP-complete problems are considered intractable.** *Rubric:* No polynomial-time algorithm known; if any P-time algorithm exists for one NPC problem, then $P=NP$; thousands of problems are NPC; widely believed $P \neq NP$.

Diagram Label

1. **Red-black tree insertion cases:**
 - Case 1: Uncle is red $\rightarrow$ recolor parent/uncle/grandparent, move z up
 - Case 2: Uncle is black, z is right child $\rightarrow$ left rotate, fall through to Case 3
 - Case 3: Uncle is black, z is left child $\rightarrow$ right rotate + recolor
2. **BFS on undirected graph:**
 - Queue-based level-order traversal
 - Vertices colored WHITE (undiscovered), GRAY (discovered, in queue), BLACK (finished)
 - Distances increase monotonically
3. **Ford-Fulkerson residual network:**
 - Forward edges with residual capacity $c_f(u,v) = c(u,v) - f(u,v)$
 - Backward edges with residual capacity $c_f(v,u) = f(u,v)$
 - Augmenting path in $G_f \rightarrow$ flow increase

Essay / Long-Form

1. **Compare and contrast Dijkstra's algorithm and the Bellman-Ford algorithm. When would you use each?** *Key points:*

Dijkstra: $O(E \lg V)$, non-negative weights only, greedy.
 Bellman-Ford: $O(VE)$, handles negative weights, detects negative cycles. Dijkstra for road networks (non-negative), Bellman-Ford for currency arbitrage or graphs with negative edges.

2. ****Describe the structure of a reduction proof showing a problem B is NP-complete. Use 3-CNF-SAT $\leq_P$ CLIQUE as an example.**** *Key points:* Show $B \in \text{NP}$ (give certificate verifier), reduce from known NPC problem A, construct f such that $x \in A \iff f(x) \in B$, show polynomial time. For SAT $\rightarrow$ CLIQUE: construct 3-CNF formula ϕ , create graph with vertices for each literal in each clause, connect all literals except those in same clause or contradictory, ϕ satisfiable $\iff G$ has clique of size k .
3. **Explain the theory of NP-completeness: the classes P, NP, NPC, co-NP, the Cook-Levin theorem, and the implications of P vs NP.** *Key points:* P = polynomial-time decidable, NP = polynomial-time verifiable, NPC = hardest in NP. Cook-Levin: CIRCUIT-SAT is NP-complete (first such proof). If any NPC problem is in P, then $P = \text{NP}$. Open problem since 1971, \$1M prize. Practical implications: exponential time for exact solutions to NPC problems $\rightarrow$ use approximation, heuristics, or special cases.